

POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Core Python File Report

Hierarchical Reinforcement Learning for Opportunistic Aircraft Maintenance

A line-by-line guide to the maintenance tutorial

Soroush Asadian
Student Number: 270334

Supervisor: Emanuele Petriconi

26 October 2026

Contents

1 Purpose, source version, and reading instructions **7**

2 What changed since the earlier code report **7**

3 The environment, policy, trainer, and evaluator **8**

3.1 Four responsibilities 8

3.2 The notebook's library trainer versus the custom fleet trainers 9

3.3 One day is the physical time unit 9

4 State, observations, identities, and maintenance actions **9**

4.1 Hidden plant and observed belief 9

4.2 Bitwise selection and aircraft plates 10

4.3 Full eight-field action and duration 10

4.4 Reservations, costs, and observation updates 11

5 Single-agent, shared policies, CTDE, and global component actors **11**

5.1 Four independent design questions 11

5.2 Why one shared actor is not the original joint fleet controller 12

5.3 Why single-agent flat PPO is not DCMAC despite similar network counts . . 12

5.4 Why DDMAC has N networks and still needs global observations 12

5.5 Critic sharing is a separate choice 12

6 Every implemented method and its concrete differences **13**

6.1 flat 13

6.2 conditional 13

6.3 shared 14

6.4 options 14

6.5 learned_options 14

6.6 mappo_aircraft 14

6.7 mappo_component 14

6.8 dcmac_aircraft 14

6.9 ddmac_aircraft 15

6.10 dcmac_component 15

6.11 ddmac_component 15

6.12 concurrent_options 15

6.13 planning_hierarchy 15

6.14 aggregate_targets 16

6.15 maxq 16

6.16 option_critic 16

6.17 hierarchical_vdn . . . 16
6.18 hrl_flat . . . 16

7 Training equations and what their axes mean **16**
7.1 Fleet-joint PPO and per-agent MAPPO . . . 16
7.2 Replay-based DCMAC and DDMAC . . . 17
7.3 Aircraft option durations and completion values . . . 17
7.4 Option-critic termination and two-level VDN . . . 17

8 Comparison design, verification, and remaining work **18**
8.1 Which comparisons isolate which changes . . . 18
8.2 Recorded software checks . . . 18
8.3 Important physical and methodological limits . . . 18

9 Source walkthrough conventions **19**
9.1 Module header and imports . . . 19
9.2 MiniMaintenanceEnv . . . 20
9.3 MiniMaintenanceEnv.___init___ . . . 22
9.4 MiniMaintenanceEnv.reset . . . 22
9.5 MiniMaintenanceEnv.step . . . 24
9.6 MiniMaintenanceEnv._observation . . . 25
9.7 MiniMaintenanceEnv.render_text . . . 26
9.8 random_policy . . . 26
9.9 threshold_policy . . . 27
9.10 rollout . . . 27
9.11 Module declarations and integration . . . 28
9.12 valid_action . . . 29
9.13 all_actions . . . 30
9.14 ConditionalMaintenanceEnv . . . 30
9.15 ConditionalMaintenanceEnv.___init___ . . . 31
9.16 ConditionalMaintenanceEnv._observation . . . 31
9.17 ConditionalMaintenanceEnv.reset . . . 32
9.18 ConditionalMaintenanceEnv.step . . . 32
9.19 Module declarations and integration . . . 34
9.20 GymMaintenanceEnv . . . 35
9.21 GymMaintenanceEnv.___init___ . . . 35
9.22 GymMaintenanceEnv._encode . . . 36
9.23 GymMaintenanceEnv.reset . . . 36
9.24 GymMaintenanceEnv.step . . . 37
9.25 GymMaintenanceEnv.render . . . 37
9.26 GymMaintenanceEnv.close . . . 38

9.27 Module declarations and integration . . . 38
9.28 Config . . . 39
9.29 Config.___post_init___ . . . 41
9.30 Plan . . . 42
9.31 Plan.crews . . . 43
9.32 Job . . . 44
9.33 FleetMaintenanceEnv . . . 44
9.34 FleetMaintenanceEnv.___init___ . . . 46
9.35 FleetMaintenanceEnv.reset . . . 47
9.36 FleetMaintenanceEnv.split_observation . . . 50
9.37 FleetMaintenanceEnv.idle_action . . . 50
9.38 FleetMaintenanceEnv.available_crew_mask . . . 50
9.39 FleetMaintenanceEnv.resource_state . . . 51
9.40 FleetMaintenanceEnv.resource_details . . . 51
9.41 FleetMaintenanceEnv.sensor_noise . . . 52
9.42 FleetMaintenanceEnv._sensor_model . . . 52
9.43 FleetMaintenanceEnv._clip_measurements . . . 53
9.44 FleetMaintenanceEnv._invert_sensor_metrics . . . 54
9.45 FleetMaintenanceEnv.observation_metrics . . . 54
9.46 FleetMaintenanceEnv._observation . . . 55
9.47 FleetMaintenanceEnv.validate_row . . . 57
9.48 FleetMaintenanceEnv._start_job . . . 58
9.49 FleetMaintenanceEnv._update_filter . . . 60
9.50 FleetMaintenanceEnv.step . . . 62
9.51 FleetMaintenanceEnv.render . . . 68
9.52 threshold_action . . . 69
9.53 Module declarations and integration . . . 72
9.54 FlattenFleetAction . . . 72
9.55 FlattenFleetAction.___init___ . . . 72
9.56 FlattenFleetAction.action . . . 73
9.57 FlattenFleetAction.reverse_action . . . 73
9.58 Module declarations and integration . . . 74
9.59 FleetActorCritic . . . 74
9.60 FleetActorCritic.___init___ . . . 75
9.61 FleetActorCritic.value . . . 77
9.62 FleetActorCritic.act . . . 77
9.63 OptionActorCritic . . . 81
9.64 OptionActorCritic.___init___ . . . 82
9.65 OptionActorCritic.value . . . 82
9.66 OptionActorCritic.act . . . 83

9.67 LearnedOptionActorCritic 83
9.68 LearnedOptionActorCritic.___init___ 84
9.69 LearnedOptionActorCritic.value 85
9.70 LearnedOptionActorCritic.act 85
9.71 Module declarations and integration 86
9.72 mappo_agent_inputs 86
9.73 MultiAgentFleetEnv 88
9.74 MultiAgentFleetEnv.___init___ 89
9.75 MultiAgentFleetEnv.observations 91
9.76 MultiAgentFleetEnv.reset 91
9.77 MultiAgentFleetEnv.compose_action 92
9.78 MultiAgentFleetEnv.step 96
9.79 MultiAgentActorCritic 97
9.80 MultiAgentActorCritic.___init___ 97
9.81 MultiAgentActorCritic.agent_inputs 99
9.82 MultiAgentActorCritic.policy 99
9.83 MultiAgentActorCritic.active_agents 102
9.84 MultiAgentActorCritic.value 102
9.85 MultiAgentActorCritic.act 103
9.86 MultiAgentActorCritic.action_dict 103
9.87 Module declarations and integration 104
9.88 request_assignment 104
9.89 DCMACActorCritic 104
9.90 DCMACActorCritic.___init___ 105
9.91 DCMACActorCritic.agent_inputs 107
9.92 DCMACActorCritic._busy_fields 108
9.93 DCMACActorCritic.active_agents 108
9.94 DCMACActorCritic.policy 109
9.95 DCMACActorCritic.value 110
9.96 DCMACActorCritic.act 111
9.97 dcmac_losses 111
9.98 train_dcmac 113
9.99 mappo_gae 119
9.100mappo_policy_loss 121
9.101train_mappo 121
9.102Module declarations and integration 127
9.103hrl_mlp 128
9.104hrl_catalog_mask 128
9.105hrl_categorical 130
9.106AircraftOptionEnv 131

9.107AircraftOptionEnv.___init___ . . . 132
9.108AircraftOptionEnv.reset . . . 132
9.109AircraftOptionEnv.decision_mask . . . 133
9.110AircraftOptionEnv.step . . . 134
9.111AircraftPlanningHierarchy . . . 136
9.112AircraftPlanningHierarchy.___init___ . . . 137
9.113AircraftPlanningHierarchy.features . . . 139
9.114AircraftPlanningHierarchy.value . . . 139
9.115AircraftPlanningHierarchy.actor_parameters . . . 140
9.116AircraftPlanningHierarchy.act . . . 140
9.117AircraftPlanningHierarchy.catalog_actions . . . 144
9.118hrl_option_target . . . 145
9.119train_aircraft_hrl . . . 146
9.120MAXQMaintenance . . . 151
9.121MAXQMaintenance.___init___ . . . 152
9.122MAXQMaintenance.state . . . 153
9.123MAXQMaintenance.task_children . . . 153
9.124MAXQMaintenance.value . . . 155
9.125MAXQMaintenance.q . . . 155
9.126MAXQMaintenance.update_completion . . . 156
9.127MAXQMaintenance.execute . . . 156
9.128MAXQMaintenance.get_extra_state . . . 159
9.129MAXQMaintenance.set_extra_state . . . 159
9.130train_maxq . . . 159
9.131FleetOptionCritic . . . 161
9.132FleetOptionCritic.___init___ . . . 161
9.133FleetOptionCritic.features . . . 162
9.134FleetOptionCritic.q_values . . . 163
9.135FleetOptionCritic.beta . . . 163
9.136FleetOptionCritic.policy . . . 164
9.137option_critic_target . . . 165
9.138hrl_epsilon_choice . . . 165
9.139train_option_critic . . . 166
9.140HierarchicalVDN . . . 171
9.141HierarchicalVDN.___init___ . . . 172
9.142HierarchicalVDN.features . . . 172
9.143HierarchicalVDN.high_values . . . 173
9.144HierarchicalVDN.low_values . . . 173
9.145HierarchicalVDN.mix . . . 174
9.146HierarchicalVDN.choose_goals . . . 174

9.147HierarchicalVDN.choose_actions 175
9.148train_hierarchical_vdn 176
9.149evaluate_hrl 181
9.150Module declarations and integration 184
9.151execute_option 184
9.152Module declarations and integration 187
9.153make_model 188
9.154load_checkpoint 189
9.155smdp_gae 190
9.156train 191
9.157evaluate 197
9.158main 200
9.159Module declarations and integration 203

10 Closing interpretation **204**

1. Purpose, source version, and reading instructions

This report documents the current core Python implementation of the aircraft-maintenance tutorial, including the environment and all implemented learning methods. It supersedes the implementation descriptions in the report dated 22 October 2026. That earlier report correctly described a single fleet controller at its own source version; its statement that multi-agent learning was not implemented no longer describes the current code.

The visual template is retained from the earlier report: the same cover arrangement, institutional artwork, author details, margins, fonts, blue headings, Minted code listings, and numbered explanation lists. The narrative first establishes what the simulator and learning algorithms do. It then compares the methods and finally explains the entire core source in order. This is an implementation report, not a rewritten literature review or a claim of operational validation.

Code scope. As in the previous report, the detailed walkthrough covers `maintenance_tutorial_annotated_core.py`. The notebook assembles this file with the separate exercises/solutions and behavioral-test files into one in-memory module. Their integration and verification are explained, but the separate files and every notebook cell are not additional line-by-line appendices. All code needed to read the core walkthrough is embedded in this TeX file. The institutional artwork is also embedded and is extracted automatically when this document is compiled.

Line numbers. Each listing displays the corresponding line numbers in the current annotated Python file, including gaps where redundant `# Explanation:` comments have been removed. Every nonblank retained code, import, decorator, declaration, documentation, and ordinary comment line has a numbered explanation. Blank lines only separate source visually. Continuation lines are explained individually. The original explanatory comments are not additional executed instructions; their purpose is replaced by the prose explanations here.

How to read. Begin with the observation, action, time, and information conventions. Then read the method comparisons before the long source walkthrough. A network count alone does not determine whether a controller is single-agent, multi-agent, or hierarchical. The decisive questions are which information it uses, which decisions it owns, how its probability or value is represented, and which update is applied.

Frozen source. `maintenance_tutorial_annotated_core.py`; 4273 physical lines; 2099 nonblank retained lines explained. SHA-256:

`a626792817b8f870a1ebe644f25cdf8783d97ed16eaff8652b7fc4060aec7628`

The source snapshot is frozen for this report; later edits require regeneration of the line mapping.

2. What changed since the earlier code report

1. Aircraft have stable fleet-local plate labels, such as `AC-001`, linking their array indices, actions, maintenance jobs, and event diagnostics. These labels do not by themselves create a persistent flight-history database or a fleet-status bitmask.

2. Crew members and workshops have identities and individual duration factors. Crew selection is an identity bitmask. Selected crew work rates add; a workshop factor changes component work duration. A job stores cumulative component-completion days so accepted scheduling assumptions are used during actual execution.

3. Observation version 3 retains 70 values per aircraft and extends public information with individual resource availability and factor encodings. Fleet and resource counts are configurable at model construction.

4. Multi-agent proximal policy optimization (MAPPO) supports aircraft ownership with shared actor weights and component-type ownership with six independent actors. Its centralized training critic produces one value per agent; actors use restricted observed information.

5. Deep centralized multi-agent actor–critic (DCMAC) and deep decentralized multi-agent actor–critic (DDMAC) have separate off-policy fleet adaptations under aircraft and component-type ownership. They receive global observed beliefs, use replay and joint importance-weighted temporal-difference (TD) updates, and have one scalar fleet critic.

6. A daily aircraft-option wrapper permits new plans for free aircraft while other aircraft remain in committed repairs. Additional methods implement separate learned planning stages, replacement-budget targets, a single-aircraft MAXQ-0 baseline, safe planning-mode option-critic, and a two-level value-decomposition adaptation. A restricted-catalog flat comparator accompanies them.

7. The common model factory, training dispatcher, evaluation, checkpoints, runner, tests, and comprehensive notebook recognize the additions. The notebook has ten learning parts and 122 executed code cells; the behavioral suite contains 43 passing tests at the recorded verification.

Not implemented by these changes. Explicit flight legs, flight-hour duration, time/cycle-dependent wear, thermal/dust/salt condition inputs, standardized common-reference post-landing measurements, and the richer bird/hail encounter model remain pending. The current weather multiplier and incident constants must not be described as the completed literature-based upgrade.

3. The environment, policy, trainer, and evaluator

3.1. Four responsibilities

The **environment** owns hidden health, synthetic measurements, estimated condition, inventory, crews, workshops, committed jobs, rewards, and the physical clock. A policy submits a decision; the simulator decides whether and how it can be executed. The policy does not directly write true health or change a running repair.

The **policy** maps permitted information to decisions. An actor produces action probabilities. A value learner estimates future return, either as a state value or an action/option value. Reinforcement learning (RL) updates these estimates from interaction; hierarchical reinforcement learning (HRL) introduces specified planning levels or temporally extended behavior. A physically nested component structure does not alone define an HRL algorithm [1, 2].

The **trainer** collects transitions, constructs targets, computes losses, and updates network parameters or tables. Different trainers in this file use different algorithms: proximal policy optimization (PPO), MAPPO, replay-based actor–critic, MAXQ completion-value learning, option-critic gradients, or hierarchical value-decomposition targets. They share a simulator, not one universal loss [3].

The **evaluator** creates fresh seeded episodes and requests fixed-weight decisions. It does not call an optimizer. It reconstructs the required option state at episode start. For neural categorical policies it normally chooses the most likely action; option-critic also uses a deterministic threshold for termination. These deterministic evaluation conventions differ from the stochastic choices used during training.

3.2. The notebook's library trainer versus the custom fleet trainers

Stable-Baselines3 (SB3) PPO is used in the introductory notebook example with `GymMaintenanceEnv`. The custom fleet models are independently constructed by `make_model` and updated by the appropriate custom training function. SB3 does not initialize these fleet weights and then pass them to a second optimizer.

The custom implementations make conditional action likelihoods, masks, per-agent statistics, option durations, and completion values explicit. Standard SB3 policies do not automatically implement those project-specific interfaces. This is an implementation and teaching choice, not a claim that SB3 cannot be extended. `FlattenFleetAction` only reshapes an action; it supplies neither conditional sampling nor a hierarchical training algorithm [4].

3.3. One day is the physical time unit

`FleetMaintenanceEnv.step` advances one day. Each eligible aircraft performs one modeled operating flight that day; a grounded aircraft does not. This is not an explicit route or multi-leg schedule. Maintenance occupies several days. Other aircraft continue operating while a job progresses.

Grounding, engine choice, bundle choice, crew selection, and workshop selection may be calculated through several Python calls before that one day advances. Those calls are computational planning stages. The older `execute_option` instead advances the entire fleet through several days before returning to its manager. The new `AircraftOptionEnv` always returns after one day and tracks each aircraft's committed physical option independently. These two interfaces therefore provide different manager decision opportunities.

4. State, observations, identities, and maintenance actions

4.1. Hidden plant and observed belief

For N aircraft, hidden health has shape $(N, 2, 6)$. The six component types are fan, low-pressure compressor (LPC), high-pressure compressor (HPC), combustor, high-pressure turbine (HPT), and low-pressure turbine (LPT). Health is a normalized teaching variable, not a calibrated component map. A noisy synthetic efficiency/temperature model feeds a bootstrap particle filter. The controller observes estimates and uncertainty summaries, not the realized incident severity or hidden health array.

Each aircraft contributes 70 local values. The public vector has

$$P = 10 + 2C + 2W, \qquad D = 70N + P,$$

where C is crew count, W workshop count, and D total observed width. With seven aircraft, three crew members and two workshops, $P = 20$ and $D = 510$. The earlier report's 500-value example belonged to the older ten-public-value layout.

Local indices	Contents	Interpretation
0–11	Estimated health means	Two engines times six components.
12–23	Estimated standard deviations	Component uncertainty, not wear rates.
24–35	Lower health quantiles	Used for conservative urgency and grounding.
36	Exposure divided by 1.5	Fixed synthetic episode context; not flight hours.
37–38	Busy flag and remaining-job fraction	Whether a committed repair occupies the aircraft.
39–44	Committed plan fields	Engines, two bundles, crew identity mask, workshop and cyclic order start, all normalized.
45–56	Efficiency proxies	Synthetic readings, not directly observed physical health modifiers.
57–68	Normalized temperature proxies	Encoded through the fixed Celsius range.
69	Reading age divided by horizon	Increases while grounded because no new flight measurement is collected.

Public indices 0–9 contain day, free-resource fractions, replenishment timing and six stock values. Next come crew free flags, crew factor encodings, workshop free flags and workshop factor encodings. A duration factor f is encoded as $1/(1+f)$; its meaning remains a maintenance-time factor, not a direct reward bonus.

4.2. Bitwise selection and aircraft plates

For a component index c, `1 << c` sets one bit. Decimal 5 is 000101_2, selecting fan and HPC on one engine. The two engine bits identify which engine bundles are nonempty; they do not replace the component bundles. Similarly, `0b101` selects crew members 0 and 2. Bitwise AND tests membership, bitwise OR combines selections, and `bit_count()` counts selected members.

The workshop is a single integer index, not a bitmask. Aircraft plates are stable strings mapped to array indices, not a one-hot observation or a binary fleet status. Identity, availability, and operating history are separate concepts. Reset retains the plate/index mapping but starts a new episode history.

4.3. Full eight-field action and duration

The original daily fleet policies choose

```
[ground, engine_mask, bundle0, bundle1,
 crew_mask, workshop_index, order_start, priority]
```

for every aircraft. Their category counts are

$$[2, 4, 64, 64, 2^C, W, 12, N].$$

The 64 bundle categories are the 2^6 possible subsets. The crew head grows as 2^C, an explicit combinatorial limitation. A nonempty new repair requires a nonzero crew identity mask. Priority determines proposal processing order. `order_start` selects a cyclic order of engine/component slots, not an arbitrary permutation of selected tasks.

For crew set S, component labor L_c, and workshop factor f_w,

$$R_S = \sum_{i \in S} \frac{1}{f_i}, \qquad d_c = \max\left(1, \left\lceil \frac{L_c f_w}{R_S} \right\rceil\right), \qquad d_{job} = d_{setup} + \sum_c d_c.$$

Setup is fixed; component tasks execute sequentially. The stored cumulative completion days determine when each component is restored. Faster crew or workshop factors can reduce work time; they do not remove setup or create a new negative cost term. These are synthetic additive-work-rate assumptions.

4.4. Reservations, costs, and observation updates

Accepted plans reserve identified crew members and a workshop and consume selected spares. A busy aircraft's new row cannot replace the plan. Components restore to health one at their completion thresholds. The aircraft remains grounded until the entire job completes, and readings remain old until another flight.

The current cost includes setup, selected engines, parts, unused-life cost, downtime, failures, and rejected requests. Daily reward is $(10\,flights - cost)/(10N)$. The unsafe-flight hold uses the minimum observed lower health quantile over all components; it does not require HPC to fail first. Job duration is derived from the package and resources, rather than freely selected as a duration action. Spares are selected indirectly through replacement content; procurement and ordering are not learned actions.

5. Single-agent, shared policies, CTDE, and global component actors

5.1. Four independent design questions

Multi-agent reinforcement learning (MARL) requires specifying decision ownership and learning, not merely counting network objects. The following choices should be stated separately:

1. **Ownership:** one joint fleet controller, one agent per aircraft, or one agent per component type across aircraft.
2. **Information:** global observed fleet belief, aircraft-local information plus public variables, or a component-restricted view.
3. **Parameters:** shared actor weights, a central actor with distinct output blocks, or separately parameterized actors.
4. **Learning:** a fleet-joint PPO objective, per-agent MAPPO clipping, replay-based actor–critic, or a hierarchical/value-decomposition update.

Centralized training with decentralized execution (CTDE) is an information pattern, not a competing algorithm alongside PPO or DCMAC. In the present MAPPO methods the critic uses the full observed fleet input during training and the actors can execute from their restricted inputs. However, a fixed centralized broker still allocates resources. Actor proposal generation is decentralized; complete maintenance allocation is not fully decentralized.

5.2. Why one shared actor is not the original joint fleet controller

The original `conditional` policy includes a learned whole-fleet encoding when constructing each aircraft row and combines action likelihoods into one fleet likelihood. Aircraft MAPPO applies one shared actor to separate local/public observations and retains a log probability and advantage per agent. Thus the same network count can conceal different information and update rules.

The original `shared` mode is an important qualification: it also reuses local/public actor weights. It remains in the original fleet-joint update and full-action interface. It is closer in information structure to aircraft MAPPO, but it is not the new MAPPO objective or request-only action interface. In fact, `flat`, `conditional`, and `shared` all reuse their actor and field heads across aircraft. The name `shared` chiefly distinguishes omission of learned global actor context.

5.3. Why single-agent flat PPO is not DCMAC despite similar network counts

The `flat` baseline already has factorized categorical fields; it is not a network enumerating every possible joint action. DCMAC also uses factorized outputs, but the implemented DCMAC family defines aircraft/component control units, uses the replacement-request interface and fixed broker, and trains a global actor and scalar critic from replay with a joint truncated importance ratio. The original fleet PPO instead learns eight fields and optimizes clipped on-policy-style fleet likelihoods. A one-actor/one-critic count does not equate these implementations [5].

5.4. Why DDMAC has N networks and still needs global observations

Aircraft DDMAC sets its actor count from `len(env.aircraft_plates)`. Each list-comprehension iteration constructs a new network, so seven aircraft create seven independent parameter sets. All actors nevertheless receive the same global observed fleet vector; each owns only its own replacement request. This follows the global-belief actor convention discussed in the cited DDMAC formulation, rather than local-information execution [6].

DCMAC's single global actor has different output blocks for different units. Its shared trunk is not the same as repeatedly applying an identical local actor. Component-type DDMAC keeps six actors as fleet size changes, but each actor's inputs and output count still grow with N. Changing N constructs a new model and normally requires retraining; it does not dynamically add networks to an already running checkpoint.

5.5. Critic sharing is a separate choice

The original fleet PPO and DCMAC/DDMAC have one scalar state-value critic. MAPPO and the new planning-PPO modes use one centralized network with a separate output per agent/aircraft. This is one network with multiple value heads, not N separate critic networks. Option-critic has option-action values, MAXQ stores primitive/completion values, and hierarchical value-decomposition networks (VDN) sum learned agent Q values. They should not all be called the same critic architecture.

Separate local critics, a separate global critic per agent, independent local-input aircraft MAPPO actors, and shared component MAPPO actors are not additional configuration choices in the current registry. Their conceptual feasibility should not be reported as implementation.

6. Every implemented method and its concrete differences

For the table, a semi-Markov decision process (SMDP) accounts for actions spanning variable numbers of primitive days. Generalized advantage estimation (GAE) combines bootstrapped TD residuals. N denotes the configurable aircraft count; scalar V means one fleet state-value output, whereas an N-output V retains one baseline per aircraft.

Method	Actor information	Networks / value	Update
`flat`	Global + local	1 reused actor; scalar V	Joint PPO
`conditional`	Global + local	1 reused actor; scalar V	Joint PPO
`shared`	Local + public	1 reused actor; scalar V	Joint PPO
`options`	Global	4-choice manager; scalar V	SMDP PPO
`learned_options`	Global + local	Manager + conditioned worker	SMDP PPO
`mappo_aircraft`	Local + public	1 shared actor; N-output V	Per-agent PPO
`mappo_component`	Component view + public	6 actors; 6-output V	Per-agent PPO
`dcmac_aircraft`	Global belief	1 factorized actor; scalar V	Replay TD
`ddmac_aircraft`	Global belief	N actors; scalar V	Replay TD
`dcmac_component`	Global belief	1 factorized actor; scalar V	Replay TD
`ddmac_component`	Global belief	6 actors; scalar V	Replay TD
`concurrent_options`	Global + local	Shared manager; N-output V	Aircraft SMDP PPO
`planning_hierarchy`	Global + local	3 shared stage nets; N-output V	Aircraft SMDP PPO
`aggregate_targets`	Global + local	Manager + worker; N-output V	Aircraft SMDP PPO
`maxq`	Quantized global belief	Tabular primitive V and C	MAXQ-0
`option_critic`	Global + local	Worker + Q + beta	Online TD + termination
`hierarchical_vdn`	Local + public	N high + N low Q nets	Two-level additive TD
`hrl_flat`	Global + local	Shared catalog actor; N-output V	Aircraft SMDP PPO

6.1. flat

Original flat field logits; later validity masks still depend on earlier choices.

Decision authority. Full fleet plan. **Information and parameters.** Global + local; 1 reused actor; scalar V. **Learning.** Joint PPO.

6.2. conditional

Previous field choices additionally condition later logits.

Decision authority. Full fleet plan. **Information and parameters.** Global + local; 1 reused actor; scalar V. **Learning.** Joint PPO.

6.3. shared

No global actor encoding; still one fleet-joint training likelihood.

Decision authority. Full fleet plan. **Information and parameters.** Local + public; 1 reused actor; scalar V. **Learning.** Joint PPO.

6.4. options

Learned manager over four fixed threshold routines; fleet-wide decision suspension.

Decision authority. Fleet option. **Information and parameters.** Global; 4-choice manager; scalar V. **Learning.** SMDP PPO.

6.5. learned_options

Both levels learn; one full plan is held by the fixed repair executor.

Decision authority. Fleet option + plan. **Information and parameters.** Global + local; Manager + conditioned worker. **Learning.** SMDP PPO.

6.6. mappo_aircraft

N aircraft decision roles; two 64-way bundle heads per role.

Decision authority. 2 bundles / aircraft. **Information and parameters.** Local + public; 1 shared actor; N-output V. **Learning.** Per-agent PPO.

These policies request replacements; the fixed broker assigns crews, workshops, ordering and priority. Resource arbitration is centralized even when actor execution uses restricted views.

6.7. mappo_component

Six component types; each actor makes N four-way engine decisions.

Decision authority. Engine masks / type. **Information and parameters.** Component view + public; 6 actors; 6-output V. **Learning.** Per-agent PPO.

These policies request replacements; the fixed broker assigns crews, workshops, ordering and priority. Resource arbitration is centralized even when actor execution uses restricted views.

6.8. dcmac_aircraft

Output blocks represent N aircraft agents; no local-view execution.

Decision authority. 2 bundles / aircraft. **Information and parameters.** Global belief; 1 factorized actor; scalar V. **Learning.** Replay TD.

These policies request replacements; the fixed broker assigns crews, workshops, ordering and priority. Resource arbitration is centralized even when actor execution uses restricted views.

6.9. ddmac_aircraft

An independent global-input network is constructed for every aircraft.

Decision authority. 2 bundles / aircraft. **Information and parameters.** Global belief; N actors; scalar V. **Learning.** Replay TD.

These policies request replacements; the fixed broker assigns crews, workshops, ordering and priority. Resource arbitration is centralized even when actor execution uses restricted views.

The independent actor count follows ownership: N for aircraft assignment, six for component-type assignment. Each network still receives the global observed vector; the word decentralized here describes parameter separation, not local information.

6.10. dcmac_component

Six output blocks; N four-way fields within each component block.

Decision authority. Engine masks / type. **Information and parameters.** Global belief; 1 factorized actor; scalar V. **Learning.** Replay TD.

These policies request replacements; the fixed broker assigns crews, workshops, ordering and priority. Resource arbitration is centralized even when actor execution uses restricted views.

6.11. ddmac_component

One independent global-input actor for each component type.

Decision authority. Engine masks / type. **Information and parameters.** Global belief; 6 actors; scalar V. **Learning.** Replay TD.

These policies request replacements; the fixed broker assigns crews, workshops, ordering and priority. Resource arbitration is centralized even when actor execution uses restricted views.

The independent actor count follows ownership: N for aircraft assignment, six for component-type assignment. Each network still receives the global observed vector; the word decentralized here describes parameter separation, not local information.

6.12. concurrent_options

Fixed condition-based worker; free aircraft replan while others remain busy.

Decision authority. Operate / maintain. **Information and parameters.** Global + local; Shared manager; N-output V. **Learning.** Aircraft SMDP PPO.

A free aircraft starts a decision; its return spans the actual time until it becomes free again. Other free aircraft continue deciding on intervening days. Structural masks do not guarantee inventory acceptance.

6.13. planning_hierarchy

Three conditioned planning stages, followed by committed execution.

Decision authority. Ground / engine / bundle. **Information and parameters.** Global + local; 3 shared stage nets; N-output V. **Learning.** Aircraft SMDP PPO.

A free aircraft starts a decision; its return spans the actual time until it becomes free again. Other free aircraft continue deciding on intervening days. Structural masks do not guarantee inventory acceptance.

6.14. aggregate_targets

Goals 0/1/2/12 limit component instances; targets do not ensure broker acceptance.

Decision authority. Budget then catalog. **Information and parameters.** Global + local; Manager + worker; N-output V. **Learning.** Aircraft SMDP PPO.

A free aircraft starts a decision; its return spans the actual time until it becomes free again. Other free aircraft continue deciding on intervening days. Structural masks do not guarantee inventory acceptance.

6.15. maxq

One aircraft only; child completion is learned with actual duration.

Decision authority. Recursive task tree. **Information and parameters.** Quantized global belief; Tabular primitive V and C. **Learning.** MAXQ-0.

6.16. option_critic

Three modes; safe beta is zero while physical repair remains committed.

Decision authority. Mode and catalog. **Information and parameters.** Global + local; Worker + Q + beta. **Learning.** Online TD + termination.

6.17. hierarchical_vdn

Three-day upper clock, daily lower clock, disclosed advantage reward shaping.

Decision authority. Budget and catalog. **Information and parameters.** Local + public; N high + N low Q nets. **Learning.** Two-level additive TD.

6.18. hrl_flat

Comparator for staged planning using the same catalog, broker and decision timing.

Decision authority. 22-entry catalog. **Information and parameters.** Global + local; Shared catalog actor; N-output V. **Learning.** Aircraft SMDP PPO.

A free aircraft starts a decision; its return spans the actual time until it becomes free again. Other free aircraft continue deciding on intervening days. Structural masks do not guarantee inventory acceptance.

7. Training equations and what their axes mean

7.1. Fleet-joint PPO and per-agent MAPPO

The original full-action fleet methods sum log probabilities across fields and aircraft. PPO compares the same stored fleet action under current and collection-time policies. MAPPO

sums only fields belonging to one agent, preserving the agent axis before clipping. Their ratios are respectively

$$r_t^{fleet} = \exp\left(\sum_i \log \pi_i^{new} - \sum_i \log \pi_i^{old}\right), \qquad r_{t,i} = \exp\left(\log \pi_i^{new} - \log \pi_i^{old}\right).$$

Clipping the product is not the same operation as clipping each ratio. For positive unit advantages and individual ratios 2 and 0.5, the averaged per-agent clipped surrogate is $(1.2 + 0.5)/2 = 0.85$, while the joint ratio is one. This explains a concrete code difference without attributing fleet performance to a benchmark paper [7].

7.2. Replay-based DCMAC and DDMAC

The code stores original submitted proposal likelihoods even when the broker defers the request. For replay behavior policy μ and current policy π, it calculates

$$w = \min\left(2, \exp\left[\sum_i \log \pi_i(a_i \mid b) - \sum_i \log \mu_i(a_i \mid b)\right]\right),$$

$$y = r + \gamma(1-d)V(b'), \qquad \delta = y - V(b).$$

The actor minimizes $-\mathbb{E}[w \log \pi(a \mid b)\,\mathrm{stopgrad}(\delta)]$, and the critic minimizes $\frac{1}{2}\mathbb{E}[w\delta^2]$. Weights and next-state targets are detached. The cap 2 and small-network settings are tutorial choices. Replay truncation trades variance for bias; no replay state-distribution correction is implemented. These weights correct policy likelihood mismatch, not oversampling of rare hazard scenarios.

7.3. Aircraft option durations and completion values

For rewards accumulated over k physical days, the new aircraft-option target is

$$y = \sum_{j=0}^{k-1} \gamma^j r_j + \gamma^k (1-d) V(b_{next}).$$

A training-budget cutoff can bootstrap from a still-busy state; it does not cancel the job or claim option completion. Pending decisions retain their collection-time likelihoods across rollout updates. This is a delayed-decision PPO adaptation, not an assertion that all later samples are freshly on-policy. The critic also learns daily targets, including busy states.

MAXQ-0 uses a different decomposition, $Q(i, s, a) = V(a, s) + C(i, s, a)$, where primitive V values estimate immediate reward and C estimates future reward needed to complete the parent after the child. The code updates C with the parent's continuation value discounted over the actual child duration. Quantizing observed beliefs can alias states, so exact-state guarantees are not claimed [8].

7.4. Option-critic termination and two-level VDN

Option-critic constructs a continuation value

$$U(b', o) = (1 - \beta_{safe}) Q_\Omega(b', o) + \beta_{safe} \max_{o'} Q_\Omega(b', o').$$

Here $\beta_{safe} = 0$ while physical repair is committed. The internal policy receives a detached TD advantage. The termination network uses an option-advantage loss with a 0.01 margin

at permitted nonterminal free-aircraft states only. This learns planning-mode termination without acquiring permission to interrupt repair [9].

The hierarchical VDN adaptation sums agent values at the high and low levels. The high target uses a three-day-or-shorter macro return. The low reward adds $0.1\delta_H/k$, using the detached high-level TD advantage δ_H over macro duration k. This is a disclosed inter-level learning coupling and changes the low-level objective. It is inspired by hierarchical cooperative coordination, but is not the complete HAVEN dual-coordination algorithm or a nonlinear QMIX mixer [10, 11].

8. Comparison design, verification, and remaining work

8.1. Which comparisons isolate which changes

The closest new representation comparison is `hrl_flat` versus `planning_hierarchy`: both have global/local inputs, the same 22 catalog requests, the same fixed resource broker, the same aircraft-specific decision opportunities, critic shape, duration accounting and PPO update. Their network parameterization and parameter counts differ. Aggregate targets retain the same union of possible requests but change the upper action semantics. The concurrent-options worker is fixed and therefore restricts realized choices further.

The six MAPPO/DCMAC/DDMAC variants share their broader replacement-request interface and broker, but differ in actor information, sharing, critic outputs, or update. They cannot be treated as a single-factor agent-count ablation. The original full-action PPO methods also learn resource fields, whereas all request methods and the new catalog methods delegate those fields. Older fleet options reduce manager opportunities. VDN changes macro timing and shaped reward. MAXQ supports one aircraft only. Equal simulation days do not equalize gradient updates, network size, or wall-clock effort.

8.2. Recorded software checks

The latest consolidated suite passes 43 behavioral tests. It checks environment contracts, identities, crew/workshop duration and reservations, commitment, observation boundaries, sampling/reconstruction likelihoods, option targets, per-agent clipping, replay importance arithmetic, stage gradients, budget masks, MAXQ V+C, safe learned termination, VDN mixing, parameter updates, reproducible seeds, and checkpoint round trips. All 122 code cells in the comprehensive notebook executed without errors. A separate all-method runner smoke test trained 17 neural configurations on a two-aircraft case and explicitly recorded MAXQ as skipped; MAXQ's one-aircraft checkpoint was checked separately.

These checks establish the behaviors they assert, not convergence, numerical reproduction of the papers, operational validity, or superior maintenance performance. Short notebook runs use two training seeds and held-out evaluation seeds. They are demonstrations, not a completed statistical study. Saved checkpoints contain model weights/tables and configuration, not an exact optimizer/replay resumption.

8.3. Important physical and methodological limits

The particle filter returns condition summaries, not a remaining-useful-life forecast. It resamples component marginals independently, so common physical weather does not create a fully dependency-aware Bayesian filter. Temperature still includes the aircraft-specific exposure offset. Bird/hail events are sampled independently per engine, with uniform severities

and fixed component weight vectors; there is no aircraft encounter stage, explicit no-damage branch, or separate operability state.

The 22-entry menu is structurally defined, not a guarantee of stock feasibility. For example, one aircraft initially has one spare per component type, so a request for the same component on both engines needs more stock than is present under that configuration. The broker can defer it. Repair order changes component restoration timing, but the current whole-job grounding rule can leave total return unchanged for a fixed package. An earlier read-only two-task probe observed exactly that behavior.

Calendar-based staffing, graph/attention/hypergraph policies, learned peer messaging, arbitrary task permutations, procurement decisions, rolling-horizon or mixed-integer baselines, a fleet exact-search oracle, calibrated hazard parameters, rare-event confidence intervals, and unchanged-weight transfer across fleet sizes remain outside the implementation described here. The report does not promote a code prototype into an empirical literature result.

9. Source walkthrough conventions

The following sections cover the current core in source order. A class introduction states what it owns. Each function introduction explains its purpose, inputs, side effects and calls where relevant. Listings remove redundant explanation-only comments but preserve original line labels through a source mapping. The prose describes every nonblank retained physical line, including continuation lines. For a multi-line expression, later fragments must be read together with the initiating statement; they are not independent operations.

9.1. Module header and imports

Source lines 1–27.

```
1  # Annotated teaching copy: every executable source line has an explanation immediately above it.
2  # Updated simulator: fixed aircraft identities and individually assigned heterogeneous maintenance
   ↪  resources.
4  """One supporting module for the comprehensive RL/HRL maintenance course.
6  OBSERVATION_VERSION=3 adds individual resource availability and duration factors.
7  Fleet action field 4 is now a crew-selection bitmask, not a crew count minus one.
8  The first examples and user's exercise answers are namespaced by tutorial part.
9  The fleet classes/functions keep their public names. No import depends on Code.
10 All figures are teaching assumptions rather than real turbofan calibration.
11 """
13 from __future__ import annotations
17 # ============================================================================
18 # mini_maintenance.py
19 # ============================================================================
23 import random
25 from typing import TypeAlias
27 Observation: TypeAlias = tuple[int, int]
```

Line-by-line explanation.

1 Source comment: Annotated teaching copy: every executable source line has an explanation immediately above it. This line documents the following code and does not execute.

2 Source comment: Updated simulator: fixed aircraft identities and individually assigned heterogeneous maintenance resources. This line documents the following code and does not execute.

4 Documentation text: One supporting module for the comprehensive RL/HRL maintenance course.. This states the contract or qualification for `module`; it does not update the simulator or model.

6 Documentation text: OBSERVATION_VERSION=3 adds individual resource availability and duration factors.. This states the contract or qualification for `module`; it does not update the simulator or model.

7 Documentation text: Fleet action field 4 is now a crew-selection bitmask, not a crew count minus one.. This states the contract or qualification for `module`; it does not update the simulator or model.

8 Documentation text: The first examples and user's exercise answers are namespaced by tutorial part.. This states the contract or qualification for `module`; it does not update the simulator or model.

9 Documentation text: The fleet classes/functions keep their public names. No import depends on Code.. This states the contract or qualification for `module`; it does not update the simulator or model.

10 Documentation text: All figures are teaching assumptions rather than real turbofan calibration.. This states the contract or qualification for `module`; it does not update the simulator or model.

11 Close the documentation string for `module`. This delimiter ends the descriptive text and performs no simulator or learning update.

13 Import `from __future__ import annotations`. The imported names are used by the definitions below; importing does not start a training run.

17 A visual section separator in the source. It has no executable effect.

18 Source comment: mini_maintenance.py This line documents the following code and does not execute.

19 A visual section separator in the source. It has no executable effect.

23 Import `import random`. Random sampling supports the small simulator and reproducible policies.

25 Import `from typing import TypeAlias`. The imported names are used by the definitions below; importing does not start a training run.

27 Set `Observation` by select the indexed/sliced entries of tuple. Declare Observation as a type-level nickname for a two-integer tuple; runtime values are ordinary tuples.

9.2. MiniMaintenanceEnv

A one-item teaching Markov decision process. Operate and replace actions expose the basic observation, transition, reward and episode contract before the fleet model is introduced.

Source lines 30–49.

```
30  class MiniMaintenanceEnv:
32      """One aircraft with one replaceable item and two high-level actions.
34      Actions:
35          0 = operate for one day
36          1 = ground for one day and replace the item
38      Health is an integer in {0, 1, 2, 3}; 3 is new and 0 is failed.
39      A successful operation earns +5. If operation ends at health 0, it earns
```

```
    -12 instead. Grounding/replacement earns -4 and restores health to 3.
    Operating wear is 1, plus one extra unit with probability shock_prob.
    The episode ends after ``horizon`` days. Failure itself does not end it:
    the policy can ground and restore the item on a later day.
    """
    OPERATE = 0
    GROUND_AND_REPLACE = 1
```

Line-by-line explanation.

30 Define `MiniMaintenanceEnv` with the stated inheritance. A one-item teaching Markov decision process. Operate and replace actions expose the basic observation, transition, reward and episode contract before the fleet model is introduced.

32 Documentation text: One aircraft with one replaceable item and two high-level actions.. This states the contract or qualification for `MiniMaintenanceEnv`; it does not update the simulator or model.

34 Documentation text: Actions:. This states the contract or qualification for `MiniMaintenanceEnv`; it does not update the simulator or model.

35 Documentation text: 0 = operate for one day. This states the contract or qualification for `MiniMaintenanceEnv`; it does not update the simulator or model.

36 Documentation text: 1 = ground for one day and replace the item. This states the contract or qualification for `MiniMaintenanceEnv`; it does not update the simulator or model.

38 Documentation text: Health is an integer in {0, 1, 2, 3}; 3 is new and 0 is failed.. This states the contract or qualification for `MiniMaintenanceEnv`; it does not update the simulator or model.

39 Documentation text: A successful operation earns +5. If operation ends at health 0, it earns. This states the contract or qualification for `MiniMaintenanceEnv`; it does not update the simulator or model.

40 Documentation text: -12 instead. Grounding/replacement earns -4 and restores health to 3.. This states the contract or qualification for `MiniMaintenanceEnv`; it does not update the simulator or model.

41 Documentation text: Operating wear is 1, plus one extra unit with probability shock_prob.. This states the contract or qualification for `MiniMaintenanceEnv`; it does not update the simulator or model.

43 Documentation text: The episode ends after “horizon“ days. Failure itself does not end it:. This states the contract or qualification for `MiniMaintenanceEnv`; it does not update the simulator or model.

44 Documentation text: the policy can ground and restore the item on a later day.. This states the contract or qualification for `MiniMaintenanceEnv`; it does not update the simulator or model.

45 Close the documentation string for `MiniMaintenanceEnv`. This delimiter ends the descriptive text and performs no simulator or learning update.

47 Set `OPERATE` by use the literal 0.

49 Set `GROUND_AND_REPLACE` by use the literal 1.

9.3. MiniMaintenanceEnv.___init___

Initialize the object, register its layers or allocate its state. Construction does not itself train a policy.

Source lines 52–72.

```
52      def __init__(self, horizon: int=12, shock_prob: float=0.0) -> None:
54          if not isinstance(horizon, int) or horizon <= 0:
56              raise ValueError('horizon must be a positive integer')
58          if not 0.0 <= shock_prob <= 1.0:
60              raise ValueError('shock_prob must be between 0 and 1')
62          self.horizon = horizon
64          self.shock_prob = float(shock_prob)
66          self._rng = random.Random()
68          self.health = 3
70          self.day = 0
72          self._needs_reset = True
```

Line-by-line explanation.

52 Define `__init__(self, horizon: int=12, shock_prob: float=0.0) -> None`. This ___init___ method is part of MiniMaintenanceEnv.___init___; its parameters and defaults are the interface used by the calls in this section.

54 Test `not isinstance(horizon, int) or horizon <= 0`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

56 Raise `ValueError('horizon must be a positive integer')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

58 Test `not 0.0 <= shock_prob <= 1.0`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

60 Raise `ValueError('shock_prob must be between 0 and 1')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

62 Set `self.horizon` by evaluate horizon.

64 Set `self.shock_prob` by convert the result to a Python floating-point number.

66 Set `self._rng` by call random.Random with the shown positional and keyword arguments.

68 Set `self.health` by use the literal 3. It holds hidden physical component condition.

70 Set `self.day` by use the literal 0.

72 Set `self._needs_reset` by use the literal True.

9.4. MiniMaintenanceEnv.reset

Start an episode, initialize random streams/state, and return the initial observation and diagnostic information.

Source lines 75–97.

```
75      def reset(self, *, seed: int | None=None, initial_health: int=3):
77          """Start an episode and return (observation, info).
79          A seed makes subsequent stochastic wear reproducible. Calling reset
80          without a seed keeps the current generator stream.
81          """
83          if initial_health not in (0, 1, 2, 3):
85              raise ValueError('initial_health must be 0, 1, 2, or 3')
87          if seed is not None:
89              self._rng.seed(seed)
91          self.health = initial_health
93          self.day = 0
95          self._needs_reset = False
97          return (self._observation(), {'day': self.day})
```

Line-by-line explanation.

75 Define `reset(self, *, seed: int | None=None, initial_health: int=3)`. This reset method is part of MiniMaintenanceEnv.reset; its parameters and defaults are the interface used by the calls in this section.

77 Documentation text: Start an episode and return (observation, info).. This states the contract or qualification for `MiniMaintenanceEnv.reset`; it does not update the simulator or model.

79 Documentation text: A seed makes subsequent stochastic wear reproducible. Calling reset. This states the contract or qualification for `MiniMaintenanceEnv.reset`; it does not update the simulator or model.

80 Documentation text: without a seed keeps the current generator stream.. This states the contract or qualification for `MiniMaintenanceEnv.reset`; it does not update the simulator or model.

81 Close the documentation string for `MiniMaintenanceEnv.reset`. This delimiter ends the descriptive text and performs no simulator or learning update.

83 Test `initial_health not in (0, 1, 2, 3)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

85 Raise `ValueError('initial_health must be 0, 1, 2, or 3')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

87 Test `seed is not None`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

89 Call self._rng.seed with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

91 Set `self.health` by evaluate initial_health. It holds hidden physical component condition.

93 Set `self.day` by use the literal 0.

95 Set `self._needs_reset` by use the literal False.

97 Return `(self._observation(), {'day': self.day})` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

9.5. MiniMaintenanceEnv.step

Apply the action through this class's transition/adapter contract and return the resulting observation, reward and episode flags.

Source lines 100–134.

```
100     def step(self, action: int):
102         """Return (next_observation, reward, terminated, truncated, info)."""
104         if self._needs_reset:
106             raise RuntimeError('call reset() before step(), including after an episode')
108         if action not in (self.OPERATE, self.GROUND_AND_REPLACE):
110             raise ValueError('action must be 0 (operate) or 1 (ground/replace)')
112         previous_health = self.health
114         if action == self.OPERATE:
116             extra_wear = int(self._rng.random() < self.shock_prob)
118             self.health = max(0, self.health - 1 - extra_wear)
120             failed = self.health == 0
122             reward = -12.0 if failed else 5.0
124             info = {'event': 'failed_operation' if failed else 'successful_operation', 'wear':
                ↪  previous_health - self.health, 'failed': failed, 'grounded': False}
126         else:
128             self.health = 3
130             reward = -4.0
132             info = {'event': 'grounded_and_replaced', 'wear': 0, 'failed': False, 'grounded': True}
134         self.day += 1
```

Line-by-line explanation.

100 Define `step(self, action: int)`. This step method is part of MiniMaintenanceEnv.step; its parameters and defaults are the interface used by the calls in this section.

102 Documentation text: Return (next_observation, reward, terminated, truncated, info).. This states the contract or qualification for `MiniMaintenanceEnv.step`; it does not update the simulator or model.

104 Test `self._needs_reset`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

106 Raise `RuntimeError('call reset() before step(), including after an episode')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

108 Test `action not in (self.OPERATE, self.GROUND_AND_REPLACE)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

110 Raise `ValueError('action must be 0 (operate) or 1 (ground/replace)')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

112 Set `previous_health` by evaluate self.health. Compute and bind the right-hand value to previous_health, naming an intermediate for the surrounding algorithm.

114 Test `action == self.OPERATE`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

116 Set `extra_wear` by convert the result to a Python integer. Compute and bind the right-hand value to extra_wear, naming an intermediate for the surrounding algorithm.

118 Set `self.health` by call max with the shown positional and keyword arguments. It holds hidden physical component condition.

120 Set `failed` by evaluate the comparison self.health == 0. Record whether wear left the item at health zero; this Boolean selects the failure penalty and event label.

122 Set `reward` by select -12.0 when failed, otherwise 5.0. Give an operation a -12 failure penalty if health reached zero; otherwise the successful flight earns +5.

124 Set `info` by construct the shown dict in its stated order. Compute and bind the right-hand value to info, naming an intermediate for the surrounding algorithm.

126 Execute this alternative only when the corresponding preceding condition did not select its branch.

128 Set `self.health` by use the literal 3. It holds hidden physical component condition.

130 Set `reward` by evaluate -4.0. Compute the immediate transition reward and store it in reward; later checks/updates use this named intermediate.

132 Set `info` by construct the shown dict in its stated order. Compute and bind the right-hand value to info, naming an intermediate for the surrounding algorithm.

134 Add `1` to `self.day`, and store the result back in that same target. The change is visible to subsequent statements in this block.

Source lines 136–144.

```
136         terminated = self.day >= self.horizon
138         truncated = False
140         self._needs_reset = terminated
142         info['day'] = self.day
144         return (self._observation(), reward, terminated, truncated, info)
```

Line-by-line explanation.

136 Set `terminated` by evaluate the comparison self.day >= self.horizon. Mark the episode terminal once the number of simulated days reaches the configured horizon.

138 Set `truncated` by use the literal False. Compute whether an external time limit stopped the episode and store it in truncated; later checks/updates use this named intermediate.

140 Set `self._needs_reset` by evaluate terminated.

142 Set `info['day']` by evaluate self.day. Compute and bind the right-hand value to info['day'], naming an intermediate for the surrounding algorithm.

144 Return `(self._observation(), reward, terminated, truncated, info)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

9.6. MiniMaintenanceEnv._observation

Encode this environment's current policy-visible state. In the fleet model this includes belief summaries, committed plans, sensor age and public resources, rather than true health.

Source lines 147–149.

```
147     def _observation(self) -> Observation:
149         return (self.health, self.horizon - self.day)
```

Line-by-line explanation.

147 Define `_observation(self) -> Observation`. Encode this environment's current policy-visible state. In the fleet model this includes belief summaries, committed plans, sensor age and public resources, rather than true health.

149 Return `(self.health, self.horizon - self.day)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

9.7. MiniMaintenanceEnv.render_text

This method implements MiniMaintenanceEnv.render_text in the class described above. Read its inputs, validity branches and returned objects together; persistent writes use the instance or simulator records.

Source lines 152–156.

```
152     def render_text(self) -> str:
154         """Small human-readable display; useful for tracing a policy."""
156         return f'day={self.day:2d}  health={self.health}  steps_left={self.horizon - self.day}'
```

Line-by-line explanation.

152 Define `render_text(self) -> str`. This render_text method is part of MiniMaintenanceEnv.render_text; its parameters and defaults are the interface used by the calls in this section.

154 Documentation text: Small human-readable display; useful for tracing a policy.. This states the contract or qualification for `MiniMaintenanceEnv.render_text`; it does not update the simulator or model.

156 Return `f'day={self.day:2d} health={self.health} steps_left={self.horizon - self.day}'` to the caller by evaluate f'day={self.day:2d} health={self.health} steps_left={self.horizon - self.day}'. No further statements in this call execute.

9.8. random_policy

Select one of the tiny environment actions with an explicitly supplied random generator.

Source lines 159–163.

```
159 def random_policy(observation: Observation, *, rng: random.Random) -> int:
161     """Random baseline, independent of health."""
163     return rng.randrange(2)
```

Line-by-line explanation.

159 Define `random_policy(observation: Observation, *, rng: random.Random) -> int`. Select one of the tiny environment actions with an explicitly supplied random generator.

161 Documentation text: Random baseline, independent of health.. This states the contract or qualification for `random_policy`; it does not update the simulator or model.

163 Return `rng.randrange(2)` to the caller by call rng.randrange with the shown positional and keyword arguments. No further statements in this call execute.

9.9. threshold_policy

Use observed tiny-environment health to choose operation or replacement without learning.

Source lines 166–172.

```
166 def threshold_policy(observation: Observation) -> int:
168     """Simple rule baseline: replace at health 0 or 1."""
170     health, _steps_left = observation
172     return MiniMaintenanceEnv.GROUND_AND_REPLACE if health <= 1 else MiniMaintenanceEnv.OPERATE
```

Line-by-line explanation.

166 Define `threshold_policy(observation: Observation) -> int`. Use observed tiny-environment health to choose operation or replacement without learning.

168 Documentation text: Simple rule baseline: replace at health 0 or 1.. This states the contract or qualification for `threshold_policy`; it does not update the simulator or model.

170 Set `(health, _steps_left)` by evaluate observation. Read health from the observation and intentionally ignore the remaining-day field when this policy only needs health.

172 Return `MiniMaintenanceEnv.GROUND_AND_REPLACE if health <= 1 else MiniMaintenanceEnv.OPERATE` to the caller by select MiniMaintenanceEnv.GROUND_AND_REPLACE when health <= 1, otherwise MiniMaintenanceEnv.OPERATE. No further statements in this call execute.

9.10. rollout

Run a tiny-environment episode, retaining transitions so state, action, reward and termination can be inspected.

Source lines 175–197.

```
175 def rollout(env: MiniMaintenanceEnv, policy, *, seed: int=0):
177     """Execute one episode; policy receives observation and returns an action."""
179     observation, _info = env.reset(seed=seed)
181     rows: list[dict] = []
183     while True:
185         action = policy(observation)
187         next_observation, reward, terminated, truncated, info = env.step(action)
189         rows.append({'observation': observation, 'action': action, 'reward': reward,
            ↪  'next_observation': next_observation, 'event': info['event'], 'terminated': terminated,
            ↪  'truncated': truncated})
191         observation = next_observation
193         if terminated or truncated:
195             break
197     return rows
```

Line-by-line explanation.

175 Define `rollout(env: MiniMaintenanceEnv, policy, *, seed: int=0)`. Run a tiny-environment episode, retaining transitions so state, action, reward and termination can be inspected.

177 Documentation text: Execute one episode; policy receives observation and returns an action.. This states the contract or qualification for `rollout`; it does not update the simulator or model.

179 Set `(observation, _info)` by initialize a new episode using the stated seed/options. Unpack reset's pair into the starting observation and auxiliary info; the underscore means this caller does not need the info dictionary.

181 Set `rows` by construct the shown list in its stated order. Annotate rows as list[dict] to document the expected kind of value for readers and type checkers.

183 Repeat the indented block while `True` remains true; this condition controls the simulation/collection loop here.

185 Set `action` by call policy with the shown positional and keyword arguments. It holds the sampled or stored action whose probability is evaluated. Compute the proposed or selected action and store it in action; later checks/updates use this named intermediate.

187 Set `(next_observation, reward, terminated, truncated, info)` by execute this object's step with the shown arguments. Unpack the environment step result in Gym order: next observation, reward, terminated, truncated, and info. The underscore field is intentionally discarded when present.

189 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

191 Set `observation` by evaluate next_observation. It holds the policy-visible belief and public context. Compute the state visible to the policy and store it in observation; later checks/updates use this named intermediate.

193 Test `terminated or truncated`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

195 Exit the nearest loop immediately; execution resumes after that loop.

197 Return `rows` to the caller by evaluate rows. No further statements in this call execute.

9.11. Module declarations and integration

Source lines 200–208.

```
200 # ======================================================================
201 # hierarchical_maintenance.py
202 # ======================================================================
206 from itertools import product
208 TOY_COMPONENTS = ('fan', 'compressor')
```

Line-by-line explanation.

200 A visual section separator in the source. It has no executable effect.

201 Source comment: hierarchical_maintenance.py This line documents the following code and does not execute.

202 A visual section separator in the source. It has no executable effect.

206 Import `from itertools import product`. The imported names are used by the definitions below; importing does not start a training run.

208 Set `TOY_COMPONENTS` by construct the shown tuple in its stated order.

9.12. valid_action

Check the conditional tuple contract before a small-example transition can execute.

Source lines 211–237.

```
211 def valid_action(action: tuple[int, int, int]) -> bool:
213     """Action is (ground, engine_mask, component_mask).
215     Engine bits 0 and 1 represent engines 0 and 1. Component bits 0-1
216     represent engine 0; bits 2-3 represent engine 1. A selected engine must
217     contain at least one selected component, and no unselected engine may
218     receive component work.
219     """
221     if len(action) != 3 or any((type(x) is not int for x in action)):
223         return False
225     ground, engines, components = action
227     if ground == 0:
229         return engines == 0 and components == 0
231     if ground != 1 or not 1 <= engines <= 3 or (not 1 <= components <= 15):
233         return False
235     selected_engines = sum((1 << engine for engine in range(2) if components & 3 << 2 * engine))
237     return engines == selected_engines
```

Line-by-line explanation.

211 Define `valid_action(action: tuple[int, int, int]) -> bool`. Check the conditional tuple contract before a small-example transition can execute.

213 Documentation text: Action is (ground, engine_mask, component_mask).. This states the contract or qualification for `valid_action`; it does not update the simulator or model.

215 Documentation text: Engine bits 0 and 1 represent engines 0 and 1. Component bits 0-1. This states the contract or qualification for `valid_action`; it does not update the simulator or model.

216 Documentation text: represent engine 0; bits 2-3 represent engine 1. A selected engine must. This states the contract or qualification for `valid_action`; it does not update the simulator or model.

217 Documentation text: contain at least one selected component, and no unselected engine may. This states the contract or qualification for `valid_action`; it does not update the simulator or model.

218 Documentation text: receive component work.. This states the contract or qualification for `valid_action`; it does not update the simulator or model.

219 Close the documentation string for `valid_action`. This delimiter ends the descriptive text and performs no simulator or learning update.

221 Test `len(action) != 3 or any((type(x) is not int for x in action))`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

223 Return `False` to the caller by use the literal False. No further statements in this call execute.

225 Set `(ground, engines, components)` by evaluate action. Decode the action tuple by position into grounding flag, engine mask, and component mask; this gives the later bit checks meaningful names.

227 Test `ground == 0`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

229 Return `engines == 0 and components == 0` to the caller by evaluate engines == 0 and components == 0. No further statements in this call execute.

231 Test `ground != 1 or not 1 <= engines <= 3 or (not 1 <= components <= 15)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

233 Return `False` to the caller by use the literal False. No further statements in this call execute.

235 Set `selected_engines` by add the terms of the supplied iterable. This toy action has one four-bit component mask: bits 0–1 belong to engine 0 and bits 2–3 to engine 1. Decimal 3 is 00112; shifting it by 2*engine creates that engine's pair mask. Shift binds more tightly than &: components & 3 « 2*engine means components & (3 « (2*engine)). The AND asks whether that pair contains any selected component; 1 « engine contributes engine flag 1 or 2. Summing distinct flags reconstructs the separate two-bit engine mask; it does not store component identities.

237 Return `engines == selected_engines` to the caller by evaluate the comparison engines == selected_engines. No further statements in this call execute.

9.13. all_actions

Enumerate all structurally valid tuples in the small conditional example.

Source lines 240–244.

```
240 def all_actions():
242     """List the 16 feasible composite actions, including operate."""
244     return [(0, 0, 0)] + [(1, engines, components) for engines, components in product(range(1, 4),
        ↪  range(1, 16)) if valid_action((1, engines, components))]
```

Line-by-line explanation.

240 Define `all_actions()`. Enumerate all structurally valid tuples in the small conditional example.

242 Documentation text: List the 16 feasible composite actions, including operate.. This states the contract or qualification for `all_actions`; it does not update the simulator or model.

244 Return `[(0, 0, 0)] + [(1, engines, components) for engines, components in product(range(1, 4), range(1, 16)) if valid_action((1, engines, components))]` to the caller by add [(0, 0, 0)] and [(1, engines, components) for engines, components in product(range(1, 4), range(1, 16)) if valid_action((1, engines, components))]. No further statements in this call execute.

9.14. ConditionalMaintenanceEnv

A small structural-action example. Grounding gates engine and component choices; it demonstrates valid conditional tuples, rather than the later fleet resource scheduler.

Source lines 247–249.

```
247 class ConditionalMaintenanceEnv:
249     """Tiny deterministic simulator for one aircraft with two engines."""
```

Line-by-line explanation.

247 Define `ConditionalMaintenanceEnv` with the stated inheritance. A small structural-action example. Grounding gates engine and component choices; it demonstrates valid conditional tuples, rather than the later fleet resource scheduler.

249 Documentation text: Tiny deterministic simulator for one aircraft with two engines.. This states the contract or qualification for `ConditionalMaintenanceEnv`; it does not update the simulator or model.

9.15. ConditionalMaintenanceEnv.___init___

Initialize the object, register its layers or allocate its state. Construction does not itself train a policy.

Source lines 252–264.

```
252     def __init__(self, horizon: int=6):
254         if not isinstance(horizon, int) or horizon <= 0:
256             raise ValueError('horizon must be positive')
258         self.horizon = horizon
260         self.health = [3, 3, 3, 3]
262         self.day = 0
264         self._needs_reset = True
```

Line-by-line explanation.

252 Define `__init__(self, horizon: int=6)`. This ___init___ method is part of ConditionalMaintenanceEnv.___init___; its parameters and defaults are the interface used by the calls in this section.

254 Test `not isinstance(horizon, int) or horizon <= 0`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

256 Raise `ValueError('horizon must be positive')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

258 Set `self.horizon` by evaluate horizon.

260 Set `self.health` by construct the shown list in its stated order. It holds hidden physical component condition.

262 Set `self.day` by use the literal 0.

264 Set `self._needs_reset` by use the literal True.

9.16. ConditionalMaintenanceEnv._observation

Encode this environment's current policy-visible state. In the fleet model this includes belief summaries, committed plans, sensor age and public resources, rather than true health.

Source lines 267–269.

```
267     def _observation(self):
269         return tuple(self.health) + (self.horizon - self.day,)
```

Line-by-line explanation.

267 Define `_observation(self)`. Encode this environment's current policy-visible state. In the fleet model this includes belief summaries, committed plans, sensor age and public resources, rather than true health.

269 Return `tuple(self.health) + (self.horizon - self.day,)` to the caller by add tuple(self.health) and (self.horizon - self.day,). No further statements in this call execute.

9.17. ConditionalMaintenanceEnv.reset

Start an episode, initialize random streams/state, and return the initial observation and diagnostic information.

Source lines 272–284.

```
272     def reset(self, initial_health=(3, 3, 3, 3)):
274         if len(initial_health) != 4 or any((h not in range(4) for h in initial_health)):
276             raise ValueError('provide four health values in 0..3')
278         self.health = list(initial_health)
280         self.day = 0
282         self._needs_reset = False
284         return (self._observation(), {})
```

Line-by-line explanation.

272 Define `reset(self, initial_health=(3, 3, 3, 3))`. This reset method is part of ConditionalMaintenanceEnv.reset; its parameters and defaults are the interface used by the calls in this section.

274 Test `len(initial_health) != 4 or any((h not in range(4) for h in initial_health))`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

276 Raise `ValueError('provide four health values in 0..3')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

278 Set `self.health` by call list with the shown positional and keyword arguments. It holds hidden physical component condition.

280 Set `self.day` by use the literal 0.

282 Set `self._needs_reset` by use the literal False.

284 Return `(self._observation(), {})` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

9.18. ConditionalMaintenanceEnv.step

Apply the action through this class's transition/adapter contract and return the resulting observation, reward and episode flags.

Source lines 287–321.

```
287     def step(self, action):
289         if self._needs_reset:
291             raise RuntimeError('reset before stepping')
293         if not valid_action(action):
295             raise ValueError(f'infeasible hierarchical action: {action}')
297         ground, engines, components = action
299         if ground:
301             for index in range(4):
303                 if components & 1 << index:
305                     self.health[index] = 3
307             n_engines = engines.bit_count()
309             n_components = components.bit_count()
311             reward = -3.0 - 2.0 * n_engines - 1.0 * n_components
313             event = 'grounded_and_replaced'
315         else:
317             self.health = [max(0, h - 1) for h in self.health]
319             reward = -15.0 if 0 in self.health else 8.0
321             event = 'failed_operation' if reward < 0 else 'successful_operation'
```

Line-by-line explanation.

287 Define `step(self, action)`. This step method is part of ConditionalMaintenanceEnv.step; its parameters and defaults are the interface used by the calls in this section.

289 Test `self._needs_reset`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

291 Raise `RuntimeError('reset before stepping')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

293 Test `not valid_action(action)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

295 Raise `ValueError(f'infeasible hierarchical action: {action}')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

297 Set `(ground, engines, components)` by evaluate action. Decode the action tuple by position into grounding flag, engine mask, and component mask; this gives the later bit checks meaningful names.

299 Test `ground`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

301 Iterate `index` over `range(4)`. The following indented statements run once for each selected element.

303 Test `components & 1 « index`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

305 Set `self.health[index]` by use the literal 3. It holds hidden physical component condition.

307 Set `n_engines` by call engines.bit_count with the shown positional and keyword arguments. Compute and bind the right-hand value to n_engines, naming an intermediate for the surrounding algorithm.

309 Set `n_components` by call components.bit_count with the shown positional and keyword arguments. Compute and bind the right-hand value to n_components, naming an intermediate for the surrounding algorithm.

311 Set `reward` by subtract -3.0 - 2.0 * n_engines and 1.0 * n_components. Charge a base grounding cost of 3, plus 2 per selected engine and 1 per replaced component; larger work bundles cost more.

313 Set `event` by use the literal 'grounded_and_replaced'. Compute and bind the right-hand value to event, naming an intermediate for the surrounding algorithm.

315 Execute this alternative only when the corresponding preceding condition did not select its branch.

317 Set `self.health` by construct the collection by iterating the shown comprehension and applying its filters. It holds hidden physical component condition.

319 Set `reward` by select -15.0 when 0 in self.health, otherwise 8.0. Compute the immediate transition reward and store it in reward; later checks/updates use this named intermediate.

321 Set `event` by select 'failed_operation' when reward < 0, otherwise 'successful_operation'. Compute and bind the right-hand value to event, naming an intermediate for the surrounding algorithm.

Source lines 323–329.

```
        self.day += 1
        terminated = self.day >= self.horizon
        self._needs_reset = terminated
        return (self._observation(), reward, terminated, False, {'event': event})
```

Line-by-line explanation.

323 Add `1` to `self.day`, and store the result back in that same target. The change is visible to subsequent statements in this block.

325 Set `terminated` by evaluate the comparison self.day >= self.horizon. Mark the episode terminal once the number of simulated days reaches the configured horizon.

327 Set `self._needs_reset` by evaluate terminated.

329 Return `(self._observation(), reward, terminated, False, {'event': event})` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

9.19. Module declarations and integration

Source lines 332–342.

```
# ================================================================
# gym_maintenance.py
# ================================================================
import gymnasium as gym
import numpy as np
from gymnasium import spaces
```

Line-by-line explanation.

332 A visual section separator in the source. It has no executable effect.

333 Source comment: gym_maintenance.py This line documents the following code and does not execute.

334 A visual section separator in the source. It has no executable effect.

338 Import `import gymnasium as gym`. The imported names are used by the definitions below; importing does not start a training run.

340 Import `import numpy as np`. The imported names are used by the definitions below; importing does not start a training run.

342 Import `from gymnasium import spaces`. Gymnasium supplies the environment/wrapper contracts.

9.20. GymMaintenanceEnv

A Gymnasium adapter for the small example. It declares library-compatible spaces and reset/step signatures so a library trainer can interact with the teaching process.

Source lines 345–353.

```
345 class GymMaintenanceEnv(gym.Env):
347     """Same decisions and rewards as MiniMaintenanceEnv, with Gymnasium spaces.
349     The observation is [health / 3, days_left / horizon]. Normalization keeps
350     both input features between zero and one for a neural-network policy.
351     """
353     metadata = {'render_modes': ['ansi']}
```

Line-by-line explanation.

345 Define `GymMaintenanceEnv` with the stated inheritance. A Gymnasium adapter for the small example. It declares library-compatible spaces and reset/step signatures so a library trainer can interact with the teaching process.

347 Documentation text: Same decisions and rewards as MiniMaintenanceEnv, with Gymnasium spaces.. This states the contract or qualification for `GymMaintenanceEnv`; it does not update the simulator or model.

349 Documentation text: The observation is [health / 3, days_left / horizon]. Normalization keeps. This states the contract or qualification for `GymMaintenanceEnv`; it does not update the simulator or model.

350 Documentation text: both input features between zero and one for a neural-network policy.. This states the contract or qualification for `GymMaintenanceEnv`; it does not update the simulator or model.

351 Close the documentation string for `GymMaintenanceEnv`. This delimiter ends the descriptive text and performs no simulator or learning update.

353 Set `metadata` by construct the shown dict in its stated order. Compute and bind the right-hand value to metadata, naming an intermediate for the surrounding algorithm.

9.21. GymMaintenanceEnv.___init___

Initialize the object, register its layers or allocate its state. Construction does not itself train a policy.

Source lines 356–364.

```
356     def __init__(self, horizon: int=12, shock_prob: float=0.0):
358         super().__init__()
360         self.core = MiniMaintenanceEnv(horizon=horizon, shock_prob=shock_prob)
362         self.action_space = spaces.Discrete(2)
364         self.observation_space = spaces.Box(low=np.zeros(2, dtype=np.float32), high=np.ones(2,
            ↪  dtype=np.float32), dtype=np.float32)
```

Line-by-line explanation.

356 Define `__init__(self, horizon: int=12, shock_prob: float=0.0)`. This ___init___ method is part of GymMaintenanceEnv.___init___; its parameters and defaults are the interface used by the calls in this section.

358 Call super().___init___ with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

360 Set `self.core` by call MiniMaintenanceEnv with the shown positional and keyword arguments.

362 Set `self.action_space` by call spaces.Discrete with the shown positional and keyword arguments.

364 Set `self.observation_space` by call spaces.Box with the shown positional and keyword arguments.

9.22. GymMaintenanceEnv._encode

Convert the small example's observation into the declared Gymnasium observation encoding.

Source lines 367–371.

```
367     def _encode(self, observation):
369         health, days_left = observation
371         return np.array([health / 3.0, days_left / self.core.horizon], dtype=np.float32)
```

Line-by-line explanation.

367 Define `_encode(self, observation)`. Convert the small example's observation into the declared Gymnasium observation encoding.

369 Set `(health, days_left)` by evaluate observation. Read health from the observation and intentionally ignore the remaining-day field when this policy only needs health.

371 Return `np.array([health / 3.0, days_left / self.core.horizon], dtype=np.float32)` to the caller by call np.array with the shown positional and keyword arguments. No further statements in this call execute.

9.23. GymMaintenanceEnv.reset

Start an episode, initialize random streams/state, and return the initial observation and diagnostic information.

Source lines 374–382.

```
374     def reset(self, *, seed=None, options=None):
376         super().reset(seed=seed)
378         initial_health = 3 if options is None else options.get('initial_health', 3)
380         observation, info = self.core.reset(seed=seed, initial_health=initial_health)
382         return (self._encode(observation), info)
```

Line-by-line explanation.

374 Define `reset(self, *, seed=None, options=None)`. This reset method is part of GymMaintenanceEnv.reset; its parameters and defaults are the interface used by the calls in this section.

376 Initialize a new episode using the stated seed/options. The exact receiver and arguments are shown on this source line.

378 Set `initial_health` by select 3 when options is None, otherwise options.get('initial_health', 3). Compute and bind the right-hand value to initial_health, naming an intermediate for the surrounding algorithm.

380 Set `(observation, info)` by initialize a new episode using the stated seed/options. Unpack reset's pair into the starting observation and auxiliary info; the underscore means this caller does not need the info dictionary.

382 Return `(self._encode(observation), info)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

9.24. GymMaintenanceEnv.step

Apply the action through this class's transition/adapter contract and return the resulting observation, reward and episode flags.

Source lines 385–389.

```
385     def step(self, action):
387         observation, reward, terminated, truncated, info = self.core.step(int(action))
389         return (self._encode(observation), float(reward), terminated, truncated, info)
```

Line-by-line explanation.

385 Define `step(self, action)`. This step method is part of GymMaintenanceEnv.step; its parameters and defaults are the interface used by the calls in this section.

387 Set `(observation, reward, terminated, truncated, info)` by execute this object's step with the shown arguments. Unpack the environment step result in Gym order: next observation, reward, terminated, truncated, and info. The underscore field is intentionally discarded when present.

389 Return `(self._encode(observation), float(reward), terminated, truncated, info)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

9.25. GymMaintenanceEnv.render

This method implements GymMaintenanceEnv.render in the class described above. Read its inputs, validity branches and returned objects together; persistent writes use the instance or simulator records.

Source lines 392–394.

```
392     def render(self):
394         return self.core.render_text()
```

Line-by-line explanation.

392 Define `render(self)`. This render method is part of GymMaintenanceEnv.render; its parameters and defaults are the interface used by the calls in this section.

394 Return `self.core.render_text()` to the caller by call self.core.render_text with the shown positional and keyword arguments. No further statements in this call execute.

9.26. GymMaintenanceEnv.close

This method implements GymMaintenanceEnv.close in the class described above. Read its inputs, validity branches and returned objects together; persistent writes use the instance or simulator records.

Source lines 397–399.

```
397     def close(self):
399         pass
```

Line-by-line explanation.

397 Define `close(self)`. This close method is part of GymMaintenanceEnv.close; its parameters and defaults are the interface used by the calls in this section.

399 Use an explicit empty statement; it performs no state or parameter update.

9.27. Module declarations and integration

Source lines 402–436.

```
402 # ============================================================================
403 # fleet_maintenance/fleet_env.py
404 # ============================================================================
408 from dataclasses import dataclass
410 import math
412 import numpy as np
414 import gymnasium as gym
416 from gymnasium import spaces
418 COMPONENTS = ('fan', 'LPC', 'HPC', 'combustor', 'HPT', 'LPT')
420 WEAR = np.array([0.002, 0.0025, 0.004, 0.002, 0.003, 0.0025])
422 LABOR_DAYS = np.array([2, 3, 4, 2, 3, 3])
424 PART_COST = np.array([8.0, 10.0, 16.0, 8.0, 12.0, 10.0])
426 EFFICIENCY_BASE = 0.72
428 EFFICIENCY_GAIN = 0.24
430 TEMPERATURE_BASE_C = np.array([40.0, 120.0, 350.0, 900.0, 650.0, 450.0])
432 TEMPERATURE_HEALTH_GAIN_C = 120.0
434 TEMPERATURE_EXPOSURE_GAIN_C = 30.0
436 TEMPERATURE_MIN_C = -50.0
```

Line-by-line explanation.

402 A visual section separator in the source. It has no executable effect.

403 Source comment: fleet_maintenance/fleet_env.py This line documents the following code and does not execute.

404 A visual section separator in the source. It has no executable effect.

408 Import `from dataclasses import dataclass`. Dataclasses provide structured configuration and repair records.

410 Import `import math`. Scalar mathematics supports finite-value checks and rounded durations.

412 Import `import numpy as np`. The imported names are used by the definitions below; importing does not start a training run.

414 Import `import gymnasium as gym`. The imported names are used by the definitions below; importing does not start a training run.

416 Import `from gymnasium import spaces`. Gymnasium supplies the environment/wrapper contracts.

418 Set `COMPONENTS` by construct the shown tuple in its stated order.

420 Set `WEAR` by call np.array with the shown positional and keyword arguments.

422 Set `LABOR_DAYS` by call np.array with the shown positional and keyword arguments.

424 Set `PART_COST` by call np.array with the shown positional and keyword arguments.

426 Set `EFFICIENCY_BASE` by use the literal 0.72.

428 Set `EFFICIENCY_GAIN` by use the literal 0.24.

430 Set `TEMPERATURE_BASE_C` by call np.array with the shown positional and keyword arguments.

432 Set `TEMPERATURE_HEALTH_GAIN_C` by use the literal 120.0.

434 Set `TEMPERATURE_EXPOSURE_GAIN_C` by use the literal 30.0.

436 Set `TEMPERATURE_MIN_C` by evaluate -50.0.

Source lines 438–440.

```
438 TEMPERATURE_MAX_C = 1400.0
440 OBSERVATION_VERSION = 3
```

Line-by-line explanation.

438 Set `TEMPERATURE_MAX_C` by use the literal 1400.0.

440 Set `OBSERVATION_VERSION` by use the literal 3.

9.28. Config

Immutable simulation inputs and validation. Fleet size changes array sizes and model construction; changing it does not automatically resize an already trained checkpoint.

Source lines 443–477.

```
443 @dataclass(frozen=True)
445 class Config:
447     aircraft: int = 7
449     horizon: int = 90
451     crews: int = 3
453     workshops: int = 2
455     crew_time_factors: tuple[float, ...] = ()
457     workshop_time_factors: tuple[float, ...] = ()
459     particles: int = 64
461     setup_days: int = 7
463     replenish_days: int = 14
465     bird_probability: float = 0.002
467     hail_probability: float = 0.003
469     weather_probability: float = 0.03
471     observation_noise: float = 0.025
473     wear_scale: float = 1.0
475     mandatory_health: float = 0.08
477     initial_health: float = 0.88
```

Line-by-line explanation.

443 Apply the decorator `dataclass(frozen=True)` to the following definition. Dataclass decorators generate field initialization/representation; staticmethod omits automatic instance binding where used.

445 Define `Config` with the stated inheritance. Immutable simulation inputs and validation. Fleet size changes array sizes and model construction; changing it does not automatically resize an already trained checkpoint.

447 Set `aircraft` by use the literal 7. Declare Config field aircraft as int with default 7; it controls fleet size.

449 Set `horizon` by use the literal 90. Declare Config field horizon as int with default 90; it controls episode duration in days.

451 Set `crews` by use the literal 3. Declare Config field crews as int with default 3; it controls available maintenance crews.

453 Set `workshops` by use the literal 2. Declare Config field workshops as int with default 2; it controls parallel maintenance shops.

455 Set `crew_time_factors` by construct the shown tuple in its stated order. A factor below one makes this crew member faster than standard; above one is slower. Empty uses a repeating expert/standard/novice teaching profile (0.8, 1.0, 1.25).

457 Set `workshop_time_factors` by construct the shown tuple in its stated order. Each workshop has its own component-work duration multiplier; empty uses repeating improved/standard/slower teaching values (0.8, 1.0, 1.2).

459 Set `particles` by use the literal 64. It holds samples representing uncertain component health. Declare Config field particles as int with default 64; it controls health-belief samples per component.

461 Set `setup_days` by use the literal 7. Declare Config field setup_days as int with default 7; it controls fixed setup delay before component work.

463 Set `replenish_days` by use the literal 14. Declare Config field replenish_days as int with default 14; it controls inventory replenishment interval.

465 Set `bird_probability` by use the literal 0.002. Declare Config field bird_probability as float with default 0.002; it controls bird-event probability.

467 Set `hail_probability` by use the literal 0.003. Declare Config field hail_probability as float with default 0.003; it controls hail-event probability.

469 Set `weather_probability` by use the literal 0.03. Declare Config field weather_probability as float with default 0.03; it controls weather-event probability.

471 Set `observation_noise` by use the literal 0.025. Declare Config field observation_noise as float with default 0.025; it controls sensor noise scale.

473 Set `wear_scale` by use the literal 1.0. Declare Config field wear_scale as float with default 1.0; it controls wear multiplier.

475 Set `mandatory_health` by use the literal 0.08. Declare Config field mandatory_health as float with default 0.08; it controls safety threshold for estimated lower health.

477 Set `initial_health` by use the literal 0.88. Declare Config field initial_health as float with default 0.88; it controls mean component health at reset.

9.29. Config.___post_init___

Validate configuration values after dataclass initialization, reject malformed resource profiles, and normalize accepted profiles to immutable tuples.

Source lines 480–514.

```
480     def __post_init__(self):
482         for k in ('aircraft', 'horizon', 'crews', 'workshops', 'particles', 'setup_days',
            ↪  'replenish_days'):
484             if not isinstance(getattr(self, k), int) or getattr(self, k) < 1:
486                 raise ValueError(f'{k} must be a positive integer')
488         for name, count in (('crew_time_factors', self.crews), ('workshop_time_factors',
            ↪  self.workshops)):
490             factors = getattr(self, name)
492             if not isinstance(factors, (tuple, list)):
494                 raise ValueError(f'{name} must be a tuple or list of duration factors')
496             if len(factors) and (len(factors) != count or any(not isinstance(x, (int, float)) or
                ↪  isinstance(x, bool) or not math.isfinite(x) or x <= 0 for x in factors)):
498                 raise ValueError(f'{name} must be empty or contain {count} finite positive numbers')
500             object.__setattr__(self, name, tuple(factors))
502         for k in ('bird_probability', 'hail_probability', 'weather_probability'):
504             if not 0 <= getattr(self, k) <= 1:
506                 raise ValueError(f'{k} must be in [0,1]')
508         if not 0 < self.observation_noise or not 0 < self.wear_scale:
510             raise ValueError('noise and wear_scale must be positive')
512         if not 0 <= self.mandatory_health < 1 or not 0 < self.initial_health <= 1:
514             raise ValueError('invalid health bounds')
```

Line-by-line explanation.

480 Define `__post_init__(self)`. Validate configuration values after dataclass initialization, reject malformed resource profiles, and normalize accepted profiles to immutable tuples.

482 Iterate `k` over `('aircraft', 'horizon', 'crews', 'workshops', 'particles', 'setup_days', 'replenish_days')`. The following indented statements run once for each selected element.

484 Test `not isinstance(getattr(self, k), int) or getattr(self, k) < 1`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

Raise `ValueError(f'{k} must be a positive integer')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

Iterate `(name, count)` over `(('crew_time_factors', self.crews), ('workshop_time_factors', self.workshops))`. The following indented statements run once for each selected element.

Set `factors` by call getattr with the shown positional and keyword arguments. Obtain the optional profile; an empty tuple requests the documented illustrative defaults.

Test `not isinstance(factors, (tuple, list))`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

Raise `ValueError(f'{name} must be a tuple or list of duration factors')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

Test `len(factors) and (len(factors) != count or any((not isinstance(x, (int, float)) or isinstance(x, bool) or (not math.isfinite(x)) or (x <= 0) for x in factors)))`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

Raise `ValueError(f'{name} must be empty or contain {count} finite positive numbers')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

Call object.___setattr___ with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

Iterate `k` over `('bird_probability', 'hail_probability', 'weather_probability')`. The following indented statements run once for each selected element.

Test `not 0 <= getattr(self, k) <= 1`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

Raise `ValueError(f'{k} must be in [0,1]')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

Test `not 0 < self.observation_noise or not 0 < self.wear_scale`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

Raise `ValueError('noise and wear_scale must be positive')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

Test `not 0 <= self.mandatory_health < 1 or not 0 < self.initial_health <= 1`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

Raise `ValueError('invalid health bounds')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

9.30. Plan

The decoded full-action request. It records aircraft, grounding, engine/component selections, crew identities, workshop, cyclic order and priority before acceptance.

Source lines 517–533.

```
@dataclass(frozen=True)
class Plan:
    aircraft: int
    engine_mask: int
    bundles: tuple[int, int]
    crew_mask: int
    workshop: int
    order_start: int
    priority: int
```

Line-by-line explanation.

517 Apply the decorator `dataclass(frozen=True)` to the following definition. Dataclass decorators generate field initialization/representation; staticmethod omits automatic instance binding where used.

519 Define `Plan` with the stated inheritance. The decoded full-action request. It records aircraft, grounding, engine/component selections, crew identities, workshop, cyclic order and priority before acceptance.

521 Declare `aircraft` with type `int`. The dataclass initializer requires this field when constructing the record; this declaration does not supply a default value.

523 Declare `engine_mask` with type `int`. The dataclass initializer requires this field when constructing the record; this declaration does not supply a default value.

525 Declare `bundles` with type `tuple[int, int]`. The dataclass initializer requires this field when constructing the record; this declaration does not supply a default value.

527 Declare `crew_mask` with type `int`. The dataclass initializer requires this field when constructing the record; this declaration does not supply a default value.

529 Declare `workshop` with type `int`. The dataclass initializer requires this field when constructing the record; this declaration does not supply a default value.

531 Declare `order_start` with type `int`. The dataclass initializer requires this field when constructing the record; this declaration does not supply a default value.

533 Declare `priority` with type `int`. The dataclass initializer requires this field when constructing the record; this declaration does not supply a default value.

9.31. Plan.crews

Count selected crew identities using the number of set bits in the crew-selection mask.

Source lines 536–540.

```
    @property
    def crews(self):
        return self.crew_mask.bit_count()
```

Line-by-line explanation.

536 Apply the decorator `property` to the following definition. Dataclass decorators generate field initialization/representation; staticmethod omits automatic instance binding where used.

538 Define `crews(self)`. Count selected crew identities using the number of set bits in the crew-selection mask.

540 Return `self.crew_mask.bit_count()` to the caller by call self.crew_mask.bit_count with the shown positional and keyword arguments. No further statements in this call execute.

9.32. Job

The accepted, committed repair record. Remaining time, ordered tasks, component-completion offsets and assigned resources belong to the simulator after acceptance.

Source lines 543–555.

```
543 @dataclass
545 class Job:
547     plan: Plan
549     tasks: tuple[tuple[int, int], ...]
551     remaining: int
553     total_duration: int
555     completion_days: tuple[int, ...]
```

Line-by-line explanation.

543 Apply the decorator `dataclass` to the following definition. Dataclass decorators generate field initialization/representation; staticmethod omits automatic instance binding where used.

545 Define `Job` with the stated inheritance. The accepted, committed repair record. Remaining time, ordered tasks, component-completion offsets and assigned resources belong to the simulator after acceptance.

547 Declare `plan` with type `Plan`. The dataclass initializer requires this field when constructing the record; this declaration does not supply a default value.

549 Declare `tasks` with type `tuple[tuple[int, int], ...]`. The dataclass initializer requires this field when constructing the record; this declaration does not supply a default value.

551 Declare `remaining` with type `int`. The dataclass initializer requires this field when constructing the record; this declaration does not supply a default value.

553 Declare `total_duration` with type `int`. The dataclass initializer requires this field when constructing the record; this declaration does not supply a default value.

555 Declare `completion_days` with type `tuple[int, ...]`. The dataclass initializer requires this field when constructing the record; this declaration does not supply a default value.

9.33. FleetMaintenanceEnv

The physical fleet simulator. It owns true health, synthetic sensors, particle beliefs, inventory, heterogeneous resources, jobs, stable aircraft identities, event metrics and the daily clock. Policy networks read observations and cannot replace committed job definitions.

Source lines 558–575.

```
558 class FleetMaintenanceEnv(gym.Env):
560     """Action row: ground, engines, bundle0, bundle1, crew_mask, shop, order, priority.
562     Engine mask bits 0/1 select engines. Each bundle has six component bits.
563     Crew bit i selects member CR-(i+1); workshop remains a single resource index.
564     Crew work rates add; workshop factors multiply component work, not setup.
565     Resource factors are synthetic teaching assumptions, not calibrated estimates.
566     aircraft_plates[i] identifies aircraft i in every state array and action row.
567     Plates are synthetic fleet-local identifiers, independent of operating status.
568     Reset starts a new simulated history while retaining the index-to-plate mapping.
569     Resources are assigned to complete plans, in selected priority order.
570     Unfunded/conflicting proposals are rejected and logged; no partial repairs.
571     A busy aircraft's row is ignored: the committed plan cannot be overwritten.
572     Mandatory grounding uses observed lower health quantiles, not hidden health.
573     """
575     metadata = {'render_modes': ['ansi']}
```

Line-by-line explanation.

558 Define `FleetMaintenanceEnv` with the stated inheritance. The physical fleet simulator. It owns true health, synthetic sensors, particle beliefs, inventory, heterogeneous resources, jobs, stable aircraft identities, event metrics and the daily clock. Policy networks read observations and cannot replace committed job definitions.

560 Documentation text: Action row: ground, engines, bundle0, bundle1, crew_mask, shop, order, priority.. This states the contract or qualification for `FleetMaintenanceEnv`; it does not update the simulator or model.

562 Documentation text: Engine mask bits 0/1 select engines. Each bundle has six component bits.. This states the contract or qualification for `FleetMaintenanceEnv`; it does not update the simulator or model.

563 Documentation text: Crew bit i selects member CR-(i+1); workshop remains a single resource index.. This states the contract or qualification for `FleetMaintenanceEnv`; it does not update the simulator or model.

564 Documentation text: Crew work rates add; workshop factors multiply component work, not setup.. This states the contract or qualification for `FleetMaintenanceEnv`; it does not update the simulator or model.

565 Documentation text: Resource factors are synthetic teaching assumptions, not calibrated estimates.. This states the contract or qualification for `FleetMaintenanceEnv`; it does not update the simulator or model.

566 Documentation text: aircraft_plates[i] identifies aircraft i in every state array and action row.. This states the contract or qualification for `FleetMaintenanceEnv`; it does not update the simulator or model.

567 Documentation text: Plates are synthetic fleet-local identifiers, independent of operating status.. This states the contract or qualification for `FleetMaintenanceEnv`; it does not update the simulator or model.

568 Documentation text: Reset starts a new simulated history while retaining the index-to-plate mapping.. This states the contract or qualification for `FleetMaintenanceEnv`; it does not update the simulator or model.

569 Documentation text: Resources are assigned to complete plans, in selected priority order.. This states the contract or qualification for `FleetMaintenanceEnv`; it does not update the simulator or model.

570 Documentation text: Unfunded/conflicting proposals are rejected and logged; no partial repairs.. This states the contract or qualification for `FleetMaintenanceEnv`; it does not update the simulator or model.

571 Documentation text: A busy aircraft's row is ignored: the committed plan cannot be overwritten.. This states the contract or qualification for `FleetMaintenanceEnv`; it does not update the simulator or model.

572 Documentation text: Mandatory grounding uses observed lower health quantiles, not hidden health.. This states the contract or qualification for `FleetMaintenanceEnv`; it does not update the simulator or model.

573 Close the documentation string for `FleetMaintenanceEnv`. This delimiter ends the descriptive text and performs no simulator or learning update.

575 Set `metadata` by construct the shown dict in its stated order. Compute and bind the right-hand value to metadata, naming an intermediate for the surrounding algorithm.

9.34. FleetMaintenanceEnv.___init___

Initialize the object, register its layers or allocate its state. Construction does not itself train a policy.

Source lines 578–608.

```
578     def __init__(self, config: Config | None=None, render_mode=None):
580         super().__init__()
582         self.config = config or Config()
584         self.render_mode = render_mode
586         n = self.config.aircraft
588         self.aircraft_plates = tuple(f'AC-{i + 1:03d}' for i in range(n))
590         self.crew_ids = tuple(f'CR-{i + 1:03d}' for i in range(self.config.crews))
592         self.workshop_ids = tuple(f'WS-{i + 1:03d}' for i in range(self.config.workshops))
594         self.crew_time_factors = tuple(self.config.crew_time_factors) or tuple((0.8, 1.0, 1.25)[i % 3]
        ↪  for i in range(self.config.crews))
596         self.workshop_time_factors = tuple(self.config.workshop_time_factors) or tuple((0.8, 1.0,
        ↪  1.2)[i % 3] for i in range(self.config.workshops))
598         self.row_dims = np.array([2, 4, 64, 64, 1 << self.config.crews, self.config.workshops, 12, n])
600         self.action_space = spaces.MultiDiscrete(np.tile(self.row_dims, (n, 1)))
602         self.local_size = 70
604         self.public_size = 10 + 2 * self.config.crews + 2 * self.config.workshops
606         self.observation_space = spaces.Box(0.0, 1.0, shape=(n * self.local_size + self.public_size,),
        ↪  dtype=np.float32)
608         self._done = True
```

Line-by-line explanation.

578 Define `__init__(self, config: Config | None=None, render_mode=None)`. This ___init___ method is part of FleetMaintenanceEnv.___init___; its parameters and defaults are the interface used by the calls in this section.

580 Call super().___init___ with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

582 Set `self.config` by evaluate config or Config().

584 Set `self.render_mode` by evaluate render_mode.

586 Set `n` by evaluate self.config.aircraft. Compute and bind the right-hand value to n, naming an intermediate for the surrounding algorithm.

588 Set `self.aircraft_plates` by call tuple with the shown positional and keyword arguments. Give each aircraft index a fixed synthetic plate; tuple order matches health, exposure, action rows, and any future fleet-mask bit positions.

590 Set `self.crew_ids` by call tuple with the shown positional and keyword arguments. Give every crew member a stable fleet-local identity in crew-bit order.

592 Set `self.workshop_ids` by call tuple with the shown positional and keyword arguments. Give every workshop a stable identity in workshop-index order.

594 Set `self.crew_time_factors` by evaluate tuple(self.config.crew_time_factors) or tuple(((0.8, 1.0, 1.25)[i % 3] for i in range(self.config.crews))). Resolve explicit crew profiles or repeat the documented expert/standard/novice teaching factors.

596 Set `self.workshop_time_factors` by evaluate tuple(self.config.workshop_time_factors) or tuple(((0.8, 1.0, 1.2)[i % 3] for i in range(self.config.workshops))). Resolve explicit workshop profiles or repeat the documented improved/standard/slower teaching factors.

598 Set `self.row_dims` by call np.array with the shown positional and keyword arguments.

600 Set `self.action_space` by call spaces.MultiDiscrete with the shown positional and keyword arguments.

602 Set `self.local_size` by use the literal 70.

604 Set `self.public_size` by add 10 + 2 * self.config.crews and 2 * self.config.workshops. Keep the original ten public fields, then add availability and duration-factor encodings for every crew member and workshop.

606 Set `self.observation_space` by call spaces.Box with the shown positional and keyword arguments.

608 Set `self._done` by use the literal True.

9.35. FleetMaintenanceEnv.reset

Start an episode, initialize random streams/state, and return the initial observation and diagnostic information.

Source lines 611–645.

```
611     def reset(self, *, seed=None, options=None):
613         super().reset(seed=seed)
615         cfg = self.config
617         seeds = np.random.SeedSequence(seed).spawn(3)
619         self.physical_rng = np.random.default_rng(seeds[0])
621         self.filter_rng = np.random.default_rng(seeds[1])
623         tape_rng = np.random.default_rng(seeds[2])
625         self.day = 0
627         self.health = np.clip(cfg.initial_health + self.physical_rng.normal(0, 0.025, (cfg.aircraft,
            ↪  2, 6)), 0.1, 1)
629         if options and 'health' in options:
631             health = np.asarray(options['health'], dtype=float)
633             if health.shape != self.health.shape or np.any((health < 0) | (health > 1)):
635                 raise ValueError('initial health must have shape (aircraft,2,6), in [0,1]')
637             self.health = health.copy()
639         self.exposure = tape_rng.uniform(0.75, 1.35, cfg.aircraft)
641         self.measurements = self._sensor_model(self.health, self.exposure[:, None, None])
643         self.measurements += self.physical_rng.normal(0, 1, self.measurements.shape) *
            ↪  self.sensor_noise
645         self._clip_measurements(self.measurements)
```

Line-by-line explanation.

611 Define `reset(self, *, seed=None, options=None)`. This reset method is part of FleetMaintenanceEnv.reset; its parameters and defaults are the interface used by the calls in this section.

613 Initialize a new episode using the stated seed/options. The exact receiver and arguments are shown on this source line.

615 Set `cfg` by evaluate self.config. Compute and bind the right-hand value to cfg, naming an intermediate for the surrounding algorithm.

617 Set `seeds` by call np.random.SeedSequence(seed).spawn with the shown positional and keyword arguments. Compute and bind the right-hand value to seeds, naming an intermediate for the surrounding algorithm.

619 Set `self.physical_rng` by call np.random.default_rng with the shown positional and keyword arguments.

621 Set `self.filter_rng` by call np.random.default_rng with the shown positional and keyword arguments.

623 Set `tape_rng` by call np.random.default_rng with the shown positional and keyword arguments. Compute and bind the right-hand value to tape_rng, naming an intermediate for the surrounding algorithm.

625 Set `self.day` by use the literal 0.

627 Set `self.health` by bound the numeric result elementwise. It holds hidden physical component condition.

629 Test `options and 'health' in options`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

631 Set `health` by call np.asarray with the shown positional and keyword arguments. It holds hidden physical component condition. Convert the expression to a NumPy array and store it as health (health values held by this toy state or simulator).

633 Test `health.shape != self.health.shape or np.any((health < 0) | (health > 1))`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

635 Raise `ValueError('initial health must have shape (aircraft,2,6), in [0,1]')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

637 Set `self.health` by make a separate array/dictionary copy. It holds hidden physical component condition.

639 Set `self.exposure` by call tape_rng.uniform with the shown positional and keyword arguments. Compute the aircraft-level multiplier that changes modeled wear and heat and store it in self.exposure; later checks/updates use this named intermediate.

641 Set `self.measurements` by call self._sensor_model with the shown positional and keyword arguments.

643 Add `self.physical_rng.normal(0, 1, self.measurements.shape) * self.sensor_noise` to `self.measurements`, and store the result back in that same target. The change is visible to subsequent statements in this block.

645 Call self._clip_measurements with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

Source lines 647–671.

```
647         self.last_flight_day = np.zeros(cfg.aircraft, dtype=int)
649         prior = self._invert_sensor_metrics(self.measurements, self.exposure[:, None, None])
651         self.particles = np.clip(prior[..., None] + self.filter_rng.normal(0, 0.04,
            ↪  (*self.health.shape, cfg.particles)), 0, 1)
653         self.jobs: dict[int, Job] = {}
655         self.stock = np.full(6, cfg.aircraft, dtype=int)
657         self.event_tape = tape_rng.random((cfg.horizon, cfg.aircraft, 2, 2))
659         self.severity_tape = tape_rng.uniform(0.1, 0.35, (cfg.horizon, cfg.aircraft, 2, 2))
661         self.weather_tape = tape_rng.random(cfg.horizon)
663         self.wear_tape = tape_rng.normal(1.0, 0.12, (cfg.horizon, cfg.aircraft, 2, 6))
665         self.noise_tape = tape_rng.normal(0, cfg.observation_noise, (cfg.horizon, cfg.aircraft, 2, 6,
            ↪  2))
667         self.metrics = {k: 0.0 for k in ('return', 'cost', 'flight_failures', 'flights',
            ↪  'downtime_aircraft_days', 'groundings', 'replacements', 'rejections', 'bird_events',
            ↪  'hail_events', 'weather_exposures', 'unsafe_attempts')}
669         self._done = False
671         return (self._observation(), {'day': self.day, 'aircraft_plates': self.aircraft_plates,
            ↪  'resources': self.resource_details()})
```

Line-by-line explanation.

647 Set `self.last_flight_day` by call np.zeros with the shown positional and keyword arguments.

649 Set `prior` by call self._invert_sensor_metrics with the shown positional and keyword arguments. Compute and bind the right-hand value to prior, naming an intermediate for the surrounding algorithm.

651 Set `self.particles` by bound the numeric result elementwise. It holds samples representing uncertain component health.

653 Set `self.jobs` by construct the shown dict in its stated order. It holds accepted repair records held by the simulator. Unpack {} into named values self.jobs: dict[int, Job] so each part can be used separately below.

655 Set `self.stock` by call np.full with the shown positional and keyword arguments. It holds spare inventory by component type. Create self.stock as a fixed-shape array filled with the given starting value; it represents the spare-part inventory by component type.

657 Set `self.event_tape` by call tape_rng.random with the shown positional and keyword arguments.

659 Set `self.severity_tape` by call tape_rng.uniform with the shown positional and keyword arguments.

661 Set `self.weather_tape` by call tape_rng.random with the shown positional and keyword arguments.

663 Set `self.wear_tape` by call tape_rng.normal with the shown positional and keyword arguments.

665 Set `self.noise_tape` by call tape_rng.normal with the shown positional and keyword arguments.

667 Set `self.metrics` by construct the collection by iterating the shown comprehension and applying its filters.

669 Set `self._done` by use the literal False.

671 Return `(self._observation(), {'day': self.day, 'aircraft_plates': self.aircraft_plates, 'resources': self.resource_details()})` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

9.36. FleetMaintenanceEnv.split_observation

Separate the flattened observed vector into N local aircraft rows and the public resource/inventory vector. This is a view/layout operation, not access to true health.

Source lines 674–678.

```
674     def split_observation(self, observation):
676         a = np.asarray(observation)
678         return (a[:-self.public_size].reshape(self.config.aircraft, self.local_size),
            ↪  a[-self.public_size:])
```

Line-by-line explanation.

674 Define `split_observation(self, observation)`. Separate the flattened observed vector into N local aircraft rows and the public resource/inventory vector. This is a view/layout operation, not access to true health.

676 Set `a` by call np.asarray with the shown positional and keyword arguments. Compute and bind the right-hand value to a, naming an intermediate for the surrounding algorithm.

678 Return `(a[:-self.public_size].reshape(self.config.aircraft, self.local_size), a[-self.public_size:])` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

9.37. FleetMaintenanceEnv.idle_action

Construct the default fleet action, preserving the required representation for aircraft already in committed repair.

Source lines 681–683.

```
681     def idle_action(self):
683         return np.zeros((self.config.aircraft, 8), dtype=np.int64)
```

Line-by-line explanation.

681 Define `idle_action(self)`. Construct the default fleet action, preserving the required representation for aircraft already in committed repair.

683 Return `np.zeros((self.config.aircraft, 8), dtype=np.int64)` to the caller by call np.zeros with the shown positional and keyword arguments. No further statements in this call execute.

9.38. FleetMaintenanceEnv.available_crew_mask

Set one bit for each currently free crew identity. This mask represents availability, while the selected crew mask specifies an assignment.

Source lines 686–694.

```
686     def available_crew_mask(self):
688         available = (1 << self.config.crews) - 1
690         for job in self.jobs.values():
692             available &= ~job.plan.crew_mask
694         return available
```

Line-by-line explanation.

686 Define `available_crew_mask(self)`. Set one bit for each currently free crew identity. This mask represents availability, while the selected crew mask specifies an assignment.

688 Set `available` by subtract 1 « self.config.crews and 1. Start with every configured crew bit set, meaning all members are available.

690 Iterate `job` over `self.jobs.values()`. The following indented statements run once for each selected element.

692 Bitwise-AND with `~job.plan.crew_mask` in `available`, and store the result back in that same target. The change is visible to subsequent statements in this block.

694 Return `available` to the caller by evaluate available. No further statements in this call execute.

9.39. FleetMaintenanceEnv.resource_state

Determine which identified resources are reserved by active jobs and which remain free.

Source lines 697–701.

```
697     def resource_state(self):
699         busy_shops = {j.plan.workshop for j in self.jobs.values()}
701         return (self.available_crew_mask().bit_count(), [s for s in range(self.config.workshops) if s
            ↪  not in busy_shops])
```

Line-by-line explanation.

697 Define `resource_state(self)`. Determine which identified resources are reserved by active jobs and which remain free.

699 Set `busy_shops` by evaluate {j.plan.workshop for j in self.jobs.values()}. Compute and bind the right-hand value to busy_shops, naming an intermediate for the surrounding algorithm.

701 Return `(self.available_crew_mask().bit_count(), [s for s in range(self.config.workshops) if s not in busy_shops])` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

9.40. FleetMaintenanceEnv.resource_details

Expose identified crew/workshop availability and configured time factors for diagnostics and reporting.

Source lines 704–706.

```
704     def resource_details(self):
706         return {'crew_ids': self.crew_ids, 'crew_time_factors': self.crew_time_factors,
            ↪  'available_crew_mask': self.available_crew_mask(), 'workshop_ids': self.workshop_ids,
            ↪  'workshop_time_factors': self.workshop_time_factors, 'available_workshops':
            ↪  tuple(self.resource_state()[1])}
```

Line-by-line explanation.

704 Define `resource_details(self)`. Expose identified crew/workshop availability and configured time factors for diagnostics and reporting.

706 Return `{'crew_ids': self.crew_ids, 'crew_time_factors': self.crew_time_factors, 'available_crew_mask': self.available_crew_mask(), 'workshop_ids': self.workshop_ids, 'workshop_time_factors': self.workshop_time_factors, 'available_workshops': tuple(self.resource_state()[1])}` to the caller by construct the shown dict in its stated order. No further statements in this call execute.

9.41. FleetMaintenanceEnv.sensor_noise

Return the configured efficiency/temperature uncertainty scales used by the synthetic sensor likelihood.

Source lines 709–715.

```
709     @property
711     def sensor_noise(self):
713         """Independent sensor noise scales in efficiency fraction and Celsius."""
715         return np.array([EFFICIENCY_GAIN, TEMPERATURE_HEALTH_GAIN_C]) * self.config.observation_noise
```

Line-by-line explanation.

709 Apply the decorator `property` to the following definition. Dataclass decorators generate field initialization/representation; staticmethod omits automatic instance binding where used.

711 Define `sensor_noise(self)`. Return the configured efficiency/temperature uncertainty scales used by the synthetic sensor likelihood.

713 Documentation text: Independent sensor noise scales in efficiency fraction and Celsius.. This states the contract or qualification for `FleetMaintenanceEnv.sensor_noise`; it does not update the simulator or model.

715 Return `np.array([EFFICIENCY_GAIN, TEMPERATURE_HEALTH_GAIN_C]) * self.config.observation_noise` to the caller by multiply np.array([EFFICIENCY_GAIN, TEMPERATURE_HEALTH_GAIN_C]) and self.config.observation_noise. No further statements in this call execute.

9.42. FleetMaintenanceEnv._sensor_model

Map hidden normalized health and exposure to synthetic efficiency and temperature means. This is a teaching observation model rather than a turbofan performance map.

Source lines 718–728.

```
718     @staticmethod
720     def _sensor_model(health, exposure):
722         """Toy performance mapping; health decreases efficiency and raises heat."""
724         efficiency = EFFICIENCY_BASE + EFFICIENCY_GAIN * health
726         temperature = TEMPERATURE_BASE_C + TEMPERATURE_HEALTH_GAIN_C * (1 - health) +
            ↪  TEMPERATURE_EXPOSURE_GAIN_C * (exposure - 1)
728         return np.stack([efficiency, temperature], axis=-1)
```

Line-by-line explanation.

718 Apply the decorator `staticmethod` to the following definition. Dataclass decorators generate field initialization/representation; staticmethod omits automatic instance binding where used.

720 Define `_sensor_model(health, exposure)`. Map hidden normalized health and exposure to synthetic efficiency and temperature means. This is a teaching observation model rather than a turbofan performance map.

722 Documentation text: Toy performance mapping; health decreases efficiency and raises heat.. This states the contract or qualification for `FleetMaintenanceEnv._sensor_model`; it does not update the simulator or model.

724 Set `efficiency` by add EFFICIENCY_BASE and EFFICIENCY_GAIN * health. Compute and bind the right-hand value to efficiency, naming an intermediate for the surrounding algorithm.

726 Set `temperature` by add TEMPERATURE_BASE_C + TEMPERATURE_HEALTH_GAIN_C * (1 - health) and TEMPERATURE_EXPOSURE_GAIN_C * (exposure - 1). Compute and bind the right-hand value to temperature, naming an intermediate for the surrounding algorithm.

728 Return `np.stack([efficiency, temperature], axis=-1)` to the caller by call np.stack with the shown positional and keyword arguments. No further statements in this call execute.

9.43. FleetMaintenanceEnv._clip_measurements

Bound synthetic sensor values to the accepted measurement ranges before encoding/likelihood use.

Source lines 731–737.

```
731     @staticmethod
733     def _clip_measurements(measurements):
735         np.clip(measurements[..., 0], 0, 1, out=measurements[..., 0])
737         np.clip(measurements[..., 1], TEMPERATURE_MIN_C, TEMPERATURE_MAX_C, out=measurements[..., 1])
```

Line-by-line explanation.

731 Apply the decorator `staticmethod` to the following definition. Dataclass decorators generate field initialization/representation; staticmethod omits automatic instance binding where used.

733 Define `_clip_measurements(measurements)`. Bound synthetic sensor values to the accepted measurement ranges before encoding/likelihood use.

735 Bound the numeric result elementwise. The exact receiver and arguments are shown on this source line.

737 Bound the numeric result elementwise. The exact receiver and arguments are shown on this source line.

9.44. FleetMaintenanceEnv._invert_sensor_metrics

Map observed synthetic performance back to an approximate health-scale estimate used in filter initialization or correction.

Source lines 740–750.

```
740     @staticmethod
742     def _invert_sensor_metrics(measurements, exposure):
744         """Initialization only: fuse two noisy inverse mappings with equal weight."""
746         from_efficiency = (measurements[..., 0] - EFFICIENCY_BASE) / EFFICIENCY_GAIN
748         from_temperature = 1 - (measurements[..., 1] - TEMPERATURE_BASE_C -
            ↪  TEMPERATURE_EXPOSURE_GAIN_C * (exposure - 1)) / TEMPERATURE_HEALTH_GAIN_C
750         return np.clip(0.5 * (from_efficiency + from_temperature), 0, 1)
```

Line-by-line explanation.

740 Apply the decorator `staticmethod` to the following definition. Dataclass decorators generate field initialization/representation; staticmethod omits automatic instance binding where used.

742 Define `_invert_sensor_metrics(measurements, exposure)`. Map observed synthetic performance back to an approximate health-scale estimate used in filter initialization or correction.

744 Documentation text: Initialization only: fuse two noisy inverse mappings with equal weight.. This states the contract or qualification for `FleetMaintenanceEnv._invert_sensor_metrics`; it does not update the simulator or model.

746 Set `from_efficiency` by divide measurements[..., 0] - EFFICIENCY_BASE and EFFICIENCY_GAIN. Compute and bind the right-hand value to from_efficiency, naming an intermediate for the surrounding algorithm.

748 Set `from_temperature` by subtract 1 and (measurements[..., 1] - TEMPERATURE_BASE_C - TEMPERATURE_EXPOSURE_GAIN_C * (exposure - 1)) / TEMPERATURE_HEALTH_GAIN_C. Compute and bind the right-hand value to from_temperature, naming an intermediate for the surrounding algorithm.

750 Return `np.clip(0.5 * (from_efficiency + from_temperature), 0, 1)` to the caller by bound the numeric result elementwise. No further statements in this call execute.

9.45. FleetMaintenanceEnv.observation_metrics

Decode the observed synthetic performance fields for diagnostics without exposing hidden physical condition.

Source lines 753–759.

```
753     def observation_metrics(self, observation):
755         """Decode observed sensor metrics without exposing hidden health."""
757         rows, _ = self.split_observation(observation)
759         return {'efficiency': rows[:, 45:57].reshape(self.config.aircraft, 2, 6).copy(),
            ↪  'temperature_c': rows[:, 57:69].reshape(self.config.aircraft, 2, 6) * (TEMPERATURE_MAX_C -
            ↪  TEMPERATURE_MIN_C) + TEMPERATURE_MIN_C, 'reading_age_days': np.rint(rows[:, 69] *
            ↪  self.config.horizon).astype(int)}
```

Line-by-line explanation.

753 Define `observation_metrics(self, observation)`. Decode the observed synthetic performance fields for diagnostics without exposing hidden physical condition.

755 Documentation text: Decode observed sensor metrics without exposing hidden health.. This states the contract or qualification for `FleetMaintenanceEnv.observation_metrics`; it does not update the simulator or model.

757 Set `(rows, _)` by call self.split_observation with the shown positional and keyword arguments. Unpack the return values from self.split_observation(observation) into rows; underscore entries are intentionally ignored.

759 Return `{'efficiency': rows[:, 45:57].reshape(self.config.aircraft, 2, 6).copy(), 'temperature_c': rows[:, 57:69].reshape(self.config.aircraft, 2, 6) * (TEMPERATURE_MAX_C - TEMPERATURE_MIN_C) + TEMPERATURE_MIN_C, 'reading_age_days': np.rint(rows[:, 69] * self.config.horizon).astype(int)}` to the caller by construct the shown dict in its stated order. No further statements in this call execute.

9.46. FleetMaintenanceEnv._observation

Encode this environment's current policy-visible state. In the fleet model this includes belief summaries, committed plans, sensor age and public resources, rather than true health.

Source lines 762–790.

```
762     def _observation(self):
764         mean = self.particles.mean(-1)
766         std = self.particles.std(-1)
768         low = np.quantile(self.particles, 0.1, axis=-1)
770         rows = []
772         for a in range(self.config.aircraft):
774             j = self.jobs.get(a)
776             committed = [j.plan.engine_mask / 3, j.plan.bundles[0] / 63, j.plan.bundles[1] / 63,
                ↪  j.plan.crew_mask / ((1 << self.config.crews) - 1), j.plan.workshop / max(1,
                ↪  self.config.workshops - 1), j.plan.order_start / 11] if j else [0.0] * 6
778             rows.append(np.concatenate([mean[a].ravel(), std[a].ravel(), low[a].ravel(),
                ↪  [self.exposure[a] / 1.5, float(j is not None), min(1, j.remaining / 60) if j else 0],
                ↪  committed, self.measurements[a, :, :, 0].ravel(), ((self.measurements[a, :, :, 1] -
                ↪  TEMPERATURE_MIN_C) / (TEMPERATURE_MAX_C - TEMPERATURE_MIN_C)).ravel(), [(self.day -
                ↪  self.last_flight_day[a]) / self.config.horizon]]))
780         free, shops = self.resource_state()
782         public = [self.day / self.config.horizon, free / self.config.crews, len(shops) /
            ↪  self.config.workshops, (self.config.replenish_days - self.day %
            ↪  self.config.replenish_days) / self.config.replenish_days, *np.clip(self.stock /
            ↪  self.config.aircraft, 0, 1)]
784         crew_mask = self.available_crew_mask()
786         public.extend([float(bool(crew_mask & (1 << i))) for i in range(self.config.crews)] + [1 / (1
            ↪  + factor) for factor in self.crew_time_factors])
```

```
788         public.extend([float(i in shops) for i in range(self.config.workshops)] + [1 / (1 + factor)
            ↪  for factor in self.workshop_time_factors])
790         return np.asarray(np.concatenate([np.asarray(rows).ravel(), public]), dtype=np.float32)
```

Line-by-line explanation.

762 Define `_observation(self)`. Encode this environment's current policy-visible state. In the fleet model this includes belief summaries, committed plans, sensor age and public resources, rather than true health.

764 Set `mean` by average the selected entries. Compute and bind the right-hand value to mean, naming an intermediate for the surrounding algorithm.

766 Set `std` by call self.particles.std with the shown positional and keyword arguments. Compute and bind the right-hand value to std, naming an intermediate for the surrounding algorithm.

768 Set `low` by call np.quantile with the shown positional and keyword arguments. Compute the conservative 10th-percentile health estimate and store it in low; later checks/updates use this named intermediate.

770 Set `rows` by construct the shown list in its stated order. Compute and bind the right-hand value to rows, naming an intermediate for the surrounding algorithm.

772 Iterate `a` over `range(self.config.aircraft)`. The following indented statements run once for each selected element.

774 Set `j` by call self.jobs.get with the shown positional and keyword arguments. Compute and bind the right-hand value to j, naming an intermediate for the surrounding algorithm.

776 Set `committed` by select [j.plan.engine_mask / 3, j.plan.bundles[0] / 63, j.plan.bundles[1] / 63, j.plan.crew_mask / ((1 « self.config.crews) - 1), j.plan.workshop / max(1, self.config.workshops - 1), j.plan.order_start / 11] when j, otherwise [0.0] * 6. Compute and bind the right-hand value to committed, naming an intermediate for the surrounding algorithm.

778 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

780 Set `(free, shops)` by call self.resource_state with the shown positional and keyword arguments. Unpack self.resource_state() into named values free, shops so each part can be used separately below.

782 Set `public` by construct the shown list in its stated order. Compute fleet-wide observation values shared by aircraft and store it in public; later checks/updates use this named intermediate.

784 Set `crew_mask` by call self.available_crew_mask with the shown positional and keyword arguments. It holds the binary selection of individual crew identities. Read the individual availability mask after all current reservations.

786 Call public.extend with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

788 Call public.extend with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

790 Return `np.asarray(np.concatenate([np.asarray(rows).ravel(), public]), dtype=np.float32)` to the caller by call np.asarray with the

shown positional and keyword arguments. No further statements in this call execute.

9.47. FleetMaintenanceEnv.validate_row

Decode and validate a single aircraft action row, including bitmask/category limits and the structural contract for a proposed plan.

Source lines 793–811.

```
793     def validate_row(self, aircraft, row):
795         ground, engines, b0, b1, crew_mask, shop, order, priority = map(int, row)
797         if not ground:
799             return None if engines == b0 == b1 == 0 else 'work_without_grounding'
801         actual = int(b0 != 0) + 2 * int(b1 != 0)
803         if not actual or engines != actual:
805             return 'inconsistent_engine_bundle'
807         if not 0 < crew_mask < (1 << self.config.crews):
809             return 'invalid_crew_mask'
811         return None
```

Line-by-line explanation.

793 Define `validate_row(self, aircraft, row)`. Decode and validate a single aircraft action row, including bitmask/category limits and the structural contract for a proposed plan.

795 Set `(ground, engines, b0, b1, crew_mask, shop, order, priority)` by call map with the shown positional and keyword arguments. Decode the action tuple by position into grounding flag, engine mask, and component mask; this gives the later bit checks meaningful names.

797 Test `not ground`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

799 Return `None if engines == b0 == b1 == 0 else 'work_without_grounding'` to the caller by select None when engines == b0 == b1 == 0, otherwise 'work_without_grounding'. No further statements in this call execute.

801 Set `actual` by add int(b0 != 0) and 2 * int(b1 != 0). Derive the engine mask from whether each bundle is nonempty. int(b0 != 0) is engine 0's low bit (1); 2 * int(b1 != 0) places engine 1 in bit 1 (2). Examples: [5,0] to 012 to 1, [0,8] to 102 to 2, [5,8] to 112 to 3. Both zero gives 0 and cannot describe grounded work. The factor 2 is a bit-position weight, not extra priority; a busy aircraft's committed job is separate and is not canceled by this proposal.

803 Test `not actual or engines != actual`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

805 Return `'inconsistent_engine_bundle'` to the caller by use the literal 'inconsistent_engine_bundle'. No further statements in this call execute.

807 Test `not 0 < crew_mask < 1 « self.config.crews`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

809 Return `'invalid_crew_mask'` to the caller by use the literal 'invalid_crew_mask'. No further statements in this call execute.

811 Return `None` to the caller by use the literal None. No further statements in this call execute.

9.48. FleetMaintenanceEnv._start_job

Check proposed work against aircraft/resource/stock availability; accept and reserve a committed job or record rejection. Store resource-derived task completion timing at acceptance.

Source lines 814–848.

```
814     def _start_job(self, aircraft, row):
816         _, engines, b0, b1, crew_mask, shop, order, priority = map(int, row)
818         plan = Plan(aircraft, engines, (b0, b1), crew_mask, shop, order, priority)
820         tasks = [(e, c) for e, b in enumerate((b0, b1)) for c in range(6) if b & 1 << c]
822         tasks.sort(key=lambda x: (x[0] * 6 + x[1] - order) % 12)
824         needed = np.bincount([c for _, c in tasks], minlength=6)
826         _, shops = self.resource_state()
828         if crew_mask & self.available_crew_mask() != crew_mask or shop not in shops:
830             return ('resource_conflict', 0.0)
832         if np.any(needed > self.stock):
834             return ('spare_shortage', 0.0)
836         work_rate = sum(1 / factor for i, factor in enumerate(self.crew_time_factors) if crew_mask &
            ↪  (1 << i))
838         duration = self.config.setup_days
840         completion_days = []
842         for _, component in tasks:
844             duration += max(1, math.ceil(LABOR_DAYS[component] * self.workshop_time_factors[shop] /
                ↪  work_rate))
846             completion_days.append(duration)
848         self.stock -= needed
```

Line-by-line explanation.

814 Define `_start_job(self, aircraft, row)`. Check proposed work against aircraft/resource/stock availability; accept and reserve a committed job or record rejection. Store resource-derived task completion timing at acceptance.

816 Set `(_, engines, b0, b1, crew_mask, shop, order, priority)` by call map with the shown positional and keyword arguments. Decode engine/component selections, the individual crew bitmask, workshop index, task ordering, and priority.

818 Set `plan` by call Plan with the shown positional and keyword arguments. Compute the proposed or accepted maintenance plan and store it in plan; later checks/updates use this named intermediate.

820 Set `tasks` by construct the collection by iterating the shown comprehension and applying its filters. Decode bundle bits into (engine, component) tasks. enumerate maps b0/b1 to engine e; b & (1 « c) tests component c. Each set bit adds one repair task; a zero bit adds none.

822 Call tasks.sort with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

824 Set `needed` by call np.bincount with the shown positional and keyword arguments. Count requested repairs by component type, then compare those needs with inventory before committing anything; this prevents partial plans when stock is short.

826 Set `(_, shops)` by call self.resource_state with the shown positional and keyword arguments. Unpack self.resource_state() into named values free, shops so each part can be used separately below.

828 Test `crew_mask & self.available_crew_mask() != crew_mask or shop not in shops`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

830 Return `('resource_conflict', 0.0)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

832 Test `np.any(needed > self.stock)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

834 Return `('spare_shortage', 0.0)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

836 Set `work_rate` by add the terms of the supplied iterable. Combine selected members' work rates; two standard members supply twice the rate of one standard member under this teaching assumption.

838 Set `duration` by evaluate self.config.setup_days. It holds the number of primitive days in this decision. Begin with the fixed setup delay, which resource expertise does not shorten.

840 Set `completion_days` by construct the shown list in its stated order. Build one cumulative completion threshold per component before committing any inventory or resources.

842 Iterate `(_, component)` over `tasks`. The following indented statements run once for each selected element.

844 Add `max(1, math.ceil(LABOR_DAYS[component] * self.workshop_time_factors[shop] / work_rate))` to `duration`, and store the result back in that same target. This updates the number of primitive days in this decision.

846 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

848 Subtract `needed` from `self.stock`, and store the result back in that same target. This updates spare inventory by component type.

Source lines 850–856.

```
850         self.jobs[aircraft] = Job(plan, tuple(tasks), duration, duration, tuple(completion_days))
852         cost = 12.0 + engines.bit_count() * 5 + sum((PART_COST[c] + 3 * self.health[aircraft, e, c]
            ↪  for e, c in tasks))
854         self.metrics['groundings'] += 1
856         return (None, cost)
```

Line-by-line explanation.

850 Set `self.jobs[aircraft]` by call Job with the shown positional and keyword arguments. It holds accepted repair records held by the simulator.

852 Set `cost` by add 12.0 + engines.bit_count() * 5 and sum((PART_COST[c] + 3 * self.health[aircraft, e, c] for e, c in tasks)). Compute and bind the right-hand value to cost, naming an intermediate for the surrounding algorithm.

854 Add `1` to `self.metrics['groundings']`, and store the result back in that same target. The change is visible to subsequent statements in this block.

856 Return `(None, cost)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

9.49. FleetMaintenanceEnv._update_filter

Predict component-health particles under the assumed wear/shock prior, score the synthetic measurements and resample. This is approximate independent marginal filtering, not an RUL forecast.

Source lines 859–891.

```
859     def _update_filter(self, aircraft, measurements):
861         """Bootstrap particle filter with noisy observations and shock-mixture prior.
863         The estimator does not receive true event type, severity or hidden health.
864         It is a deliberately simple independent-component filter, not Emanuele's.
865         """
867         cfg = self.config
869         p = self.particles[aircraft]
871         wear = WEAR[None, :, None] * self.exposure[aircraft] * cfg.wear_scale
873         p -= wear * self.filter_rng.normal(1, 0.12, p.shape)
875         bird = self.filter_rng.random((2, 1, cfg.particles)) < cfg.bird_probability
877         hail = self.filter_rng.random((2, 1, cfg.particles)) < cfg.hail_probability
879         bseverity = self.filter_rng.uniform(0.1, 0.35, (2, 1, cfg.particles))
881         hseverity = self.filter_rng.uniform(0.1, 0.35, (2, 1, cfg.particles))
883         p -= bird * bseverity * np.array([1, 0.5, 0.15, 0, 0, 0])[None, :, None]
885         p -= hail * hseverity * np.array([0.7, 0.4, 0.3, 0.1, 0.1, 0.1])[None, :, None]
887         p -= (self.filter_rng.random((1, 1, cfg.particles)) < cfg.weather_probability) * wear * 2
889         np.clip(p, 0, 1, out=p)
891         predicted_efficiency = EFFICIENCY_BASE + EFFICIENCY_GAIN * p
```

Line-by-line explanation.

859 Define `_update_filter(self, aircraft, measurements)`. Predict component-health particles under the assumed wear/shock prior, score the synthetic measurements and resample. This is approximate independent marginal filtering, not an RUL forecast.

861 Documentation text: Bootstrap particle filter with noisy observations and shock-mixture prior.. This states the contract or qualification for `FleetMaintenanceEnv._update_filter`; it does not update the simulator or model.

863 Documentation text: The estimator does not receive true event type, severity or hidden health.. This states the contract or qualification for `FleetMaintenanceEnv._update_filter`; it does not update the simulator or model.

864 Documentation text: It is a deliberately simple independent-component filter, not Emanuele's.. This states the contract or qualification for `FleetMaintenanceEnv._update_filter`; it does not update the simulator or model.

865 Close the documentation string for `FleetMaintenanceEnv._update_filter`. This delimiter ends the descriptive text and performs no simulator or learning update.

867 Set `cfg` by evaluate self.config. Compute and bind the right-hand value to cfg, naming an intermediate for the surrounding algorithm.

869 Set `p` by select the indexed/sliced entries of self.particles. Compute and bind the right-hand value to p, naming an intermediate for the surrounding algorithm.

871 Set `wear` by multiply WEAR[None, :, None] * self.exposure[aircraft] and cfg.wear_scale. Compute and bind the right-hand value to wear, naming an intermediate for the surrounding algorithm.

873 Subtract `wear * self.filter_rng.normal(1, 0.12, p.shape)` from `p`, and store the result back in that same target. The change is visible to subsequent statements in this block.

875 Set `bird` by evaluate the comparison self.filter_rng.random((2, 1, cfg.particles)) < cfg.bird_probability. Compute and bind the right-hand value to bird, naming an intermediate for the surrounding algorithm.

877 Set `hail` by evaluate the comparison self.filter_rng.random((2, 1, cfg.particles)) < cfg.hail_probability. Compute and bind the right-hand value to hail, naming an intermediate for the surrounding algorithm.

879 Set `bseverity` by call self.filter_rng.uniform with the shown positional and keyword arguments. Compute and bind the right-hand value to bseverity, naming an intermediate for the surrounding algorithm.

881 Set `hseverity` by call self.filter_rng.uniform with the shown positional and keyword arguments. Compute and bind the right-hand value to hseverity, naming an intermediate for the surrounding algorithm.

883 Subtract `bird * bseverity * np.array([1, 0.5, 0.15, 0, 0, 0])[None, :, None]` from `p`, and store the result back in that same target. The change is visible to subsequent statements in this block.

885 Subtract `hail * hseverity * np.array([0.7, 0.4, 0.3, 0.1, 0.1, 0.1])[None, :, None]` from `p`, and store the result back in that same target. The change is visible to subsequent statements in this block.

887 Subtract `(self.filter_rng.random((1, 1, cfg.particles)) < cfg.weather_probability) * wear * 2` from `p`, and store the result back in that same target. The change is visible to subsequent statements in this block.

889 Bound the numeric result elementwise. The exact receiver and arguments are shown on this source line.

891 Set `predicted_efficiency` by add EFFICIENCY_BASE and EFFICIENCY_GAIN * p. Compute and bind the right-hand value to predicted_efficiency, naming an intermediate for the surrounding algorithm.

Source lines 893–911.

```
893         predicted_temperature = TEMPERATURE_BASE_C[None, :, None] + TEMPERATURE_HEALTH_GAIN_C * (1 -
            ↪  p) + TEMPERATURE_EXPOSURE_GAIN_C * (self.exposure[aircraft] - 1)
895         efficiency_error = (predicted_efficiency - measurements[:, :, 0, None]) / self.sensor_noise[0]
897         temperature_error = (predicted_temperature - measurements[:, :, 1, None]) /
            ↪  self.sensor_noise[1]
899         logits = -0.5 * (efficiency_error ** 2 + temperature_error ** 2)
901         weights = np.exp(logits - logits.max(-1, keepdims=True))
903         weights /= weights.sum(-1, keepdims=True)
905         for e in range(2):
907             for c in range(6):
909                 idx = self.filter_rng.choice(cfg.particles, cfg.particles, p=weights[e, c])
911                 p[e, c] = np.clip(p[e, c, idx] + self.filter_rng.normal(0, 0.006, cfg.particles), 0, 1)
```

Line-by-line explanation.

893 Set `predicted_temperature` by add TEMPERATURE_BASE_C[None, :, None] + TEMPERATURE_HEALTH_GAIN_C * (1 - p) and TEMPERATURE_EXPOSURE_GAIN_C * (self.exposure[aircraft] - 1). Compute and bind the right-hand value to predicted_temperature, naming an intermediate for the surrounding algorithm.

895 Set `efficiency_error` by divide predicted_efficiency - measurements[:, :, 0, None] and self.sensor_noise[0]. Compute and bind the right-hand value to efficiency_error, naming an intermediate for the surrounding algorithm.

897 Set `temperature_error` by divide predicted_temperature - measurements[:, :, 1, None] and self.sensor_noise[1]. Compute and bind the right-hand value to temperature_error, naming an intermediate for the surrounding algorithm.

899 Set `logits` by multiply -0.5 and efficiency_error ** 2 + temperature_error ** 2. Compute unnormalized scores later converted to choice probabilities and store it in logits; later checks/updates use this named intermediate.

901 Set `weights` by exponentiate log values, including log-probability ratios. It holds the detached capped importance weights for replay. Compute and bind the right-hand value to weights, naming an intermediate for the surrounding algorithm.

903 Divide by `weights.sum(-1, keepdims=True)` in `weights`, and store the result back in that same target. This updates the detached capped importance weights for replay.

905 Iterate `e` over `range(2)`. The following indented statements run once for each selected element.

907 Iterate `c` over `range(6)`. The following indented statements run once for each selected element.

909 Set `idx` by call self.filter_rng.choice with the shown positional and keyword arguments. Compute and bind the right-hand value to idx, naming an intermediate for the surrounding algorithm.

911 Set `p[e, c]` by bound the numeric result elementwise. Unpack np.clip(p[e, c, idx] + self.filter_rng.normal(0, 0.006, cfg.particles), 0, 1) into named values p[e, c] so each part can be used separately below.

9.50. FleetMaintenanceEnv.step

Apply the action through this class's transition/adapter contract and return the resulting observation, reward and episode flags.

Source lines 914–948.

```
914     def step(self, action):
916         if self._done:
918             raise RuntimeError('reset before step, including after episode completion')
920         a = np.asarray(action)
922         if not np.issubdtype(a.dtype, np.integer) or not self.action_space.contains(a):
924             raise ValueError('action outside MultiDiscrete space')
926         cfg = self.config
928         low = np.quantile(self.particles, 0.1, axis=-1)
930         mandatory = low.min(axis=(1, 2)) <= cfg.mandatory_health
932         costs = 0.0
934         events = []
936         rejected = []
938         completed = []
940         grounded = set(self.jobs)
942         for aircraft in sorted(range(cfg.aircraft), key=lambda i: (-int(a[i, 7]), i)):
944             if aircraft in self.jobs:
946                 continue
948             reason = self.validate_row(aircraft, a[aircraft])
```

Line-by-line explanation.

914 Define `step(self, action)`. This step method is part of FleetMaintenanceEnv.step; its parameters and defaults are the interface used by the calls in this section.

916 Test `self._done`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

918 Raise `RuntimeError('reset before step, including after episode completion')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

920 Set `a` by call np.asarray with the shown positional and keyword arguments. Compute and bind the right-hand value to a, naming an intermediate for the surrounding algorithm.

922 Test `not np.issubdtype(a.dtype, np.integer) or not self.action_space.contains(a)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

924 Raise `ValueError('action outside MultiDiscrete space')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

926 Set `cfg` by evaluate self.config. Compute and bind the right-hand value to cfg, naming an intermediate for the surrounding algorithm.

928 Set `low` by call np.quantile with the shown positional and keyword arguments. Compute the conservative 10th-percentile health estimate and store it in low; later checks/updates use this named intermediate.

930 Set `mandatory` by evaluate the comparison low.min(axis=(1, 2)) <= cfg.mandatory_health. Compute whether a lower-health safety alarm is active and store it in mandatory; later checks/updates use this named intermediate.

932 Set `costs` by use the literal 0.0. Compute repair, downtime, rejection, and failure costs and store it in costs; later checks/updates use this named intermediate.

934 Set `events` by construct the shown list in its stated order. Compute event records generated during the transition and store it in events; later checks/updates use this named intermediate.

936 Set `rejected` by construct the shown list in its stated order. Compute proposed plans that were not accepted and why and store it in rejected; later checks/updates use this named intermediate.

938 Set `completed` by construct the shown list in its stated order. Compute aircraft whose jobs finished today and store it in completed; later checks/updates use this named intermediate.

940 Set `grounded` by call set with the shown positional and keyword arguments. Compute and bind the right-hand value to grounded, naming an intermediate for the surrounding algorithm.

942 Iterate `aircraft` over `sorted(range(cfg.aircraft), key=lambda i: (-int(a[i, 7]), i))`. The following indented statements run once for each selected element.

944 Test `aircraft in self.jobs`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

946 Skip the remainder of this loop iteration and begin the next iteration.

948 Set `reason` by call self.validate_row with the shown positional and keyword arguments. Compute the validation or resource-conflict reason and store it in reason; later checks/updates use this named intermediate.

Source lines 950–984.

```
            if reason is None and int(a[aircraft, 0]):
                reason, cost = self._start_job(aircraft, a[aircraft])
                costs += cost
            if reason:
                rejected.append((aircraft, reason))
                costs += 2.0
            if mandatory[aircraft] and aircraft not in self.jobs:
                grounded.add(aircraft)
                self.metrics['unsafe_attempts'] += float(not int(a[aircraft, 0]))
            if aircraft in self.jobs:
                grounded.add(aircraft)
        self.metrics['rejections'] += len(rejected)
        flights = 0
        failures = 0
        weather = self.weather_tape[self.day] < cfg.weather_probability
        for aircraft in range(cfg.aircraft):
            if aircraft in grounded:
                continue
```

Line-by-line explanation.

950 Test `reason is None and int(a[aircraft, 0])`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

952 Set `(reason, cost)` by call self._start_job with the shown positional and keyword arguments. Unpack self._start_job(aircraft, a[aircraft]) into named values reason, cost so each part can be used separately below.

954 Add `cost` to `costs`, and store the result back in that same target. The change is visible to subsequent statements in this block.

956 Test `reason`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

958 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

960 Add `2.0` to `costs`, and store the result back in that same target. The change is visible to subsequent statements in this block.

962 Test `mandatory[aircraft] and aircraft not in self.jobs`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

964 Call grounded.add with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

966 Add `float(not int(a[aircraft, 0]))` to `self.metrics['unsafe_attempts']`, and store the result back in that same target. The change is visible to subsequent statements in this block.

968 Test `aircraft in self.jobs`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

970 Call grounded.add with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

972 Add `len(rejected)` to `self.metrics['rejections']`, and store the result back in that same target. The change is visible to subsequent statements in this block.

974 Set `flights` by use the literal 0. Compute the number of aircraft that flew today and store it in flights; later checks/updates use this named intermediate.

976 Set `failures` by use the literal 0. Compute the number of failures in this transition or episode and store it in failures; later checks/updates use this named intermediate.

978 Set `weather` by evaluate the comparison self.weather_tape[self.day] < cfg.weather_probability. Compute whether the shared weather event occurs today and store it in weather; later checks/updates use this named intermediate.

980 Iterate `aircraft` over `range(cfg.aircraft)`. The following indented statements run once for each selected element.

982 Test `aircraft in grounded`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

984 Skip the remainder of this loop iteration and begin the next iteration.

Source lines 986–1020.

```
986             flights += 1
988             self.health[aircraft] -= WEAR * self.exposure[aircraft] * cfg.wear_scale * np.maximum(0,
                ↪  self.wear_tape[self.day, aircraft]) * (3 if weather else 1)
990             if weather:
992                 self.metrics['weather_exposures'] += 1
994             for e in range(2):
996                 for kind, p, impact in [('bird', cfg.bird_probability, [1, 0.5, 0.15, 0, 0, 0]),
                    ↪  ('hail', cfg.hail_probability, [0.7, 0.4, 0.3, 0.1, 0.1, 0.1])]:
998                     k = int(kind == 'hail')
1000                    if self.event_tape[self.day, aircraft, e, k] < p:
1002                        severity = self.severity_tape[self.day, aircraft, e, k]
1004                        self.health[aircraft, e] -= severity * np.array(impact)
1006                        events.append({'aircraft': aircraft, 'aircraft_plate':
                            ↪  self.aircraft_plates[aircraft], 'engine': e, 'type': kind})
1008                        self.metrics[kind + '_events'] += 1
1010            np.clip(self.health[aircraft], 0, 1, out=self.health[aircraft])
1012            failed = bool(np.any(self.health[aircraft] <= 0))
1014            failures += failed
1016            self.measurements[aircraft] = self._sensor_model(self.health[aircraft],
                ↪  self.exposure[aircraft])
1018            self.measurements[aircraft] += self.noise_tape[self.day, aircraft] *
                ↪  np.array([EFFICIENCY_GAIN, TEMPERATURE_HEALTH_GAIN_C])
1020            self._clip_measurements(self.measurements[aircraft])
```

Line-by-line explanation.

986 Add `1` to `flights`, and store the result back in that same target. The change is visible to subsequent statements in this block.

988 Subtract `WEAR * self.exposure[aircraft] * cfg.wear_scale * np.maximum(0, self.wear_tape[self.day, aircraft]) * (3 if weather else 1)` from `self.health[aircraft]`, and store the result back in that same target. This updates hidden physical component condition.

990 Test `weather`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

992 Add `1` to `self.metrics['weather_exposures']`, and store the result back in that same target. The change is visible to subsequent statements in this block.

994 Iterate `e` over `range(2)`. The following indented statements run once for each selected element.

996 Iterate `(kind, p, impact)` over `[('bird', cfg.bird_probability, [1, 0.5, 0.15, 0, 0, 0]), ('hail', cfg.hail_probability, [0.7, 0.4, 0.3, 0.1, 0.1, 0.1])]`. The following indented statements run once for each selected element.

998 Set `k` by convert the result to a Python integer. Compute and bind the right-hand value to k, naming an intermediate for the surrounding algorithm.

1000 Test `self.event_tape[self.day, aircraft, e, k] < p`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1002 Set `severity` by select the indexed/sliced entries of self.severity_tape. Compute and bind the right-hand value to severity, naming an intermediate for the surrounding algorithm.

1004 Subtract `severity * np.array(impact)` from `self.health[aircraft, e]`, and store the result back in that same target. This updates hidden physical component condition.

1006 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

1008 Add `1` to `self.metrics[kind + '_events']`, and store the result back in that same target. The change is visible to subsequent statements in this block.

1010 Bound the numeric result elementwise. The exact receiver and arguments are shown on this source line.

1012 Set `failed` by convert the condition to a Python Boolean. Compute and bind the right-hand value to failed, naming an intermediate for the surrounding algorithm.

1014 Add `failed` to `failures`, and store the result back in that same target. The change is visible to subsequent statements in this block.

1016 Set `self.measurements[aircraft]` by call self._sensor_model with the shown positional and keyword arguments.

1018 Add `self.noise_tape[self.day, aircraft] * np.array([EFFICIENCY_GAIN, TEMPERATURE_HEALTH_GAIN_C])` to `self.measurements[aircraft]`, and store the result back in that same target. The change is visible to subsequent statements in this block.

1020 Call self._clip_measurements with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

Source lines 1022–1056.

```
            self.last_flight_day[aircraft] = self.day + 1
            self._update_filter(aircraft, self.measurements[aircraft])
        for aircraft, j in list(self.jobs.items()):
            elapsed = j.total_duration - j.remaining + 1
            for (e, c), task_day in zip(j.tasks, j.completion_days):
                if elapsed == task_day:
                    self.health[aircraft, e, c] = 1.0
                    self.particles[aircraft, e, c] = 1.0
                    self.metrics['replacements'] += 1
            j.remaining -= 1
            if j.remaining == 0:
                completed.append(aircraft)
```

```
                del self.jobs[aircraft]
        downtime = len(grounded)
        costs += downtime * 2 + failures * 80
        reward = (flights * 10 - costs) / (cfg.aircraft * 10)
        self.metrics['return'] += reward
        self.metrics['cost'] += costs
```

Line-by-line explanation.

1022 Set `self.last_flight_day[aircraft]` by add self.day and 1.

1024 Call self._update_filter with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

1026 Iterate `(aircraft, j)` over `list(self.jobs.items())`. The following indented statements run once for each selected element.

1028 Set `elapsed` by add j.total_duration - j.remaining and 1. Convert total duration minus remaining days into a one-based elapsed-day count for locating the current work stage.

1030 Iterate `((e, c), task_day)` over `zip(j.tasks, j.completion_days)`. The following indented statements run once for each selected element.

1032 Test `elapsed == task_day`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1034 Set `self.health[aircraft, e, c]` by use the literal 1.0. It holds hidden physical component condition. When the scheduled task completes, set the replaced component's hidden health to 1.0, the simulator's fully-new value.

1036 Set `self.particles[aircraft, e, c]` by use the literal 1.0. It holds samples representing uncertain component health. Reset the belief particles for the replaced component to fully healthy because the repair outcome is known.

1038 Add `1` to `self.metrics['replacements']`, and store the result back in that same target. The change is visible to subsequent statements in this block.

1040 Subtract `1` from `j.remaining`, and store the result back in that same target. The change is visible to subsequent statements in this block.

1042 Test `j.remaining == 0`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1044 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

1046 Execute the Python fragment `del self.jobs[aircraft]` in `FleetMaintenanceEnv.step`. Its indentation places it in the surrounding control block.

1048 Set `downtime` by count entries. Compute and bind the right-hand value to downtime, naming an intermediate for the surrounding algorithm.

1050 Add `downtime * 2 + failures * 80` to `costs`, and store the result back in that same target. The change is visible to subsequent statements in this block.

1052 Set `reward` by divide flights * 10 - costs and cfg.aircraft * 10. Compute the immediate transition reward and store it in reward; later checks/updates use this named intermediate.

1054 Add `reward` to `self.metrics['return']`, and store the result back in that same target. The change is visible to subsequent statements in this block.

1056 Add `costs` to `self.metrics['cost']`, and store the result back in that same target. The change is visible to subsequent statements in this block.

Source lines 1058–1074.

```
1058            self.metrics['flights'] += flights
1060            self.metrics['flight_failures'] += failures
1062            self.metrics['downtime_aircraft_days'] += downtime
1064            self.day += 1
1066            if self.day % cfg.replenish_days == 0:
1068                self.stock = np.minimum(cfg.aircraft, self.stock + max(1, cfg.aircraft // 2))
1070            self._done = self.day >= cfg.horizon
1072            info = {'day': self.day, 'aircraft_plates': self.aircraft_plates, 'resources':
                ↪  self.resource_details(), 'events': events, 'rejected': rejected, 'completed': completed,
                ↪  'reward_terms': {'flight_revenue': flights * 10, 'cost': costs}, 'metrics':
                ↪  dict(self.metrics)}
1074            return (self._observation(), float(reward), self._done, False, info)
```

Line-by-line explanation.

1058 Add `flights` to `self.metrics['flights']`, and store the result back in that same target. The change is visible to subsequent statements in this block.

1060 Add `failures` to `self.metrics['flight_failures']`, and store the result back in that same target. The change is visible to subsequent statements in this block.

1062 Add `downtime` to `self.metrics['downtime_aircraft_days']`, and store the result back in that same target. The change is visible to subsequent statements in this block.

1064 Add `1` to `self.day`, and store the result back in that same target. The change is visible to subsequent statements in this block.

1066 Test `self.day % cfg.replenish_days == 0`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1068 Set `self.stock` by call np.minimum with the shown positional and keyword arguments. It holds spare inventory by component type.

1070 Set `self._done` by evaluate the comparison self.day >= cfg.horizon.

1072 Set `info` by construct the shown dict in its stated order. Include the fixed plate mapping so aircraft indices in events, rejected proposals, and completed jobs resolve to the same named assets.

1074 Return `(self._observation(), float(reward), self._done, False, info)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

9.51. FleetMaintenanceEnv.render

This method implements FleetMaintenanceEnv.render in the class described above. Read its inputs, validity branches and returned objects together; persistent writes use the instance or simulator records.

Source lines 1077–1079.

```
1077        def render(self):
1079            return f'day={self.day} aircraft_plates={self.aircraft_plates} busy={[self.aircraft_plates[i]
                ↪  for i in sorted(self.jobs)]} crews/shops={self.resource_state()}
                ↪  stock={self.stock.tolist()}'
```

Line-by-line explanation.

1077 Define `render(self)`. This render method is part of FleetMaintenanceEnv.render; its parameters and defaults are the interface used by the calls in this section.

1079 Return `f'day={self.day} aircraft_plates={self.aircraft_plates} busy={[self.aircraft_plates[i] for i in sorted(self.jobs)]} crews/shops={self.resource_state()} stock={self.stock.tolist()}'` to the caller by evaluate f'day={self.day} aircraft_plates={self.aircraft_plates} busy={[self.aircraft_plates[i] for i in sorted(self.jobs)]} crews/shops={self.resource_state()} stock={self.stock.tolist()}'. No further statements in this call execute.

9.52. threshold_action

A fixed threshold controller for the original full-action interface. It selects observed-condition-driven repairs and feasible resources; it contains no trained neural parameters.

Source lines 1082–1116.

```
1082 def threshold_action(env, observation, threshold=0.65, bundle_threshold=0.75):
1084     """Feasible rule using only observations, not hidden health or event diagnostics."""
1086     rows, public = env.split_observation(observation)
1088     action = env.idle_action()
1090     free_mask = sum(1 << i for i in range(env.config.crews) if public[10 + i] > 0.5)
1092     shop_offset = 10 + 2 * env.config.crews
1094     shops = sorted((i for i in range(env.config.workshops) if public[shop_offset + i] > 0.5),
         ↪  key=lambda i: env.workshop_time_factors[i])
1096     stock = np.rint(public[4:10] * env.config.aircraft).astype(int)
1098     scores = rows[:, 24:36].min(1)
1100     rank = np.argsort(scores)
1102     for priority, aircraft in enumerate(rank[::-1]):
1104         action[aircraft, 7] = priority
1106     for aircraft in rank:
1108         if rows[aircraft, 37] > 0.5 or scores[aircraft] > threshold or not free_mask or (not shops):
1110             continue
1112         health = rows[aircraft, :12].reshape(2, 6)
1114         selected = health <= bundle_threshold
1116         if not selected.any():
```

Line-by-line explanation.

1082 Define `threshold_action(env, observation, threshold=0.65, bundle_threshold=0.75)`. A fixed threshold controller for the original full-action interface. It selects observed-condition-driven repairs and feasible resources; it contains no trained neural parameters.

1084 Documentation text: Feasible rule using only observations, not hidden health or event diagnostics.. This states the contract or qualification for `threshold_action`; it does not update the simulator or model.

1086 Set `(rows, public)` by call env.split_observation with the shown positional and keyword arguments. Unpack env.split_observation(observation) into named values rows, public so each part can be used separately below.

1088 Set `action` by call env.idle_action with the shown positional and keyword arguments. It holds the sampled or stored action whose probability is evaluated. Compute the proposed or selected action and store it in action; later checks/updates use this named intermediate.

1090 Set `free_mask` by add the terms of the supplied iterable. Decode individual crew availability from observed flags rather than reading hidden simulator state.

1092 Set `shop_offset` by add 10 and 2 * env.config.crews. Workshop flags follow crew flags and crew-factor encodings in the public observation.

1094 Set `shops` by call sorted with the shown positional and keyword arguments. Rank available workshops by their known duration factors, preferring the faster workshop first.

1096 Set `stock` by call np.rint(public[4:10] * env.config.aircraft).astype with the shown positional and keyword arguments. It holds spare inventory by component type. Compute the spare-part inventory by component type and store it in stock; later checks/updates use this named intermediate.

1098 Set `scores` by call rows[:, 24:36].min with the shown positional and keyword arguments. Compute the candidate action or option values being compared and store it in scores; later checks/updates use this named intermediate.

1100 Set `rank` by call np.argsort with the shown positional and keyword arguments. Compute and bind the right-hand value to rank, naming an intermediate for the surrounding algorithm.

1102 Iterate `(priority, aircraft)` over `enumerate(rank[::-1])`. The following indented statements run once for each selected element.

1104 Set `action[aircraft, 7]` by evaluate priority. It holds the sampled or stored action whose probability is evaluated. Unpack priority into named values action[aircraft, 7] so each part can be used separately below.

1106 Iterate `aircraft` over `rank`. The following indented statements run once for each selected element.

1108 Test `rows[aircraft, 37] > 0.5 or scores[aircraft] > threshold or (not free_mask) or (not shops)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1110 Skip the remainder of this loop iteration and begin the next iteration.

1112 Set `health` by reinterpret the shape while retaining element order. It holds hidden physical component condition. Reshape this aircraft's first 12 observation values into [engine, component]. They are policy-visible estimates, not hidden simulator truth.

1114 Set `selected` by evaluate the comparison health <= bundle_threshold. Build a 2 times 6 Boolean grid selecting each component whose estimated mean health is at or below the bundle threshold.

1116 Test `not selected.any()`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

Source lines 1118–1144.

```
            selected.flat[rows[aircraft, 24:36].argmin()] = True
        bundles = [0, 0]
        for e in range(2):
            for c in range(6):
                if selected[e, c] and stock[c] > 0:
                    bundles[e] |= 1 << c
                    stock[c] -= 1
        mask = int(bundles[0] != 0) + 2 * int(bundles[1] != 0)
```

```
        if not mask:
            continue
        crew = min((i for i in range(env.config.crews) if free_mask & (1 << i)), key=lambda i:
        ↪  env.crew_time_factors[i])
        action[aircraft] = [1, mask, *bundles, 1 << crew, shops.pop(0), int(health.argmin()),
        ↪  action[aircraft, 7]]
        free_mask &= ~(1 << crew)
    return action
```

Line-by-line explanation.

1118 Set `selected.flat[rows[aircraft, 24:36].argmin()]` by use the literal True. If the mean-health threshold selected nothing, flatten the 2 times 6 grid and force its weakest lower-health estimate to True. This fallback asks for at least one repair, if stock permits.

1120 Set `bundles` by construct the shown list in its stated order. Compute the separate six-bit component masks, one per fleet engine and store it in bundles; later checks/updates use this named intermediate.

1122 Iterate `e` over `range(2)`. The following indented statements run once for each selected element.

1124 Iterate `c` over `range(6)`. The following indented statements run once for each selected element.

1126 Test `selected[e, c] and stock[c] > 0`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1128 Bitwise-OR in `1 << c` in `bundles[e]`, and store the result back in that same target. The change is visible to subsequent statements in this block.

1130 Subtract `1` from `stock[c]`, and store the result back in that same target. This updates spare inventory by component type.

1132 Set `mask` by add int(bundles[0] != 0) and 2 * int(bundles[1] != 0). Turn two nonempty-bundle tests into engine flags: engine 0 contributes bit 0/value 1; engine 1 contributes bit 1/value 2. [5,0] to 012 (engine 0 only), [0,8] to 102 (engine 1 only), [5,8] to 112 (both). Each bundle is its own six-bit component mask; this two-bit mask only records which engines have work.

1134 Test `not mask`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1136 Skip the remainder of this loop iteration and begin the next iteration.

1138 Set `crew` by call min with the shown positional and keyword arguments. Select one available member with the fastest known individual rate; this rule baseline keeps one member per new job.

1140 Set `action[aircraft]` by construct the shown list in its stated order. It holds the sampled or stored action whose probability is evaluated. Assign this specific member by bitmask and reserve the chosen workshop in the proposed joint action.

1142 Bitwise-AND with `~(1 << crew)` in `free_mask`, and store the result back in that same target. The change is visible to subsequent statements in this block.

1144 Return `action` to the caller by evaluate action. No further statements in this call execute.

9.53. Module declarations and integration

Source lines 1147–1157.

```
1147  # ====================================================================
1148  # fleet_maintenance/adapters.py
1149  # ====================================================================
1153  import numpy as np
1155  import gymnasium as gym
1157  from gymnasium import spaces
```

Line-by-line explanation.

1147 A visual section separator in the source. It has no executable effect.

1148 Source comment: fleet_maintenance/adapters.py This line documents the following code and does not execute.

1149 A visual section separator in the source. It has no executable effect.

1153 Import `import numpy as np`. The imported names are used by the definitions below; importing does not start a training run.

1155 Import `import gymnasium as gym`. The imported names are used by the definitions below; importing does not start a training run.

1157 Import `from gymnasium import spaces`. Gymnasium supplies the environment/wrapper contracts.

9.54. FlattenFleetAction

A reversible action-shape adapter. It exposes the fleet matrix as a single MultiDiscrete vector for library interfaces; it is not a separate control algorithm.

Source lines 1160–1162.

```
1160  class FlattenFleetAction(gym.ActionWrapper):
1162      """Stable-Baselines3 expects a one-dimensional MultiDiscrete nvec."""
```

Line-by-line explanation.

1160 Define `FlattenFleetAction` with the stated inheritance. A reversible action-shape adapter. It exposes the fleet matrix as a single MultiDiscrete vector for library interfaces; it is not a separate control algorithm.

1162 Documentation text: Stable-Baselines3 expects a one-dimensional MultiDiscrete nvec.. This states the contract or qualification for `FlattenFleetAction`; it does not update the simulator or model.

9.55. FlattenFleetAction.__init__

Initialize the object, register its layers or allocate its state. Construction does not itself train a policy.

Source lines 1165–1171.

```
1165    def __init__(self, env):
1167        super().__init__(env)
1169        self.action_shape = env.action_space.nvec.shape
1171        self.action_space = spaces.MultiDiscrete(env.action_space.nvec.ravel())
```

Line-by-line explanation.

`1165` Define `__init__(self, env)`. This ___init___ method is part of FlattenFleetAction.___init___; its parameters and defaults are the interface used by the calls in this section.

`1167` Call super().___init___ with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

`1169` Set `self.action_shape` by evaluate env.action_space.nvec.shape.

`1171` Set `self.action_space` by call spaces.MultiDiscrete with the shown positional and keyword arguments.

9.56. FlattenFleetAction.action

Reshape a flattened library action into the simulator's aircraft-by-field matrix.

Source lines 1174–1176.

```
1174    def action(self, action):
1176        return np.asarray(action, dtype=np.int64).reshape(self.action_shape)
```

Line-by-line explanation.

`1174` Define `action(self, action)`. Reshape a flattened library action into the simulator's aircraft-by-field matrix.

`1176` Return `np.asarray(action, dtype=np.int64).reshape(self.action_shape)` to the caller by reinterpret the shape while retaining element order. No further statements in this call execute.

9.57. FlattenFleetAction.reverse_action

Flatten a simulator fleet action back into the wrapper's one-dimensional representation.

Source lines 1179–1181.

```
1179    def reverse_action(self, action):
1181        return np.asarray(action, dtype=np.int64).ravel()
```

Line-by-line explanation.

`1179` Define `reverse_action(self, action)`. Flatten a simulator fleet action back into the wrapper's one-dimensional representation.

`1181` Return `np.asarray(action, dtype=np.int64).ravel()` to the caller by call np.asarray(action, dtype=np.int64).ravel with the shown positional and keyword arguments. No further statements in this call execute.

9.58. Module declarations and integration

Source lines 1184–1196.

```
1184  # ======================================================================
1185  # fleet_maintenance/policies.py
1186  # ======================================================================
1190  import numpy as np
1192  import torch
1194  from torch import nn
1196  from torch.distributions import Categorical
```

Line-by-line explanation.

`1184` A visual section separator in the source. It has no executable effect.

`1185` Source comment: fleet_maintenance/policies.py This line documents the following code and does not execute.

`1186` A visual section separator in the source. It has no executable effect.

`1190` Import `import numpy as np`. The imported names are used by the definitions below; importing does not start a training run.

`1192` Import `import torch`. Torch supplies tensors, automatic differentiation and model persistence.

`1194` Import `from torch import nn`. Torch supplies tensors, automatic differentiation and model persistence.

`1196` Import `from torch.distributions import Categorical`. Torch supplies tensors, automatic differentiation and model persistence.

9.59. FleetActorCritic

The full-action policy family. One actor is reused across aircraft; optional global encoding and previous-field conditioning distinguish flat, conditional and shared modes. A scalar global critic and a fleet-joint likelihood support the original PPO trainer.

Source lines 1199–1210.

```
1199  class FleetActorCritic(nn.Module):
1201      """Central critic + flat, conditional, or parameter-shared aircraft actors.
1203      shared: local aircraft estimates + public resource counts. Parameters reused
1204      across aircraft; a central deterministic resolver enforces resource capacity.
1205      conditional: global actor context and masked dependent action distributions.
1206      flat: shared global features with no learned conditioning on preceding choices.
1207      All methods use the same logical masks; scarce-resource conflicts are resolved
1208      by the simulator and reported separately from structural feasibility.
1209      All act daily and obey pre-start commitment; this is not temporal HRL.
1210      """
```

Line-by-line explanation.

`1199` Define `FleetActorCritic` with the stated inheritance. The full-action policy family. One actor is reused across aircraft; optional global encoding and previous-field conditioning distinguish flat, conditional and shared modes. A scalar global critic and a fleet-joint likelihood support the original PPO trainer.

`1201` Documentation text: Central critic + flat, conditional, or parameter-shared aircraft actors.. This states the contract or qualification for `FleetActorCritic`; it does not update the simulator or model.

1203 Documentation text: shared: local aircraft estimates + public resource counts. Parameters reused. This states the contract or qualification for `FleetActorCritic`; it does not update the simulator or model.

1204 Documentation text: across aircraft; a central deterministic resolver enforces resource capacity.. This states the contract or qualification for `FleetActorCritic`; it does not update the simulator or model.

1205 Documentation text: conditional: global actor context and masked dependent action distributions.. This states the contract or qualification for `FleetActorCritic`; it does not update the simulator or model.

1206 Documentation text: flat: shared global features with no learned conditioning on preceding choices.. This states the contract or qualification for `FleetActorCritic`; it does not update the simulator or model.

1207 Documentation text: All methods use the same logical masks; scarce-resource conflicts are resolved. This states the contract or qualification for `FleetActorCritic`; it does not update the simulator or model.

1208 Documentation text: by the simulator and reported separately from structural feasibility.. This states the contract or qualification for `FleetActorCritic`; it does not update the simulator or model.

1209 Documentation text: All act daily and obey pre-start commitment; this is not temporal HRL.. This states the contract or qualification for `FleetActorCritic`; it does not update the simulator or model.

1210 Close the documentation string for `FleetActorCritic`. This delimiter ends the descriptive text and performs no simulator or learning update.

9.60. FleetActorCritic.___init___

Initialize the object, register its layers or allocate its state. Construction does not itself train a policy.

Source lines 1213–1247.

```
1213    def __init__(self, env: FleetMaintenanceEnv, mode='conditional', goal_size=0):
1215        super().__init__()
1217        if mode not in ('flat', 'conditional', 'shared'):
1219            raise ValueError(mode)
1221        self.mode = mode
1223        self.n = env.config.aircraft
1225        self.local = env.local_size
1227        self.public = env.public_size
1229        self.dims = env.row_dims.tolist()
1231        self.mandatory = env.config.mandatory_health
1233        self.goal_size = goal_size
1235        size = env.observation_space.shape[0]
1237        self.global_encoder = nn.Sequential(nn.Linear(size, 64), nn.Tanh())
1239        in_size = self.local + self.public + (0 if mode == 'shared' else 64) + goal_size
1241        self.actor = nn.Sequential(nn.Linear(in_size, 64), nn.Tanh(), nn.Linear(64, 64), nn.Tanh())
1243        self.heads = nn.ModuleList((nn.Linear(64, d) for d in self.dims))
1245        self.condition = nn.ModuleList((nn.Linear(8, 64, bias=False) for _ in self.dims))
1247        self.critic = nn.Sequential(nn.Linear(size, 64), nn.Tanh(), nn.Linear(64, 64), nn.Tanh(),
        ↪  nn.Linear(64, 1))
```

Line-by-line explanation.

1213 Define `__init__(self, env: FleetMaintenanceEnv, mode='conditional', goal_size=0)`. This ___init___ method is part of FleetActorCritic.___init___; its parameters and defaults are the interface used by the calls in this section.

1215 Call super().___init___ with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

1217 Test `mode not in ('flat', 'conditional', 'shared')`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1219 Raise `ValueError(mode)` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1221 Set `self.mode` by evaluate mode.

1223 Set `self.n` by evaluate env.config.aircraft.

1225 Set `self.local` by evaluate env.local_size. Compute each aircraft's own observation slice and store it in self.local; later checks/updates use this named intermediate.

1227 Set `self.public` by evaluate env.public_size. Compute fleet-wide observation values shared by aircraft and store it in self.public; later checks/updates use this named intermediate.

1229 Set `self.dims` by call env.row_dims.tolist with the shown positional and keyword arguments.

1231 Set `self.mandatory` by evaluate env.config.mandatory_health. Compute whether a lower-health safety alarm is active and store it in self.mandatory; later checks/updates use this named intermediate.

1233 Set `self.goal_size` by evaluate goal_size.

1235 Set `size` by select the indexed/sliced entries of env.observation_space.shape. Compute and bind the right-hand value to size, naming an intermediate for the surrounding algorithm.

1237 Set `self.global_encoder` by compose the listed layers in execution order. Build a shared 64-unit encoder for the full fleet observation. It converts the global state into a compact context vector; Tanh bounds each encoded feature.

1239 Set `in_size` by add self.local + self.public + (0 if mode == 'shared' else 64) and goal_size. Calculate the per-aircraft actor input width: local row + broadcast public fleet data + optional 64-feature global context (omitted in shared mode) + optional worker goal.

1241 Set `self.actor` by compose the listed layers in execution order. Build the actor's two-layer feature network. It maps each aircraft's assembled inputs to a 64-number representation used by every action-field head.

1243 Set `self.heads` by register the listed independent networks so parameters and checkpoints include them. Create one output head per action field; each emits as many logits as that field has categories. ModuleList registers all heads so PyTorch trains/saves their parameters.

1245 Set `self.condition` by register the listed independent networks so parameters and checkpoints include them. Create one conditioning projection per action field. It maps the eight previously selected field values into the current 64-feature state; bias=False keeps this as a pure learned interaction.

1247 Set `self.critic` by compose the listed layers in execution order. Build the critic on the complete fleet observation; the final single output is V(s), the estimated return from this state.

9.61. FleetActorCritic.value

Evaluate the centralized state-value or action-value path using the input contract of this class.

Source lines 1250–1252.

```
1250     def value(self, obs):
1252         return self.critic(obs).squeeze(-1)
```

Line-by-line explanation.

1250 Define `value(self, obs)`. This value method is part of FleetActorCritic.value; its parameters and defaults are the interface used by the calls in this section.

1252 Return `self.critic(obs).squeeze(-1)` to the caller by call self.critic(obs).squeeze with the shown positional and keyword arguments. No further statements in this call execute.

9.62. FleetActorCritic.act

Sample or reconstruct actions and compute their log probabilities, entropy and values as defined by this policy family.

Source lines 1255–1289.

```
1255     def act(self, obs, actions=None, deterministic=False, goal=None):
1257         if obs.ndim == 1:
1259             obs = obs.unsqueeze(0)
1261         batch = obs.shape[0]
1263         local = obs[:, :-self.public].reshape(batch, self.n, self.local)
1265         public = obs[:, -self.public:].unsqueeze(1).expand(-1, self.n, -1)
1267         features = [local, public]
1269         if self.goal_size:
1271             if goal is None:
1273                 raise ValueError('goal-conditioned worker requires a goal')
1275             features.append(goal.unsqueeze(1).expand(-1, self.n, -1))
1277         if self.mode != 'shared':
1279             features.append(self.global_encoder(obs).unsqueeze(1).expand(-1, self.n, -1))
1281         h = self.actor(torch.cat(features, -1))
1283         previous = torch.zeros(batch, self.n, 8, device=obs.device)
1285         logprob = torch.zeros(batch, device=obs.device)
1287         entropy = torch.zeros_like(logprob)
1289         result = []
```

Line-by-line explanation.

1255 Define `act(self, obs, actions=None, deterministic=False, goal=None)`. This act method is part of FleetActorCritic.act; its parameters and defaults are the interface used by the calls in this section.

1257 Test `obs.ndim == 1`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

`1259` Set `obs` by call obs.unsqueeze with the shown positional and keyword arguments. It holds the observed belief vector, rather than hidden physical health. Compute the current encoded state and store it in obs; later checks/updates use this named intermediate.

`1261` Set `batch` by select the indexed/sliced entries of obs.shape. Compute and bind the right-hand value to batch, naming an intermediate for the surrounding algorithm.

`1263` Set `local` by reinterpret the shape while retaining element order. Compute each aircraft's own observation slice and store it in local; later checks/updates use this named intermediate.

`1265` Set `public` by call obs[:, -self.public:].unsqueeze(1).expand with the shown positional and keyword arguments. Compute fleet-wide observation values shared by aircraft and store it in public; later checks/updates use this named intermediate.

`1267` Set `features` by construct the shown list in its stated order. Start each aircraft's actor input with its own local row and the shared public-resource vector; both are needed to choose a feasible plan.

`1269` Test `self.goal_size`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

`1271` Test `goal is None`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

`1273` Raise `ValueError('goal-conditioned worker requires a goal')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

`1275` Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

`1277` Test `self.mode != 'shared'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

`1279` Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

`1281` Set `h` by call self.actor with the shown positional and keyword arguments. Concatenate each aircraft's local/public/context/goal features and pass them through the actor MLP to get per-aircraft hidden features.

`1283` Set `previous` by call torch.zeros with the shown positional and keyword arguments. Initialize the eight action-field conditioning slots to zero; as fields are chosen, their normalized values are filled in for later heads.

`1285` Set `logprob` by call torch.zeros with the shown positional and keyword arguments. Start one joint log-probability accumulator per observation in the batch; each chosen field contributes to it.

`1287` Set `entropy` by call torch.zeros_like with the shown positional and keyword arguments. Start an entropy accumulator with the same batch shape/device as logprob.

`1289` Set `result` by construct the shown list in its stated order. Compute and bind the right-hand value to result, naming an intermediate for the surrounding algorithm.

Source lines 1291–1325.

```
        busy = local[:, :, 37] > 0.5
        if self.goal_size:
            busy = busy | (goal[:, 0] > 0.5).unsqueeze(-1)
        urgent = local[:, :, 24:36].amin(-1) <= self.mandatory
        for k, head in enumerate(self.heads):
            logits = head(h if self.mode == 'flat' else torch.tanh(h + self.condition[k](previous)))
            valid = torch.ones_like(logits, dtype=torch.bool)
            valid[busy] = False
            valid[:, :, 0] |= busy
            if k == 0:
                valid[:, :, 0] &= ~(urgent & ~busy)
            else:
                ground = result[0] == 1
                if k == 1:
                    valid[:, :, 0] &= ~ground
                if k in (2, 3):
                    engine_on = result[1] & 1 << k - 2 != 0
                    active = ground & engine_on & ~busy
```

Line-by-line explanation.

1291 Set `busy` by evaluate the comparison local[:, :, 37] > 0.5. Read the busy indicator from local observation column 37 for every batch item and aircraft; busy aircraft already have a committed plan.

1293 Test `self.goal_size`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1295 Set `busy` by apply a bitwise union to busy and (goal[:, 0] > 0.5).unsqueeze(-1). Compute whether an aircraft already has a committed job and store it in busy; later checks/updates use this named intermediate.

1297 Set `urgent` by evaluate the comparison local[:, :, 24:36].amin(-1) <= self.mandatory. Take the minimum of the 12 per-component lower-health estimates in columns 24–35; mark the aircraft urgent if any estimate crosses the mandatory threshold.

1299 Iterate `(k, head)` over `enumerate(self.heads)`. The following indented statements run once for each selected element.

1301 Set `logits` by call head with the shown positional and keyword arguments. Produce this field's category scores. Flat mode uses h directly; conditional/shared modes add a learned projection of earlier choices and pass the sum through Tanh.

1303 Set `valid` by call torch.ones_like with the shown positional and keyword arguments. It holds the categorical structural-validity mask. Initially allow every category for every aircraft; subsequent mask rules remove choices that violate action dependencies or safety constraints.

1305 Set `valid[busy]` by use the literal False. It holds the categorical structural-validity mask. Temporarily invalidate every category for busy aircraft; their committed job must not be overwritten by a new plan.

1307 Bitwise-OR in `busy` in `valid[:, :, 0]`, and store the result back in that same target. This updates the categorical structural-validity mask.

1309 Test `k == 0`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1311 Bitwise-AND with `~(urgent & ~busy)` in `valid[:, :, 0]`, and store the result back in that same target. This updates the categorical structural-validity mask.

1313 Execute this alternative only when the corresponding preceding condition did not select its branch.

1315 Set `ground` by evaluate the comparison result[0] == 1. Read the already-selected first field and convert it to a Boolean: this aircraft was chosen for grounding/maintenance.

1317 Test `k == 1`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1319 Bitwise-AND with `~ground` in `valid[:, :, 0]`, and store the result back in that same target. This updates the categorical structural-validity mask.

1321 Test `k in (2, 3)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1323 Set `engine_on` by evaluate the comparison result[1] & 1 « k - 2 != 0. For bundle heads k=2/3, select bit 0/1 from the chosen engine mask. Shift 1 by k-2, AND it with result[1], then compare with zero. Python evaluates shifts before &, so this is result[1] & (1 « (k-2)) != 0.

1325 Set `active` by apply a bitwise intersection to ground & engine_on and ~busy. Compute whether a selected engine may receive a new component bundle and store it in active; later checks/updates use this named intermediate.

Source lines 1327–1359.

```
                    valid[:, :, 0] &= ~active
                    valid[:, :, 1:] &= active.unsqueeze(-1)
                elif k >= 4:
                    valid[:, :, 1:] &= (ground & ~busy).unsqueeze(-1)
                    if k == 4:
                        valid[:, :, 0] &= ~(ground & ~busy)
                if k == 1:
                    valid[:, :, 1:] &= (ground & ~busy).unsqueeze(-1)
            dist = Categorical(logits=logits.masked_fill(~valid, -1000000000.0))
            choice = actions[:, :, k] if actions is not None else dist.probs.argmax(-1) if
            ↪  deterministic else dist.sample()
            result.append(choice)
            logprob += dist.log_prob(choice).sum(-1)
            entropy += dist.entropy().sum(-1)
            addition = torch.zeros_like(previous)
            addition[:, :, k] = choice / float(max(1, self.dims[k] - 1))
            previous = previous + addition
        return (torch.stack(result, -1), logprob, entropy, self.value(obs))
```

Line-by-line explanation.

1327 Bitwise-AND with `~active` in `valid[:, :, 0]`, and store the result back in that same target. This updates the categorical structural-validity mask.

1329 Bitwise-AND with `active.unsqueeze(-1)` in `valid[:, :, 1:]`, and store the result back in that same target. This updates the categorical structural-validity mask.

1331 Test `k >= 4`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1333 Bitwise-AND with `(ground & ~busy).unsqueeze(-1)` in `valid[:, :, 1:]`, and store the result back in that same target. This updates the categorical structural-validity mask.

1335 Test `k == 4`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1337 Bitwise-AND with `~(ground & ~busy)` in `valid[:, :, 0]`, and store the result back in that same target. This updates the categorical structural-validity mask.

1339 Test `k == 1`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1341 Bitwise-AND with `(ground & ~busy).unsqueeze(-1)` in `valid[:, :, 1:]`, and store the result back in that same target. This updates the categorical structural-validity mask.

1343 Set `dist` by construct a discrete distribution from the supplied logits/probabilities. Give invalid choices an extremely negative logit, making their softmax probability effectively zero before constructing the categorical distribution.

1345 Set `choice` by select actions[:, :, k] when actions is not None, otherwise dist.probs.argmax(-1) if deterministic else dist.sample(). When training, reuse the stored action to recompute its likelihood. Otherwise take argmax for deterministic evaluation or sample a categorical choice for exploration.

1347 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

1349 Add `dist.log_prob(choice).sum(-1)` to `logprob`, and store the result back in that same target. The change is visible to subsequent statements in this block.

1351 Add `dist.entropy().sum(-1)` to `entropy`, and store the result back in that same target. The change is visible to subsequent statements in this block.

1353 Set `addition` by call torch.zeros_like with the shown positional and keyword arguments. Create a zero tensor shaped like the prior-field buffer so only this action coordinate will be written before accumulation.

1355 Set `addition[:, :, k]` by divide choice and float(max(1, self.dims[k] - 1)). Store this selected category in slot k, scaled into roughly 0–1; normalization lets later conditioning layers consume fields with different category counts.

1357 Set `previous` by add previous and addition. Add the newly selected normalized field to the conditioning buffer without changing earlier field choices.

1359 Return `(torch.stack(result, -1), logprob, entropy, self.value(obs))` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

9.63. OptionActorCritic

The original fleet-level learned manager. Its four actions invoke fixed threshold routines; the manager chooses an option rather than individual replacement/resource fields.

Source lines 1362–1364.

```
1362 class OptionActorCritic(nn.Module):
1364     """Learned manager selects one of four fixed fleet planning options."""
```

Line-by-line explanation.

1362 Define `OptionActorCritic` with the stated inheritance. The original fleet-level learned manager. Its four actions invoke fixed threshold routines; the manager chooses an option rather than individual replacement/resource fields.

1364 Documentation text: Learned manager selects one of four fixed fleet planning options.. This states the contract or qualification for `OptionActorCritic`; it does not update the simulator or model.

9.64. OptionActorCritic.___init___

Initialize the object, register its layers or allocate its state. Construction does not itself train a policy.

Source lines 1367–1377.

```
1367    def __init__(self, env):
1369        super().__init__()
1371        size = env.observation_space.shape[0]
1373        self.encoder = nn.Sequential(nn.Linear(size, 64), nn.Tanh(), nn.Linear(64, 64), nn.Tanh())
1375        self.actor = nn.Linear(64, 4)
1377        self.critic = nn.Linear(64, 1)
```

Line-by-line explanation.

1367 Define `__init__(self, env)`. This ___init___ method is part of OptionActorCritic.___init___; its parameters and defaults are the interface used by the calls in this section.

1369 Call super().___init___ with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

1371 Set `size` by select the indexed/sliced entries of env.observation_space.shape. Compute and bind the right-hand value to size, naming an intermediate for the surrounding algorithm.

1373 Set `self.encoder` by compose the listed layers in execution order. Encode the whole fleet observation into a 64-feature manager representation using two Linear/Tanh stages.

1375 Set `self.actor` by create a trainable affine layer with the shown input/output widths. Map the manager representation to four option logits, one for each high-level option ID.

1377 Set `self.critic` by create a trainable affine layer with the shown input/output widths. Map the same manager representation to one scalar state-value estimate.

9.65. OptionActorCritic.value

Evaluate the centralized state-value or action-value path using the input contract of this class.

Source lines 1380–1382.

```
1380    def value(self, obs):
1382        return self.critic(self.encoder(obs)).squeeze(-1)
```

Line-by-line explanation.

1380 Define `value(self, obs)`. This value method is part of OptionActorCritic.value; its parameters and defaults are the interface used by the calls in this section.

1382 Return `self.critic(self.encoder(obs)).squeeze(-1)` to the caller by call self.critic(self.encoder(obs)).squeeze with the shown positional and keyword arguments. No further statements in this call execute.

9.66. OptionActorCritic.act

Sample or reconstruct actions and compute their log probabilities, entropy and values as defined by this policy family.

Source lines 1385–1397.

```
1385    def act(self, obs, actions=None, deterministic=False):
1387        if obs.ndim == 1:
1389            obs = obs.unsqueeze(0)
1391        h = self.encoder(obs)
1393        dist = Categorical(logits=self.actor(h))
1395        action = actions if actions is not None else dist.probs.argmax(-1) if deterministic else
            ↪  dist.sample()
1397        return (action, dist.log_prob(action), dist.entropy(), self.critic(h).squeeze(-1))
```

Line-by-line explanation.

1385 Define `act(self, obs, actions=None, deterministic=False)`. This act method is part of OptionActorCritic.act; its parameters and defaults are the interface used by the calls in this section.

1387 Test `obs.ndim == 1`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1389 Set `obs` by call obs.unsqueeze with the shown positional and keyword arguments. It holds the observed belief vector, rather than hidden physical health. Compute the current encoded state and store it in obs; later checks/updates use this named intermediate.

1391 Set `h` by call self.encoder with the shown positional and keyword arguments. Compute the manager's learned feature representation of the current fleet state.

1393 Set `dist` by construct a discrete distribution from the supplied logits/probabilities. Convert the four manager logits into a categorical option distribution; softmax probabilities are implicit.

1395 Set `action` by select actions when actions is not None, otherwise dist.probs.argmax(-1) if deterministic else dist.sample(). It holds the sampled or stored action whose probability is evaluated. Compute the proposed or selected action and store it in action; later checks/updates use this named intermediate.

1397 Return `(action, dist.log_prob(action), dist.entropy(), self.critic(h).squeeze(-1))` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

9.67. LearnedOptionActorCritic

The original learned fleet manager plus option-conditioned full-action worker. A sampled goal is passed to the worker and their log probabilities are added. The manager value supplies the training baseline.

Source lines 1400–1408.

```
class LearnedOptionActorCritic(nn.Module):
    """Jointly trained manager + goal-conditioned plan worker + fixed executor.
    Macro 0 waits; macros 1..3 are latent planning contexts, not certified skills.
    The worker selects engines, components and resources once before execution.
    Both levels learn from fleet return. This is not MAXQ or option-critic and
    option collapse/credit assignment require later ablations and diagnostics.
    """
```

Line-by-line explanation.

1400 Define `LearnedOptionActorCritic` with the stated inheritance. The original learned fleet manager plus option-conditioned full-action worker. A sampled goal is passed to the worker and their log probabilities are added. The manager value supplies the training baseline.

1402 Documentation text: Jointly trained manager + goal-conditioned plan worker + fixed executor.. This states the contract or qualification for `LearnedOptionActorCritic`; it does not update the simulator or model.

1404 Documentation text: Macro 0 waits; macros 1..3 are latent planning contexts, not certified skills.. This states the contract or qualification for `LearnedOptionActorCritic`; it does not update the simulator or model.

1405 Documentation text: The worker selects engines, components and resources once before execution.. This states the contract or qualification for `LearnedOptionActorCritic`; it does not update the simulator or model.

1406 Documentation text: Both levels learn from fleet return. This is not MAXQ or option-critic and. This states the contract or qualification for `LearnedOptionActorCritic`; it does not update the simulator or model.

1407 Documentation text: option collapse/credit assignment require later ablations and diagnostics.. This states the contract or qualification for `LearnedOptionActorCritic`; it does not update the simulator or model.

1408 Close the documentation string for `LearnedOptionActorCritic`. This delimiter ends the descriptive text and performs no simulator or learning update.

9.68. LearnedOptionActorCritic.___init___

Initialize the object, register its layers or allocate its state. Construction does not itself train a policy.

Source lines 1411–1419.

```
    def __init__(self, env):
        super().__init__()
        self.manager = OptionActorCritic(env)
        self.worker = FleetActorCritic(env, 'conditional', goal_size=4)
        self.n = env.config.aircraft
```

Line-by-line explanation.

1411 Define `__init__(self, env)`. This ___init___ method is part of LearnedOptionActorCritic.___init___; its parameters and defaults are the interface used by the calls in this section.

1413 Call super().__init__ with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

1415 Set `self.manager` by call OptionActorCritic with the shown positional and keyword arguments.

1417 Set `self.worker` by call FleetActorCritic with the shown positional and keyword arguments.

1419 Set `self.n` by evaluate env.config.aircraft.

9.69. LearnedOptionActorCritic.value

Evaluate the centralized state-value or action-value path using the input contract of this class.

Source lines 1422–1424.

```
1422     def value(self, obs):
1424         return self.manager.value(obs)
```

Line-by-line explanation.

1422 Define `value(self, obs)`. This value method is part of LearnedOptionActorCritic.value; its parameters and defaults are the interface used by the calls in this section.

1424 Return `self.manager.value(obs)` to the caller by call self.manager.value with the shown positional and keyword arguments. No further statements in this call execute.

9.70. LearnedOptionActorCritic.act

Sample or reconstruct actions and compute their log probabilities, entropy and values as defined by this policy family.

Source lines 1427–1445.

```
1427     def act(self, obs, actions=None, deterministic=False):
1429         if obs.ndim == 1:
1431             obs = obs.unsqueeze(0)
1433         macro = None if actions is None else actions[:, 0]
1435         plan = None if actions is None else actions[:, 1:].reshape(-1, self.n, 8)
1437         option, lp, entropy, value = self.manager.act(obs, macro, deterministic)
1439         goal = torch.nn.functional.one_hot(option, num_classes=4).float()
1441         plan, worker_lp, worker_entropy, _ = self.worker.act(obs, plan, deterministic, goal)
1443         composite = torch.cat([option.unsqueeze(-1), plan.flatten(1)], -1)
1445         return (composite, lp + worker_lp, entropy + worker_entropy, value)
```

Line-by-line explanation.

1427 Define `act(self, obs, actions=None, deterministic=False)`. This act method is part of LearnedOptionActorCritic.act; its parameters and defaults are the interface used by the calls in this section.

1429 Test `obs.ndim == 1`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1431 Set `obs` by call obs.unsqueeze with the shown positional and keyword arguments. It holds the observed belief vector, rather than hidden physical health. Compute the current encoded state and store it in obs; later checks/updates use this named intermediate.

1433 Set `macro` by select None when actions is None, otherwise actions[:, 0]. Compute the manager's high-level option ID and store it in macro; later checks/updates use this named intermediate.

1435 Set `plan` by select None when actions is None, otherwise actions[:, 1:].reshape(-1, self.n, 8). Compute the proposed or accepted maintenance plan and store it in plan; later checks/updates use this named intermediate.

1437 Set `(option, lp, entropy, value)` by call self.manager.act with the shown positional and keyword arguments. Unpack the policy result into its selected action, likelihood, entropy, and value outputs; underscore positions deliberately discard unused statistics.

1439 Set `goal` by call torch.nn.functional.one_hot(option, num_classes=4).float with the shown positional and keyword arguments. Represent each selected option ID as a four-element one-hot goal, which becomes the worker's conditioning input.

1441 Set `(plan, worker_lp, worker_entropy, _)` by call self.worker.act with the shown positional and keyword arguments. Unpack the policy result into its selected action, likelihood, entropy, and value outputs; underscore positions deliberately discard unused statistics.

1443 Set `composite` by concatenate features along the specified existing axis. Flatten each worker plan to one row, then prepend its manager option ID so the stored composite action can later be split and rescored.

1445 Return `(composite, lp + worker_lp, entropy + worker_entropy, value)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

9.71. Module declarations and integration

Source lines 1446–1447.

```
1446  # Multi-agent maintenance: explicit proposal ownership and a common resource broker.
1447  MAPPO_METHODS = ('mappo_aircraft', 'mappo_component')
```

Line-by-line explanation.

1446 Source comment: Multi-agent maintenance: explicit proposal ownership and a common resource broker. This line documents the following code and does not execute.

1447 Set `MAPPO_METHODS` by construct the shown tuple in its stated order.

9.72. mappo_agent_inputs

Construct exactly the actor information views used by the two MAPPO assignments. Aircraft views contain one local row plus public state; component views collect their component-specific features across the fleet.

Source lines 1451–1481.

```
1451 def mappo_agent_inputs(observation, aircraft, local_size, public_size, assignment):
1453     """Build restricted actor inputs from observed beliefs, never hidden health."""
1455     obs = torch.as_tensor(observation, dtype=torch.float32)
1457     if obs.ndim == 1:
1459         obs = obs.unsqueeze(0)
1461     if obs.ndim != 2 or obs.shape[1] != aircraft * local_size + public_size:
1463         raise ValueError('incorrect fleet observation shape')
1465     local = obs[:, :aircraft * local_size].reshape(-1, aircraft, local_size)
1467     public = obs[:, -public_size:]
1469     if assignment == 'aircraft':
1470         # Each aircraft sees only its own row plus the common resource board.
1471         return torch.cat([local, public.unsqueeze(1).expand(-1, aircraft, -1)], dim=-1)
1473     if assignment != 'component':
1475         raise ValueError('assignment must be aircraft or component')
1476     # Ten own-component values: two engines x mean/std/lower/efficiency/temperature.
1477     indices = [[c, c + 6, c + 12, c + 18, c + 24, c + 30,
1479                 c + 45, c + 51, c + 57, c + 63, 36, 37, 38, 69] for c in range(6)]
1481     own = torch.stack([local[:, :, ix] for ix in indices], dim=1)
```

Line-by-line explanation.

1451 Define `mappo_agent_inputs(observation, aircraft, local_size, public_size, assignment)`. Construct exactly the actor information views used by the two MAPPO assignments. Aircraft views contain one local row plus public state; component views collect their component-specific features across the fleet.

1453 Documentation text: Build restricted actor inputs from observed beliefs, never hidden health.. This states the contract or qualification for `mappo_agent_inputs`; it does not update the simulator or model.

1455 Set `obs` by convert inputs to a tensor with the requested dtype. It holds the observed belief vector, rather than hidden physical health. Execute this step of the multi-agent observation, proposal, or learning calculation.

1457 Test `obs.ndim == 1`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1459 Set `obs` by call obs.unsqueeze with the shown positional and keyword arguments. It holds the observed belief vector, rather than hidden physical health. Execute this step of the multi-agent observation, proposal, or learning calculation.

1461 Test `obs.ndim != 2 or obs.shape[1] != aircraft * local_size + public_size`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1463 Raise `ValueError('incorrect fleet observation shape')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1465 Set `local` by reinterpret the shape while retaining element order. Execute this step of the multi-agent observation, proposal, or learning calculation.

1467 Set `public` by select the indexed/sliced entries of obs. Execute this step of the multi-agent observation, proposal, or learning calculation.

1469 Test `assignment == 'aircraft'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1470 Source comment: Each aircraft sees only its own row plus the common resource board. This line documents the following code and does not execute.

1471 Return `torch.cat([local, public.unsqueeze(1).expand(-1, aircraft, -1)], dim=-1)` to the caller by concatenate features along the specified existing axis. No further statements in this call execute.

1473 Test `assignment != 'component'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1475 Raise `ValueError('assignment must be aircraft or component')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1476 Source comment: Ten own-component values: two engines x mean/std/lower/efficiency/temperature. This line documents the following code and does not execute.

1477 Set `indices` by construct the collection by iterating the shown comprehension and applying its filters.

1479 Continue the assignment begun at line 1477 with `c + 45, c + 51, c + 57, c + 63, 36, 37, 38, 69] for c in range(6)]`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

1481 Set `own` by join equal-shaped tensors along the specified new axis. Execute this step of the multi-agent observation, proposal, or learning calculation.

Source lines 1482–1489.

```
1482     # A broadcast safety-criticality report is an explicit cross-component message.
1483     criticality = local[:, :, 24:36].amin(-1)
1485     board = criticality[:, None, :, None].expand(-1, 6, -1, -1)
1487     features = torch.cat([own, board], dim=-1).flatten(2)
1489     return torch.cat([features, public.unsqueeze(1).expand(-1, 6, -1)], dim=-1)
```

Line-by-line explanation.

1482 Source comment: A broadcast safety-criticality report is an explicit cross-component message. This line documents the following code and does not execute.

1483 Set `criticality` by call local[:, :, 24:36].amin with the shown positional and keyword arguments.

1485 Set `board` by call criticality[:, None, :, None].expand with the shown positional and keyword arguments. Execute this step of the multi-agent observation, proposal, or learning calculation.

1487 Set `features` by call torch.cat([own, board], dim=-1).flatten with the shown positional and keyword arguments. Execute this step of the multi-agent observation, proposal, or learning calculation.

1489 Return `torch.cat([features, public.unsqueeze(1).expand(-1, 6, -1)], dim=-1)` to the caller by concatenate features along the specified existing axis. No further statements in this call execute.

9.73. MultiAgentFleetEnv

A request adapter with aircraft or component-type ownership. It converts agent proposals into full fleet actions through a deterministic stock/resource broker. Shared team reward does not remove the broker or create learned communication.

Source lines 1493–1504.

```
class MultiAgentFleetEnv:
    """Cooperative dictionary API around the same physical fleet simulator.
    Aircraft agents propose two six-bit bundles; six component agents each
    propose a two-engine mask for every aircraft. Both cover the same twelve
    replacement bits per aircraft. They do not learn crew/shop/order choices.
    A public request board and deterministic centralized dispatcher coordinate
    execution. This is decentralized proposal generation with centralized
    arbitration, not fully decentralized resource allocation or learned messaging.
    This small adapter is not a PettingZoo/RLlib API implementation.
    """
```

Line-by-line explanation.

1493 Define `MultiAgentFleetEnv` with the stated inheritance. A request adapter with aircraft or component-type ownership. It converts agent proposals into full fleet actions through a deterministic stock/resource broker. Shared team reward does not remove the broker or create learned communication.

1495 Documentation text: Cooperative dictionary API around the same physical fleet simulator.. This states the contract or qualification for `MultiAgentFleetEnv`; it does not update the simulator or model.

1497 Documentation text: Aircraft agents propose two six-bit bundles; six component agents each. This states the contract or qualification for `MultiAgentFleetEnv`; it does not update the simulator or model.

1498 Documentation text: propose a two-engine mask for every aircraft. Both cover the same twelve. This states the contract or qualification for `MultiAgentFleetEnv`; it does not update the simulator or model.

1499 Documentation text: replacement bits per aircraft. They do not learn crew/shop/order choices.. This states the contract or qualification for `MultiAgentFleetEnv`; it does not update the simulator or model.

1500 Documentation text: A public request board and deterministic centralized dispatcher coordinate. This states the contract or qualification for `MultiAgentFleetEnv`; it does not update the simulator or model.

1501 Documentation text: execution. This is decentralized proposal generation with centralized. This states the contract or qualification for `MultiAgentFleetEnv`; it does not update the simulator or model.

1502 Documentation text: arbitration, not fully decentralized resource allocation or learned messaging.. This states the contract or qualification for `MultiAgentFleetEnv`; it does not update the simulator or model.

1503 Documentation text: This small adapter is not a PettingZoo/RLlib API implementation.. This states the contract or qualification for `MultiAgentFleetEnv`; it does not update the simulator or model.

1504 Close the documentation string for `MultiAgentFleetEnv`. This delimiter ends the descriptive text and performs no simulator or learning update.

9.74. MultiAgentFleetEnv.___init___

Initialize the object, register its layers or allocate its state. Construction does not itself train a policy.

Source lines 1507–1531.

```
1507    def __init__(self, config=None, assignment='aircraft'):
1509        if assignment not in ('aircraft', 'component'):
1511            raise ValueError('assignment must be aircraft or component')
1513        self.env = FleetMaintenanceEnv(config)
1515        self.assignment = assignment
1517        self.agent_ids = self.env.aircraft_plates if assignment == 'aircraft' else COMPONENTS
1519        self.num_agents = len(self.agent_ids)
1521        dims = [64, 64] if assignment == 'aircraft' else [4] * self.env.config.aircraft
1523        self.action_spaces = {agent: spaces.MultiDiscrete(dims) for agent in self.agent_ids}
1525        width = self.env.local_size + self.env.public_size if assignment == 'aircraft' else 15 *
        ↪  self.env.config.aircraft + self.env.public_size
1527        self.observation_spaces = {agent: spaces.Box(0, 1, shape=(width,), dtype=np.float32) for agent
        ↪  in self.agent_ids}
1529        self.central_observation = None
1531        self.coordination_deferrals = 0
```

Line-by-line explanation.

1507 Define `__init__(self, config=None, assignment='aircraft')`. This ___init___ method is part of MultiAgentFleetEnv.___init___; its parameters and defaults are the interface used by the calls in this section.

1509 Test `assignment not in ('aircraft', 'component')`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1511 Raise `ValueError('assignment must be aircraft or component')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1513 Set `self.env` by call FleetMaintenanceEnv with the shown positional and keyword arguments. It holds the environment or wrapper that owns physical transitions. Execute this step of the multi-agent observation, proposal, or learning calculation.

1515 Set `self.assignment` by evaluate assignment. Execute this step of the multi-agent observation, proposal, or learning calculation.

1517 Set `self.agent_ids` by select self.env.aircraft_plates when assignment == 'aircraft', otherwise COMPONENTS. Execute this step of the multi-agent observation, proposal, or learning calculation.

1519 Set `self.num_agents` by count entries. Execute this step of the multi-agent observation, proposal, or learning calculation.

1521 Set `dims` by select [64, 64] when assignment == 'aircraft', otherwise [4] * self.env.config.aircraft. Execute this step of the multi-agent observation, proposal, or learning calculation.

1523 Set `self.action_spaces` by construct the collection by iterating the shown comprehension and applying its filters. Execute this step of the multi-agent observation, proposal, or learning calculation.

1525 Set `width` by select self.env.local_size + self.env.public_size when assignment == 'aircraft', otherwise 15 * self.env.config.aircraft + self.env.public_size. Execute this step of the multi-agent observation, proposal, or learning calculation.

1527 Set `self.observation_spaces` by construct the collection by iterating the shown comprehension and applying its filters. Execute this step of the multi-agent observation, proposal, or learning calculation.

1529 Set `self.central_observation` by use the literal None. Execute this step of the multi-agent observation, proposal, or learning calculation.

1531 Set `self.coordination_deferrals` by use the literal 0. Execute this step of the multi-agent observation, proposal, or learning calculation.

9.75. MultiAgentFleetEnv.observations

Build the agent-keyed observation dictionary using the restricted-view information contract for the selected ownership.

Source lines 1534–1540.

```
1534    def observations(self):
1536        views = mappo_agent_inputs(self.central_observation, self.env.config.aircraft,
1538                                   self.env.local_size, self.env.public_size,
                                       ↪  self.assignment)[0].numpy()
1540        return {agent: views[i].copy() for i, agent in enumerate(self.agent_ids)}
```

Line-by-line explanation.

1534 Define `observations(self)`. Build the agent-keyed observation dictionary using the restricted-view information contract for the selected ownership.

1536 Set `views` by expose this CPU tensor as a NumPy array. Execute this step of the multi-agent observation, proposal, or learning calculation.

1538 Continue the assignment begun at line 1536 with `self.env.local_size, self.env.public_size, self.assignment)[0].numpy()`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

1540 Return `{agent: views[i].copy() for i, agent in enumerate(self.agent_ids)}` to the caller by construct the collection by iterating the shown comprehension and applying its filters. No further statements in this call execute.

9.76. MultiAgentFleetEnv.reset

Start an episode, initialize random streams/state, and return the initial observation and diagnostic information.

Source lines 1543–1551.

```
1543    def reset(self, *, seed=None, options=None):
1545        self.central_observation, info = self.env.reset(seed=seed, options=options)
1547        self.coordination_deferrals = 0
1549        info.update(agent_ids=self.agent_ids, central_observation=self.central_observation.copy())
1551        return self.observations(), info
```

Line-by-line explanation.

1543 Define `reset(self, *, seed=None, options=None)`. This reset method is part of MultiAgentFleetEnv.reset; its parameters and defaults are the interface used by the calls in this section.

1545 Set `(self.central_observation, info)` by initialize a new episode using the stated seed/options. Execute this step of the multi-agent observation, proposal, or learning calculation.

1547 Set `self.coordination_deferrals` by use the literal 0. Execute this step of the multi-agent observation, proposal, or learning calculation.

1549 Call info.update with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

1551 Return `(self.observations(), info)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

9.77. MultiAgentFleetEnv.compose_action

Resolve agent replacement requests through the fixed broker, preserve active plans, prioritize urgency and allocate feasible stock, crew and workshop selections.

Source lines 1554–1586.

```
1554     def compose_action(self, actions):
1556         """Merge owned requests and allocate resources using observed data only."""
1558         if self.central_observation is None:
1560             raise RuntimeError('reset before composing agent proposals')
1562         if not isinstance(actions, dict) or set(actions) != set(self.agent_ids):
1564             raise ValueError('supply exactly one proposal for each agent ID')
1566         for agent in self.agent_ids:
1568             proposal = np.asarray(actions[agent])
1570             if not np.issubdtype(proposal.dtype, np.integer) or not
                 ↪  self.action_spaces[agent].contains(proposal):
1572                 raise ValueError(f'invalid proposal for {agent}')
1574         raw = np.stack([actions[agent] for agent in self.agent_ids])
1576         if self.assignment == 'aircraft':
1578             bundles = raw.copy()
1580         else:
1581             # Specialist c can set only component bit c, independently per engine.
1582             bundles = np.zeros((self.env.config.aircraft, 2), dtype=np.int64)
1584             for c in range(6):
1586                 for engine in range(2):
```

Line-by-line explanation.

1554 Define `compose_action(self, actions)`. Resolve agent replacement requests through the fixed broker, preserve active plans, prioritize urgency and allocate feasible stock, crew and workshop selections.

1556 Documentation text: Merge owned requests and allocate resources using observed data only.. This states the contract or qualification for `MultiAgentFleetEnv.compose_action`; it does not update the simulator or model.

1558 Test `self.central_observation is None`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1560 Raise `RuntimeError('reset before composing agent proposals')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1562 Test `not isinstance(actions, dict) or set(actions) != set(self.agent_ids)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1564 Raise `ValueError('supply exactly one proposal for each agent ID')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1566 Iterate `agent` over `self.agent_ids`. The following indented statements run once for each selected element.

1568 Set `proposal` by call np.asarray with the shown positional and keyword arguments. Execute this step of the multi-agent observation, proposal, or learning calculation.

1570 Test `not np.issubdtype(proposal.dtype, np.integer) or not self.action_spaces[agent].contains(proposal)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1572 Raise `ValueError(f'invalid proposal for {agent}')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1574 Set `raw` by call np.stack with the shown positional and keyword arguments. Execute this step of the multi-agent observation, proposal, or learning calculation.

1576 Test `self.assignment == 'aircraft'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1578 Set `bundles` by make a separate array/dictionary copy. Execute this step of the multi-agent observation, proposal, or learning calculation.

1580 Execute this alternative only when the corresponding preceding condition did not select its branch.

1581 Source comment: Specialist c can set only component bit c, independently per engine. This line documents the following code and does not execute.

1582 Set `bundles` by call np.zeros with the shown positional and keyword arguments.

1584 Iterate `c` over `range(6)`. The following indented statements run once for each selected element.

1586 Iterate `engine` over `range(2)`. The following indented statements run once for each selected element.

Source lines 1588–1620.

```
                bundles[:, engine] |= ((raw[c] >> engine) & 1) << c
    rows, public = self.env.split_observation(self.central_observation)
    free_mask = sum(1 << i for i in range(self.env.config.crews) if public[10 + i] > 0.5)
    offset = 10 + 2 * self.env.config.crews
    shops = [i for i in range(self.env.config.workshops) if public[offset + i] > 0.5]
    stock = np.rint(public[4:10] * self.env.config.aircraft).astype(int)
    scores = rows[:, 24:36].min(1)
    rank = np.argsort(scores, kind='stable')
    fleet_action = self.env.idle_action()
    deferred = []
    for position, aircraft in enumerate(rank):
        # The broker collects per-aircraft criticality reports; it is not a critic.
        fleet_action[aircraft, 7] = self.env.config.aircraft - position - 1
        if rows[aircraft, 37] > 0.5 or not bundles[aircraft].any():
            continue
        b0, b1 = map(int, bundles[aircraft])
        tasks = [(e, c) for e, b in enumerate((b0, b1)) for c in range(6) if b & (1 << c)]
        needed = np.bincount([c for _, c in tasks], minlength=6)
```

Line-by-line explanation.

1588 Bitwise-OR in `(raw[c] » engine & 1) « c` in `bundles[:, engine]`, and store the result back in that same target. The change is visible to subsequent statements in this block.

`1590` Set `(rows, public)` by call self.env.split_observation with the shown positional and keyword arguments. Execute this step of the multi-agent observation, proposal, or learning calculation.

`1592` Set `free_mask` by add the terms of the supplied iterable. Execute this step of the multi-agent observation, proposal, or learning calculation.

`1594` Set `offset` by add 10 and 2 * self.env.config.crews. Execute this step of the multi-agent observation, proposal, or learning calculation.

`1596` Set `shops` by construct the collection by iterating the shown comprehension and applying its filters. Execute this step of the multi-agent observation, proposal, or learning calculation.

`1598` Set `stock` by call np.rint(public[4:10] * self.env.config.aircraft).astype with the shown positional and keyword arguments. It holds spare inventory by component type. Execute this step of the multi-agent observation, proposal, or learning calculation.

`1600` Set `scores` by call rows[:, 24:36].min with the shown positional and keyword arguments. Execute this step of the multi-agent observation, proposal, or learning calculation.

`1602` Set `rank` by call np.argsort with the shown positional and keyword arguments. Execute this step of the multi-agent observation, proposal, or learning calculation.

`1604` Set `fleet_action` by call self.env.idle_action with the shown positional and keyword arguments. Execute this step of the multi-agent observation, proposal, or learning calculation.

`1606` Set `deferred` by construct the shown list in its stated order. Execute this step of the multi-agent observation, proposal, or learning calculation.

`1608` Iterate `(position, aircraft)` over `enumerate(rank)`. The following indented statements run once for each selected element.

`1609` Source comment: The broker collects per-aircraft criticality reports; it is not a critic. This line documents the following code and does not execute.

`1610` Set `fleet_action[aircraft, 7]` by subtract self.env.config.aircraft - position and 1.

`1612` Test `rows[aircraft, 37] > 0.5 or not bundles[aircraft].any()`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

`1614` Skip the remainder of this loop iteration and begin the next iteration.

`1616` Set `(b0, b1)` by call map with the shown positional and keyword arguments. Execute this step of the multi-agent observation, proposal, or learning calculation.

`1618` Set `tasks` by construct the collection by iterating the shown comprehension and applying its filters. Execute this step of the multi-agent observation, proposal, or learning calculation.

`1620` Set `needed` by call np.bincount with the shown positional and keyword arguments. Execute this step of the multi-agent observation, proposal, or learning calculation.

Source lines 1622–1650.

```
            if not free_mask or not shops:
                deferred.append((int(aircraft), 'resource_conflict'))
                continue
            if np.any(needed > stock):
                deferred.append((int(aircraft), 'spare_shortage'))
                continue
            crew = min((i for i in range(self.env.config.crews) if free_mask & (1 << i)), key=lambda
            ↪  i: self.env.crew_time_factors[i])
            shop = min(shops, key=lambda i: self.env.workshop_time_factors[i])
            first = min(tasks, key=lambda pair: rows[aircraft, pair[0] * 6 + pair[1]])
            engines = int(b0 != 0) + 2 * int(b1 != 0)
            fleet_action[aircraft, :7] = [1, engines, b0, b1, 1 << crew, shop, first[0] * 6 + first[1]]
            free_mask &= ~(1 << crew)
            shops.remove(shop)
            stock -= needed
        return fleet_action, deferred
```

Line-by-line explanation.

1622 Test `not free_mask or not shops`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1624 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

1626 Skip the remainder of this loop iteration and begin the next iteration.

1628 Test `np.any(needed > stock)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1630 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

1632 Skip the remainder of this loop iteration and begin the next iteration.

1634 Set `crew` by call min with the shown positional and keyword arguments. Execute this step of the multi-agent observation, proposal, or learning calculation.

1636 Set `shop` by call min with the shown positional and keyword arguments. Execute this step of the multi-agent observation, proposal, or learning calculation.

1638 Set `first` by call min with the shown positional and keyword arguments. Execute this step of the multi-agent observation, proposal, or learning calculation.

1640 Set `engines` by add int(b0 != 0) and 2 * int(b1 != 0). Execute this step of the multi-agent observation, proposal, or learning calculation.

1642 Set `fleet_action[aircraft, :7]` by construct the shown list in its stated order. Execute this step of the multi-agent observation, proposal, or learning calculation.

1644 Bitwise-AND with `~(1 << crew)` in `free_mask`, and store the result back in that same target. The change is visible to subsequent statements in this block.

1646 Call shops.remove with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

1648 Subtract `needed` from `stock`, and store the result back in that same target. This updates spare inventory by component type.

1650 Return `(fleet_action, deferred)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

9.78. MultiAgentFleetEnv.step

Apply the action through this class's transition/adapter contract and return the resulting observation, reward and episode flags.

Source lines 1653–1671.

```
1653    def step(self, actions):
1655        fleet_action, deferred = self.compose_action(actions)
1657        self.central_observation, reward, terminated, truncated, info = self.env.step(fleet_action)
1659        self.coordination_deferrals += len(deferred)
1661        info['metrics']['coordination_deferrals'] = self.coordination_deferrals
1663        info.update(agent_ids=self.agent_ids, agent_proposals={key: np.asarray(value).tolist() for
            ↪  key, value in actions.items()},
1665                    fleet_action=fleet_action.tolist(), deferred_requests=deferred,
1667                    central_observation=self.central_observation.copy())
1668        # The cooperative reward is replicated, not divided by the number of agents.
1669        rewards = {agent: reward for agent in self.agent_ids}
1671        return self.observations(), rewards, terminated, truncated, info
```

Line-by-line explanation.

1653 Define `step(self, actions)`. This step method is part of MultiAgentFleetEnv.step; its parameters and defaults are the interface used by the calls in this section.

1655 Set `(fleet_action, deferred)` by call self.compose_action with the shown positional and keyword arguments. Execute this step of the multi-agent observation, proposal, or learning calculation.

1657 Set `(self.central_observation, reward, terminated, truncated, info)` by execute this object's step with the shown arguments. Execute this step of the multi-agent observation, proposal, or learning calculation.

1659 Add `len(deferred)` to `self.coordination_deferrals`, and store the result back in that same target. The change is visible to subsequent statements in this block.

1661 Set `info['metrics']['coordination_deferrals']` by evaluate self.coordination_deferrals. Execute this step of the multi-agent observation, proposal, or learning calculation.

1663 Call info.update with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

1665 Continue the expression begun at line 1663 with `fleet_action=fleet_action.tolist(), deferred_requests=deferred,`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

1667 Continue the expression begun at line 1663 with `central_observation=self.central_observation.copy())`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

1668 Source comment: The cooperative reward is replicated, not divided by the number of agents. This line documents the following code and does not execute.

1669 Set `rewards` by construct the collection by iterating the shown comprehension and applying its filters. It holds the primitive reward sequence used for return accounting.

1671 Return `(self.observations(), rewards, terminated, truncated, info)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

9.79. MultiAgentActorCritic

The local-view MAPPO controller. Aircraft actors share parameters; component actors have separate parameters. The centralized observed-belief critic returns one value per agent, while actor-only execution consumes restricted inputs.

Source lines 1675–1683.

```
1675 class MultiAgentActorCritic(nn.Module):
1677     """Feed-forward MAPPO prototype with explicit per-agent policy statistics.
1679     Aircraft agents share actor weights; six specialist actors have separate
1680     weights. The centralized critic has one value head per agent and receives
1681     the full observed fleet belief, never the simulator's hidden health.
1682     Actors can execute through policy(agent_observations) without the critic.
1683     """
```

Line-by-line explanation.

1675 Define `MultiAgentActorCritic` with the stated inheritance. The local-view MAPPO controller. Aircraft actors share parameters; component actors have separate parameters. The centralized observed-belief critic returns one value per agent, while actor-only execution consumes restricted inputs.

1677 Documentation text: Feed-forward MAPPO prototype with explicit per-agent policy statistics.. This states the contract or qualification for `MultiAgentActorCritic`; it does not update the simulator or model.

1679 Documentation text: Aircraft agents share actor weights; six specialist actors have separate. This states the contract or qualification for `MultiAgentActorCritic`; it does not update the simulator or model.

1680 Documentation text: weights. The centralized critic has one value head per agent and receives. This states the contract or qualification for `MultiAgentActorCritic`; it does not update the simulator or model.

1681 Documentation text: the full observed fleet belief, never the simulator's hidden health.. This states the contract or qualification for `MultiAgentActorCritic`; it does not update the simulator or model.

1682 Documentation text: Actors can execute through policy(agent_observations) without the critic.. This states the contract or qualification for `MultiAgentActorCritic`; it does not update the simulator or model.

1683 Close the documentation string for `MultiAgentActorCritic`. This delimiter ends the descriptive text and performs no simulator or learning update.

9.80. MultiAgentActorCritic.___init___

Initialize the object, register its layers or allocate its state. Construction does not itself train a policy.

Source lines 1686–1720.

```
1686    def __init__(self, env, assignment='aircraft'):
1688        super().__init__()
1690        if assignment not in ('aircraft', 'component'):
1692            raise ValueError('assignment must be aircraft or component')
1694        self.assignment = assignment
1696        self.n = env.config.aircraft
1698        self.local = env.local_size
1700        self.public = env.public_size
1702        self.agent_ids = env.aircraft_plates if assignment == 'aircraft' else COMPONENTS
1704        self.num_agents = len(self.agent_ids)
1706        self.input_size = self.local + self.public if assignment == 'aircraft' else 15 * self.n +
        ↪  self.public
1708        self.fields = 2 if assignment == 'aircraft' else self.n
1710        self.categories = 64 if assignment == 'aircraft' else 4
1712        count = 1 if assignment == 'aircraft' else 6
1714        self.actors = nn.ModuleList([nn.Sequential(nn.Linear(self.input_size, 64), nn.Tanh(),
1716                                                   nn.Linear(64, 64), nn.Tanh(),
1718                                                   nn.Linear(64, self.fields * self.categories)) for _
                                                   ↪  in range(count)])
1720        self.critic = nn.Sequential(nn.Linear(env.observation_space.shape[0], 64), nn.Tanh(),
```

Line-by-line explanation.

1686 Define `__init__(self, env, assignment='aircraft')`. This ___init___ method is part of MultiAgentActorCritic.___init___; its parameters and defaults are the interface used by the calls in this section.

1688 Call super().___init___ with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

1690 Test `assignment not in ('aircraft', 'component')`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1692 Raise `ValueError('assignment must be aircraft or component')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1694 Set `self.assignment` by evaluate assignment. Execute this step of the multi-agent observation, proposal, or learning calculation.

1696 Set `self.n` by evaluate env.config.aircraft. Execute this step of the multi-agent observation, proposal, or learning calculation.

1698 Set `self.local` by evaluate env.local_size. Execute this step of the multi-agent observation, proposal, or learning calculation.

1700 Set `self.public` by evaluate env.public_size. Execute this step of the multi-agent observation, proposal, or learning calculation.

1702 Set `self.agent_ids` by select env.aircraft_plates when assignment == 'aircraft', otherwise COMPONENTS. Execute this step of the multi-agent observation, proposal, or learning calculation.

1704 Set `self.num_agents` by count entries. Execute this step of the multi-agent observation, proposal, or learning calculation.

1706 Set `self.input_size` by select self.local + self.public when assignment == 'aircraft', otherwise 15 * self.n + self.public. Execute this step of the multi-agent observation, proposal, or learning calculation.

1708 Set `self.fields` by select 2 when assignment == 'aircraft', otherwise self.n. Execute this step of the multi-agent observation, proposal, or learning calculation.

1710 Set `self.categories` by select 64 when assignment == 'aircraft', otherwise 4. Execute this step of the multi-agent observation, proposal, or learning calculation.

1712 Set `count` by select 1 when assignment == 'aircraft', otherwise 6. Execute this step of the multi-agent observation, proposal, or learning calculation.

1714 Set `self.actors` by register the listed independent networks so parameters and checkpoints include them. Execute this step of the multi-agent observation, proposal, or learning calculation.

1716 Continue the assignment begun at line 1714 with `nn.Linear(64, 64), nn.Tanh(),`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

1718 Continue the assignment begun at line 1714 with `nn.Linear(64, self.fields * self.categories)) for _ in range(count)])`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

1720 Set `self.critic` by compose the listed layers in execution order. Execute this step of the multi-agent observation, proposal, or learning calculation.

Source lines 1722–1722.

```
1722                                    nn.Linear(64, 64), nn.Tanh(), nn.Linear(64, self.num_agents))
```

Line-by-line explanation.

1722 Continue the assignment begun at line 1720 with `nn.Linear(64, 64), nn.Tanh(), nn.Linear(64, self.num_agents))`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

9.81. MultiAgentActorCritic.agent_inputs

Prepare the actor inputs for this policy family. MAPPO restricts inputs by owner; the DCMAC override supplies the global observed fleet belief.

Source lines 1725–1727.

```
1725     def agent_inputs(self, obs):
1727         return mappo_agent_inputs(obs, self.n, self.local, self.public, self.assignment)
```

Line-by-line explanation.

1725 Define `agent_inputs(self, obs)`. Prepare the actor inputs for this policy family. MAPPO restricts inputs by owner; the DCMAC override supplies the global observed fleet belief.

1727 Return `mappo_agent_inputs(obs, self.n, self.local, self.public, self.assignment)` to the caller by call mappo_agent_inputs with the shown positional and keyword arguments. No further statements in this call execute.

9.82. MultiAgentActorCritic.policy

Run the actor path using its allowed information. The separate critic path need not be available during actor-only execution.

Source lines 1730–1764.

```
1730    def policy(self, agent_obs, actions=None, deterministic=False):
1732        """Execute only restricted actors; no central observation or critic needed."""
1734        x = torch.as_tensor(agent_obs, dtype=torch.float32)
1736        if x.ndim != 3 or x.shape[1:] != (self.num_agents, self.input_size):
1738            raise ValueError('incorrect per-agent observation shape')
1740        if self.assignment == 'aircraft':
1742            logits = self.actors[0](x).reshape(-1, self.num_agents, self.fields, self.categories)
1744            busy = (x[:, :, 37] > 0.5).unsqueeze(-1).expand(-1, -1, 2)
1746        else:
1748            logits = torch.stack([actor(x[:, i]) for i, actor in enumerate(self.actors)], dim=1)
1750            logits = logits.reshape(-1, 6, self.n, 4)
1752            busy = x[:, :, :15 * self.n].reshape(-1, 6, self.n, 15)[:, :, :, 11] > 0.5
1754        valid = (~busy).unsqueeze(-1) | (torch.arange(self.categories, device=x.device) == 0)
1756        distribution = Categorical(logits=logits.masked_fill(~valid, -1e9))
1758        if actions is None:
1760            selected = distribution.probs.argmax(-1) if deterministic else distribution.sample()
1762        else:
1764            selected = torch.as_tensor(actions, device=x.device)
```

Line-by-line explanation.

1730 Define `policy(self, agent_obs, actions=None, deterministic=False)`. This policy method is part of MultiAgentActorCritic.policy; its parameters and defaults are the interface used by the calls in this section.

1732 Documentation text: Execute only restricted actors; no central observation or critic needed.. This states the contract or qualification for `MultiAgentActorCritic.policy`; it does not update the simulator or model.

1734 Set `x` by convert inputs to a tensor with the requested dtype. Execute this step of the multi-agent observation, proposal, or learning calculation.

1736 Test `x.ndim != 3 or x.shape[1:] != (self.num_agents, self.input_size)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1738 Raise `ValueError('incorrect per-agent observation shape')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1740 Test `self.assignment == 'aircraft'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1742 Set `logits` by reinterpret the shape while retaining element order. Execute this step of the multi-agent observation, proposal, or learning calculation.

1744 Set `busy` by call (x[:, :, 37] > 0.5).unsqueeze(-1).expand with the shown positional and keyword arguments. Execute this step of the multi-agent observation, proposal, or learning calculation.

1746 Execute this alternative only when the corresponding preceding condition did not select its branch.

1748 Set `logits` by join equal-shaped tensors along the specified new axis. Execute this step of the multi-agent observation, proposal, or learning calculation.

1750 Set `logits` by reinterpret the shape while retaining element order. Execute this step of the multi-agent observation, proposal, or learning calculation.

1752 Set `busy` by evaluate the comparison x[:, :, :15 * self.n].reshape(-1, 6, self.n, 15)[:, :, :, 11] > 0.5. Execute this step of the multi-agent observation, proposal, or learning calculation.

1754 Set `valid` by apply a bitwise union to (~busy).unsqueeze(-1) and torch.arange(self.categories, device=x.device) == 0. It holds the categorical structural-validity mask. Execute this step of the multi-agent observation, proposal, or learning calculation.

1756 Set `distribution` by construct a discrete distribution from the supplied logits/probabilities. Execute this step of the multi-agent observation, proposal, or learning calculation.

1758 Test `actions is None`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1760 Set `selected` by select distribution.probs.argmax(-1) when deterministic, otherwise distribution.sample(). Execute this step of the multi-agent observation, proposal, or learning calculation.

1762 Execute this alternative only when the corresponding preceding condition did not select its branch.

1764 Set `selected` by convert inputs to a tensor with the requested dtype. Execute this step of the multi-agent observation, proposal, or learning calculation.

Source lines 1766–1774.

```
1766            if selected.shape != logits.shape[:-1] or selected.dtype not in (torch.int32, torch.int64):
1768                raise ValueError('incorrect stored agent action shape or dtype')
1770            if torch.any((selected < 0) | (selected >= self.categories)) or torch.any(busy & (selected
                ↪  != 0)):
1772                raise ValueError('stored agent action violates a category or busy mask')
1773        # Sum fields WITHIN each agent; preserve the agent axis for MAPPO clipping.
1774        return selected, distribution.log_prob(selected).sum(-1), distribution.entropy().sum(-1)
```

Line-by-line explanation.

1766 Test `selected.shape != logits.shape[:-1] or selected.dtype not in (torch.int32, torch.int64)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1768 Raise `ValueError('incorrect stored agent action shape or dtype')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1770 Test `torch.any((selected < 0) | (selected >= self.categories)) or torch.any(busy & (selected != 0))`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1772 Raise `ValueError('stored agent action violates a category or busy mask')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1773 Source comment: Sum fields WITHIN each agent; preserve the agent axis for MAPPO clipping. This line documents the following code and does not execute.

1774 Return `(selected, distribution.log_prob(selected).sum(-1), distribution.entropy().sum(-1))` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

9.83. MultiAgentActorCritic.active_agents

Identify which decision roles have an actual free-aircraft action opportunity for normalization and actor updates.

Source lines 1777–1785.

```
1777     def active_agents(self, agent_obs):
1779         if self.assignment == 'aircraft':
1781             return (agent_obs[:, :, 37] <= 0.5).float()
1783         busy = agent_obs[:, :, :15 * self.n].reshape(-1, 6, self.n, 15)[:, :, :, 11] > 0.5
1785         return (~busy).any(-1).float()
```

Line-by-line explanation.

1777 Define `active_agents(self, agent_obs)`. Identify which decision roles have an actual free-aircraft action opportunity for normalization and actor updates.

1779 Test `self.assignment == 'aircraft'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1781 Return `(agent_obs[:, :, 37] <= 0.5).float()` to the caller by call (agent_obs[:, :, 37] <= 0.5).float with the shown positional and keyword arguments. No further statements in this call execute.

1783 Set `busy` by evaluate the comparison agent_obs[:, :, :15 * self.n].reshape(-1, 6, self.n, 15)[:, :, :, 11] > 0.5. Execute this step of the multi-agent observation, proposal, or learning calculation.

1785 Return `(~busy).any(-1).float()` to the caller by call (~busy).any(-1).float with the shown positional and keyword arguments. No further statements in this call execute.

9.84. MultiAgentActorCritic.value

Evaluate the centralized state-value or action-value path using the input contract of this class.

Source lines 1788–1792.

```
1788     def value(self, obs):
1790         x = torch.as_tensor(obs, dtype=torch.float32)
1792         return self.critic(x.unsqueeze(0) if x.ndim == 1 else x)
```

Line-by-line explanation.

1788 Define `value(self, obs)`. This value method is part of MultiAgentActorCritic.value; its parameters and defaults are the interface used by the calls in this section.

1790 Set `x` by convert inputs to a tensor with the requested dtype. Execute this step of the multi-agent observation, proposal, or learning calculation.

1792 Return `self.critic(x.unsqueeze(0) if x.ndim == 1 else x)` to the caller by call self.critic with the shown positional and keyword arguments. No further statements in this call execute.

9.85. MultiAgentActorCritic.act

Sample or reconstruct actions and compute their log probabilities, entropy and values as defined by this policy family.

Source lines 1795–1799.

```
1795    def act(self, obs, actions=None, deterministic=False):
1797        selected, logp, entropy = self.policy(self.agent_inputs(obs), actions, deterministic)
1799        return selected, logp, entropy, self.value(obs)
```

Line-by-line explanation.

1795 Define `act(self, obs, actions=None, deterministic=False)`. This act method is part of MultiAgentActorCritic.act; its parameters and defaults are the interface used by the calls in this section.

1797 Set `(selected, logp, entropy)` by call self.policy with the shown positional and keyword arguments. Execute this step of the multi-agent observation, proposal, or learning calculation.

1799 Return `(selected, logp, entropy, self.value(obs))` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

9.86. MultiAgentActorCritic.action_dict

Convert batched sampled action tensors into the agent-keyed NumPy requests expected by the multi-agent adapter.

Source lines 1802–1810.

```
1802    def action_dict(self, selected):
1804        raw = torch.as_tensor(selected).detach().cpu().numpy()
1806        if raw.shape != (self.num_agents, self.fields):
1808            raise ValueError('incorrect unbatched agent action shape')
1810        return {agent: raw[i].copy() for i, agent in enumerate(self.agent_ids)}
```

Line-by-line explanation.

1802 Define `action_dict(self, selected)`. Convert batched sampled action tensors into the agent-keyed NumPy requests expected by the multi-agent adapter.

1804 Set `raw` by expose this CPU tensor as a NumPy array. Execute this step of the multi-agent observation, proposal, or learning calculation.

1806 Test `raw.shape != (self.num_agents, self.fields)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1808 Raise `ValueError('incorrect unbatched agent action shape')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1810 Return `{agent: raw[i].copy() for i, agent in enumerate(self.agent_ids)}` to the caller by construct the collection by iterating the shown comprehension and applying its filters. No further statements in this call execute.

9.87. Module declarations and integration

Source lines 1814–1816.

```
1814 DCMAC_METHODS = ('dcmac_aircraft', 'ddmac_aircraft', 'dcmac_component', 'ddmac_component')
1816 REQUEST_METHODS = (*MAPPO_METHODS, *DCMAC_METHODS)
```

Line-by-line explanation.

1814 Set `DCMAC_METHODS` by construct the shown tuple in its stated order.

1816 Set `REQUEST_METHODS` by construct the shown tuple in its stated order.

9.88. request_assignment

Translate a registered request-method suffix into aircraft versus component ownership and reject unsupported method names.

Source lines 1820–1828.

```
1820 def request_assignment(method):
1822     """Resolve ownership for registered proposal methods; reject silent fallbacks."""
1824     if method not in REQUEST_METHODS:
1826         raise ValueError('unknown request policy method')
1828     return method.rsplit('_', 1)[1]
```

Line-by-line explanation.

1820 Define `request_assignment(method)`. Translate a registered request-method suffix into aircraft versus component ownership and reject unsupported method names.

1822 Documentation text: Resolve ownership for registered proposal methods; reject silent fallbacks.. This states the contract or qualification for `request_assignment`; it does not update the simulator or model.

1824 Test `method not in REQUEST_METHODS`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1826 Raise `ValueError('unknown request policy method')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1828 Return `method.rsplit('_', 1)[1]` to the caller by select the indexed/sliced entries of method.rsplit('_', 1). No further statements in this call execute.

9.89. DCMACActorCritic

The global-view DCMAC/DDMAC adaptation. DCMAC has one joint network with factorized output blocks; DDMAC constructs one global-input actor per agent. Both use one scalar fleet critic and the replay-based trainer.

Source lines 1832–1842.

```
1832 class DCMACActorCritic(MultiAgentActorCritic):
1834     """Fleet adaptation of DCMAC/DDMAC; global observed beliefs enter all actors.
1836     DCMAC has one actor with factorized control-unit outputs. DDMAC has an
1837     independently parameterized actor per control unit. Both have one scalar
```

```
    centralized value network and use replay with joint importance weighting.
    The fixed dispatcher owns resources; paper-specific inspection actions,
    budget augmentation and primal-dual risk constraints are not reproduced.
    Inherits only the proposal-to-dictionary conversion from the MAPPO class.
    """
```

Line-by-line explanation.

1832 Define `DCMACActorCritic` with the stated inheritance. The global-view DCMAC/DDMAC adaptation. DCMAC has one joint network with factorized output blocks; DDMAC constructs one global-input actor per agent. Both use one scalar fleet critic and the replay-based trainer.

1834 Documentation text: Fleet adaptation of DCMAC/DDMAC; global observed beliefs enter all actors.. This states the contract or qualification for `DCMACActorCritic`; it does not update the simulator or model.

1836 Documentation text: DCMAC has one actor with factorized control-unit outputs. DDMAC has an. This states the contract or qualification for `DCMACActorCritic`; it does not update the simulator or model.

1837 Documentation text: independently parameterized actor per control unit. Both have one scalar. This states the contract or qualification for `DCMACActorCritic`; it does not update the simulator or model.

1838 Documentation text: centralized value network and use replay with joint importance weighting.. This states the contract or qualification for `DCMACActorCritic`; it does not update the simulator or model.

1839 Documentation text: The fixed dispatcher owns resources; paper-specific inspection actions,. This states the contract or qualification for `DCMACActorCritic`; it does not update the simulator or model.

1840 Documentation text: budget augmentation and primal-dual risk constraints are not reproduced.. This states the contract or qualification for `DCMACActorCritic`; it does not update the simulator or model.

1841 Documentation text: Inherits only the proposal-to-dictionary conversion from the MAPPO class.. This states the contract or qualification for `DCMACActorCritic`; it does not update the simulator or model.

1842 Close the documentation string for `DCMACActorCritic`. This delimiter ends the descriptive text and performs no simulator or learning update.

9.90. DCMACActorCritic.__init__

Initialize the object, register its layers or allocate its state. Construction does not itself train a policy.

Source lines 1845–1879.

```
    def __init__(self, env, assignment='aircraft', decentralized=False):
        nn.Module.__init__(self)
        if assignment not in ('aircraft', 'component'):
            raise ValueError('assignment must be aircraft or component')
        self.assignment = assignment
        self.decentralized = decentralized
        self.n = env.config.aircraft
        self.local = env.local_size
```

```
1861            self.public = env.public_size
1863            self.agent_ids = env.aircraft_plates if assignment == 'aircraft' else COMPONENTS
1865            self.num_agents = len(self.agent_ids)
1867            self.input_size = env.observation_space.shape[0]
1869            self.fields = 2 if assignment == 'aircraft' else self.n
1871            self.categories = 64 if assignment == 'aircraft' else 4
1873            count = self.num_agents if decentralized else 1
1875            output = self.fields * self.categories * (1 if decentralized else self.num_agents)
1877            self.actors = nn.ModuleList([
1879                nn.Sequential(nn.Linear(self.input_size, 64), nn.Tanh(),
```

Line-by-line explanation.

1845 Define `__init__(self, env, assignment='aircraft', decentralized=False)`. This ___init___ method is part of DCMACActor-Critic.___init___; its parameters and defaults are the interface used by the calls in this section.

1847 Call nn.module.___init___ with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

1849 Test `assignment not in ('aircraft', 'component')`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1851 Raise `ValueError('assignment must be aircraft or component')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1853 Set `self.assignment` by evaluate assignment.

1855 Set `self.decentralized` by evaluate decentralized.

1857 Set `self.n` by evaluate env.config.aircraft.

1859 Set `self.local` by evaluate env.local_size.

1861 Set `self.public` by evaluate env.public_size.

1863 Set `self.agent_ids` by select env.aircraft_plates when assignment == 'aircraft', otherwise COMPONENTS.

1865 Set `self.num_agents` by count entries.

1867 Set `self.input_size` by select the indexed/sliced entries of env.observation_space.shape.

1869 Set `self.fields` by select 2 when assignment == 'aircraft', otherwise self.n.

1871 Set `self.categories` by select 64 when assignment == 'aircraft', otherwise 4.

1873 Set `count` by select self.num_agents when decentralized, otherwise 1.

1875 Set `output` by multiply self.fields * self.categories and 1 if decentralized else self.num_agents.

1877 Set `self.actors` by register the listed independent networks so parameters and checkpoints include them. Construct or update factorized proposal policies from global observed beliefs.

1879 Continue the assignment begun at line 1877 with `nn.Sequential(nn.Linear(self.input_size, 64), nn.Tanh(),`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

Source lines 1881–1889.

```
                            nn.Linear(64, 64), nn.Tanh(), nn.Linear(64, output))
                for _ in range(count)
            ])
            self.critic = nn.Sequential(nn.Linear(self.input_size, 64), nn.Tanh(),
                                        nn.Linear(64, 64), nn.Tanh(), nn.Linear(64, 1))
```

Line-by-line explanation.

1881 Continue the assignment begun at line 1877 with `nn.Linear(64, 64), nn.Tanh(), nn.Linear(64, output))`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

1883 Continue the assignment begun at line 1877 with `for _ in range(count)`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

1885 Continue the assignment begun at line 1877 with `])`. Close the nested call/container at this indentation, completing that part of the same statement.

1887 Set `self.critic` by compose the listed layers in execution order. Calculate or update the common scalar fleet-return baseline.

1889 Continue the assignment begun at line 1887 with `nn.Linear(64, 64), nn.Tanh(), nn.Linear(64, 1))`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

9.91. DCMACActorCritic.agent_inputs

Prepare the actor inputs for this policy family. MAPPO restricts inputs by owner; the DCMAC override supplies the global observed fleet belief.

Source lines 1892–1906.

```
    def agent_inputs(self, obs):
        """Return global actor inputs; unlike MAPPO, execution needs fleet beliefs."""
        x = torch.as_tensor(obs, dtype=torch.float32)
        if x.ndim == 1:
            x = x.unsqueeze(0)
        if x.ndim != 2 or x.shape[1] != self.input_size:
            raise ValueError('incorrect global actor observation shape')
        return x
```

Line-by-line explanation.

1892 Define `agent_inputs(self, obs)`. Prepare the actor inputs for this policy family. MAPPO restricts inputs by owner; the DCMAC override supplies the global observed fleet belief.

1894 Documentation text: Return global actor inputs; unlike MAPPO, execution needs fleet beliefs.. This states the contract or qualification for `DCMACActorCritic.agent_inputs`; it does not update the simulator or model.

1896 Set `x` by convert inputs to a tensor with the requested dtype.

1898 Test `x.ndim == 1`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1900 Set `x` by call x.unsqueeze with the shown positional and keyword arguments.

1902 Test `x.ndim != 2 or x.shape[1] != self.input_size`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1904 Raise `ValueError('incorrect global actor observation shape')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1906 Return `x` to the caller by evaluate x. No further statements in this call execute.

9.92. DCMACActorCritic._busy_fields

Identify action fields whose owning aircraft is physically committed so their distributions can be forced to the no-operation category.

Source lines 1909–1917.

```
1909     def _busy_fields(self, obs):
1911         busy = obs[:, :self.n * self.local].reshape(-1, self.n, self.local)[:, :, 37] > 0.5
1913         if self.assignment == 'aircraft':
1915             return busy.unsqueeze(-1).expand(-1, -1, 2)
1917         return busy.unsqueeze(1).expand(-1, 6, -1)
```

Line-by-line explanation.

1909 Define `_busy_fields(self, obs)`. Identify action fields whose owning aircraft is physically committed so their distributions can be forced to the no-operation category.

1911 Set `busy` by evaluate the comparison obs[:, :self.n * self.local].reshape(-1, self.n, self.local)[:, :, 37] $>$ 0.5.

1913 Test `self.assignment == 'aircraft'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1915 Return `busy.unsqueeze(-1).expand(-1, -1, 2)` to the caller by call busy.unsqueeze(-1).expand with the shown positional and keyword arguments. No further statements in this call execute.

1917 Return `busy.unsqueeze(1).expand(-1, 6, -1)` to the caller by call busy.unsqueeze(1).expand with the shown positional and keyword arguments. No further statements in this call execute.

9.93. DCMACActorCritic.active_agents

Identify which decision roles have an actual free-aircraft action opportunity for normalization and actor updates.

Source lines 1920–1922.

```
1920     def active_agents(self, obs):
1922         return (~self._busy_fields(self.agent_inputs(obs))).any(-1).float()
```

Line-by-line explanation.

1920 Define `active_agents(self, obs)`. Identify which decision roles have an actual free-aircraft action opportunity for normalization and actor updates.

1922 Return `(~self._busy_fields(self.agent_inputs(obs))).any(-1).float()` to the caller by call (~self._busy_fields(self.agent_inputs(obs))).any(-1).float with the shown positional and keyword arguments. No further statements in this call execute.

9.94. DCMACActorCritic.policy

Run the actor path using its allowed information. The separate critic path need not be available during actor-only execution.

Source lines 1925–1959.

```
1925    def policy(self, obs, actions=None, deterministic=False):
1927        """Factorized categorical proposals; no hidden state or critic evaluation."""
1929        x = self.agent_inputs(obs)
1931        if self.decentralized:
1933            logits = torch.stack([actor(x) for actor in self.actors], dim=1)
1935        else:
1937            logits = self.actors[0](x)
1939        logits = logits.reshape(-1, self.num_agents, self.fields, self.categories)
1941        busy = self._busy_fields(x)
1943        valid = (~busy).unsqueeze(-1) | (torch.arange(self.categories, device=x.device) == 0)
1945        distribution = Categorical(logits=logits.masked_fill(~valid, -1e9))
1947        if actions is None:
1949            selected = distribution.probs.argmax(-1) if deterministic else distribution.sample()
1951        else:
1953            selected = torch.as_tensor(actions, device=x.device)
1955            if selected.shape != logits.shape[:-1] or selected.dtype not in (torch.int32, torch.int64):
1957                raise ValueError('incorrect stored agent action shape or dtype')
1959            if torch.any((selected < 0) | (selected >= self.categories)) or torch.any(busy & (selected
                ↪  != 0)):
```

Line-by-line explanation.

1925 Define `policy(self, obs, actions=None, deterministic=False)`. This policy method is part of DCMACActorCritic.policy; its parameters and defaults are the interface used by the calls in this section.

1927 Documentation text: Factorized categorical proposals; no hidden state or critic evaluation.. This states the contract or qualification for `DCMACActorCritic.policy`; it does not update the simulator or model.

1929 Set `x` by call self.agent_inputs with the shown positional and keyword arguments.

1931 Test `self.decentralized`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1933 Set `logits` by join equal-shaped tensors along the specified new axis. Construct or update factorized proposal policies from global observed beliefs.

1935 Execute this alternative only when the corresponding preceding condition did not select its branch.

1937 Set `logits` by call self.actors[0] with the shown positional and keyword arguments. Construct or update factorized proposal policies from global observed beliefs.

1939 Set `logits` by reinterpret the shape while retaining element order. Construct or update factorized proposal policies from global observed beliefs.

1941 Set `busy` by call self._busy_fields with the shown positional and keyword arguments.

1943 Set `valid` by apply a bitwise union to (~busy).unsqueeze(-1) and torch.arange(self.categories, device=x.device) == 0. It holds the categorical structural-validity mask.

1945 Set `distribution` by construct a discrete distribution from the supplied logits/probabilities. Construct or update factorized proposal policies from global observed beliefs.

1947 Test `actions is None`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1949 Set `selected` by select distribution.probs.argmax(-1) when deterministic, otherwise distribution.sample(). Construct or update factorized proposal policies from global observed beliefs.

1951 Execute this alternative only when the corresponding preceding condition did not select its branch.

1953 Set `selected` by convert inputs to a tensor with the requested dtype.

1955 Test `selected.shape != logits.shape[:-1] or selected.dtype not in (torch.int32, torch.int64)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1957 Raise `ValueError('incorrect stored agent action shape or dtype')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1959 Test `torch.any((selected < 0) | (selected >= self.categories)) or torch.any(busy & (selected != 0))`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

Source lines 1961–1963.

```
1961                raise ValueError('stored agent action violates a category or busy mask')
1963        return selected, distribution.log_prob(selected).sum(-1), distribution.entropy().sum(-1)
```

Line-by-line explanation.

1961 Raise `ValueError('stored agent action violates a category or busy mask')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1963 Return `(selected, distribution.log_prob(selected).sum(-1), distribution.entropy().sum(-1))` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

9.95. DCMACActorCritic.value

Evaluate the centralized state-value or action-value path using the input contract of this class.

Source lines 1966–1968.

```
1966    def value(self, obs):
1968        return self.critic(self.agent_inputs(obs)).squeeze(-1)
```

Line-by-line explanation.

1966 Define `value(self, obs)`. This value method is part of DCMACActorCritic.value; its parameters and defaults are the interface used by the calls in this section.

1968 Return `self.critic(self.agent_inputs(obs)).squeeze(-1)` to the caller by call self.critic(self.agent_inputs(obs)).squeeze with the shown positional and keyword arguments. No further statements in this call execute.

9.96. DCMACActorCritic.act

Sample or reconstruct actions and compute their log probabilities, entropy and values as defined by this policy family.

Source lines 1971–1975.

```
1971     def act(self, obs, actions=None, deterministic=False):
1973         selected, logp, entropy = self.policy(obs, actions, deterministic)
1975         return selected, logp, entropy, self.value(obs)
```

Line-by-line explanation.

1971 Define `act(self, obs, actions=None, deterministic=False)`. This act method is part of DCMACActorCritic.act; its parameters and defaults are the interface used by the calls in this section.

1973 Set `(selected, logp, entropy)` by call self.policy with the shown positional and keyword arguments.

1975 Return `(selected, logp, entropy, self.value(obs))` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

9.97. dcmac_losses

Compute the joint importance ratio, capped detached replay weight, scalar TD residual, weighted policy loss and weighted value loss. Agent log probabilities are summed before importance weighting.

Source lines 1979–2010.

```
1979 def dcmac_losses(logp, behavior_logp, values, next_values, rewards, dones,
1981                  gamma=0.99, importance_clip=2.0):
1983     """One-step replay update: joint truncated ratio and detached TD advantage.
1985     The ratio is pi(joint proposal|belief)/mu(joint proposal|belief), not the
1986     product of separately truncated ratios. Truncation introduces bias but
1987     bounds the high-variance replay weights. No PPO clipping or GAE is used.
1988     """
1990     if logp.ndim != 2 or behavior_logp.shape != logp.shape:
1992         raise ValueError('proposal log probabilities must have batch x agent shape')
1994     if any(t.shape != (len(logp),) for t in (values, next_values, rewards, dones)):
1996         raise ValueError('one scalar value, reward and terminal flag per replay transition required')
1998     if not np.isfinite(importance_clip) or importance_clip <= 0 or not 0 <= gamma <= 1:
2000         raise ValueError('positive finite importance cap and gamma in [0,1] required')
2002     if not all(bool(torch.isfinite(t).all()) for t in (logp, behavior_logp, values, next_values,
     ↪  rewards, dones)):
2004         raise ValueError('replay tensors must be finite')
```

```
    joint_logp = logp.sum(-1)
    log_ratio = (joint_logp - behavior_logp.sum(-1)).detach()
    weights = torch.exp(log_ratio.clamp(max=float(np.log(importance_clip))))
```

Line-by-line explanation.

1979 Define `dcmac_losses(logp, behavior_logp, values, next_values, rewards, dones,`. Compute the joint importance ratio, capped detached replay weight, scalar TD residual, weighted policy loss and weighted value loss. Agent log probabilities are summed before importance weighting.

1981 Execute the Python fragment `gamma=0.99, importance_clip=2.0):` in `dcmac_losses`. Its indentation places it in the surrounding control block.

1983 Documentation text: One-step replay update: joint truncated ratio and detached TD advantage.. This states the contract or qualification for `dcmac_losses`; it does not update the simulator or model.

1985 Documentation text: The ratio is pi(joint proposal|belief)/mu(joint proposal|belief), not the. This states the contract or qualification for `dcmac_losses`; it does not update the simulator or model.

1986 Documentation text: product of separately truncated ratios. Truncation introduces bias but. This states the contract or qualification for `dcmac_losses`; it does not update the simulator or model.

1987 Documentation text: bounds the high-variance replay weights. No PPO clipping or GAE is used.. This states the contract or qualification for `dcmac_losses`; it does not update the simulator or model.

1988 Close the documentation string for `dcmac_losses`. This delimiter ends the descriptive text and performs no simulator or learning update.

1990 Test `logp.ndim != 2 or behavior_logp.shape != logp.shape`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1992 Raise `ValueError('proposal log probabilities must have batch x agent shape')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1994 Test `any((t.shape != (len(logp),) for t in (values, next_values, rewards, dones)))`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

1996 Raise `ValueError('one scalar value, reward and terminal flag per replay transition required')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

1998 Test `not np.isfinite(importance_clip) or importance_clip <= 0 or (not 0 <= gamma <= 1)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2000 Raise `ValueError('positive finite importance cap and gamma in [0,1] required')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

2002 Test `not all((bool(torch.isfinite(t).all()) for t in (logp, behavior_logp, values, next_values, rewards, dones)))`. Execute the

indented branch only when it is true; subsequent alternatives handle the other case.

2004 Raise `ValueError('replay tensors must be finite')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

2006 Set `joint_logp` by sum across the specified tensor/array axis.

2008 Set `log_ratio` by stop gradients through this result. Use the joint behavior-to-target likelihood correction for replay.

2010 Set `weights` by exponentiate log values, including log-probability ratios. It holds the detached capped importance weights for replay. Use the joint behavior-to-target likelihood correction for replay.

Source lines 2012–2020.

```
2012      target = rewards + gamma * (1 - dones.float()) * next_values.detach()
2014      delta = target - values
2016      actor_loss = -(weights * joint_logp * delta.detach()).mean()
2018      critic_loss = 0.5 * (weights * delta.square()).mean()
2020      return actor_loss, critic_loss, weights, target.detach()
```

Line-by-line explanation.

2012 Set `target` by add rewards and gamma * (1 - dones.float()) * next_values.detach(). It holds a bootstrapped learning target. Calculate or update the common scalar fleet-return baseline.

2014 Set `delta` by subtract target and values. Calculate or update the common scalar fleet-return baseline.

2016 Set `actor_loss` by evaluate -(weights * joint_logp * delta.detach()).mean(). It holds the objective differentiated through policy parameters. Use the joint behavior-to-target likelihood correction for replay.

2018 Set `critic_loss` by multiply 0.5 and (weights * delta.square()).mean(). It holds the prediction-error objective for the value estimator. Use the joint behavior-to-target likelihood correction for replay.

2020 Return `(actor_loss, critic_loss, weights, target.detach())` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

9.98. train_dcmac

Collect daily joint request transitions into bounded uniform replay, then update global-input actor(s) and the scalar critic with importance-weighted TD losses. This function does not apply PPO clipping or GAE.

Source lines 2024–2054.

```
2024  def train_dcmac(config, method='dcmac_aircraft', steps=4096, seed=0,
2026                  rollout_size=128, replay_capacity=4096, importance_clip=2.0):
2028      """Off-policy DCMAC/DDMAC adaptation with uniform bounded experience replay.
2030      Stochastic actor sampling is the behavior policy; its original proposal
2031      likelihood is stored even when the dispatcher defers that proposal.
2032      One replay minibatch is updated per day after two samples are available.
2033      rollout_size names minibatch size here, preserving the common train API.
```

```
    """
    if method not in DCMAC_METHODS or steps < 1 or rollout_size < 2 or replay_capacity < 2:
        raise ValueError('registered DCMAC/DDMAC method, positive steps and batch/capacity >= 2
        ↪  required')
    if not np.isfinite(importance_clip) or importance_clip <= 0:
        raise ValueError('importance_clip must be positive and finite')
    torch.set_num_threads(1)
    torch.manual_seed(seed)
    rng = np.random.default_rng(seed)
    assignment = request_assignment(method)
    env = MultiAgentFleetEnv(config, assignment)
    model = DCMACActorCritic(env.env, assignment, method.startswith('ddmac_'))
```

Line-by-line explanation.

2024 Define `train_dcmac(config, method='dcmac_aircraft', steps=4096, seed=0,`. Collect daily joint request transitions into bounded uniform replay, then update global-input actor(s) and the scalar critic with importance-weighted TD losses. This function does not apply PPO clipping or GAE.

2026 Execute the Python fragment `rollout_size=128, replay_capacity=4096, importance_clip=2.0):` in `train_dcmac`. Its indentation places it in the surrounding control block.

2028 Documentation text: Off-policy DCMAC/DDMAC adaptation with uniform bounded experience replay.. This states the contract or qualification for `train_dcmac`; it does not update the simulator or model.

2030 Documentation text: Stochastic actor sampling is the behavior policy; its original proposal. This states the contract or qualification for `train_dcmac`; it does not update the simulator or model.

2031 Documentation text: likelihood is stored even when the dispatcher defers that proposal.. This states the contract or qualification for `train_dcmac`; it does not update the simulator or model.

2032 Documentation text: One replay minibatch is updated per day after two samples are available.. This states the contract or qualification for `train_dcmac`; it does not update the simulator or model.

2033 Documentation text: rollout_size names minibatch size here, preserving the common train API.. This states the contract or qualification for `train_dcmac`; it does not update the simulator or model.

2034 Close the documentation string for `train_dcmac`. This delimiter ends the descriptive text and performs no simulator or learning update.

2036 Test `method not in DCMAC_METHODS or steps < 1 or rollout_size < 2 or (replay_capacity < 2)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2038 Raise `ValueError('registered DCMAC/DDMAC method, positive steps and batch/capacity >= 2 required')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

2040 Test `not np.isfinite(importance_clip) or importance_clip <= 0`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2042 Raise `ValueError('importance_clip must be positive and finite')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

2044 Set the cpu thread count for these small models. The exact receiver and arguments are shown on this source line.

2046 Seed neural initialization and torch sampling. The exact receiver and arguments are shown on this source line.

2048 Set `rng` by call np.random.default_rng with the shown positional and keyword arguments.

2050 Set `assignment` by call request_assignment with the shown positional and keyword arguments.

2052 Set `env` by call MultiAgentFleetEnv with the shown positional and keyword arguments. It holds the environment or wrapper that owns physical transitions.

2054 Set `model` by call DCMACActorCritic with the shown positional and keyword arguments. It holds the chosen controller architecture.

Source lines 2056–2090.

```
    actor_optimizer = torch.optim.Adam(model.actors.parameters(), lr=3e-4)
    critic_optimizer = torch.optim.Adam(model.critic.parameters(), lr=3e-4)
    replay, logs = [], []
    _, info = env.reset(seed=seed)
    obs = info['central_observation']
    episode = 0
    for day in range(1, steps + 1):
        with torch.no_grad():
            action, behavior_logp, _ = model.policy(obs)
        _, rewards, done, truncated, info = env.step(model.action_dict(action[0]))
        next_obs = info['central_observation']
        terminal = done or truncated
        transition = (obs.copy(), action[0].numpy().copy(), behavior_logp[0].numpy().copy(),
                      rewards[model.agent_ids[0]], next_obs.copy(), terminal, day)
        if len(replay) < replay_capacity:
            replay.append(transition)
        else:
            replay[(day - 1) % replay_capacity] = transition
```

Line-by-line explanation.

2056 Set `actor_optimizer` by create an Adam optimizer for the specified parameter collection. It holds the optimizer restricted to actor parameters. Construct or update factorized proposal policies from global observed beliefs.

2058 Set `critic_optimizer` by create an Adam optimizer for the specified parameter collection. It holds the optimizer for the centralized value estimator. Calculate or update the common scalar fleet-return baseline.

2060 Set `(replay, logs)` by construct the shown tuple in its stated order. Store or sample original proposal transitions and their collection days.

2062 Set `(_, info)` by initialize a new episode using the stated seed/options.

2064 Set `obs` by select the indexed/sliced entries of info. It holds the observed belief vector, rather than hidden physical health.

2066 Set `episode` by use the literal 0.

2068 Iterate `day` over `range(1, steps + 1)`. The following indented statements run once for each selected element.

2070 Enter `torch.no_grad()`. The block records no gradients, keeping rollout/target calculations outside backpropagation.

2072 Set `(action, behavior_logp, _)` by call model.policy with the shown positional and keyword arguments.

2074 Set `(_, rewards, done, truncated, info)` by execute this object's step with the shown arguments.

2076 Set `next_obs` by select the indexed/sliced entries of info.

2078 Set `terminal` by evaluate done or truncated. It holds the episode boundary used to suppress continuation value.

2080 Set `transition` by construct the shown tuple in its stated order. Store or sample original proposal transitions and their collection days.

2082 Continue the assignment begun at line 2080 with `rewards[model.agent_ids[0]], next_obs.copy(), terminal, day)`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

2084 Test `len(replay) < replay_capacity`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2086 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

2088 Execute this alternative only when the corresponding preceding condition did not select its branch.

2090 Set `replay[(day - 1) % replay_capacity]` by evaluate transition. It holds the bounded off-policy transition buffer. Store or sample original proposal transitions and their collection days.

Source lines 2092–2126.

```
        if len(replay) >= 2:
            indices = rng.choice(len(replay), size=min(rollout_size, len(replay)), replace=False)
            batch = [replay[int(i)] for i in indices]
            states, actions, behavior, reward, following, terminal_flags, sampled_days = [
                torch.as_tensor(np.asarray(items)) for items in zip(*batch)
            ]
            _, logp, entropy = model.policy(states, actions)
            values = model.value(states)
            with torch.no_grad():
                next_values = model.value(following)
            actor_loss, value_loss, weights, _ = dcmac_losses(
                logp, behavior, values, next_values, reward.float(), terminal_flags,
                importance_clip=importance_clip
            )
            active = model.active_agents(states)
            entropy_bonus = (entropy * active).sum() / active.sum().clamp_min(1)
            policy_loss = actor_loss - 0.001 * entropy_bonus
            if not bool(torch.isfinite(policy_loss + value_loss)):
```

Line-by-line explanation.

2092 Test `len(replay) >= 2`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2094 Set `indices` by call rng.choice with the shown positional and keyword arguments. Store or sample original proposal transitions and their collection days.

2096 Set `batch` by construct the collection by iterating the shown comprehension and applying its filters. Store or sample original proposal transitions and their collection days.

2098 Set `(states, actions, behavior, reward, following, terminal_flags, sampled_days)` by construct the collection by iterating the shown comprehension and applying its filters. Store or sample original proposal transitions and their collection days.

2100 Continue the assignment begun at line 2098 with `torch.as_tensor(np.asarray(items)) for items in zip(*batch)`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

2102 Continue the assignment begun at line 2098 with `]`. Close the nested call/container at this indentation, completing that part of the same statement.

2104 Set `(_, logp, entropy)` by call model.policy with the shown positional and keyword arguments.

2106 Set `values` by call model.value with the shown positional and keyword arguments. It holds critic predictions used for baselines or TD residuals. Calculate or update the common scalar fleet-return baseline.

2108 Enter `torch.no_grad()`. The block records no gradients, keeping rollout/target calculations outside backpropagation.

2110 Set `next_values` by call model.value with the shown positional and keyword arguments. It holds the bootstrap value estimate at the following observation. Calculate or update the common scalar fleet-return baseline.

2112 Set `(actor_loss, value_loss, weights, _)` by compute capped joint replay-weighted actor and scalar-critic losses. Use the joint behavior-to-target likelihood correction for replay.

2114 Continue the assignment begun at line 2112 with `logp, behavior, values, next_values, reward.float(), terminal_flags,`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

2116 Continue the assignment begun at line 2112 with `importance_clip=importance_clip`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

2118 Continue the assignment begun at line 2112 with `)`. Close the nested call/container at this indentation, completing that part of the same statement.

2120 Set `active` by call model.active_agents with the shown positional and keyword arguments.

2122 Set `entropy_bonus` by divide (entropy * active).sum() and active.sum().clamp_min(1).

2124 Set `policy_loss` by subtract actor_loss and 0.001 * entropy_bonus. Construct or update factorized proposal policies from global observed beliefs.

2126 Test `not bool(torch.isfinite(policy_loss + value_loss))`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

Source lines 2128–2162.

```
2128                raise RuntimeError('nonfinite DCMAC/DDMAC loss')
2130            actor_optimizer.zero_grad()
2132            policy_loss.backward()
```

```
            torch.nn.utils.clip_grad_norm_(model.actors.parameters(), 0.5)
            if bool(active.any()):
                actor_optimizer.step()
            critic_optimizer.zero_grad()
            value_loss.backward()
            torch.nn.utils.clip_grad_norm_(model.critic.parameters(), 0.5)
            critic_optimizer.step()
            logs.append({'record_type': 'update', 'method': method, 'simulated_days': day,
                         'policy_loss': float(actor_loss.detach()), 'value_loss':
                         ↪  float(value_loss.detach()),
                         'importance_mean': float(weights.mean()), 'importance_max':
                         ↪  float(weights.max()),
                         'replay_size': len(replay), 'batch_size': len(batch),
                         'sampled_transition_days': sampled_days.tolist(),
                         'num_agents': model.num_agents})
        obs = next_obs
        if terminal:
```

Line-by-line explanation.

2128 Raise `RuntimeError('nonfinite DCMAC/DDMAC loss')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

2130 Clear gradients from the previous optimization step. The exact receiver and arguments are shown on this source line.

2132 Differentiate the loss and accumulate parameter gradients. The exact receiver and arguments are shown on this source line.

2134 Cap the joint gradient norm before the optimizer changes weights. The exact receiver and arguments are shown on this source line.

2136 Test `bool(active.any())`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2138 Apply accumulated gradients to the optimizer's registered parameters. The exact receiver and arguments are shown on this source line.

2140 Clear gradients from the previous optimization step. The exact receiver and arguments are shown on this source line.

2142 Differentiate the loss and accumulate parameter gradients. The exact receiver and arguments are shown on this source line.

2144 Cap the joint gradient norm before the optimizer changes weights. The exact receiver and arguments are shown on this source line.

2146 Apply accumulated gradients to the optimizer's registered parameters. The exact receiver and arguments are shown on this source line.

2148 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

2150 Continue the expression begun at line 2148 with `'policy_loss': float(actor_loss.detach()), 'value_loss': float(value_loss.detach()),`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

2152 Continue the expression begun at line 2148 with `'importance_mean': float(weights.mean()), 'importance_max': float(weights.max()),`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

2154 Continue the expression begun at line 2148 with `'replay_size': len(replay), 'batch_size': len(batch),`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

2156 Continue the expression begun at line 2148 with `'sampled_transition_days': sampled_days.tolist(),`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

2158 Continue the expression begun at line 2148 with `'num_agents': model.num_agents})`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

2160 Set `obs` by evaluate next_obs. It holds the observed belief vector, rather than hidden physical health.

2162 Test `terminal`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

Source lines 2164–2172.

```
2164            logs.append({'record_type': 'episode', 'episode': episode, 'simulated_days': day,
                ↪  **info['metrics']})
2166            episode += 1
2168            _, info = env.reset(seed=seed + episode)
2170            obs = info['central_observation']
2172    return model, logs, steps
```

Line-by-line explanation.

2164 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

2166 Add `1` to `episode`, and store the result back in that same target. The change is visible to subsequent statements in this block.

2168 Set `(_, info)` by initialize a new episode using the stated seed/options.

2170 Set `obs` by select the indexed/sliced entries of info. It holds the observed belief vector, rather than hidden physical health.

2172 Return `(model, logs, steps)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

9.99. mappo_gae

Calculate bootstrapped generalized advantage estimates while retaining a separate agent axis and masking terminal continuations.

Source lines 2176–2206.

```
2176 def mappo_gae(rewards, values, next_values, dones, gamma=0.99, lam=0.95):
2178     """Daily generalized advantages with a separate value baseline per agent."""
2180     values = np.asarray(values, dtype=np.float32)
2182     next_values = np.asarray(next_values, dtype=np.float32)
2184     rewards = np.asarray(rewards, dtype=np.float32)
2186     dones = np.asarray(dones, dtype=np.float32)
2188     if values.ndim != 2 or next_values.shape != values.shape or rewards.shape != (len(values),) or
         ↪  dones.shape != rewards.shape:
2190         raise ValueError('MAPPO rollout shapes must be time x agent, with one team reward and done per
             ↪  time')
```

```
    advantages = np.zeros_like(values)
    carry = np.zeros(values.shape[1], dtype=np.float32)
    for t in range(len(rewards) - 1, -1, -1):
        live = 1 - dones[t]
        delta = rewards[t] + gamma * next_values[t] * live - values[t]
        carry = delta + gamma * lam * live * carry
        advantages[t] = carry
    return advantages, advantages + values
```

Line-by-line explanation.

2176 Define `mappo_gae(rewards, values, next_values, dones, gamma=0.99, lam=0.95)`. Calculate bootstrapped generalized advantage estimates while retaining a separate agent axis and masking terminal continuations.

2178 Documentation text: Daily generalized advantages with a separate value baseline per agent.. This states the contract or qualification for `mappo_gae`; it does not update the simulator or model.

2180 Set `values` by call np.asarray with the shown positional and keyword arguments. It holds critic predictions used for baselines or TD residuals. Execute this step of the multi-agent observation, proposal, or learning calculation.

2182 Set `next_values` by call np.asarray with the shown positional and keyword arguments. It holds the bootstrap value estimate at the following observation. Execute this step of the multi-agent observation, proposal, or learning calculation.

2184 Set `rewards` by call np.asarray with the shown positional and keyword arguments. It holds the primitive reward sequence used for return accounting. Execute this step of the multi-agent observation, proposal, or learning calculation.

2186 Set `dones` by call np.asarray with the shown positional and keyword arguments. Execute this step of the multi-agent observation, proposal, or learning calculation.

2188 Test `values.ndim != 2 or next_values.shape != values.shape or rewards.shape != (len(values),) or (dones.shape != rewards.shape)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2190 Raise `ValueError('MAPPO rollout shapes must be time x agent, with one team reward and done per time')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

2192 Set `advantages` by call np.zeros_like with the shown positional and keyword arguments. It holds the policy credit signal with its retained aircraft/agent axis. Execute this step of the multi-agent observation, proposal, or learning calculation.

2194 Set `carry` by call np.zeros with the shown positional and keyword arguments. Execute this step of the multi-agent observation, proposal, or learning calculation.

2196 Iterate `t` over `range(len(rewards) - 1, -1, -1)`. The following indented statements run once for each selected element.

2198 Set `live` by subtract 1 and dones[t]. Execute this step of the multi-agent observation, proposal, or learning calculation.

2200 Set `delta` by subtract rewards[t] + gamma * next_values[t] * live and values[t]. Execute this step of the multi-agent observation, proposal, or learning calculation.

2202 Set `carry` by add delta and gamma * lam * live * carry. Execute this step of the multi-agent observation, proposal, or learning calculation.

2204 Set `advantages[t]` by evaluate carry. It holds the policy credit signal with its retained aircraft/agent axis. Execute this step of the multi-agent observation, proposal, or learning calculation.

2206 Return `(advantages, advantages + values)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

9.100. mappo_policy_loss

Form individual-agent probability ratios and apply PPO clipping before averaging over active decisions; this differs from clipping a fleet-joint ratio.

Source lines 2210–2222.

```
2210 def mappo_policy_loss(new_logp, old_logp, advantages, active, clip=0.2):
2212     """Clip each agent ratio BEFORE averaging; never form a fleet joint ratio."""
2214     if new_logp.ndim != 2 or not (new_logp.shape == old_logp.shape == advantages.shape ==
     ↪  active.shape):
2216         raise ValueError('MAPPO policy tensors must have identical batch x agent shapes')
2218     ratio = torch.exp(new_logp - old_logp)
2220     surrogate = torch.minimum(ratio * advantages, ratio.clamp(1 - clip, 1 + clip) * advantages)
2222     return -(surrogate * active).sum() / active.sum().clamp_min(1)
```

Line-by-line explanation.

2210 Define `mappo_policy_loss(new_logp, old_logp, advantages, active, clip=0.2)`. Form individual-agent probability ratios and apply PPO clipping before averaging over active decisions; this differs from clipping a fleet-joint ratio.

2212 Documentation text: Clip each agent ratio BEFORE averaging; never form a fleet joint ratio.. This states the contract or qualification for `mappo_policy_loss`; it does not update the simulator or model.

2214 Test `new_logp.ndim != 2 or not new_logp.shape == old_logp.shape == advantages.shape == active.shape`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2216 Raise `ValueError('MAPPO policy tensors must have identical batch x agent shapes')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

2218 Set `ratio` by exponentiate log values, including log-probability ratios. It holds the current-to-behavior probability ratio. Execute this step of the multi-agent observation, proposal, or learning calculation.

2220 Set `surrogate` by call torch.minimum with the shown positional and keyword arguments. Execute this step of the multi-agent observation, proposal, or learning calculation.

2222 Return `-(surrogate * active).sum() / active.sum().clamp_min(1)` to the caller by divide -(surrogate * active).sum() and active.sum().clamp_min(1). No further statements in this call execute.

9.101. train_mappo

Collect daily restricted-view agent requests and shared rewards, estimate per-agent advantages, and run clipped policy/value updates with separate actor and critic optimizers.

Source lines 2226–2257.

```
2226 def train_mappo(config, method='mappo_aircraft', steps=4096, seed=0, rollout_size=128):
2228     """Train decentralized proposals with centralized observed-belief value heads.
2230     Educational adaptation of MAPPO, not an exact benchmark reproduction:
2231     feed-forward actors, four full-rollout epochs, clipped values, no recurrence,
2232     reward/value normalization, learned communication, or hierarchical options.
2233     """
2235     if method not in MAPPO_METHODS or steps < 1 or rollout_size < 2:
2237         raise ValueError('valid MAPPO method, positive steps, and rollout_size >= 2 required')
2239     torch.set_num_threads(1)
2241     torch.manual_seed(seed)
2243     assignment = 'aircraft' if method == 'mappo_aircraft' else 'component'
2245     env = MultiAgentFleetEnv(config, assignment)
2247     model = MultiAgentActorCritic(env.env, assignment)
2249     actor_optimizer = torch.optim.Adam(model.actors.parameters(), lr=3e-4)
2251     critic_optimizer = torch.optim.Adam(model.critic.parameters(), lr=3e-4)
2253     _, info = env.reset(seed=seed)
2255     obs = info['central_observation']
2257     days, episode = 0, 0
```

Line-by-line explanation.

2226 Define `train_mappo(config, method='mappo_aircraft', steps=4096, seed=0, rollout_size=128)`. Collect daily restricted-view agent requests and shared rewards, estimate per-agent advantages, and run clipped policy/value updates with separate actor and critic optimizers.

2228 Documentation text: Train decentralized proposals with centralized observed-belief value heads.. This states the contract or qualification for `train_mappo`; it does not update the simulator or model.

2230 Documentation text: Educational adaptation of MAPPO, not an exact benchmark reproduction:. This states the contract or qualification for `train_mappo`; it does not update the simulator or model.

2231 Documentation text: feed-forward actors, four full-rollout epochs, clipped values, no recurrence,. This states the contract or qualification for `train_mappo`; it does not update the simulator or model.

2232 Documentation text: reward/value normalization, learned communication, or hierarchical options.. This states the contract or qualification for `train_mappo`; it does not update the simulator or model.

2233 Close the documentation string for `train_mappo`. This delimiter ends the descriptive text and performs no simulator or learning update.

2235 Test `method not in MAPPO_METHODS or steps < 1 or rollout_size < 2`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2237 Raise `ValueError('valid MAPPO method, positive steps, and rollout_size >= 2 required')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

2239 Set the cpu thread count for these small models. The exact receiver and arguments are shown on this source line.

2241 Seed neural initialization and torch sampling. The exact receiver and arguments are shown on this source line.

2243 Set `assignment` by select 'aircraft' when method == 'mappo_aircraft', otherwise 'component'. Execute this step of the multi-agent observation, proposal, or learning calculation.

2245 Set `env` by call MultiAgentFleetEnv with the shown positional and keyword arguments. It holds the environment or wrapper that owns physical transitions. Execute this step of the multi-agent observation, proposal, or learning calculation.

2247 Set `model` by call MultiAgentActorCritic with the shown positional and keyword arguments. It holds the chosen controller architecture. Execute this step of the multi-agent observation, proposal, or learning calculation.

2249 Set `actor_optimizer` by create an Adam optimizer for the specified parameter collection. It holds the optimizer restricted to actor parameters. Execute this step of the multi-agent observation, proposal, or learning calculation.

2251 Set `critic_optimizer` by create an Adam optimizer for the specified parameter collection. It holds the optimizer for the centralized value estimator. Execute this step of the multi-agent observation, proposal, or learning calculation.

2253 Set `(_, info)` by initialize a new episode using the stated seed/options. Execute this step of the multi-agent observation, proposal, or learning calculation.

2255 Set `obs` by select the indexed/sliced entries of info. It holds the observed belief vector, rather than hidden physical health. Execute this step of the multi-agent observation, proposal, or learning calculation.

2257 Set `(days, episode)` by construct the shown tuple in its stated order. Execute this step of the multi-agent observation, proposal, or learning calculation.

Source lines 2259–2293.

```
    logs = []
    while days < steps:
        data = {key: [] for key in ('obs', 'action', 'logp', 'value', 'next_value', 'reward', 'done',
        ↪  'active')}
        while len(data['obs']) < rollout_size and days < steps:
            with torch.no_grad():
                action, logp, _, values = model.act(torch.tensor(obs))
                active = model.active_agents(model.agent_inputs(obs))[0]
            _, rewards, done, truncated, info = env.step(model.action_dict(action[0]))
            next_obs = info['central_observation']
            with torch.no_grad():
                next_value = model.value(next_obs)[0].numpy()
            team_reward = rewards[model.agent_ids[0]]
            terminal = done or truncated
            items = (obs.copy(), action[0].numpy(), logp[0].numpy(), values[0].numpy(), next_value,
            ↪  team_reward, terminal, active.numpy())
            for key, item in zip(data, items):
                data[key].append(item)
            days += 1
            obs = next_obs
```

Line-by-line explanation.

2259 Set `logs` by construct the shown list in its stated order. It holds episode/update diagnostics for later inspection. Execute this step of the multi-agent observation, proposal, or learning calculation.

2261 Repeat the indented block while `days < steps` remains true; this condition controls the simulation/collection loop here.

2263 Set `data` by construct the collection by iterating the shown comprehension and applying its filters. Execute this step of the multi-agent observation, proposal, or learning calculation.

2265 Repeat the indented block while `len(data['obs']) < rollout_size and days < steps` remains true; this condition controls the simulation/collection loop here.

2267 Enter `torch.no_grad()`. The block records no gradients, keeping rollout/target calculations outside backpropagation.

2269 Set `(action, logp, _, values)` by call model.act with the shown positional and keyword arguments. Execute this step of the multi-agent observation, proposal, or learning calculation.

2271 Set `active` by select the indexed/sliced entries of model.active_agents(model.agent_inputs(obs)). Execute this step of the multi-agent observation, proposal, or learning calculation.

2273 Set `(_, rewards, done, truncated, info)` by execute this object's step with the shown arguments. Execute this step of the multi-agent observation, proposal, or learning calculation.

2275 Set `next_obs` by select the indexed/sliced entries of info. Execute this step of the multi-agent observation, proposal, or learning calculation.

2277 Enter `torch.no_grad()`. The block records no gradients, keeping rollout/target calculations outside backpropagation.

2279 Set `next_value` by expose this CPU tensor as a NumPy array. Execute this step of the multi-agent observation, proposal, or learning calculation.

2281 Set `team_reward` by select the indexed/sliced entries of rewards. Execute this step of the multi-agent observation, proposal, or learning calculation.

2283 Set `terminal` by evaluate done or truncated. It holds the episode boundary used to suppress continuation value. Execute this step of the multi-agent observation, proposal, or learning calculation.

2285 Set `items` by construct the shown tuple in its stated order. Execute this step of the multi-agent observation, proposal, or learning calculation.

2287 Iterate `(key, item)` over `zip(data, items)`. The following indented statements run once for each selected element.

2289 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

2291 Add `1` to `days`, and store the result back in that same target. The change is visible to subsequent statements in this block.

2293 Set `obs` by evaluate next_obs. It holds the observed belief vector, rather than hidden physical health. Execute this step of the multi-agent observation, proposal, or learning calculation.

Source lines 2295–2329.

```
2295            if terminal:
2297                logs.append({'record_type': 'episode', 'episode': episode, 'simulated_days': days,
                    ↪  **info['metrics']})
2299                episode += 1
2301                _, info = env.reset(seed=seed + episode)
2303                obs = info['central_observation']
2305        advantages, targets = mappo_gae(data['reward'], data['value'], data['next_value'],
            ↪  data['done'])
2307        tensors = {key: torch.as_tensor(np.asarray(value)) for key, value in data.items() if key in
            ↪  ('obs', 'action', 'logp', 'value', 'active')}
```

```
        advantages = torch.tensor(advantages)
        targets = torch.tensor(targets)
        active = tensors['active']
        counts = active.sum(0).clamp_min(1)
        mean = (advantages * active).sum(0) / counts
        variance = ((advantages - mean).square() * active).sum(0) / counts
        advantages = (advantages - mean) / torch.sqrt(variance + 1e-8) * active
        for _ in range(4):
            _, newlog, entropy, values = model.act(tensors['obs'], tensors['action'])
            policy_loss = mappo_policy_loss(newlog, tensors['logp'], advantages, active)
            entropy_bonus = (entropy * active).sum() / active.sum().clamp_min(1)
```

Line-by-line explanation.

2295 Test `terminal`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2297 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

2299 Add `1` to `episode`, and store the result back in that same target. The change is visible to subsequent statements in this block.

2301 Set `(_, info)` by initialize a new episode using the stated seed/options. Execute this step of the multi-agent observation, proposal, or learning calculation.

2303 Set `obs` by select the indexed/sliced entries of info. It holds the observed belief vector, rather than hidden physical health. Execute this step of the multi-agent observation, proposal, or learning calculation.

2305 Set `(advantages, targets)` by compute advantages with the agent axis retained. Execute this step of the multi-agent observation, proposal, or learning calculation.

2307 Set `tensors` by construct the collection by iterating the shown comprehension and applying its filters. Execute this step of the multi-agent observation, proposal, or learning calculation.

2309 Set `advantages` by call torch.tensor with the shown positional and keyword arguments. It holds the policy credit signal with its retained aircraft/agent axis. Execute this step of the multi-agent observation, proposal, or learning calculation.

2311 Set `targets` by call torch.tensor with the shown positional and keyword arguments. Execute this step of the multi-agent observation, proposal, or learning calculation.

2313 Set `active` by select the indexed/sliced entries of tensors. Execute this step of the multi-agent observation, proposal, or learning calculation.

2315 Set `counts` by call active.sum(0).clamp_min with the shown positional and keyword arguments. Execute this step of the multi-agent observation, proposal, or learning calculation.

2317 Set `mean` by divide (advantages * active).sum(0) and counts. Execute this step of the multi-agent observation, proposal, or learning calculation.

2319 Set `variance` by divide ((advantages - mean).square() * active).sum(0) and counts. Execute this step of the multi-agent observation, proposal, or learning calculation.

2321 Set `advantages` by multiply (advantages - mean) / torch.sqrt(variance + 1e-08) and active. It holds the policy credit signal with its retained aircraft/agent axis. Execute this step of the multi-agent observation, proposal, or learning calculation.

2323 Iterate `_` over `range(4)`. The following indented statements run once for each selected element.

2325 Set `(_, newlog, entropy, values)` by call model.act with the shown positional and keyword arguments. Execute this step of the multi-agent observation, proposal, or learning calculation.

2327 Set `policy_loss` by clip per-agent PPO ratios before the active-agent average. Execute this step of the multi-agent observation, proposal, or learning calculation.

2329 Set `entropy_bonus` by divide (entropy * active).sum() and active.sum().clamp_min(1). Execute this step of the multi-agent observation, proposal, or learning calculation.

Source lines 2331–2366.

```
            clipped_values = tensors['value'] + (values - tensors['value']).clamp(-0.2, 0.2)
            value_loss = torch.maximum((values - targets).square(), (clipped_values -
            ↪  targets).square()).mean()
            actor_loss = policy_loss - 0.001 * entropy_bonus
            if not torch.isfinite(actor_loss + value_loss):
                raise RuntimeError('nonfinite MAPPO loss')
            actor_optimizer.zero_grad()
            actor_loss.backward()
            torch.nn.utils.clip_grad_norm_(model.actors.parameters(), 0.5)
            if bool(active.any()):
                actor_optimizer.step()
            critic_optimizer.zero_grad()
            (0.5 * value_loss).backward()
            torch.nn.utils.clip_grad_norm_(model.critic.parameters(), 0.5)
            critic_optimizer.step()
        logs.append({'record_type': 'update', 'simulated_days': days, 'policy_loss':
        ↪  float(policy_loss.detach()),
                     'value_loss': float(value_loss.detach()), 'entropy':
                     ↪  float(entropy_bonus.detach()),
                     'num_agents': model.num_agents, 'method': method})
    return model, logs, days
```

Line-by-line explanation.

2331 Set `clipped_values` by add tensors['value'] and (values - tensors['value']).clamp(-0.2, 0.2). Execute this step of the multi-agent observation, proposal, or learning calculation.

2333 Set `value_loss` by average the selected entries. Execute this step of the multi-agent observation, proposal, or learning calculation.

2335 Set `actor_loss` by subtract policy_loss and 0.001 * entropy_bonus. It holds the objective differentiated through policy parameters. Execute this step of the multi-agent observation, proposal, or learning calculation.

2337 Test `not torch.isfinite(actor_loss + value_loss)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2339 Raise `RuntimeError('nonfinite MAPPO loss')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

2341 Clear gradients from the previous optimization step. The exact receiver and arguments are shown on this source line.

2343 Differentiate the loss and accumulate parameter gradients. The exact receiver and arguments are shown on this source line.

2345 Cap the joint gradient norm before the optimizer changes weights. The exact receiver and arguments are shown on this source line.

2348 Test `bool(active.any())`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2350 Apply accumulated gradients to the optimizer's registered parameters. The exact receiver and arguments are shown on this source line.

2352 Clear gradients from the previous optimization step. The exact receiver and arguments are shown on this source line.

2354 Differentiate the loss and accumulate parameter gradients. The exact receiver and arguments are shown on this source line.

2356 Cap the joint gradient norm before the optimizer changes weights. The exact receiver and arguments are shown on this source line.

2358 Apply accumulated gradients to the optimizer's registered parameters. The exact receiver and arguments are shown on this source line.

2360 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

2362 Continue the expression begun at line 2360 with `'value_loss': float(value_loss.detach()), 'entropy': float(entropy_bonus.detach()),`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

2364 Continue the expression begun at line 2360 with `'num_agents': model.num_agents, 'method': method})`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

2366 Return `(model, logs, days)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

9.102. Module declarations and integration

Source lines 2369–2384.

```
OPTION_NAMES = ('wait_one_day', 'targeted_maintenance', 'opportunistic_bundle', 'preventive_bundle')
HRL_METHODS = ('concurrent_options', 'planning_hierarchy', 'aggregate_targets',
               'maxq', 'option_critic', 'hierarchical_vdn', 'hrl_flat')
HRL_PPO_METHODS = ('concurrent_options', 'planning_hierarchy', 'aggregate_targets', 'hrl_flat')
HRL_BUNDLES = (1, 2, 4, 8, 16, 32, 63)
HRL_CATALOG = ((0, 0),) + tuple((b if e & 1 else 0, b if e & 2 else 0)
                                for e in (1, 2, 3) for b in HRL_BUNDLES)
HRL_BUDGETS = (0, 1, 2, 12)
```

Line-by-line explanation.

2369 Set `OPTION_NAMES` by construct the shown tuple in its stated order.

2372 Set `HRL_METHODS` by construct the shown tuple in its stated order.

2374 Continue the assignment begun at line 2372 with `'maxq', 'option_critic', 'hierarchical_vdn', 'hrl_flat')`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

2376 Set `HRL_PPO_METHODS` by construct the shown tuple in its stated order.

2378 Set `HRL_BUNDLES` by construct the shown tuple in its stated order.

2380 Set `HRL_CATALOG` by add ((0, 0),) and tuple(((b if e & 1 else 0, b if e & 2 else 0) for e in (1, 2, 3) for b in HRL_BUNDLES)).

2382 Continue the assignment begun at line 2380 with `for e in (1, 2, 3) for b in HRL_BUNDLES)`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

2384 Set `HRL_BUDGETS` by construct the shown tuple in its stated order.

9.103. hrl_mlp

Construct the common two-hidden-layer neural block used by the new hierarchy implementations.

Source lines 2388–2392.

```
2388 def hrl_mlp(width, output):
2390     return nn.Sequential(nn.Linear(width, 64), nn.Tanh(), nn.Linear(64, 64),
2392                          nn.Tanh(), nn.Linear(64, output))
```

Line-by-line explanation.

2388 Define `hrl_mlp(width, output)`. Construct the common two-hidden-layer neural block used by the new hierarchy implementations.

2390 Return `nn.Sequential(nn.Linear(width, 64), nn.Tanh(), nn.Linear(64, 64), nn.Tanh(), nn.Linear(64, output))` to the caller by compose the listed layers in execution order. No further statements in this call execute.

2392 Continue the expression begun at line 2390 with `nn.Tanh(), nn.Linear(64, output))`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

9.104. hrl_catalog_mask

Build structural feasibility masks for the 22-entry catalog using busy status, observed mandatory grounding and optional replacement-count budgets. Joint stock/resource feasibility remains the broker responsibility.

Source lines 2396–2430.

```
2396 def hrl_catalog_mask(observation, aircraft, local_size=70, mandatory=0.08, goals=None):
2398     """Structural masks only; the shared broker resolves stock/resource competition."""
2400     obs = torch.as_tensor(observation, dtype=torch.float32)
2402     if obs.ndim == 1:
2404         obs = obs.unsqueeze(0)
2406     rows = obs[:, :aircraft * local_size].reshape(-1, aircraft, local_size)
2408     busy = rows[:, :, 37] > 0.5
2410     urgent = rows[:, :, 24:36].amin(-1) <= mandatory
2412     valid = torch.ones((*busy.shape, len(HRL_CATALOG)), dtype=torch.bool, device=obs.device)
2414     valid[:, :, 0] = ~urgent | busy
2416     valid[:, :, 1:] &= ~busy.unsqueeze(-1)
2418     if goals is not None:
2420         goals = torch.as_tensor(goals, device=obs.device, dtype=torch.long)
2422         if goals.shape != busy.shape or torch.any((goals < 0) | (goals >= len(HRL_BUDGETS))):
2424             raise ValueError('one valid budget goal per aircraft required')
2426         counts = torch.tensor([sum(int(b).bit_count() for b in pair) for pair in HRL_CATALOG],
         ↪  device=obs.device)
2428         budgets = torch.tensor(HRL_BUDGETS, device=obs.device)[goals]
2430         valid &= counts <= budgets.unsqueeze(-1)
```

Line-by-line explanation.

2396 Define `hrl_catalog_mask(observation, aircraft, local_size=70, mandatory=0.08, goals=None)`. Build structural feasibility masks for the 22-entry catalog using busy status, observed mandatory grounding and optional replacement-count budgets. Joint stock/resource feasibility remains the broker responsibility.

2398 Documentation text: Structural masks only; the shared broker resolves stock/resource competition.. This states the contract or qualification for `hrl_catalog_mask`; it does not update the simulator or model.

2400 Set `obs` by convert inputs to a tensor with the requested dtype. It holds the observed belief vector, rather than hidden physical health.

2402 Test `obs.ndim == 1`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2404 Set `obs` by call obs.unsqueeze with the shown positional and keyword arguments. It holds the observed belief vector, rather than hidden physical health.

2406 Set `rows` by reinterpret the shape while retaining element order.

2408 Set `busy` by evaluate the comparison rows[:, :, 37] $>$ 0.5.

2410 Set `urgent` by evaluate the comparison rows[:, :, 24:36].amin(-1) $<=$ mandatory.

2412 Set `valid` by call torch.ones with the shown positional and keyword arguments. It holds the categorical structural-validity mask.

2414 Set `valid[:, :, 0]` by apply a bitwise union to ~urgent and busy. It holds the categorical structural-validity mask.

2416 Bitwise-AND with `~busy.unsqueeze(-1)` in `valid[:, :, 1:]`, and store the result back in that same target. This updates the categorical structural-validity mask.

2418 Test `goals is not None`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2420 Set `goals` by convert inputs to a tensor with the requested dtype. It holds per-aircraft upper-level replacement budgets.

2422 Test `goals.shape != busy.shape or torch.any((goals < 0) | (goals >= len(HRL_BUDGETS)))`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2424 Raise `ValueError('one valid budget goal per aircraft required')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

2426 Set `counts` by call torch.tensor with the shown positional and keyword arguments.

2428 Set `budgets` by select the indexed/sliced entries of torch.tensor(HRL_BUDGETS, device=obs.device).

2430 Bitwise-AND with `counts <= budgets.unsqueeze(-1)` in `valid`, and store the result back in that same target. This updates the categorical structural-validity mask.

Source lines 2432–2434.

```
2432            valid[:, :, 0] |= busy
2434        return valid
```

Line-by-line explanation.

2432 Bitwise-OR in `busy` in `valid[:, :, 0]`, and store the result back in that same target. This updates the categorical structural-validity mask.

2434 Return `valid` to the caller by evaluate valid. No further statements in this call execute.

9.105. hrl_categorical

Sample or reconstruct a masked categorical choice. Stored choices are validated so training evaluates the same action under the current parameters.

Source lines 2438–2466.

```
2438 def hrl_categorical(logits, valid, selected=None, deterministic=False):
2440     if logits.shape != valid.shape or not bool(valid.any(-1).all()):
2442         raise ValueError('categorical choices require matching masks with a feasible option')
2444     dist = Categorical(logits=logits.masked_fill(~valid, -1e9))
2446     if selected is None:
2448         selected = dist.probs.argmax(-1) if deterministic else dist.sample()
2450     else:
2452         selected = torch.as_tensor(selected, device=logits.device)
2454         if selected.shape != logits.shape[:-1] or selected.dtype not in (torch.int32, torch.int64):
2456             raise ValueError('incorrect hierarchical stored choice shape/dtype')
2458         if torch.any((selected < 0) | (selected >= logits.shape[-1])):
2460             raise ValueError('hierarchical choice outside action categories')
2462         if not bool(valid.gather(-1, selected.unsqueeze(-1)).all()):
2464             raise ValueError('hierarchical choice violates a feasibility mask')
2466     return selected, dist.log_prob(selected), dist.entropy()
```

Line-by-line explanation.

2438 Define `hrl_categorical(logits, valid, selected=None, deterministic=False)`. Sample or reconstruct a masked categorical choice. Stored choices are validated so training evaluates the same action under the current parameters.

2440 Test `logits.shape != valid.shape or not bool(valid.any(-1).all())`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2442 Raise `ValueError('categorical choices require matching masks with a feasible option')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

2444 Set `dist` by construct a discrete distribution from the supplied logits/probabilities.

2446 Test `selected is None`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2448 Set `selected` by select dist.probs.argmax(-1) when deterministic, otherwise dist.sample().

2450 Execute this alternative only when the corresponding preceding condition did not select its branch.

2452 Set `selected` by convert inputs to a tensor with the requested dtype.

2454 Test `selected.shape != logits.shape[:-1] or selected.dtype not in (torch.int32, torch.int64)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2456 Raise `ValueError('incorrect hierarchical stored choice shape/dtype')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

2458 Test `torch.any((selected < 0) | (selected >= logits.shape[-1]))`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2460 Raise `ValueError('hierarchical choice outside action categories')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

2462 Test `not bool(valid.gather(-1, selected.unsqueeze(-1)).all())`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2464 Raise `ValueError('hierarchical choice violates a feasibility mask')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

2466 Return `(selected, dist.log_prob(selected), dist.entropy())` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

9.106. AircraftOptionEnv

One-day concurrent-option wrapper. It keeps individual aircraft jobs committed, records start/completion events, and exposes decision opportunities for free aircraft without waiting for all aircraft to become free.

Source lines 2470–2479.

```
2470 class AircraftOptionEnv:
2472     """Daily fleet interface with independently committed aircraft repair options.
2474     Catalog 0 operates; other entries request one engine, the other, or both,
2475     with one component type or a full six-component bundle. Both engines use
2476     the same bundle if selected. This is a strict subset of the older action
2477     space. Every new method shares this catalog and the same fixed broker.
2478     A new decision for a free aircraft never waits for another aircraft's job.
2479     """
```

Line-by-line explanation.

2470 Define `AircraftOptionEnv` with the stated inheritance. One-day concurrent-option wrapper. It keeps individual aircraft jobs committed, records start/completion events, and exposes decision opportunities for free aircraft without waiting for all aircraft to become free.

2472 Documentation text: Daily fleet interface with independently committed aircraft repair options.. This states the contract or qualification for `AircraftOptionEnv`; it does not update the simulator or model.

2474 Documentation text: Catalog 0 operates; other entries request one engine, the other, or both,. This states the contract or qualification for `AircraftOptionEnv`; it does not update the simulator or model.

2475 Documentation text: with one component type or a full six-component bundle. Both engines use. This states the contract or qualification for `AircraftOptionEnv`; it does not update the simulator or model.

2476 Documentation text: the same bundle if selected. This is a strict subset of the older action. This states the contract or qualification for `AircraftOptionEnv`; it does not update the simulator or model.

2477 Documentation text: space. Every new method shares this catalog and the same fixed broker.. This states the contract or qualification for `AircraftOptionEnv`; it does not update the simulator or model.

2478 Documentation text: A new decision for a free aircraft never waits for another aircraft's job.. This states the contract or qualification for `AircraftOptionEnv`; it does not update the simulator or model.

2479 Close the documentation string for `AircraftOptionEnv`. This delimiter ends the descriptive text and performs no simulator or learning update.

9.107. AircraftOptionEnv.___init___

Initialize the object, register its layers or allocate its state. Construction does not itself train a policy.

Source lines 2482–2494.

```
2482     def __init__(self, config=None):
2484         self.adapter = MultiAgentFleetEnv(config, 'aircraft')
2486         self.env = self.adapter.env
2488         self.n = self.env.config.aircraft
2490         self.active_options = {}
2492         self.option_history = []
2494         self.central_observation = None
```

Line-by-line explanation.

2482 Define `__init__(self, config=None)`. This ___init___ method is part of AircraftOptionEnv.___init___; its parameters and defaults are the interface used by the calls in this section.

2484 Set `self.adapter` by call MultiAgentFleetEnv with the shown positional and keyword arguments.

2486 Set `self.env` by evaluate self.adapter.env. It holds the environment or wrapper that owns physical transitions.

2488 Set `self.n` by evaluate self.env.config.aircraft.

2490 Set `self.active_options` by construct the shown dict in its stated order.

2492 Set `self.option_history` by construct the shown list in its stated order.

2494 Set `self.central_observation` by use the literal None.

9.108. AircraftOptionEnv.reset

Start an episode, initialize random streams/state, and return the initial observation and diagnostic information.

Source lines 2497–2509.

```
2497    def reset(self, *, seed=None, options=None):
2499        _, info = self.adapter.reset(seed=seed, options=options)
2501        self.central_observation = info['central_observation']
2503        self.active_options.clear()
2505        self.option_history.clear()
2507        self.last_info = info
2509        return self.central_observation.copy(), info
```

Line-by-line explanation.

2497 Define `reset(self, *, seed=None, options=None)`. This reset method is part of AircraftOptionEnv.reset; its parameters and defaults are the interface used by the calls in this section.

2499 Set `(_, info)` by initialize a new episode using the stated seed/options.

2501 Set `self.central_observation` by select the indexed/sliced entries of info.

2503 Call self.active_options.clear with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

2505 Call self.option_history.clear with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

2507 Set `self.last_info` by evaluate info.

2509 Return `(self.central_observation.copy(), info)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

9.109. AircraftOptionEnv.decision_mask

Return one Boolean per aircraft indicating that it is free to receive a new plan. This permission does not certify joint resource feasibility.

Source lines 2512–2520.

```
2512    def decision_mask(self):
2514        if self.central_observation is None:
2516            raise RuntimeError('reset before reading aircraft options')
2518        rows, _ = self.env.split_observation(self.central_observation)
2520        return rows[:, 37] <= 0.5
```

Line-by-line explanation.

2512 Define `decision_mask(self)`. Return one Boolean per aircraft indicating that it is free to receive a new plan. This permission does not certify joint resource feasibility.

2514 Test `self.central_observation is None`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2516 Raise `RuntimeError('reset before reading aircraft options')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

2518 Set `(rows, _)` by call self.env.split_observation with the shown positional and keyword arguments.

2520 Return `rows[:, 37] <= 0.5` to the caller by evaluate the comparison rows[:, 37] <= 0.5. No further statements in this call execute.

9.110. AircraftOptionEnv.step

Apply the action through this class's transition/adapter contract and return the resulting observation, reward and episode flags.

Source lines 2523–2557.

```
2523     def step(self, choices):
2525         choices = np.asarray(choices)
2527         if choices.shape != (self.n,) or not np.issubdtype(choices.dtype, np.integer):
2529             raise ValueError('one integer catalog action per aircraft required')
2531         if np.any((choices < 0) | (choices >= len(HRL_CATALOG))):
2533             raise ValueError('unknown maintenance catalog action')
2535         free = self.decision_mask()
2537         if np.any((~free) & (choices != 0)):
2539             raise ValueError('committed aircraft options accept no new planning request')
2541         previous = {i: job.plan for i, job in self.env.jobs.items()}
2543         day = self.env.day
2545         proposals = {plate: np.asarray(HRL_CATALOG[int(choices[i])], dtype=np.int64)
2547                      for i, plate in enumerate(self.env.aircraft_plates)}
2549         _, rewards, done, truncated, info = self.adapter.step(proposals)
2551         for i, plan in previous.items():
2553             if i in self.env.jobs and self.env.jobs[i].plan != plan:
2555                 raise RuntimeError('a committed aircraft plan was changed')
2557         started, finished = [], []
```

Line-by-line explanation.

2523 Define `step(self, choices)`. This step method is part of AircraftOptionEnv.step; its parameters and defaults are the interface used by the calls in this section.

2525 Set `choices` by call np.asarray with the shown positional and keyword arguments.

2527 Test `choices.shape != (self.n,) or not np.issubdtype(choices.dtype, np.integer)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2529 Raise `ValueError('one integer catalog action per aircraft required')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

2531 Test `np.any((choices < 0) | (choices >= len(HRL_CATALOG)))`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2533 Raise `ValueError('unknown maintenance catalog action')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

2535 Set `free` by call self.decision_mask with the shown positional and keyword arguments. It holds the current per-aircraft permission to issue a new plan.

2537 Test `np.any(~free & (choices != 0))`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2539 Raise `ValueError('committed aircraft options accept no new planning request')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

2541 Set `previous` by construct the collection by iterating the shown comprehension and applying its filters.

2543 Set `day` by evaluate self.env.day.

2545 Set `proposals` by construct the collection by iterating the shown comprehension and applying its filters.

2547 Continue the assignment begun at line 2545 with `for i, plate in enumerate(self.env.aircraft_plates)}`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

2549 Set `(_, rewards, done, truncated, info)` by execute this object's step with the shown arguments.

2551 Iterate `(i, plan)` over `previous.items()`. The following indented statements run once for each selected element.

2553 Test `i in self.env.jobs and self.env.jobs[i].plan != plan`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2555 Raise `RuntimeError('a committed aircraft plan was changed')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

2557 Set `(started, finished)` by construct the shown tuple in its stated order.

Source lines 2559–2593.

```
2559        for i, job in self.env.jobs.items():
2561            if i not in previous:
2563                record = {'aircraft': i, 'aircraft_plate': self.env.aircraft_plates[i],
2565                          'start_day': day, 'catalog_action': int(choices[i]),
2567                          'planned_duration': job.total_duration}
2569                self.active_options[i] = record
2571                started.append(dict(record))
2573        for i in info['completed']:
2575            record = dict(self.active_options.pop(i))
2577            record.update(end_day=self.env.day, duration=self.env.day - record['start_day'])
2579            self.option_history.append(record)
2581            finished.append(record)
2583        self.central_observation = info['central_observation']
2585        info.update(aircraft_option_started=started, aircraft_option_completed=finished,
2587                    decision_mask_before=free.tolist(),
                        ↪  active_aircraft_options=sorted(self.active_options))
2589        info['metrics'].update(option_starts=len(self.option_history) + len(self.active_options),
2591                               option_completions=len(self.option_history))
2593        self.last_info = info
```

Line-by-line explanation.

2559 Iterate `(i, job)` over `self.env.jobs.items()`. The following indented statements run once for each selected element.

2561 Test `i not in previous`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2563 Set `record` by construct the shown dict in its stated order.

2565 Continue the assignment begun at line 2563 with `'start_day': day, 'catalog_action': int(choices[i]),`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

2567 Continue the assignment begun at line 2563 with `'planned_duration': job.total_duration}`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

2569 Set `self.active_options[i]` by evaluate record.

2571 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

2573 Iterate `i` over `info['completed']`. The following indented statements run once for each selected element.

2575 Set `record` by call dict with the shown positional and keyword arguments.

2577 Call record.update with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

2579 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

2581 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

2583 Set `self.central_observation` by select the indexed/sliced entries of info.

2585 Call info.update with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

2587 Continue the expression begun at line 2585 with `decision_mask_before=free.tolist(), active_aircraft_options=sorted(self.active_options))`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

2589 Call info['metrics'].update with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

2591 Continue the expression begun at line 2589 with `option_completions=len(self.option_history))`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

2593 Set `self.last_info` by evaluate info.

Source lines 2595–2595.

```
2595         return self.central_observation.copy(), rewards[self.env.aircraft_plates[0]], done, truncated,
             ↪  info
```

Line-by-line explanation.

2595 Return `(self.central_observation.copy(), rewards[self.env.aircraft_plates[0]], done, truncated, info)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

9.111. AircraftPlanningHierarchy

Shared stage policies for concurrent options, three-stage planning, aggregate targets and the catalog flat comparator. It produces per-aircraft decision likelihoods and a centralized value vector. The catalog request is still executed by the fixed broker.

Source lines 2599–2606.

```
2599 class AircraftPlanningHierarchy(nn.Module):
2601     """Independent aircraft options with learned planning stages or explicit targets.
2603     Ground/engine/bundle selections are computational planning stages, not
2604     three elapsed physical epochs. Temporal abstraction is the committed job.
2605     All actors use observed global context; this is not local-information CTDE.
2606     """
```

Line-by-line explanation.

2599 Define `AircraftPlanningHierarchy` with the stated inheritance. Shared stage policies for concurrent options, three-stage planning, aggregate targets and the catalog flat comparator. It produces per-aircraft decision likelihoods and a centralized value vector. The catalog request is still executed by the fixed broker.

2601 Documentation text: Independent aircraft options with learned planning stages or explicit targets.. This states the contract or qualification for `AircraftPlanningHierarchy`; it does not update the simulator or model.

2603 Documentation text: Ground/engine/bundle selections are computational planning stages, not. This states the contract or qualification for `AircraftPlanningHierarchy`; it does not update the simulator or model.

2604 Documentation text: three elapsed physical epochs. Temporal abstraction is the committed job.. This states the contract or qualification for `AircraftPlanningHierarchy`; it does not update the simulator or model.

2605 Documentation text: All actors use observed global context; this is not local-information CTDE.. This states the contract or qualification for `AircraftPlanningHierarchy`; it does not update the simulator or model.

2606 Close the documentation string for `AircraftPlanningHierarchy`. This delimiter ends the descriptive text and performs no simulator or learning update.

9.112. AircraftPlanningHierarchy.__init__

Initialize the object, register its layers or allocate its state. Construction does not itself train a policy.

Source lines 2609–2643.

```
2609     def __init__(self, env, mode='planning_hierarchy'):
2611         super().__init__()
2613         if mode not in HRL_PPO_METHODS:
2615             raise ValueError('unknown aircraft planning hierarchy')
2617         self.mode = mode
2619         self.n = env.config.aircraft
2621         self.local = env.local_size
2623         self.width = env.observation_space.shape[0]
2625         self.mandatory = env.config.mandatory_health
2627         features = self.width + self.local
2629         if mode == 'hrl_flat':
2631             self.flat_policy = hrl_mlp(features, len(HRL_CATALOG))
2633         else:
2635             self.manager = hrl_mlp(features, 4 if mode == 'aggregate_targets' else 2)
2637             if mode == 'planning_hierarchy':
2639                 self.engine_policy = hrl_mlp(features + 2, 3)
2641                 self.component_policy = hrl_mlp(features + 5, 7)
2643             if mode == 'aggregate_targets':
```

Line-by-line explanation.

2609 Define `__init__(self, env, mode='planning_hierarchy')`. This ___init___ method is part of AircraftPlanningHierarchy.___init___; its parameters and defaults are the interface used by the calls in this section.

2611 Call super().___init___ with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

2613 Test `mode not in HRL_PPO_METHODS`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2615 Raise `ValueError('unknown aircraft planning hierarchy')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

2617 Set `self.mode` by evaluate mode.

2619 Set `self.n` by evaluate env.config.aircraft.

2621 Set `self.local` by evaluate env.local_size.

2623 Set `self.width` by select the indexed/sliced entries of env.observation_space.shape.

2625 Set `self.mandatory` by evaluate env.config.mandatory_health.

2627 Set `features` by add self.width and self.local.

2629 Test `mode == 'hrl_flat'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2631 Set `self.flat_policy` by call hrl_mlp with the shown positional and keyword arguments.

2633 Execute this alternative only when the corresponding preceding condition did not select its branch.

2635 Set `self.manager` by call hrl_mlp with the shown positional and keyword arguments.

2637 Test `mode == 'planning_hierarchy'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2639 Set `self.engine_policy` by call hrl_mlp with the shown positional and keyword arguments.

2641 Set `self.component_policy` by call hrl_mlp with the shown positional and keyword arguments.

2643 Test `mode == 'aggregate_targets'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

Source lines 2645–2647.

```
2645                 self.worker = hrl_mlp(features + 4, len(HRL_CATALOG))
2647         self.critic = hrl_mlp(self.width, self.n)
```

Line-by-line explanation.

2645 Set `self.worker` by call hrl_mlp with the shown positional and keyword arguments.

2647 Set `self.critic` by call hrl_mlp with the shown positional and keyword arguments.

9.113. AircraftPlanningHierarchy.features

Assemble the local/public or global/local features required by this hierarchy, retaining the aircraft axis and the declared information boundary.

Source lines 2650–2664.

```
2650     def features(self, observation):
2652         obs = torch.as_tensor(observation, dtype=torch.float32)
2654         if obs.ndim == 1:
2656             obs = obs.unsqueeze(0)
2658         if obs.ndim != 2 or obs.shape[1] != self.width:
2660             raise ValueError('incorrect hierarchical fleet observation')
2662         local = obs[:, :self.n * self.local].reshape(-1, self.n, self.local)
2664         return obs, local, torch.cat([obs[:, None].expand(-1, self.n, -1), local], -1)
```

Line-by-line explanation.

2650 Define `features(self, observation)`. Assemble the local/public or global/local features required by this hierarchy, retaining the aircraft axis and the declared information boundary.

2652 Set `obs` by convert inputs to a tensor with the requested dtype. It holds the observed belief vector, rather than hidden physical health.

2654 Test `obs.ndim == 1`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2656 Set `obs` by call obs.unsqueeze with the shown positional and keyword arguments. It holds the observed belief vector, rather than hidden physical health.

2658 Test `obs.ndim != 2 or obs.shape[1] != self.width`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2660 Raise `ValueError('incorrect hierarchical fleet observation')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

2662 Set `local` by reinterpret the shape while retaining element order.

2664 Return `(obs, local, torch.cat([obs[:, None].expand(-1, self.n, -1), local], -1))` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

9.114. AircraftPlanningHierarchy.value

Evaluate the centralized state-value or action-value path using the input contract of this class.

Source lines 2667–2669.

```
2667     def value(self, observation):
2669         return self.critic(self.features(observation)[0])
```

Line-by-line explanation.

2667 Define `value(self, observation)`. This value method is part of AircraftPlanningHierarchy.value; its parameters and defaults are the interface used by the calls in this section.

2669 Return `self.critic(self.features(observation)[0])` to the caller by call self.critic with the shown positional and keyword arguments. No further statements in this call execute.

9.115. AircraftPlanningHierarchy.actor_parameters

Expose only the parameters updated by the actor optimizer, excluding the separately optimized value estimator.

Source lines 2672–2674.

```
2672     def actor_parameters(self):
2674         return [p for name, p in self.named_parameters() if not name.startswith('critic.')]
```

Line-by-line explanation.

2672 Define `actor_parameters(self)`. This actor_parameters method is part of AircraftPlanningHierarchy.actor_parameters; its parameters and defaults are the interface used by the calls in this section.

2674 Return `[p for name, p in self.named_parameters() if not name.startswith('critic.')]` to the caller by construct the collection by iterating the shown comprehension and applying its filters. No further statements in this call execute.

9.116. AircraftPlanningHierarchy.act

Sample or reconstruct actions and compute their log probabilities, entropy and values as defined by this policy family.

Source lines 2677–2711.

```
2677     def act(self, observation, actions=None, deterministic=False):
2679         obs, local, features = self.features(observation)
2681         if actions is not None:
2683             actions = torch.as_tensor(actions)
2685             if actions.shape != (len(obs), self.n, 3) or actions.dtype not in (torch.int32,
                 ↪  torch.int64):
2687                 raise ValueError('hierarchical actions must be integer batch x aircraft x 3')
2689         mask = hrl_catalog_mask(obs, self.n, self.local, self.mandatory)
2691         busy = local[:, :, 37] > 0.5
2693         urgent = local[:, :, 24:36].amin(-1) <= self.mandatory
2695         zero = torch.zeros_like(busy, dtype=torch.long)
2697         if self.mode == 'hrl_flat':
2699             choice, logp, entropy = hrl_categorical(self.flat_policy(features), mask,
2701                 None if actions is None else actions[:, :, 0], deterministic)
2703             stored = torch.stack([choice, zero, zero], -1)
2705         else:
2707             count = 4 if self.mode == 'aggregate_targets' else 2
2709             valid = (~busy).unsqueeze(-1).expand(-1, -1, count).clone()
2711             valid[:, :, 0] = ~urgent | busy
```

Line-by-line explanation.

2677 Define `act(self, observation, actions=None, deterministic=False)`. This act method is part of AircraftPlanningHierarchy.act; its parameters and defaults are the interface used by the calls in this section.

2679 Set `(obs, local, features)` by call self.features with the shown positional and keyword arguments.

2681 Test `actions is not None`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2683 Set `actions` by convert inputs to a tensor with the requested dtype.

2685 Test `actions.shape != (len(obs), self.n, 3) or actions.dtype not in (torch.int32, torch.int64)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2687 Raise `ValueError('hierarchical actions must be integer batch x aircraft x 3')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

2689 Set `mask` by compute permitted catalog entries under commitment and optional budget constraints.

2691 Set `busy` by evaluate the comparison local[:, :, 37] > 0.5.

2693 Set `urgent` by evaluate the comparison local[:, :, 24:36].amin(-1) <= self.mandatory.

2695 Set `zero` by call torch.zeros_like with the shown positional and keyword arguments.

2697 Test `self.mode == 'hrl_flat'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2699 Set `(choice, logp, entropy)` by sample or score a structurally masked categorical decision.

2701 Continue the assignment begun at line 2699 with `None if actions is None else actions[:, :, 0], deterministic)`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

2703 Set `stored` by join equal-shaped tensors along the specified new axis.

2705 Execute this alternative only when the corresponding preceding condition did not select its branch.

2707 Set `count` by select 4 when self.mode == 'aggregate_targets', otherwise 2.

2709 Set `valid` by call (~busy).unsqueeze(-1).expand(-1, -1, count).clone with the shown positional and keyword arguments. It holds the categorical structural-validity mask.

2711 Set `valid[:, :, 0]` by apply a bitwise union to ~urgent and busy. It holds the categorical structural-validity mask.

Source lines 2713–2747.

```
2713            goal, logp, entropy = hrl_categorical(self.manager(features), valid,
2715                None if actions is None else actions[:, :, 0], deterministic)
2717            if self.mode == 'planning_hierarchy':
2719                active = (goal == 1) & ~busy
2721                engine_valid = active.unsqueeze(-1).expand(-1, -1, 3).clone()
2723                engine_valid[:, :, 0] |= ~active
2725                engine, lp, ent = hrl_categorical(self.engine_policy(torch.cat([
2727                    features, nn.functional.one_hot(goal, 2).float()], -1)), engine_valid,
2729                    None if actions is None else actions[:, :, 1], deterministic)
```

```
                logp, entropy = logp + lp, entropy + ent
                bundle_valid = active.unsqueeze(-1).expand(-1, -1, 7).clone()
                bundle_valid[:, :, 0] |= ~active
                bundle, lp, ent = hrl_categorical(self.component_policy(torch.cat([
                    features, nn.functional.one_hot(goal, 2).float(),
                    nn.functional.one_hot(engine, 3).float()], -1)), bundle_valid,
                    None if actions is None else actions[:, :, 2], deterministic)
                logp, entropy = logp + lp, entropy + ent
                stored = torch.stack([goal, engine, bundle], -1)
```

Line-by-line explanation.

2713 Set `(goal, logp, entropy)` by sample or score a structurally masked categorical decision.

2715 Continue the assignment begun at line 2713 with `None if actions is None else actions[:, :, 0], deterministic)`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

2717 Test `self.mode == 'planning_hierarchy'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2719 Set `active` by apply a bitwise intersection to goal == 1 and ~busy.

2721 Set `engine_valid` by call active.unsqueeze(-1).expand(-1, -1, 3).clone with the shown positional and keyword arguments.

2723 Bitwise-OR in `~active` in `engine_valid[:, :, 0]`, and store the result back in that same target. The change is visible to subsequent statements in this block.

2725 Set `(engine, lp, ent)` by sample or score a structurally masked categorical decision.

2727 Continue the assignment begun at line 2725 with `features, nn.functional.one_hot(goal, 2).float()], -1)), engine_valid,`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

2729 Continue the assignment begun at line 2725 with `None if actions is None else actions[:, :, 1], deterministic)`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

2731 Set `(logp, entropy)` by construct the shown tuple in its stated order.

2733 Set `bundle_valid` by call active.unsqueeze(-1).expand(-1, -1, 7).clone with the shown positional and keyword arguments.

2735 Bitwise-OR in `~active` in `bundle_valid[:, :, 0]`, and store the result back in that same target. The change is visible to subsequent statements in this block.

2737 Set `(bundle, lp, ent)` by sample or score a structurally masked categorical decision.

2739 Continue the assignment begun at line 2737 with `features, nn.functional.one_hot(goal, 2).float(),`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

2741 Continue the assignment begun at line 2737 with `nn.functional.one_hot(engine, 3).float()], -1)), bundle_valid,`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

2743 Continue the assignment begun at line 2737 with `None if actions is None else actions[:, :, 2], deterministic)`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

2745 Set `(logp, entropy)` by construct the shown tuple in its stated order.

2747 Set `stored` by join equal-shaped tensors along the specified new axis.

Source lines 2749–2774.

```
2749            elif self.mode == 'aggregate_targets':
2750                # A positive goal is a maximum replacement count, not a guaranteed improvement.
2752                worker_valid = hrl_catalog_mask(obs, self.n, self.local, self.mandatory, goal)
2754                worker_valid[:, :, 0] = (goal == 0) | busy
2756                choice, lp, ent = hrl_categorical(self.worker(torch.cat([
2758                    features, nn.functional.one_hot(goal, 4).float()], -1)), worker_valid,
2760                    None if actions is None else actions[:, :, 1], deterministic)
2762                logp, entropy = logp + lp, entropy + ent
2764                stored = torch.stack([goal, choice, zero], -1)
2766            else:
2768                stored = torch.stack([goal, zero, zero], -1)
2770        if actions is not None and not torch.equal(stored, actions):
2772            raise ValueError('unused hierarchical action fields must be zero')
2774        return stored, logp, entropy, self.value(obs)
```

Line-by-line explanation.

2749 Test `self.mode == 'aggregate_targets'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2750 Source comment: A positive goal is a maximum replacement count, not a guaranteed improvement. This line documents the following code and does not execute.

2752 Set `worker_valid` by compute permitted catalog entries under commitment and optional budget constraints.

2754 Set `worker_valid[:, :, 0]` by apply a bitwise union to goal == 0 and busy.

2756 Set `(choice, lp, ent)` by sample or score a structurally masked categorical decision.

2758 Continue the assignment begun at line 2756 with `features, nn.functional.one_hot(goal, 4).float()], -1)), worker_valid,`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

2760 Continue the assignment begun at line 2756 with `None if actions is None else actions[:, :, 1], deterministic)`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

2762 Set `(logp, entropy)` by construct the shown tuple in its stated order.

2764 Set `stored` by join equal-shaped tensors along the specified new axis.

2766 Execute this alternative only when the corresponding preceding condition did not select its branch.

2768 Set `stored` by join equal-shaped tensors along the specified new axis.

2770 Test `actions is not None and (not torch.equal(stored, actions))`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2772 Raise `ValueError('unused hierarchical action fields must be zero')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

2774 Return `(stored, logp, entropy, self.value(obs))` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

9.117. AircraftPlanningHierarchy.catalog_actions

Convert the stored hierarchical planning choices into the catalog indices consumed by the common option wrapper.

Source lines 2777–2811.

```
2777     def catalog_actions(self, observation, actions):
2779         actions = torch.as_tensor(actions)
2781         if actions.ndim == 3:
2783             if actions.shape[0] != 1:
2785                 raise ValueError('decode one fleet action at a time')
2787             actions = actions[0]
2789         if actions.shape != (self.n, 3):
2791             raise ValueError('incorrect hierarchical action for decoding')
2793         if self.mode == 'hrl_flat':
2795             return actions[:, 0].numpy().copy()
2797         if self.mode == 'aggregate_targets':
2799             return actions[:, 1].numpy().copy()
2801         if self.mode == 'planning_hierarchy':
2803             return torch.where(actions[:, 0] != 0, 1 + actions[:, 1] * 7 + actions[:, 2], 0).numpy()
2805         _, local, _ = self.features(observation)
2807         means = local[0, :, :12].reshape(self.n, 2, 6)
2809         engine = means.amin(-1).argmin(-1)
2811         selected = means[torch.arange(self.n), engine]
```

Line-by-line explanation.

2777 Define `catalog_actions(self, observation, actions)`. This catalog_actions method is part of AircraftPlanningHierarchy.catalog_actions; its parameters and defaults are the interface used by the calls in this section.

2779 Set `actions` by convert inputs to a tensor with the requested dtype.

2781 Test `actions.ndim == 3`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2783 Test `actions.shape[0] != 1`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2785 Raise `ValueError('decode one fleet action at a time')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

2787 Set `actions` by select the indexed/sliced entries of actions.

2789 Test `actions.shape != (self.n, 3)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2791 Raise `ValueError('incorrect hierarchical action for decoding')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

2793 Test `self.mode == 'hrl_flat'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2795 Return `actions[:, 0].numpy().copy()` to the caller by make a separate array/dictionary copy. No further statements in this call execute.

2797 Test `self.mode == 'aggregate_targets'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2799 Return `actions[:, 1].numpy().copy()` to the caller by make a separate array/dictionary copy. No further statements in this call execute.

2801 Test `self.mode == 'planning_hierarchy'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2803 Return `torch.where(actions[:, 0] != 0, 1 + actions[:, 1] * 7 + actions[:, 2], 0).numpy()` to the caller by expose this CPU tensor as a NumPy array. No further statements in this call execute.

2805 Set `(_, local, _)` by call self.features with the shown positional and keyword arguments.

2807 Set `means` by reinterpret the shape while retaining element order.

2809 Set `engine` by call means.amin(-1).argmin with the shown positional and keyword arguments.

2811 Set `selected` by select the indexed/sliced entries of means.

Source lines 2813–2817.

```
2813         component = selected.argmin(-1)
2815         bundle = torch.where((selected < .75).sum(-1) >= 3, 6, component)
2817         return torch.where(actions[:, 0] != 0, 1 + engine * 7 + bundle, 0).numpy()
```

Line-by-line explanation.

2813 Set `component` by call selected.argmin with the shown positional and keyword arguments.

2815 Set `bundle` by select values elementwise using the Boolean condition.

2817 Return `torch.where(actions[:, 0] != 0, 1 + engine * 7 + bundle, 0).numpy()` to the caller by expose this CPU tensor as a NumPy array. No further statements in this call execute.

9.118. hrl_option_target

Compute the discounted sum of primitive daily rewards and a bootstrap discounted by the actual option duration; a genuine terminal state has no bootstrap.

Source lines 2821–2827.

```
2821 def hrl_option_target(rewards, next_value, terminal, gamma=.99):
2823     """Discount each primitive day once, including bootstrap over the actual duration."""
2825     return sum(gamma ** t * float(r) for t, r in enumerate(rewards)) + (
2827         0. if terminal else gamma ** len(rewards) * float(next_value))
```

Line-by-line explanation.

2821 Define `hrl_option_target(rewards, next_value, terminal, gamma=.99)`. Compute the discounted sum of primitive daily rewards and a bootstrap discounted by the actual option duration; a genuine terminal state has no bootstrap.

2823 Documentation text: Discount each primitive day once, including bootstrap over the actual duration.. This states the contract or qualification for `hrl_option_target`; it does not update the simulator or model.

2825 Return `sum((gamma ** t * float(r) for t, r in enumerate(rewards))) + (0.0 if terminal else gamma ** len(rewards) * float(next_value))` to the caller by add sum((gamma ** t * float(r) for t, r in enumerate(rewards))) and 0.0 if terminal else gamma ** len(rewards) * float(next_value). No further statements in this call execute.

2827 Continue the expression begun at line 2825 with `0. if terminal else gamma ** len(rewards) * float(next_value))`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

9.119. train_aircraft_hrl

Keep a pending decision record per aircraft, append daily team rewards until that decision completes, and update stage actors from duration-aware decision targets. The critic also learns from daily targets. A budget cutoff bootstraps unfinished work without canceling it.

Source lines 2831–2865.

```
2831 def train_aircraft_hrl(config, method='planning_hierarchy', steps=4096, seed=0, rollout_size=128):
2833     if method not in HRL_PPO_METHODS or steps < 1 or rollout_size < 2:
2835         raise ValueError('registered hierarchy, positive steps, rollout_size >= 2 required')
2837     torch.set_num_threads(1)
2839     torch.manual_seed(seed)
2841     env = AircraftOptionEnv(config)
2843     model = AircraftPlanningHierarchy(env.env, method)
2845     actors = model.actor_parameters()
2847     actor_optimizer = torch.optim.Adam(actors, lr=3e-4)
2849     critic_optimizer = torch.optim.Adam(model.critic.parameters(), lr=3e-4)
2851     obs, _ = env.reset(seed=seed)
2853     pending, ready, daily, logs = {}, [], [], []
2855     episode = 0
2857     for day in range(1, steps + 1):
2859         with torch.no_grad():
2861             action, logp, _, values = model.act(obs)
2863         free = env.decision_mask()
2865         for i in np.flatnonzero(free):
```

Line-by-line explanation.

2831 Define `train_aircraft_hrl(config, method='planning_hierarchy', steps=4096, seed=0, rollout_size=128)`. Keep a pending decision record per aircraft, append daily team rewards until that decision completes, and update stage actors from duration-aware decision targets. The critic also learns from daily targets. A budget cutoff bootstraps unfinished work without canceling it.

2833 Test `method not in HRL_PPO_METHODS or steps < 1 or rollout_size < 2`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2835 Raise `ValueError('registered hierarchy, positive steps, rollout_size >= 2 required')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

2837 Set the cpu thread count for these small models. The exact receiver and arguments are shown on this source line.

2839 Seed neural initialization and torch sampling. The exact receiver and arguments are shown on this source line.

2841 Set `env` by call AircraftOptionEnv with the shown positional and keyword arguments. It holds the environment or wrapper that owns physical transitions.

2843 Set `model` by call AircraftPlanningHierarchy with the shown positional and keyword arguments. It holds the chosen controller architecture.

2845 Set `actors` by call model.actor_parameters with the shown positional and keyword arguments.

2847 Set `actor_optimizer` by create an Adam optimizer for the specified parameter collection. It holds the optimizer restricted to actor parameters.

2849 Set `critic_optimizer` by create an Adam optimizer for the specified parameter collection. It holds the optimizer for the centralized value estimator.

2851 Set `(obs, _)` by initialize a new episode using the stated seed/options.

2853 Set `(pending, ready, daily, logs)` by construct the shown tuple in its stated order.

2855 Set `episode` by use the literal 0.

2857 Iterate `day` over `range(1, steps + 1)`. The following indented statements run once for each selected element.

2859 Enter `torch.no_grad()`. The block records no gradients, keeping rollout/target calculations outside backpropagation.

2861 Set `(action, logp, _, values)` by call model.act with the shown positional and keyword arguments.

2863 Set `free` by call env.decision_mask with the shown positional and keyword arguments. It holds the current per-aircraft permission to issue a new plan.

2865 Iterate `i` over `np.flatnonzero(free)`. The following indented statements run once for each selected element.

Source lines 2867–2900.

```
            pending[int(i)] = {'obs': obs.copy(), 'action': action[0].numpy().copy(),
                'logp': float(logp[0, i]), 'value': float(values[0, i]), 'aircraft': int(i),
                'rewards': []}
        following, reward, done, truncated, info = env.step(model.catalog_actions(obs, action))
        terminal = done or truncated
        with torch.no_grad():
            next_values = model.value(following)[0].numpy()
        daily.append((obs.copy(), reward + .99 * (not terminal) * next_values))
        for i in list(pending):
            pending[i]['rewards'].append(reward)
            completed = terminal or bool(env.decision_mask()[i])
            # A budget cutoff bootstraps an unfinished option; it does not cancel its job.
            if completed or day == steps:
                item = pending.pop(i)
                item.update(target=hrl_option_target(item['rewards'], next_values[i], terminal),
```

```
                            duration=len(item['rewards']), option_complete=completed)
                ready.append(item)
        obs = following
        if terminal:
```

Line-by-line explanation.

2867 Set `pending[int(i)]` by construct the shown dict in its stated order. It holds unfinished per-aircraft decision records spanning primitive days.

2869 Continue the assignment begun at line 2867 with `'logp': float(logp[0, i]), 'value': float(values[0, i]), 'aircraft': int(i), 'rewards': []}`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

2871 Set `(following, reward, done, truncated, info)` by execute this object's step with the shown arguments.

2873 Set `terminal` by evaluate done or truncated. It holds the episode boundary used to suppress continuation value.

2875 Enter `torch.no_grad()`. The block records no gradients, keeping rollout/target calculations outside backpropagation.

2877 Set `next_values` by expose this CPU tensor as a NumPy array. It holds the bootstrap value estimate at the following observation.

2879 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

2881 Iterate `i` over `list(pending)`. The following indented statements run once for each selected element.

2883 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

2885 Set `completed` by evaluate terminal or bool(env.decision_mask()[i]).

2886 Source comment: A budget cutoff bootstraps an unfinished option; it does not cancel its job. This line documents the following code and does not execute.

2888 Test `completed or day == steps`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2890 Set `item` by remove and return the selected stored record.

2892 Call item.update with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

2894 Continue the expression begun at line 2892 with `duration=len(item['rewards']), option_complete=completed)`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

2896 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

2898 Set `obs` by evaluate following. It holds the observed belief vector, rather than hidden physical health.

2900 Test `terminal`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

Source lines 2902–2936.

```
            logs.append({'record_type': 'episode', 'simulated_days': day, **info['metrics']})
            episode += 1
            obs, _ = env.reset(seed=seed + episode)
        if day % rollout_size == 0 or day == steps:
            if ready:
                states = torch.tensor(np.stack([r['obs'] for r in ready]))
                actions = torch.tensor(np.stack([r['action'] for r in ready]))
                ids = torch.tensor([r['aircraft'] for r in ready])
                batch_ids = torch.arange(len(ready))
                old_logp = torch.tensor([r['logp'] for r in ready])
                advantages = torch.tensor([r['target'] - r['value'] for r in ready])
                if len(ready) > 1:
                    advantages = (advantages - advantages.mean()) /
                    ↪  advantages.std(unbiased=False).clamp_min(1e-6)
                for _ in range(4):
                    _, new_logp, entropy, _ = model.act(states, actions)
                    ratio = torch.exp(new_logp[batch_ids, ids] - old_logp)
                    loss = -torch.minimum(ratio * advantages, ratio.clamp(.8, 1.2) * advantages).mean()
                    loss -= .001 * entropy[batch_ids, ids].mean()
```

Line-by-line explanation.

2902 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

2904 Add `1` to `episode`, and store the result back in that same target. The change is visible to subsequent statements in this block.

2906 Set `(obs, _)` by initialize a new episode using the stated seed/options.

2908 Test `day % rollout_size == 0 or day == steps`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2910 Test `ready`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2912 Set `states` by call torch.tensor with the shown positional and keyword arguments.

2914 Set `actions` by call torch.tensor with the shown positional and keyword arguments.

2916 Set `ids` by call torch.tensor with the shown positional and keyword arguments.

2918 Set `batch_ids` by call torch.arange with the shown positional and keyword arguments.

2920 Set `old_logp` by call torch.tensor with the shown positional and keyword arguments.

2922 Set `advantages` by call torch.tensor with the shown positional and keyword arguments. It holds the policy credit signal with its retained aircraft/agent axis.

2924 Test `len(ready) > 1`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2926 Set `advantages` by divide advantages - advantages.mean() and advantages.std(unbiased=False).clamp_min(1e-06). It holds the policy credit signal with its retained aircraft/agent axis.

2928 Iterate `_` over `range(4)`. The following indented statements run once for each selected element.

2930 Set `(_, new_logp, entropy, _)` by call model.act with the shown positional and keyword arguments.

2932 Set `ratio` by exponentiate log values, including log-probability ratios. It holds the current-to-behavior probability ratio.

2934 Set `loss` by evaluate -torch.minimum(ratio * advantages, ratio.clamp(0.8, 1.2) * advantages).mean().

2936 Subtract `0.001 * entropy[batch_ids, ids].mean()` from `loss`, and store the result back in that same target. The change is visible to subsequent statements in this block.

Source lines 2938–2972.

```
                if not bool(torch.isfinite(loss)):
                    raise RuntimeError('nonfinite hierarchical policy loss')
                actor_optimizer.zero_grad()
                loss.backward()
                torch.nn.utils.clip_grad_norm_(actors, .5)
                actor_optimizer.step()
            logs.append({'record_type': 'option_update', 'method': method, 'simulated_days': day,
                'decisions': len(ready), 'durations': [r['duration'] for r in ready],
                'complete': [r['option_complete'] for r in ready], 'policy_loss':
                ↪  float(loss.detach())})
            ready.clear()
        states = torch.tensor(np.stack([row[0] for row in daily]))
        targets = torch.tensor(np.stack([row[1] for row in daily]))
        for _ in range(4):
            value_loss = .5 * (model.value(states) - targets).square().mean()
            if not bool(torch.isfinite(value_loss)):
                raise RuntimeError('nonfinite hierarchical value loss')
            critic_optimizer.zero_grad()
            value_loss.backward()
```

Line-by-line explanation.

2938 Test `not bool(torch.isfinite(loss))`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2940 Raise `RuntimeError('nonfinite hierarchical policy loss')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

2942 Clear gradients from the previous optimization step. The exact receiver and arguments are shown on this source line.

2944 Differentiate the loss and accumulate parameter gradients. The exact receiver and arguments are shown on this source line.

2946 Cap the joint gradient norm before the optimizer changes weights. The exact receiver and arguments are shown on this source line.

2948 Apply accumulated gradients to the optimizer's registered parameters. The exact receiver and arguments are shown on this source line.

2950 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

2952 Continue the expression begun at line 2950 with `'decisions': len(ready), 'durations': [r['duration'] for r in ready],`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

2954 Continue the expression begun at line 2950 with `'complete': [r['option_complete'] for r in ready], 'policy_loss': float(loss.detach())}`.

These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

2956 Call ready.clear with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

2958 Set `states` by call torch.tensor with the shown positional and keyword arguments.

2960 Set `targets` by call torch.tensor with the shown positional and keyword arguments.

2962 Iterate `_` over `range(4)`. The following indented statements run once for each selected element.

2964 Set `value_loss` by multiply 0.5 and (model.value(states) - targets).square().mean().

2966 Test `not bool(torch.isfinite(value_loss))`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

2968 Raise `RuntimeError('nonfinite hierarchical value loss')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

2970 Clear gradients from the previous optimization step. The exact receiver and arguments are shown on this source line.

2972 Differentiate the loss and accumulate parameter gradients. The exact receiver and arguments are shown on this source line.

Source lines 2974–2980.

```
                torch.nn.utils.clip_grad_norm_(model.critic.parameters(), .5)
                critic_optimizer.step()
            daily.clear()
    return model, logs, steps
```

Line-by-line explanation.

2974 Cap the joint gradient norm before the optimizer changes weights. The exact receiver and arguments are shown on this source line.

2976 Apply accumulated gradients to the optimizer's registered parameters. The exact receiver and arguments are shown on this source line.

2978 Call daily.clear with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

2980 Return `(model, logs, steps)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

9.120. MAXQMaintenance

Single-aircraft tabular MAXQ-0 adaptation. Primitive reward estimates V and completion estimates C are distinct tables. Recursive execution uses actual elapsed days and exports tables through PyTorch extra state.

Source lines 2984–2993.

```
class MAXQMaintenance(nn.Module):
    """Tabular MAXQ-0 baseline on one aircraft and the common repair catalog.
    Root chooses operate or Repair; Repair chooses an Engine task; each Engine
    chooses a component bundle then waits through its committed work. Primitive
    V values and composite completion C values are stored separately. No pseudo
    rewards or MAXQ-Q projected completion function is used. Beliefs are binned,
    so the original exact-state convergence guarantees are not claimed.
    """
```

Line-by-line explanation.

2984 Define `MAXQMaintenance` with the stated inheritance. Single-aircraft tabular MAXQ-0 adaptation. Primitive reward estimates V and completion estimates C are distinct tables. Recursive execution uses actual elapsed days and exports tables through PyTorch extra state.

2986 Documentation text: Tabular MAXQ-0 baseline on one aircraft and the common repair catalog.. This states the contract or qualification for `MAXQMaintenance`; it does not update the simulator or model.

2988 Documentation text: Root chooses operate or Repair; Repair chooses an Engine task; each Engine. This states the contract or qualification for `MAXQMaintenance`; it does not update the simulator or model.

2989 Documentation text: chooses a component bundle then waits through its committed work. Primitive. This states the contract or qualification for `MAXQMaintenance`; it does not update the simulator or model.

2990 Documentation text: V values and composite completion C values are stored separately. No pseudo. This states the contract or qualification for `MAXQMaintenance`; it does not update the simulator or model.

2991 Documentation text: rewards or MAXQ-Q projected completion function is used. Beliefs are binned,. This states the contract or qualification for `MAXQMaintenance`; it does not update the simulator or model.

2992 Documentation text: so the original exact-state convergence guarantees are not claimed.. This states the contract or qualification for `MAXQMaintenance`; it does not update the simulator or model.

2993 Close the documentation string for `MAXQMaintenance`. This delimiter ends the descriptive text and performs no simulator or learning update.

9.121. MAXQMaintenance.__init__

Initialize the object, register its layers or allocate its state. Construction does not itself train a policy.

Source lines 2996–3012.

```
    def __init__(self, env):
        super().__init__()
        if env.config.aircraft != 1:
            raise ValueError('MAXQ baseline currently requires Config(aircraft=1)')
        self.n = 1
        self.local = env.local_size
        self.mandatory = env.config.mandatory_health
        self.leaf_values = {}
        self.completion = {}
```

Line-by-line explanation.

2996 Define `__init__(self, env)`. This ___init___ method is part of MAXQMaintenance.___init___; its parameters and defaults are the interface used by the calls in this section.

2998 Call super().___init___ with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

3000 Test `env.config.aircraft != 1`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3002 Raise `ValueError('MAXQ baseline currently requires Config(aircraft=1)')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

3004 Set `self.n` by use the literal 1.

3006 Set `self.local` by evaluate env.local_size.

3008 Set `self.mandatory` by evaluate env.config.mandatory_health.

3010 Set `self.leaf_values` by construct the shown dict in its stated order.

3012 Set `self.completion` by construct the shown dict in its stated order.

9.122. MAXQMaintenance.state

Quantize the entire observed vector into a hashable tabular key. This aliases continuous belief states and never reads hidden health.

Source lines 3015–3018.

```
3015     def state(self, observation):
3016         # Explicit approximation: quantized observed beliefs, never true health.
3018         return tuple(np.rint(np.asarray(observation) * 8).astype(int).tolist())
```

Line-by-line explanation.

3015 Define `state(self, observation)`. Quantize the entire observed vector into a hashable tabular key. This aliases continuous belief states and never reads hidden health.

3016 Source comment: Explicit approximation: quantized observed beliefs, never true health. This line documents the following code and does not execute.

3018 Return `tuple(np.rint(np.asarray(observation) * 8).astype(int).tolist())` to the caller by call tuple with the shown positional and keyword arguments. No further statements in this call execute.

9.123. MAXQMaintenance.task_children

Return the allowed child tasks of the MAXQ parent. The dedicated name avoids colliding with PyTorch Module.children.

Source lines 3021–3049.

```
    def task_children(self, task, observation):
        busy = observation[37] > .5
        if task == 'Root':
            if busy:
                return ('Wait',)
            urgent = np.min(observation[24:36]) <= self.mandatory
            return ('Repair',) if urgent else ('Operate', 'Repair')
        if task == 'Repair':
            return ('Engine0', 'Engine1', 'EngineBoth')
        if task.startswith('Engine'):
            if busy:
                return ('Wait',)
            e = ('Engine0', 'Engine1', 'EngineBoth').index(task)
            return tuple(f'Submit{1 + 7 * e + c}' for c in range(7))
        return ()
```

Line-by-line explanation.

3021 Define `task_children(self, task, observation)`. Return the allowed child tasks of the MAXQ parent. The dedicated name avoids colliding with PyTorch Module.children.

3023 Set `busy` by evaluate the comparison observation[37] $>$ 0.5.

3025 Test `task == 'Root'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3027 Test `busy`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3029 Return `('Wait',)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

3031 Set `urgent` by evaluate the comparison np.min(observation[24:36]) <= self.mandatory.

3033 Return `('Repair',) if urgent else ('Operate', 'Repair')` to the caller by select ('Repair',) when urgent, otherwise ('Operate', 'Repair'). No further statements in this call execute.

3035 Test `task == 'Repair'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3037 Return `('Engine0', 'Engine1', 'EngineBoth')` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

3039 Test `task.startswith('Engine')`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3041 Test `busy`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3043 Return `('Wait',)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

3045 Set `e` by call ('Engine0', 'Engine1', 'EngineBoth').index with the shown positional and keyword arguments.

3047 Return `tuple((f'Submit{1 + 7 * e + c}' for c in range(7)))` to the caller by call tuple with the shown positional and keyword arguments. No further statements in this call execute.

3049 Return `()` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

9.124. MAXQMaintenance.value

Evaluate the centralized state-value or action-value path using the input contract of this class.

Source lines 3052–3062.

```
3052    def value(self, task, observation):
3054        children = self.task_children(task, observation)
3056        state = self.state(observation)
3058        if not children:
3060            return self.leaf_values.get((task, state), 0.)
3062        return max(self.q(task, observation, child) for child in children)
```

Line-by-line explanation.

3052 Define `value(self, task, observation)`. This value method is part of MAXQMaintenance.value; its parameters and defaults are the interface used by the calls in this section.

3054 Set `children` by call self.task_children with the shown positional and keyword arguments.

3056 Set `state` by call self.state with the shown positional and keyword arguments.

3058 Test `not children`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3060 Return `self.leaf_values.get((task, state), 0.0)` to the caller by call self.leaf_values.get with the shown positional and keyword arguments. No further statements in this call execute.

3062 Return `max((self.q(task, observation, child) for child in children))` to the caller by call max with the shown positional and keyword arguments. No further statements in this call execute.

9.125. MAXQMaintenance.q

Add the MAXQ child value and parent completion value for the given observed state and candidate child.

Source lines 3065–3067.

```
3065    def q(self, task, observation, child):
3067        return self.value(child, observation) + self.completion.get((task, self.state(observation),
            ↪  child), 0.)
```

Line-by-line explanation.

3065 Define `q(self, task, observation, child)`. Add the MAXQ child value and parent completion value for the given observed state and candidate child.

3067 Return `self.value(child, observation) + self.completion.get((task, self.state(observation), child), 0.0)` to the caller by add self.value(child, observation) and self.completion.get((task, self.state(observation), child), 0.0). No further statements in this call execute.

9.126. MAXQMaintenance.update_completion

Update C(parent,state,child) toward the appropriately discounted parent continuation after the child completes, using its actual primitive duration.

Source lines 3070–3076.

```
3070    def update_completion(self, task, observation, child, next_value, duration, alpha=.15, gamma=.99):
3072        key = (task, self.state(observation), child)
3074        old = self.completion.get(key, 0.)
3076        self.completion[key] = old + alpha * (gamma ** duration * next_value - old)
```

Line-by-line explanation.

3070 Define `update_completion(self, task, observation, child, next_value, duration, alpha=.15, gamma=.99)`. Update C(parent,state,child) toward the appropriately discounted parent continuation after the child completes, using its actual primitive duration.

3072 Set `key` by construct the shown tuple in its stated order.

3074 Set `old` by call self.completion.get with the shown positional and keyword arguments.

3076 Set `self.completion[key]` by add old and alpha * (gamma ** duration * next_value - old).

9.127. MAXQMaintenance.execute

Recursively execute the MAXQ task hierarchy, learn primitive/completion values when training, and respect both episode termination and the outer day budget.

Source lines 3079–3113.

```
3079    def execute(self, task, env, observation, done, budget, rng, learn=False, epsilon=.2):
3081        if budget < 1 or done:
3083            return observation, done, [], True
3085        children = self.task_children(task, observation)
3087        if not children:
3089            action = int(task[6:]) if task.startswith('Submit') else 0
3091            following, reward, done, truncated, info = env.step(np.array([action], dtype=np.int64))
3093            if learn:
3095                key = (task, self.state(observation))
3097                old = self.leaf_values.get(key, 0.)
3099                self.leaf_values[key] = old + .15 * (reward - old)
3101            return following, done or truncated, [(observation.copy(), reward)], True
3103        trajectory = []
3105        invoked = False
3107        while len(trajectory) < budget and not done:
3109            if invoked and task != 'Root' and observation[37] <= .5:
3111                break
3113            children = self.task_children(task, observation)
```

Line-by-line explanation.

3079 Define `execute(self, task, env, observation, done, budget, rng, learn=False, epsilon=.2)`. Recursively execute the MAXQ task hierarchy, learn primitive/completion values when training, and respect both episode termination and the outer day budget.

3081 Test `budget < 1 or done`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3083 Return `(observation, done, [], True)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

3085 Set `children` by call self.task_children with the shown positional and keyword arguments.

3087 Test `not children`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3089 Set `action` by select int(task[6:]) when task.startswith('Submit'), otherwise 0. It holds the sampled or stored action whose probability is evaluated.

3091 Set `(following, reward, done, truncated, info)` by execute this object's step with the shown arguments.

3093 Test `learn`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3095 Set `key` by construct the shown tuple in its stated order.

3097 Set `old` by call self.leaf_values.get with the shown positional and keyword arguments.

3099 Set `self.leaf_values[key]` by add old and 0.15 * (reward - old).

3101 Return `(following, done or truncated, [(observation.copy(), reward)], True)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

3103 Set `trajectory` by construct the shown list in its stated order.

3105 Set `invoked` by use the literal False.

3107 Repeat the indented block while `len(trajectory) < budget and (not done)` remains true; this condition controls the simulation/collection loop here.

3109 Test `invoked and task != 'Root' and (observation[37] <= 0.5)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3111 Exit the nearest loop immediately; execution resumes after that loop.

3113 Set `children` by call self.task_children with the shown positional and keyword arguments.

Source lines 3115–3149.

```
        scores = np.array([self.q(task, observation, child) for child in children])
        if learn and rng.random() < epsilon:
            selected = int(rng.integers(len(children)))
        else:
            selected = int(np.argmax(scores))
        child = children[selected]
        following, ended, trace, child_complete = self.execute(
            child, env, observation, done, budget - len(trajectory), rng, learn, epsilon)
        parent_terminal = ended or (task != 'Root' and following[37] <= .5)
        if learn and child_complete:
            next_value = 0. if parent_terminal else self.value(task, following)
            for offset, (state_obs, _) in enumerate(trace):
                self.update_completion(task, state_obs, child, next_value, len(trace) - offset)
        trajectory.extend(trace)
        observation, done, invoked = following, ended, True
```

```
            if task == 'Repair' and child_complete:
                break
            if not trace:
```

Line-by-line explanation.

3115 Set `scores` by call np.array with the shown positional and keyword arguments.

3117 Test `learn and rng.random() < epsilon`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3119 Set `selected` by convert the result to a Python integer.

3121 Execute this alternative only when the corresponding preceding condition did not select its branch.

3123 Set `selected` by convert the result to a Python integer.

3125 Set `child` by select the indexed/sliced entries of children.

3127 Set `(following, ended, trace, child_complete)` by call self.execute with the shown positional and keyword arguments.

3129 Continue the assignment begun at line 3127 with `child, env, observation, done, budget - len(trajectory), rng, learn, epsilon)`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

3131 Set `parent_terminal` by evaluate ended or (task != 'Root' and following[37] <= 0.5).

3133 Test `learn and child_complete`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3135 Set `next_value` by select 0.0 when parent_terminal, otherwise self.value(task, following).

3137 Iterate `(offset, (state_obs, _))` over `enumerate(trace)`. The following indented statements run once for each selected element.

3139 Call self.update_completion with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

3141 Call trajectory.extend with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

3143 Set `(observation, done, invoked)` by construct the shown tuple in its stated order.

3145 Test `task == 'Repair' and child_complete`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3147 Exit the nearest loop immediately; execution resumes after that loop.

3149 Test `not trace`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

Source lines 3151–3155.

```
                raise RuntimeError('MAXQ child made no physical progress')
        complete = done or (task != 'Root' and invoked and observation[37] <= .5)
        return observation, done, trajectory, complete
```

Line-by-line explanation.

3151 Raise `RuntimeError('MAXQ child made no physical progress')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

3153 Set `complete` by evaluate done or (task != 'Root' and invoked and (observation[37] <= 0.5)).

3155 Return `(observation, done, trajectory, complete)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

9.128. MAXQMaintenance.get_extra_state

Serialize MAXQ primitive and completion tables as model extra state so a checkpoint contains learned tabular values.

Source lines 3158–3160.

```
3158    def get_extra_state(self):
3160        return {'leaf_values': self.leaf_values, 'completion': self.completion}
```

Line-by-line explanation.

3158 Define `get_extra_state(self)`. Serialize MAXQ primitive and completion tables as model extra state so a checkpoint contains learned tabular values.

3160 Return `{'leaf_values': self.leaf_values, 'completion': self.completion}` to the caller by construct the shown dict in its stated order. No further statements in this call execute.

9.129. MAXQMaintenance.set_extra_state

Restore the saved MAXQ tables during checkpoint loading.

Source lines 3163–3167.

```
3163    def set_extra_state(self, state):
3165        self.leaf_values = dict(state['leaf_values'])
3167        self.completion = dict(state['completion'])
```

Line-by-line explanation.

3163 Define `set_extra_state(self, state)`. Restore the saved MAXQ tables during checkpoint loading.

3165 Set `self.leaf_values` by call dict with the shown positional and keyword arguments.

3167 Set `self.completion` by call dict with the shown positional and keyword arguments.

9.130. train_maxq

Train recursive MAXQ-0 episodes under the explicit one-aircraft restriction and the overall primitive-day budget.

Source lines 3171–3201.

```
def train_maxq(config, steps=4096, seed=0):
    if steps < 1:
        raise ValueError('positive MAXQ simulation budget required')
    env = AircraftOptionEnv(config)
    model = MAXQMaintenance(env.env)
    rng = np.random.default_rng(seed)
    days, episode, logs = 0, 0, []
    while days < steps:
        obs, _ = env.reset(seed=seed + episode)
        _, terminal, trace, _ = model.execute('Root', env, obs, False, steps - days, rng, learn=True)
        days += len(trace)
        logs.append({'record_type': 'episode' if terminal else 'budget_cutoff', 'simulated_days': days,
                     'return': sum(r for _, r in trace), 'primitive_values': len(model.leaf_values),
                     'completion_values': len(model.completion)})
        episode += 1
    return model, logs, days
```

Line-by-line explanation.

3171 Define `train_maxq(config, steps=4096, seed=0)`. Train recursive MAXQ-0 episodes under the explicit one-aircraft restriction and the overall primitive-day budget.

3173 Test `steps < 1`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3175 Raise `ValueError('positive MAXQ simulation budget required')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

3177 Set `env` by call AircraftOptionEnv with the shown positional and keyword arguments. It holds the environment or wrapper that owns physical transitions.

3179 Set `model` by call MAXQMaintenance with the shown positional and keyword arguments. It holds the chosen controller architecture.

3181 Set `rng` by call np.random.default_rng with the shown positional and keyword arguments.

3183 Set `(days, episode, logs)` by construct the shown tuple in its stated order.

3185 Repeat the indented block while `days < steps` remains true; this condition controls the simulation/collection loop here.

3187 Set `(obs, _)` by initialize a new episode using the stated seed/options.

3189 Set `(_, terminal, trace, _)` by call model.execute with the shown positional and keyword arguments.

3191 Add `len(trace)` to `days`, and store the result back in that same target. The change is visible to subsequent statements in this block.

3193 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

3195 Continue the expression begun at line 3193 with `'return': sum(r for _, r in trace), 'primitive_values': len(model.leaf_values),`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

3197 Continue the expression begun at line 3193 with `'completion_values': len(model.completion)})`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

3199 Add `1` to `episode`, and store the result back in that same target. The change is visible to subsequent statements in this block.

3201 Return `(model, logs, days)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

9.131. FleetOptionCritic

Three learned planning modes with option-conditioned catalog policies, option values and a separate termination network. Termination is forced to zero during committed repair; learned mode changes cannot cancel physical work.

Source lines 3205–3212.

```
3205 class FleetOptionCritic(nn.Module):
3207     """Option-critic adaptation with safe learned planning-mode termination.
3209     Options condition learned maintenance policies. Beta is exactly zero while
3210     a repair is committed. Changing planning mode never interrupts physical work.
3211     The greedy/epsilon-greedy policy over options is derived from Q_Omega.
3212     """
```

Line-by-line explanation.

3205 Define `FleetOptionCritic` with the stated inheritance. Three learned planning modes with option-conditioned catalog policies, option values and a separate termination network. Termination is forced to zero during committed repair; learned mode changes cannot cancel physical work.

3207 Documentation text: Option-critic adaptation with safe learned planning-mode termination.. This states the contract or qualification for `FleetOptionCritic`; it does not update the simulator or model.

3209 Documentation text: Options condition learned maintenance policies. Beta is exactly zero while. This states the contract or qualification for `FleetOptionCritic`; it does not update the simulator or model.

3210 Documentation text: a repair is committed. Changing planning mode never interrupts physical work.. This states the contract or qualification for `FleetOptionCritic`; it does not update the simulator or model.

3211 Documentation text: The greedy/epsilon-greedy policy over options is derived from Q_Omega.. This states the contract or qualification for `FleetOptionCritic`; it does not update the simulator or model.

3212 Close the documentation string for `FleetOptionCritic`. This delimiter ends the descriptive text and performs no simulator or learning update.

9.132. FleetOptionCritic.__init__

Initialize the object, register its layers or allocate its state. Construction does not itself train a policy.

Source lines 3215–3237.

```
3215     def __init__(self, env, options=3):
3217         super().__init__()
```

```
3219          if options < 2:
3221              raise ValueError('at least two planning options required')
3223          self.n = env.config.aircraft
3225          self.local = env.local_size
3227          self.width = env.observation_space.shape[0]
3229          self.mandatory = env.config.mandatory_health
3231          self.options = options
3233          self.worker = hrl_mlp(self.width + self.local + options, len(HRL_CATALOG))
3235          self.q_network = hrl_mlp(self.width, self.n * options)
3237          self.termination = hrl_mlp(self.width + self.local, options)
```

Line-by-line explanation.

3215 Define `__init__(self, env, options=3)`. This ___init___ method is part of FleetOptionCritic.___init___; its parameters and defaults are the interface used by the calls in this section.

3217 Call super().___init___ with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

3219 Test `options < 2`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3221 Raise `ValueError('at least two planning options required')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

3223 Set `self.n` by evaluate env.config.aircraft.

3225 Set `self.local` by evaluate env.local_size.

3227 Set `self.width` by select the indexed/sliced entries of env.observation_space.shape.

3229 Set `self.mandatory` by evaluate env.config.mandatory_health.

3231 Set `self.options` by evaluate options.

3233 Set `self.worker` by call hrl_mlp with the shown positional and keyword arguments.

3235 Set `self.q_network` by call hrl_mlp with the shown positional and keyword arguments.

3237 Set `self.termination` by call hrl_mlp with the shown positional and keyword arguments.

9.133. FleetOptionCritic.features

Assemble the local/public or global/local features required by this hierarchy, retaining the aircraft axis and the declared information boundary.

Source lines 3240–3254.

```
3240      def features(self, observation):
3242          obs = torch.as_tensor(observation, dtype=torch.float32)
3244          if obs.ndim == 1:
3246              obs = obs.unsqueeze(0)
3248          if obs.ndim != 2 or obs.shape[-1] != self.width:
3250              raise ValueError('incorrect option-critic observation')
3252          local = obs[:, :self.n * self.local].reshape(-1, self.n, self.local)
3254          return obs, local, torch.cat([obs[:, None].expand(-1, self.n, -1), local], -1)
```

Line-by-line explanation.

3240 Define `features(self, observation)`. Assemble the local/public or global/local features required by this hierarchy, retaining the aircraft axis and the declared information boundary.

3242 Set `obs` by convert inputs to a tensor with the requested dtype. It holds the observed belief vector, rather than hidden physical health.

3244 Test `obs.ndim == 1`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3246 Set `obs` by call obs.unsqueeze with the shown positional and keyword arguments. It holds the observed belief vector, rather than hidden physical health.

3248 Test `obs.ndim != 2 or obs.shape[-1] != self.width`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3250 Raise `ValueError('incorrect option-critic observation')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

3252 Set `local` by reinterpret the shape while retaining element order.

3254 Return `(obs, local, torch.cat([obs[:, None].expand(-1, self.n, -1), local], -1))` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

9.134. FleetOptionCritic.q_values

Evaluate the centralized option-value outputs and reshape them into aircraft by planning-mode values.

Source lines 3257–3259.

```
3257     def q_values(self, observation):
3259         return self.q_network(self.features(observation)[0]).reshape(-1, self.n, self.options)
```

Line-by-line explanation.

3257 Define `q_values(self, observation)`. Evaluate the centralized option-value outputs and reshape them into aircraft by planning-mode values.

3259 Return `self.q_network(self.features(observation)[0]).reshape(-1, self.n, self.options)` to the caller by reinterpret the shape while retaining element order. No further statements in this call execute.

9.135. FleetOptionCritic.beta

Evaluate learned planning-mode termination probabilities and force them to zero for busy aircraft.

Source lines 3262–3274.

```
3262     def beta(self, observation, option_ids):
3264         _, local, features = self.features(observation)
3266         ids = torch.as_tensor(option_ids, dtype=torch.long)
3268         if ids.shape != local.shape[:2] or torch.any((ids < 0) | (ids >= self.options)):
3270             raise ValueError('one valid option per aircraft required')
3272         raw = torch.sigmoid(self.termination(features)).gather(-1, ids.unsqueeze(-1)).squeeze(-1)
3274         return raw * (local[:, :, 37] <= .5)
```

Line-by-line explanation.

3262 Define `beta(self, observation, option_ids)`. Evaluate learned planning-mode termination probabilities and force them to zero for busy aircraft.

3264 Set `(_, local, features)` by call self.features with the shown positional and keyword arguments.

3266 Set `ids` by convert inputs to a tensor with the requested dtype.

3268 Test `ids.shape != local.shape[:2] or torch.any((ids < 0) | (ids >= self.options))`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3270 Raise `ValueError('one valid option per aircraft required')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

3272 Set `raw` by call torch.sigmoid(self.termination(features)).gather(-1, ids.unsqueeze(-1)).squeeze with the shown positional and keyword arguments.

3274 Return `raw * (local[:, :, 37] <= 0.5)` to the caller by multiply raw and local[:, :, 37] <= 0.5. No further statements in this call execute.

9.136. FleetOptionCritic.policy

Run the actor path using its allowed information. The separate critic path need not be available during actor-only execution.

Source lines 3277–3291.

```
3277     def policy(self, observation, option_ids, actions=None, deterministic=False):
3279         obs, local, features = self.features(observation)
3281         ids = torch.as_tensor(option_ids, dtype=torch.long)
3283         if ids.shape != local.shape[:2] or torch.any((ids < 0) | (ids >= self.options)):
3285             raise ValueError('one valid option per aircraft required')
3287         logits = self.worker(torch.cat([features, nn.functional.one_hot(ids, self.options).float()],
         ↪  -1))
3289         valid = hrl_catalog_mask(obs, self.n, self.local, self.mandatory)
3291         return hrl_categorical(logits, valid, actions, deterministic)
```

Line-by-line explanation.

3277 Define `policy(self, observation, option_ids, actions=None, deterministic=False)`. This policy method is part of FleetOptionCritic.policy; its parameters and defaults are the interface used by the calls in this section.

3279 Set `(obs, local, features)` by call self.features with the shown positional and keyword arguments.

3281 Set `ids` by convert inputs to a tensor with the requested dtype.

3283 Test `ids.shape != local.shape[:2] or torch.any((ids < 0) | (ids >= self.options))`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3285 Raise `ValueError('one valid option per aircraft required')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

3287 Set `logits` by call self.worker with the shown positional and keyword arguments.

3289 Set `valid` by compute permitted catalog entries under commitment and optional budget constraints. It holds the categorical structural-validity mask.

3291 Return `hrl_categorical(logits, valid, actions, deterministic)` to the caller by sample or score a structurally masked categorical decision. No further statements in this call execute.

9.137. option_critic_target

Construct the option continuation mixture using safe termination probability and the maximum next option value.

Source lines 3295–3301.

```
3295 def option_critic_target(reward, following_q, option_ids, beta, terminal, gamma=.99):
3297     current = following_q.gather(-1, option_ids.unsqueeze(-1)).squeeze(-1)
3299     continuation = (1 - beta) * current + beta * following_q.max(-1).values
3301     return reward + gamma * (1 - terminal.float()) * continuation
```

Line-by-line explanation.

3295 Define `option_critic_target(reward, following_q, option_ids, beta, terminal, gamma=.99)`. Construct the option continuation mixture using safe termination probability and the maximum next option value.

3297 Set `current` by call following_q.gather(-1, option_ids.unsqueeze(-1)).squeeze with the shown positional and keyword arguments.

3299 Set `continuation` by add (1 - beta) * current and beta * following_q.max(-1).values.

3301 Return `reward + gamma * (1 - terminal.float()) * continuation` to the caller by add reward and gamma * (1 - terminal.float()) * continuation. No further statements in this call execute.

9.138. hrl_epsilon_choice

Choose a feasible action greedily or through epsilon exploration; invalid choices never enter the exploration set.

Source lines 3305–3323.

```
3305 def hrl_epsilon_choice(values, valid, rng, epsilon=0.):
3307     values = np.asarray(values)
3309     valid = np.asarray(valid, dtype=bool)
3311     if values.ndim != 2 or values.shape != valid.shape or not valid.any(-1).all():
3313         raise ValueError('one nonempty choice mask per control unit required')
```

```
3315      choices = np.argmax(np.where(valid, values, -np.inf), axis=-1)
3317      for i in range(len(choices)):
3319          if rng.random() < epsilon:
3321              choices[i] = rng.choice(np.flatnonzero(valid[i]))
3323      return choices.astype(np.int64)
```

Line-by-line explanation.

3305 Define `hrl_epsilon_choice(values, valid, rng, epsilon=0.)`. Choose a feasible action greedily or through epsilon exploration; invalid choices never enter the exploration set.

3307 Set `values` by call np.asarray with the shown positional and keyword arguments. It holds critic predictions used for baselines or TD residuals.

3309 Set `valid` by call np.asarray with the shown positional and keyword arguments. It holds the categorical structural-validity mask.

3311 Test `values.ndim != 2 or values.shape != valid.shape or (not valid.any(-1).all())`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3313 Raise `ValueError('one nonempty choice mask per control unit required')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

3315 Set `choices` by call np.argmax with the shown positional and keyword arguments.

3317 Iterate `i` over `range(len(choices))`. The following indented statements run once for each selected element.

3319 Test `rng.random() < epsilon`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3321 Set `choices[i]` by call rng.choice with the shown positional and keyword arguments.

3323 Return `choices.astype(np.int64)` to the caller by call choices.astype with the shown positional and keyword arguments. No further statements in this call execute.

9.139. train_option_critic

Perform online internal-policy, option-value and termination updates with separate optimizers. Committed jobs mask action/termination learning where no decision is permitted.

Source lines 3327–3361.

```
3327  def train_option_critic(config, steps=4096, seed=0):
3329      if steps < 1:
3331          raise ValueError('positive option-critic simulation budget required')
3333      torch.set_num_threads(1)
3335      torch.manual_seed(seed)
3337      rng = np.random.default_rng(seed)
3339      env = AircraftOptionEnv(config)
3341      model = FleetOptionCritic(env.env)
3343      actor_opt = torch.optim.Adam(model.worker.parameters(), lr=3e-4)
3345      critic_opt = torch.optim.Adam(model.q_network.parameters(), lr=3e-4)
3347      beta_opt = torch.optim.Adam(model.termination.parameters(), lr=3e-4)
3349      obs, _ = env.reset(seed=seed)
3351      def select(observation):
3353          with torch.no_grad():
```

```
            q = model.q_values(observation)[0].numpy()
        return hrl_epsilon_choice(q, np.ones_like(q, dtype=bool), rng, .1)
    ids = select(obs)
    episode, logs = 0, []
```

Line-by-line explanation.

3327 Define `train_option_critic(config, steps=4096, seed=0)`. Perform online internal-policy, option-value and termination updates with separate optimizers. Committed jobs mask action/termination learning where no decision is permitted.

3329 Test `steps < 1`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3331 Raise `ValueError('positive option-critic simulation budget required')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

3333 Set the cpu thread count for these small models. The exact receiver and arguments are shown on this source line.

3335 Seed neural initialization and torch sampling. The exact receiver and arguments are shown on this source line.

3337 Set `rng` by call np.random.default_rng with the shown positional and keyword arguments.

3339 Set `env` by call AircraftOptionEnv with the shown positional and keyword arguments. It holds the environment or wrapper that owns physical transitions.

3341 Set `model` by call FleetOptionCritic with the shown positional and keyword arguments. It holds the chosen controller architecture.

3343 Set `actor_opt` by create an Adam optimizer for the specified parameter collection.

3345 Set `critic_opt` by create an Adam optimizer for the specified parameter collection.

3347 Set `beta_opt` by create an Adam optimizer for the specified parameter collection.

3349 Set `(obs, _)` by initialize a new episode using the stated seed/options.

3351 Define `select(observation)`. This select method is part of train_option_critic; its parameters and defaults are the interface used by the calls in this section.

3353 Enter `torch.no_grad()`. The block records no gradients, keeping rollout/target calculations outside backpropagation.

3355 Set `q` by expose this CPU tensor as a NumPy array. It holds an action or option value estimate.

3357 Return `hrl_epsilon_choice(q, np.ones_like(q, dtype=bool), rng, 0.1)` to the caller by call hrl_epsilon_choice with the shown positional and keyword arguments. No further statements in this call execute.

3359 Set `ids` by call select with the shown positional and keyword arguments.

3361 Set `(episode, logs)` by construct the shown tuple in its stated order.

Source lines 3363–3397.

```
    for day in range(1, steps + 1):
        option_ids = torch.tensor(ids).unsqueeze(0)
        actions, logp, entropy = model.policy(obs, option_ids)
        active = torch.tensor(env.decision_mask()).unsqueeze(0).float()
        following, reward, done, truncated, info = env.step(actions[0].detach().numpy())
        terminal = done or truncated
        with torch.no_grad():
            following_q = model.q_values(following)
        beta = model.beta(following, option_ids)
        target = option_critic_target(reward, following_q, option_ids, beta.detach(),
        ↪  torch.tensor(terminal))
        q = model.q_values(obs).gather(-1, option_ids.unsqueeze(-1)).squeeze(-1)
        advantage = (target - q).detach()
        actor_loss = -((logp * advantage + .001 * entropy) * active).sum() / active.sum().clamp_min(1)
        critic_loss = .5 * (q - target.detach()).square().mean()
        safe = torch.tensor(env.decision_mask()).unsqueeze(0).float() * (not terminal)
        termination_advantage = (following_q.gather(-1, option_ids.unsqueeze(-1)).squeeze(-1)
                                 - following_q.max(-1).values + .01).detach()
        termination_loss = (beta * termination_advantage * safe).sum() / safe.sum().clamp_min(1)
```

Line-by-line explanation.

3363 Iterate `day` over `range(1, steps + 1)`. The following indented statements run once for each selected element.

3365 Set `option_ids` by call torch.tensor(ids).unsqueeze with the shown positional and keyword arguments. It holds the current learned planning mode for each aircraft.

3367 Set `(actions, logp, entropy)` by call model.policy with the shown positional and keyword arguments.

3369 Set `active` by call torch.tensor(env.decision_mask()).unsqueeze(0).float with the shown positional and keyword arguments.

3371 Set `(following, reward, done, truncated, info)` by execute this object's step with the shown arguments.

3373 Set `terminal` by evaluate done or truncated. It holds the episode boundary used to suppress continuation value.

3375 Enter `torch.no_grad()`. The block records no gradients, keeping rollout/target calculations outside backpropagation.

3377 Set `following_q` by call model.q_values with the shown positional and keyword arguments.

3379 Set `beta` by call model.beta with the shown positional and keyword arguments.

3381 Set `target` by mix continuation and reselection option values using safe termination. It holds a bootstrapped learning target.

3383 Set `q` by call model.q_values(obs).gather(-1, option_ids.unsqueeze(-1)).squeeze with the shown positional and keyword arguments. It holds an action or option value estimate.

3385 Set `advantage` by stop gradients through this result.

3387 Set `actor_loss` by divide -((logp * advantage + 0.001 * entropy) * active).sum() and active.sum().clamp_min(1). It holds the objective differentiated through policy parameters.

3389 Set `critic_loss` by multiply 0.5 and (q - target.detach()).square().mean(). It holds the prediction-error objective for the value estimator.

3391 Set `safe` by multiply torch.tensor(env.decision_mask()).unsqueeze(0).float() and not terminal.

3393 Set `termination_advantage` by stop gradients through this result.

3395 Continue the assignment begun at line 3393 with `- following_q.max(-1).values + .01).detach()`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

3397 Set `termination_loss` by divide (beta * termination_advantage * safe).sum() and safe.sum().clamp_min(1).

Source lines 3399–3433.

```
3399        if not bool(torch.isfinite(actor_loss + critic_loss + termination_loss)):
3401            raise RuntimeError('nonfinite option-critic loss')
3403        actor_opt.zero_grad()
3405        actor_loss.backward()
3407        torch.nn.utils.clip_grad_norm_(model.worker.parameters(), .5)
3409        if bool(active.any()):
3411            actor_opt.step()
3413        critic_opt.zero_grad()
3415        critic_loss.backward()
3417        torch.nn.utils.clip_grad_norm_(model.q_network.parameters(), .5)
3419        critic_opt.step()
3421        beta_opt.zero_grad()
3423        termination_loss.backward()
3425        torch.nn.utils.clip_grad_norm_(model.termination.parameters(), .5)
3427        if bool(safe.any()):
3429            beta_opt.step()
3431        terminate = (rng.random(config.aircraft) < beta.detach()[0].numpy()) & env.decision_mask()
3433        next_ids = select(following)
```

Line-by-line explanation.

3399 Test `not bool(torch.isfinite(actor_loss + critic_loss + termination_loss))`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3401 Raise `RuntimeError('nonfinite option-critic loss')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

3403 Clear gradients from the previous optimization step. The exact receiver and arguments are shown on this source line.

3405 Differentiate the loss and accumulate parameter gradients. The exact receiver and arguments are shown on this source line.

3407 Cap the joint gradient norm before the optimizer changes weights. The exact receiver and arguments are shown on this source line.

3409 Test `bool(active.any())`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3411 Apply accumulated gradients to the optimizer's registered parameters. The exact receiver and arguments are shown on this source line.

3413 Clear gradients from the previous optimization step. The exact receiver and arguments are shown on this source line.

3415 Differentiate the loss and accumulate parameter gradients. The exact receiver and arguments are shown on this source line.

3417 Cap the joint gradient norm before the optimizer changes weights. The exact receiver and arguments are shown on this source line.

3419 Apply accumulated gradients to the optimizer's registered parameters. The exact receiver and arguments are shown on this source line.

3421 Clear gradients from the previous optimization step. The exact receiver and arguments are shown on this source line.

3423 Differentiate the loss and accumulate parameter gradients. The exact receiver and arguments are shown on this source line.

3425 Cap the joint gradient norm before the optimizer changes weights. The exact receiver and arguments are shown on this source line.

3427 Test `bool(safe.any())`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3429 Apply accumulated gradients to the optimizer's registered parameters. The exact receiver and arguments are shown on this source line.

3431 Set `terminate` by apply a bitwise intersection to rng.random(config.aircraft) < beta.detach()[0].numpy() and env.decision_mask().

3433 Set `next_ids` by call select with the shown positional and keyword arguments.

Source lines 3435–3457.

```
        ids = np.where(terminate, next_ids, ids)
        logs.append({'record_type': 'update', 'simulated_days': day,
                     'policy_loss': float(actor_loss.detach()), 'value_loss':
                     ↪  float(critic_loss.detach()),
                     'termination_loss': float(termination_loss.detach()),
                     'safe_termination_samples': int(safe.sum()), 'option_terminations':
                     ↪  int(terminate.sum())})
        obs = following
        if terminal:
            logs.append({'record_type': 'episode', 'simulated_days': day, **info['metrics']})
            episode += 1
            obs, _ = env.reset(seed=seed + episode)
            ids = select(obs)
    return model, logs, steps
```

Line-by-line explanation.

3435 Set `ids` by call np.where with the shown positional and keyword arguments.

3437 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

3439 Continue the expression begun at line 3437 with `'policy_loss': float(actor_loss.detach()), 'value_loss': float(critic_loss.detach()),`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

3441 Continue the expression begun at line 3437 with `'termination_loss': float(termination_loss.detach()),`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

3443 Continue the expression begun at line 3437 with `'safe_termination_samples': int(safe.sum()), 'option_terminations': int(terminate.sum())})`.

These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

3445 Set `obs` by evaluate following. It holds the observed belief vector, rather than hidden physical health.

3447 Test `terminal`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3449 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

3451 Add `1` to `episode`, and store the result back in that same target. The change is visible to subsequent statements in this block.

3453 Set `(obs, _)` by initialize a new episode using the stated seed/options.

3455 Set `ids` by call select with the shown positional and keyword arguments.

3457 Return `(model, logs, steps)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

9.140. HierarchicalVDN

Two levels of independent aircraft Q networks with additive value decomposition. The upper level sets budgets on a macro clock and the lower level requests daily catalog actions, with a detached high-level advantage shaping its reward.

Source lines 3461–3469.

```
3461 class HierarchicalVDN(nn.Module):
3463     """Two-level value-decomposition adaptation, not an exact HAVEN reproduction.
3465     Per-aircraft high policies select replacement budgets; local low policies
3466     choose catalog requests. Both levels sum agent Q values. The low reward
3467     receives a detached share of the macro TD advantage (coefficient 0.1).
3468     Macro epochs are three days; busy aircraft retain goals and committed plans.
3469     """
```

Line-by-line explanation.

3461 Define `HierarchicalVDN` with the stated inheritance. Two levels of independent aircraft Q networks with additive value decomposition. The upper level sets budgets on a macro clock and the lower level requests daily catalog actions, with a detached high-level advantage shaping its reward.

3463 Documentation text: Two-level value-decomposition adaptation, not an exact HAVEN reproduction.. This states the contract or qualification for `HierarchicalVDN`; it does not update the simulator or model.

3465 Documentation text: Per-aircraft high policies select replacement budgets; local low policies. This states the contract or qualification for `HierarchicalVDN`; it does not update the simulator or model.

3466 Documentation text: choose catalog requests. Both levels sum agent Q values. The low reward. This states the contract or qualification for `HierarchicalVDN`; it does not update the simulator or model.

3467 Documentation text: receives a detached share of the macro TD advantage (coefficient 0.1).. This states the contract or qualification for `HierarchicalVDN`; it does not update the simulator or model.

3468 Documentation text: Macro epochs are three days; busy aircraft retain goals and committed plans.. This states the contract or qualification for `HierarchicalVDN`; it does not update the simulator or model.

3469 Close the documentation string for `HierarchicalVDN`. This delimiter ends the descriptive text and performs no simulator or learning update.

9.141. HierarchicalVDN.___init___

Initialize the object, register its layers or allocate its state. Construction does not itself train a policy.

Source lines 3472–3486.

```
3472     def __init__(self, env):
3474         super().__init__()
3476         self.n = env.config.aircraft
3478         self.local = env.local_size
3480         self.public = env.public_size
3482         self.mandatory = env.config.mandatory_health
3484         self.high_agents = nn.ModuleList([hrl_mlp(self.local + self.public, 4) for _ in range(self.n)])
3486         self.low_agents = nn.ModuleList([hrl_mlp(self.local + self.public + 4, len(HRL_CATALOG)) for _
         ↪  in range(self.n)])
```

Line-by-line explanation.

3472 Define `__init__(self, env)`. This ___init___ method is part of HierarchicalVDN.___init___; its parameters and defaults are the interface used by the calls in this section.

3474 Call super().___init___ with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

3476 Set `self.n` by evaluate env.config.aircraft.

3478 Set `self.local` by evaluate env.local_size.

3480 Set `self.public` by evaluate env.public_size.

3482 Set `self.mandatory` by evaluate env.config.mandatory_health.

3484 Set `self.high_agents` by register the listed independent networks so parameters and checkpoints include them.

3486 Set `self.low_agents` by register the listed independent networks so parameters and checkpoints include them.

9.142. HierarchicalVDN.features

Assemble the local/public or global/local features required by this hierarchy, retaining the aircraft axis and the declared information boundary.

Source lines 3489–3491.

```
3489     def features(self, observation):
3491         return mappo_agent_inputs(observation, self.n, self.local, self.public, 'aircraft')
```

Line-by-line explanation.

3489 Define `features(self, observation)`. Assemble the local/public or global/local features required by this hierarchy, retaining the aircraft axis and the declared information boundary.

3491 Return `mappo_agent_inputs(observation, self.n, self.local, self.public, 'aircraft')` to the caller by call mappo_agent_inputs with the shown positional and keyword arguments. No further statements in this call execute.

9.143. HierarchicalVDN.high_values

Evaluate the independent aircraft upper-level budget Q networks from local/public features.

Source lines 3494–3498.

```
3494    def high_values(self, observation):
3496        x = self.features(observation)
3498        return torch.stack([agent(x[:, i]) for i, agent in enumerate(self.high_agents)], 1)
```

Line-by-line explanation.

3494 Define `high_values(self, observation)`. Evaluate the independent aircraft upper-level budget Q networks from local/public features.

3496 Set `x` by call self.features with the shown positional and keyword arguments.

3498 Return `torch.stack([agent(x[:, i]) for i, agent in enumerate(self.high_agents)], 1)` to the caller by join equal-shaped tensors along the specified new axis. No further statements in this call execute.

9.144. HierarchicalVDN.low_values

Evaluate independent aircraft catalog Q networks conditioned on local/public features and the retained budget goal.

Source lines 3501–3513.

```
3501    def low_values(self, observation, goals):
3503        x = self.features(observation)
3505        goals = torch.as_tensor(goals, dtype=torch.long)
3507        if goals.shape != x.shape[:2] or torch.any((goals < 0) | (goals >= 4)):
3509            raise ValueError('one valid hierarchy goal per aircraft required')
3511        x = torch.cat([x, nn.functional.one_hot(goals, 4).float()], -1)
3513        return torch.stack([agent(x[:, i]) for i, agent in enumerate(self.low_agents)], 1)
```

Line-by-line explanation.

3501 Define `low_values(self, observation, goals)`. Evaluate independent aircraft catalog Q networks conditioned on local/public features and the retained budget goal.

3503 Set `x` by call self.features with the shown positional and keyword arguments.

3505 Set `goals` by convert inputs to a tensor with the requested dtype. It holds per-aircraft upper-level replacement budgets.

3507 Test `goals.shape != x.shape[:2] or torch.any((goals < 0) | (goals >= 4))`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3509 Raise `ValueError('one valid hierarchy goal per aircraft required')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

3511 Set `x` by concatenate features along the specified existing axis.

3513 Return `torch.stack([agent(x[:, i]) for i, agent in enumerate(self.low_agents)], 1)` to the caller by join equal-shaped tensors along the specified new axis. No further statements in this call execute.

9.145. HierarchicalVDN.mix

Add the aircraft Q values to obtain the joint value-decomposition estimate. There is no learned nonlinear mixer.

Source lines 3516–3520.

```
3516     @staticmethod
3518     def mix(values):
3520         return values.sum(-1)
```

Line-by-line explanation.

3516 Apply the decorator `staticmethod` to the following definition. Dataclass decorators generate field initialization/representation; staticmethod omits automatic instance binding where used.

3518 Define `mix(values)`. Add the aircraft Q values to obtain the joint value-decomposition estimate. There is no learned nonlinear mixer.

3520 Return `values.sum(-1)` to the caller by sum across the specified tensor/array axis. No further statements in this call execute.

9.146. HierarchicalVDN.choose_goals

Choose structurally permitted upper-level budget goals with epsilon exploration during learning or greedily during evaluation.

Source lines 3523–3547.

```
3523     def choose_goals(self, observation, previous, rng, epsilon=0.):
3525         local = self.features(observation)[0]
3527         busy = local[:, 37].numpy() > .5
3529         urgent = local[:, 24:36].amin(-1).numpy() <= self.mandatory
3531         valid = np.ones((self.n, 4), dtype=bool)
3533         valid[urgent & ~busy, 0] = False
3535         with torch.no_grad():
3537             choices = hrl_epsilon_choice(self.high_values(observation)[0].numpy(), valid, rng, epsilon)
3539         if previous is not None:
3541             choices[busy] = np.asarray(previous)[busy]
3543         else:
3545             choices[busy] = 0
3547         return choices
```

Line-by-line explanation.

3523 Define `choose_goals(self, observation, previous, rng, epsilon=0.)`. Choose structurally permitted upper-level budget goals with epsilon exploration during learning or greedily during evaluation.

3525 Set `local` by select the indexed/sliced entries of self.features(observation).

3527 Set `busy` by evaluate the comparison local[:, 37].numpy() > 0.5.

3529 Set `urgent` by evaluate the comparison local[:, 24:36].amin(-1).numpy() <= self.mandatory.

3531 Set `valid` by call np.ones with the shown positional and keyword arguments. It holds the categorical structural-validity mask.

3533 Set `valid[urgent & ~busy, 0]` by use the literal False. It holds the categorical structural-validity mask.

3535 Enter `torch.no_grad()`. The block records no gradients, keeping rollout/target calculations outside backpropagation.

3537 Set `choices` by call hrl_epsilon_choice with the shown positional and keyword arguments.

3539 Test `previous is not None`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3541 Set `choices[busy]` by select the indexed/sliced entries of np.asarray(previous).

3543 Execute this alternative only when the corresponding preceding condition did not select its branch.

3545 Set `choices[busy]` by use the literal 0.

3547 Return `choices` to the caller by evaluate choices. No further statements in this call execute.

9.147. HierarchicalVDN.choose_actions

Choose structurally permitted budget-conditioned catalog requests and retain the action values required by the low-level update.

Source lines 3550–3563.

```
3550     def choose_actions(self, observation, goals, rng, epsilon=0.):
3552         goals = torch.tensor(np.asarray(goals)).unsqueeze(0)
3554         valid = hrl_catalog_mask(observation, self.n, self.local, self.mandatory, goals)
3555         # An urgent aircraft may outgrow an old zero-budget goal before the next macro epoch.
3557         if not bool(valid.any(-1).all()):
3559             raise ValueError('a stale hierarchy goal must be refreshed after an urgency alarm')
3561         with torch.no_grad():
3563             return hrl_epsilon_choice(self.low_values(observation, goals)[0].numpy(),
             ↪  valid[0].numpy(), rng, epsilon)
```

Line-by-line explanation.

3550 Define `choose_actions(self, observation, goals, rng, epsilon=0.)`. Choose structurally permitted budget-conditioned catalog requests and retain the action values required by the low-level update.

3552 Set `goals` by call torch.tensor(np.asarray(goals)).unsqueeze with the shown positional and keyword arguments. It holds per-aircraft upper-level replacement budgets.

3554 Set `valid` by compute permitted catalog entries under commitment and optional budget constraints. It holds the categorical structural-validity mask.

3555 Source comment: An urgent aircraft may outgrow an old zero-budget goal before the next macro epoch. This line documents the following code and does not execute.

3557 Test `not bool(valid.any(-1).all())`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3559 Raise `ValueError('a stale hierarchy goal must be refreshed after an urgency alarm')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

3561 Enter `torch.no_grad()`. The block records no gradients, keeping rollout/target calculations outside backpropagation.

3563 Return `hrl_epsilon_choice(self.low_values(observation, goals)[0].numpy(), valid[0].numpy(), rng, epsilon)` to the caller by call hrl_epsilon_choice with the shown positional and keyword arguments. No further statements in this call execute.

9.148. train_hierarchical_vdn

Collect macro budget choices and daily catalog requests, update additive high-level TD targets, and train the low level with the disclosed advantage-shaped reward.

Source lines 3567–3601.

```
3567 def train_hierarchical_vdn(config, steps=4096, seed=0, macro_days=3):
3569     if steps < 1 or macro_days < 1:
3571         raise ValueError('positive simulation budget and macro duration required')
3573     torch.set_num_threads(1)
3575     torch.manual_seed(seed)
3577     rng = np.random.default_rng(seed)
3579     env = AircraftOptionEnv(config)
3581     model = HierarchicalVDN(env.env)
3583     high_opt = torch.optim.Adam(model.high_agents.parameters(), lr=3e-4)
3585     low_opt = torch.optim.Adam(model.low_agents.parameters(), lr=3e-4)
3587     obs, _ = env.reset(seed=seed)
3589     goals, days, episode, logs = None, 0, 0, []
3591     while days < steps:
3593         goals = model.choose_goals(obs, goals, rng, .1)
3595         start, window = obs.copy(), []
3597         for _ in range(min(macro_days, steps - days)):
3599             actions = model.choose_actions(obs, goals, rng, .1)
3601             following, reward, done, truncated, info = env.step(actions)
```

Line-by-line explanation.

3567 Define `train_hierarchical_vdn(config, steps=4096, seed=0, macro_days=3)`. Collect macro budget choices and daily catalog requests, update additive high-level TD targets, and train the low level with the disclosed advantage-shaped reward.

3569 Test `steps < 1 or macro_days < 1`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3571 Raise `ValueError('positive simulation budget and macro duration required')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

3573 Set the cpu thread count for these small models. The exact receiver and arguments are shown on this source line.

3575 Seed neural initialization and torch sampling. The exact receiver and arguments are shown on this source line.

3577 Set `rng` by call np.random.default_rng with the shown positional and keyword arguments.

3579 Set `env` by call AircraftOptionEnv with the shown positional and keyword arguments. It holds the environment or wrapper that owns physical transitions.

3581 Set `model` by call HierarchicalVDN with the shown positional and keyword arguments. It holds the chosen controller architecture.

3583 Set `high_opt` by create an Adam optimizer for the specified parameter collection.

3585 Set `low_opt` by create an Adam optimizer for the specified parameter collection.

3587 Set `(obs, _)` by initialize a new episode using the stated seed/options.

3589 Set `(goals, days, episode, logs)` by construct the shown tuple in its stated order.

3591 Repeat the indented block while `days < steps` remains true; this condition controls the simulation/collection loop here.

3593 Set `goals` by call model.choose_goals with the shown positional and keyword arguments. It holds per-aircraft upper-level replacement budgets.

3595 Set `(start, window)` by construct the shown tuple in its stated order.

3597 Iterate `_` over `range(min(macro_days, steps - days))`. The following indented statements run once for each selected element.

3599 Set `actions` by call model.choose_actions with the shown positional and keyword arguments.

3601 Set `(following, reward, done, truncated, info)` by execute this object's step with the shown arguments.

Source lines 3603–3636.

```
            terminal = done or truncated
            window.append((obs.copy(), goals.copy(), actions.copy(), reward, following.copy(),
            ↪  terminal))
            days += 1
            obs = following
            # End the macro on a new urgency alarm so zero-budget goals never block safety.
            rows, _ = env.env.split_observation(obs)
            urgent = rows[:, 24:36].min(-1) <= config.mandatory_health
            if terminal or np.any(urgent & env.decision_mask() & (goals == 0)):
                break
        next_goals = model.choose_goals(obs, goals, rng)
        self_n = config.aircraft
        selected_goals = torch.tensor(goals).reshape(1, self_n, 1)
        high_value = model.mix(model.high_values(start).gather(-1, selected_goals).squeeze(-1))
        with torch.no_grad():
            next_high = model.mix(model.high_values(obs).gather(-1,
            ↪  torch.tensor(next_goals).reshape(1, self_n, 1)).squeeze(-1))
            target = hrl_option_target([r[3] for r in window], float(next_high[0]), terminal)
        high_advantage = (target - high_value).detach()
        high_loss = .5 * (high_value - target).square().mean()
```

Line-by-line explanation.

3603 Set `terminal` by evaluate done or truncated. It holds the episode boundary used to suppress continuation value.

3605 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

3607 Add `1` to `days`, and store the result back in that same target. The change is visible to subsequent statements in this block.

3609 Set `obs` by evaluate following. It holds the observed belief vector, rather than hidden physical health.

3610 Source comment: End the macro on a new urgency alarm so zero-budget goals never block safety. This line documents the following code and does not execute.

3612 Set `(rows, _)` by call env.env.split_observation with the shown positional and keyword arguments.

3614 Set `urgent` by evaluate the comparison rows[:, 24:36].min(-1) <= config.mandatory_health.

3616 Test `terminal or np.any(urgent & env.decision_mask() & (goals == 0))`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3618 Exit the nearest loop immediately; execution resumes after that loop.

3620 Set `next_goals` by call model.choose_goals with the shown positional and keyword arguments.

3622 Set `self_n` by evaluate config.aircraft.

3624 Set `selected_goals` by reinterpret the shape while retaining element order.

3626 Set `high_value` by call model.mix with the shown positional and keyword arguments.

3628 Enter `torch.no_grad()`. The block records no gradients, keeping rollout/target calculations outside backpropagation.

3630 Set `next_high` by call model.mix with the shown positional and keyword arguments.

3632 Set `target` by discount daily rewards and bootstrap over the actual aircraft-decision duration. It holds a bootstrapped learning target.

3634 Set `high_advantage` by stop gradients through this result.

3636 Set `high_loss` by multiply 0.5 and (high_value - target).square().mean().

Source lines 3638–3672.

```
        low_loss = torch.zeros(())
        for t, (state, old_goals, action, reward, following, ended) in enumerate(window):
            next_goal = next_goals if t == len(window) - 1 else old_goals
            q = model.mix(model.low_values(state, torch.tensor(old_goals).unsqueeze(0)).gather(
                -1, torch.tensor(action).reshape(1, self_n, 1)).squeeze(-1))
            with torch.no_grad():
                valid = hrl_catalog_mask(following, self_n, env.env.local_size,
                ↪  config.mandatory_health,
                                         torch.tensor(next_goal).unsqueeze(0))
                future = model.low_values(following, torch.tensor(next_goal).unsqueeze(0))
                next_low = model.mix(future.masked_fill(~valid, -1e9).max(-1).values)
                shaped_reward = reward + .1 * high_advantage / len(window)
                low_target = shaped_reward + .99 * (not ended) * next_low
            low_loss = low_loss + .5 * (q - low_target).square().mean() / len(window)
        if not bool(torch.isfinite(high_loss + low_loss)):
            raise RuntimeError('nonfinite hierarchical value-decomposition loss')
```

```
3668            high_opt.zero_grad()
3670            high_loss.backward()
3672            torch.nn.utils.clip_grad_norm_(model.high_agents.parameters(), .5)
```

Line-by-line explanation.

3638 Set `low_loss` by call torch.zeros with the shown positional and keyword arguments.

3640 Iterate `(t, (state, old_goals, action, reward, following, ended))` over `enumerate(window)`. The following indented statements run once for each selected element.

3642 Set `next_goal` by select next_goals when t == len(window) - 1, otherwise old_goals.

3644 Set `q` by call model.mix with the shown positional and keyword arguments. It holds an action or option value estimate.

3646 Continue the assignment begun at line 3644 with `-1, torch.tensor(action).reshape(1, self_n, 1)).squeeze(-1))`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

3648 Enter `torch.no_grad()`. The block records no gradients, keeping rollout/target calculations outside backpropagation.

3650 Set `valid` by compute permitted catalog entries under commitment and optional budget constraints. It holds the categorical structural-validity mask.

3652 Continue the assignment begun at line 3650 with `torch.tensor(next_goal).unsqueeze(0))`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

3654 Set `future` by call model.low_values with the shown positional and keyword arguments.

3656 Set `next_low` by call model.mix with the shown positional and keyword arguments.

3658 Set `shaped_reward` by add reward and 0.1 * high_advantage / len(window).

3660 Set `low_target` by add shaped_reward and 0.99 * (not ended) * next_low.

3662 Set `low_loss` by add low_loss and 0.5 * (q - low_target).square().mean() / len(window).

3664 Test `not bool(torch.isfinite(high_loss + low_loss))`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3666 Raise `RuntimeError('nonfinite hierarchical value-decomposition loss')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

3668 Clear gradients from the previous optimization step. The exact receiver and arguments are shown on this source line.

3670 Differentiate the loss and accumulate parameter gradients. The exact receiver and arguments are shown on this source line.

3672 Cap the joint gradient norm before the optimizer changes weights. The exact receiver and arguments are shown on this source line.

Source lines 3674–3700.

```
        high_opt.step()
        low_opt.zero_grad()
        low_loss.backward()
        torch.nn.utils.clip_grad_norm_(model.low_agents.parameters(), .5)
        low_opt.step()
        logs.append({'record_type': 'macro_update', 'simulated_days': days, 'duration': len(window),
                     'high_loss': float(high_loss.detach()), 'low_loss': float(low_loss.detach()),
                     'high_advantage': float(high_advantage[0]), 'goals': goals.tolist()})
        if terminal:
            logs.append({'record_type': 'episode', 'simulated_days': days, **info['metrics']})
            episode += 1
            obs, _ = env.reset(seed=seed + episode)
            goals = None
    return model, logs, days
```

Line-by-line explanation.

3674 Apply accumulated gradients to the optimizer's registered parameters. The exact receiver and arguments are shown on this source line.

3676 Clear gradients from the previous optimization step. The exact receiver and arguments are shown on this source line.

3678 Differentiate the loss and accumulate parameter gradients. The exact receiver and arguments are shown on this source line.

3680 Cap the joint gradient norm before the optimizer changes weights. The exact receiver and arguments are shown on this source line.

3682 Apply accumulated gradients to the optimizer's registered parameters. The exact receiver and arguments are shown on this source line.

3684 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

3686 Continue the expression begun at line 3684 with `'high_loss': float(high_loss.detach()), 'low_loss': float(low_loss.detach()),`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

3688 Continue the expression begun at line 3684 with `'high_advantage': float(high_advantage[0]), 'goals': goals.tolist()})`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

3690 Test `terminal`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3692 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

3694 Add `1` to `episode`, and store the result back in that same target. The change is visible to subsequent statements in this block.

3696 Set `(obs, _)` by initialize a new episode using the stated seed/options.

3698 Set `goals` by use the literal None. It holds per-aircraft upper-level replacement budgets.

3700 Return `(model, logs, days)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

9.149. evaluate_hrl

Execute each new hierarchy without learning, retaining the necessary option IDs, goals or macro clock and collecting the common physical metrics.

Source lines 3704–3738.

```
def evaluate_hrl(config, method, model, seeds):
    """Evaluate hierarchy controllers with a daily physical clock and fixed weights."""
    if method not in HRL_METHODS or model is None:
        raise ValueError('registered hierarchy and trained model required')
    records = []
    for seed in seeds:
        env = AircraftOptionEnv(config)
        obs, _ = env.reset(seed=int(seed))
        rng = np.random.default_rng(int(seed))
        decisions, free_decisions, terminations = 0, 0, 0
        if method == 'maxq':
            _, _, trace, _ = model.execute('Root', env, obs, False, config.horizon, rng, learn=False)
            decisions = len(trace)
        else:
            goals, option_ids, macro_clock = None, None, 0
            while True:
                free_decisions += int(env.decision_mask().sum())
                with torch.no_grad():
```

Line-by-line explanation.

3704 Define `evaluate_hrl(config, method, model, seeds)`. Execute each new hierarchy without learning, retaining the necessary option IDs, goals or macro clock and collecting the common physical metrics.

3706 Documentation text: Evaluate hierarchy controllers with a daily physical clock and fixed weights.. This states the contract or qualification for `evaluate_hrl`; it does not update the simulator or model.

3708 Test `method not in HRL_METHODS or model is None`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3710 Raise `ValueError('registered hierarchy and trained model required')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

3712 Set `records` by construct the shown list in its stated order.

3714 Iterate `seed` over `seeds`. The following indented statements run once for each selected element.

3716 Set `env` by call AircraftOptionEnv with the shown positional and keyword arguments. It holds the environment or wrapper that owns physical transitions.

3718 Set `(obs, _)` by initialize a new episode using the stated seed/options.

3720 Set `rng` by call np.random.default_rng with the shown positional and keyword arguments.

3722 Set `(decisions, free_decisions, terminations)` by construct the shown tuple in its stated order.

3724 Test `method == 'maxq'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3726 Set `(_, _, trace, _)` by call model.execute with the shown positional and keyword arguments.

3728 Set `decisions` by count entries.

3730 Execute this alternative only when the corresponding preceding condition did not select its branch.

3732 Set `(goals, option_ids, macro_clock)` by construct the shown tuple in its stated order.

3734 Repeat the indented block while `True` remains true; this condition controls the simulation/collection loop here.

3736 Add `int(env.decision_mask().sum())` to `free_decisions`, and store the result back in that same target. The change is visible to subsequent statements in this block.

3738 Enter `torch.no_grad()`. The block records no gradients, keeping rollout/target calculations outside backpropagation.

Source lines 3740–3774.

```
3740                    if method in HRL_PPO_METHODS:
3742                        action = model.act(obs, deterministic=True)[0]
3744                        choices = model.catalog_actions(obs, action)
3746                    elif method == 'option_critic':
3748                        if option_ids is None:
3750                            option_ids = model.q_values(obs)[0].argmax(-1).numpy()
3752                        ids = torch.tensor(option_ids).unsqueeze(0)
3754                        choices = model.policy(obs, ids, deterministic=True)[0][0].numpy()
3756                    else:
3758                        if goals is None or macro_clock == 0:
3760                            goals = model.choose_goals(obs, goals, rng)
3762                        choices = model.choose_actions(obs, goals, rng)
3764                obs, _, done, truncated, info = env.step(choices)
3766                decisions += 1
3768                if method == 'option_critic' and not (done or truncated):
3770                    with torch.no_grad():
3772                        beta = model.beta(obs, torch.tensor(option_ids).unsqueeze(0))[0].numpy()
3774                        next_ids = model.q_values(obs)[0].argmax(-1).numpy()
```

Line-by-line explanation.

3740 Test `method in HRL_PPO_METHODS`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3742 Set `action` by select the indexed/sliced entries of model.act(obs, deterministic=True). It holds the sampled or stored action whose probability is evaluated.

3744 Set `choices` by call model.catalog_actions with the shown positional and keyword arguments.

3746 Test `method == 'option_critic'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3748 Test `option_ids is None`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3750 Set `option_ids` by expose this CPU tensor as a NumPy array. It holds the current learned planning mode for each aircraft.

3752 Set `ids` by call torch.tensor(option_ids).unsqueeze with the shown positional and keyword arguments.

3754 Set `choices` by expose this CPU tensor as a NumPy array.

3756 Execute this alternative only when the corresponding preceding condition did not select its branch.

3758 Test `goals is None or macro_clock == 0`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3760 Set `goals` by call model.choose_goals with the shown positional and keyword arguments. It holds per-aircraft upper-level replacement budgets.

3762 Set `choices` by call model.choose_actions with the shown positional and keyword arguments.

3764 Set `(obs, _, done, truncated, info)` by execute this object's step with the shown arguments.

3766 Add `1` to `decisions`, and store the result back in that same target. The change is visible to subsequent statements in this block.

3768 Test `method == 'option_critic' and (not (done or truncated))`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3770 Enter `torch.no_grad()`. The block records no gradients, keeping rollout/target calculations outside backpropagation.

3772 Set `beta` by expose this CPU tensor as a NumPy array.

3774 Set `next_ids` by expose this CPU tensor as a NumPy array.

Source lines 3776–3806.

```
3776                    change = (beta >= .5) & env.decision_mask()
3778                    option_ids = np.where(change, next_ids, option_ids)
3780                    terminations += int(change.sum())
3782                if method == 'hierarchical_vdn':
3784                    macro_clock = (macro_clock + 1) % 3
3786                    rows, _ = env.env.split_observation(obs)
3788                    if np.any((rows[:, 24:36].min(-1) <= config.mandatory_health) &
                        ↪  env.decision_mask() & (goals == 0)):
3790                        macro_clock = 0
3792                if done or truncated:
3794                    break
3796        metrics = env.last_info['metrics']
3798        records.append({'seed': int(seed), 'decisions': decisions,
3800                        'learned_planning_opportunities': free_decisions,
3802                        'planning_mode_terminations': terminations,
3804                        'availability': metrics['flights'] / (config.aircraft * config.horizon),
                            ↪  **metrics})
3806    return records
```

Line-by-line explanation.

3776 Set `change` by apply a bitwise intersection to beta >= 0.5 and env.decision_mask().

3778 Set `option_ids` by call np.where with the shown positional and keyword arguments. It holds the current learned planning mode for each aircraft.

3780 Add `int(change.sum())` to `terminations`, and store the result back in that same target. The change is visible to subsequent statements in this block.

3782 Test `method == 'hierarchical_vdn'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3784 Set `macro_clock` by take the remainder of macro_clock + 1 and 3.

3786 Set `(rows, _)` by call env.env.split_observation with the shown positional and keyword arguments.

3788 Test `np.any((rows[:, 24:36].min(-1) <= config.mandatory_health) & env.decision_mask() & (goals == 0))`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3790 Set `macro_clock` by use the literal 0.

3792 Test `done or truncated`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3794 Exit the nearest loop immediately; execution resumes after that loop.

3796 Set `metrics` by select the indexed/sliced entries of env.last_info.

3798 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

3800 Continue the expression begun at line 3798 with `'learned_planning_opportunities': free_decisions,`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

3802 Continue the expression begun at line 3798 with `'planning_mode_terminations': terminations,`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

3804 Continue the expression begun at line 3798 with `'availability': metrics['flights'] / (config.aircraft * config.horizon), **metrics})`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

3806 Return `records` to the caller by evaluate records. No further statements in this call execute.

9.150. Module declarations and integration

Source lines 3810–3812.

```
3810 ALL_LEARNED_METHODS = ('flat', 'conditional', 'shared', 'options', 'learned_options',
3812                        *REQUEST_METHODS, *HRL_METHODS)
```

Line-by-line explanation.

3810 Set `ALL_LEARNED_METHODS` by construct the shown tuple in its stated order.

3812 Continue the assignment begun at line 3810 with `*REQUEST_METHODS, *HRL_METHODS)`. These arguments/entries retain their listed order and belong to the same operation; there is no additional physical-time step.

9.151. execute_option

Execute the original fleet-wide option call for several primitive days. Accepted repairs remain committed and other aircraft can fly, but no new learned fleet decision is made until this executor returns.

Source lines 3816–3844.

```
3816 def execute_option(env, option, obs, gamma=0.99, max_days=14, planned_action=None):
3818     """Plan once, then keep all active jobs committed while the fleet advances.
3820     Manager returns at first task completion, a new observed urgency alarm,
3821     episode end, or max_days. It never consumes true event diagnostics.
3822     Waiting without any accepted job lasts one day.
3823     Other operating aircraft continue their modeled flights during this option.
3824     Longer manager intervals change decision opportunities, so this is a timing
3825     experiment as well as an architecture experiment, not a matched PPO ablation.
3826     """
3828     option = int(option)
3830     if option not in range(4):
3832         raise ValueError('unknown option')
3834     if max_days < 1:
3836         raise ValueError('max_days must be positive')
3838     thresholds = {1: (0.62, 0.62), 2: (0.65, 0.77), 3: (0.78, 0.85)}
3840     action = planned_action if planned_action is not None else env.idle_action() if option == 0 else
     ↪  threshold_action(env, obs, *thresholds[option])
3842     before, _ = env.split_observation(obs)
3844     before_alarm = before[:, 24:36].min(1) <= env.config.mandatory_health
```

Line-by-line explanation.

3816 Define `execute_option(env, option, obs, gamma=0.99, max_days=14, planned_action=None)`. Execute the original fleet-wide option call for several primitive days. Accepted repairs remain committed and other aircraft can fly, but no new learned fleet decision is made until this executor returns.

3818 Documentation text: Plan once, then keep all active jobs committed while the fleet advances.. This states the contract or qualification for `execute_option`; it does not update the simulator or model.

3820 Documentation text: Manager returns at first task completion, a new observed urgency alarm,. This states the contract or qualification for `execute_option`; it does not update the simulator or model.

3821 Documentation text: episode end, or max_days. It never consumes true event diagnostics.. This states the contract or qualification for `execute_option`; it does not update the simulator or model.

3822 Documentation text: Waiting without any accepted job lasts one day.. This states the contract or qualification for `execute_option`; it does not update the simulator or model.

3823 Documentation text: Other operating aircraft continue their modeled flights during this option.. This states the contract or qualification for `execute_option`; it does not update the simulator or model.

3824 Documentation text: Longer manager intervals change decision opportunities, so this is a timing. This states the contract or qualification for `execute_option`; it does not update the simulator or model.

3825 Documentation text: experiment as well as an architecture experiment, not a matched PPO ablation.. This states the contract or qualification for `execute_option`; it does not update the simulator or model.

3826 Close the documentation string for `execute_option`. This delimiter ends the descriptive text and performs no simulator or learning update.

3828 Set `option` by convert the result to a Python integer. Compute the selected high-level action and store it in option; later checks/updates use this named intermediate.

3830 Test `option not in range(4)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3832 Raise `ValueError('unknown option')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

3834 Test `max_days < 1`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3836 Raise `ValueError('max_days must be positive')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

3838 Set `thresholds` by construct the shown dict in its stated order. Compute and bind the right-hand value to thresholds, naming an intermediate for the surrounding algorithm.

3840 Set `action` by select planned_action when planned_action is not None, otherwise env.idle_action() if option == 0 else threshold_action(env, obs, *thresholds[option]). It holds the sampled or stored action whose probability is evaluated. Compute the proposed or selected action and store it in action; later checks/updates use this named intermediate.

3842 Set `(before, _)` by call env.split_observation with the shown positional and keyword arguments. Unpack the return values from env.split_observation(obs) into before; underscore entries are intentionally ignored.

3844 Set `before_alarm` by evaluate the comparison before[:, 24:36].min(1) <= env.config.mandatory_health. Compute alarms already present before option execution and store it in before_alarm; later checks/updates use this named intermediate.

Source lines 3846–3866.

```
3846    rewards = []
3848    while True:
3850        obs, reward, done, truncated, info = env.step(action)
3852        rewards.append(reward)
3854        rows, _ = env.split_observation(obs)
3856        new_alarm = np.any((rows[:, 24:36].min(1) <= env.config.mandatory_health) & ~before_alarm)
3858        if option == 0 or done or truncated or info['completed'] or new_alarm or (len(rewards) >=
            ↪  max_days) or (not env.jobs):
3860            break
3862        action = env.idle_action()
3864    discounted = sum((gamma ** i * r for i, r in enumerate(rewards)))
3866    return (obs, discounted, done, truncated, info, len(rewards), sum(rewards))
```

Line-by-line explanation.

3846 Set `rewards` by construct the shown list in its stated order. It holds the primitive reward sequence used for return accounting. Compute the primitive rewards collected during an option and store it in rewards; later checks/updates use this named intermediate.

3848 Repeat the indented block while `True` remains true; this condition controls the simulation/collection loop here.

3850 Set `(obs, reward, done, truncated, info)` by execute this object's step with the shown arguments. Unpack the environment step result in Gym order: next observation, reward, terminated, truncated, and info. The underscore field is intentionally discarded when present.

3852 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

3854 Set `(rows, _)` by call env.split_observation with the shown positional and keyword arguments. Unpack the return values from env.split_observation(obs) into rows; underscore entries are intentionally ignored.

3856 Set `new_alarm` by call np.any with the shown positional and keyword arguments. Compute a newly appearing alarm that can end option execution and store it in new_alarm; later checks/updates use this named intermediate.

3858 Test `option == 0 or done or truncated or info['completed'] or new_alarm or (len(rewards) >= max_days) or (not env.jobs)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3860 Exit the nearest loop immediately; execution resumes after that loop.

3862 Set `action` by call env.idle_action with the shown positional and keyword arguments. It holds the sampled or stored action whose probability is evaluated. Compute the proposed or selected action and store it in action; later checks/updates use this named intermediate.

3864 Set `discounted` by add the terms of the supplied iterable. Compute the option return after applying time-discount factors and store it in discounted; later checks/updates use this named intermediate.

3866 Return `(obs, discounted, done, truncated, info, len(rewards), sum(rewards))` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

9.152. Module declarations and integration

Source lines 3869–3885.

```
3869  # ======================================================================
3870  # fleet_maintenance/train.py
3871  # ======================================================================
3875  from dataclasses import asdict
3877  from pathlib import Path
3879  import argparse
3881  import json
3883  import numpy as np
3885  import torch
```

Line-by-line explanation.

3869 A visual section separator in the source. It has no executable effect.

3870 Source comment: fleet_maintenance/train.py This line documents the following code and does not execute.

3871 A visual section separator in the source. It has no executable effect.

3875 Import `from dataclasses import asdict`. Dataclasses provide structured configuration and repair records.

3877 Import `from pathlib import Path`. Path handles output/checkpoint paths.

3879 Import `import argparse`. Argument parsing configures command-line experiments.

3881 Import `import json`. JSON stores settings and diagnostic records.

3883 Import `import numpy as np`. The imported names are used by the definitions below; importing does not start a training run.

3885 Import `import torch`. Torch supplies tensors, automatic differentiation and model persistence.

9.153. make_model

Construct the policy class that matches the method registry, observation dimensions, fleet size and ownership. A method label selects a concrete implementation, not a cosmetic plotting name.

Source lines 3888–3918.

```
3888 def make_model(env, method):
3890     if method in HRL_PPO_METHODS:
3892         return AircraftPlanningHierarchy(env, method)
3894     if method == 'maxq':
3896         return MAXQMaintenance(env)
3898     if method == 'option_critic':
3900         return FleetOptionCritic(env)
3902     if method == 'hierarchical_vdn':
3904         return HierarchicalVDN(env)
3906     if method in DCMAC_METHODS:
3908         return DCMACActorCritic(env, request_assignment(method), method.startswith('ddmac_'))
3910     if method in MAPPO_METHODS:
3912         return MultiAgentActorCritic(env, 'aircraft' if method == 'mappo_aircraft' else 'component')
3914     if method == 'learned_options':
3916         return LearnedOptionActorCritic(env)
3918     return OptionActorCritic(env) if method == 'options' else FleetActorCritic(env, method)
```

Line-by-line explanation.

3888 Define `make_model(env, method)`. Construct the policy class that matches the method registry, observation dimensions, fleet size and ownership. A method label selects a concrete implementation, not a cosmetic plotting name.

3890 Test `method in HRL_PPO_METHODS`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3892 Return `AircraftPlanningHierarchy(env, method)` to the caller by call AircraftPlanningHierarchy with the shown positional and keyword arguments. No further statements in this call execute.

3894 Test `method == 'maxq'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3896 Return `MAXQMaintenance(env)` to the caller by call MAXQMaintenance with the shown positional and keyword arguments. No further statements in this call execute.

3898 Test `method == 'option_critic'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3900 Return `FleetOptionCritic(env)` to the caller by call FleetOptionCritic with the shown positional and keyword arguments. No further statements in this call execute.

3902 Test `method == 'hierarchical_vdn'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3904 Return `HierarchicalVDN(env)` to the caller by call HierarchicalVDN with the shown positional and keyword arguments. No further statements in this call execute.

3906 Test `method in DCMAC_METHODS`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3908 Return `DCMACActorCritic(env, request_assignment(method), method.startswith('ddmac_'))` to the caller by call DCMACActorCritic with the shown positional and keyword arguments. No further statements in this call execute.

3910 Test `method in MAPPO_METHODS`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3912 Return `MultiAgentActorCritic(env, 'aircraft' if method == 'mappo_aircraft' else 'component')` to the caller by call MultiAgentActorCritic with the shown positional and keyword arguments. No further statements in this call execute.

3914 Test `method == 'learned_options'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3916 Return `LearnedOptionActorCritic(env)` to the caller by call LearnedOptionActorCritic with the shown positional and keyword arguments. No further statements in this call execute.

3918 Return `OptionActorCritic(env) if method == 'options' else FleetActorCritic(env, method)` to the caller by select OptionActorCritic(env) when method == 'options', otherwise FleetActorCritic(env, method). No further statements in this call execute.

9.154. load_checkpoint

Read saved configuration, method and model state; construct the matching architecture and restore weights or MAXQ tables.

Source lines 3921–3939.

```
3921 def load_checkpoint(path):
3923     """Reload one of this package's own checkpoints for independent evaluation."""
3925     checkpoint = torch.load(path, map_location='cpu', weights_only=True)
3927     if checkpoint.get('observation_version') != OBSERVATION_VERSION:
3929         raise ValueError('Checkpoint uses an incompatible observation/action layout. Retrain with
         ↪  individual crew masks and resource factors (version 3).')
3931     config = Config(**checkpoint['config'])
3933     model = make_model(FleetMaintenanceEnv(config), checkpoint['method'])
3935     model.load_state_dict(checkpoint['state_dict'])
3937     model.eval()
3939     return (config, checkpoint['method'], model)
```

Line-by-line explanation.

3921 Define `load_checkpoint(path)`. Read saved configuration, method and model state; construct the matching architecture and restore weights or MAXQ tables.

3923 Documentation text: Reload one of this package's own checkpoints for independent evaluation.. This states the contract or qualification for `load_checkpoint`; it does not update the simulator or model.

3925 Set `checkpoint` by call torch.load with the shown positional and keyword arguments. Compute and bind the right-hand value to checkpoint, naming an intermediate for the surrounding algorithm.

3927 Test `checkpoint.get('observation_version') != OBSERVATION_VERSION`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3929 Raise `ValueError('Checkpoint uses an incompatible observation/action layout. Retrain with individual crew masks and resource factors (version 3).')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

3931 Set `config` by call Config with the shown positional and keyword arguments. Compute and bind the right-hand value to config, naming an intermediate for the surrounding algorithm.

3933 Set `model` by call make_model with the shown positional and keyword arguments. It holds the chosen controller architecture. Compute and bind the right-hand value to model, naming an intermediate for the surrounding algorithm.

3935 Restore weights and registered extra state. The exact receiver and arguments are shown on this source line.

3937 Call model.eval with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

3939 Return `(config, checkpoint['method'], model)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

9.155. smdp_gae

Compute generalized advantages with discount and trace factors raised to each transition duration for the original fleet-level option trainer.

Source lines 3942–3964.

```
def smdp_gae(rewards, values, next_values, dones, durations, gamma=0.99, lam=0.95):
    """Backward GAE with gamma**duration and lambda**duration per decision.
    rewards already contain the discounted primitive rewards inside each option.
    A one-day decision recovers ordinary GAE. Episode boundaries block bootstrap.
    """
    advantage = np.zeros(len(rewards), dtype=np.float32)
    carry = 0.0
    for t in range(len(rewards) - 1, -1, -1):
        live = 1.0 - float(dones[t])
        delta = rewards[t] + gamma ** durations[t] * next_values[t] * live - values[t]
        carry = delta + (gamma * lam) ** durations[t] * live * carry
        advantage[t] = carry
    return (advantage, advantage + np.asarray(values))
```

Line-by-line explanation.

3942 Define `smdp_gae(rewards, values, next_values, dones, durations, gamma=0.99, lam=0.95)`. Compute generalized advantages with discount and trace factors raised to each transition duration for the original fleet-level option trainer.

3944 Documentation text: Backward GAE with gamma**duration and lambda**duration per decision.. This states the contract or qualification for `smdp_gae`; it does not update the simulator or model.

3946 Documentation text: rewards already contain the discounted primitive rewards inside each option.. This states the contract or qualification for `smdp_gae`; it does not update the simulator or model.

3947 Documentation text: A one-day decision recovers ordinary GAE. Episode boundaries block bootstrap.. This states the contract or qualification for `smdp_gae`; it does not update the simulator or model.

3948 Close the documentation string for `smdp_gae`. This delimiter ends the descriptive text and performs no simulator or learning update.

3950 Set `advantage` by call np.zeros with the shown positional and keyword arguments. Initialize advantage with zeros as a neutral container for the GAE estimate used by PPO; later lines fill or update it.

3952 Set `carry` by use the literal 0.0. Compute the later-decision advantage propagated backward and store it in carry; later checks/updates use this named intermediate.

3954 Iterate `t` over `range(len(rewards) - 1, -1, -1)`. The following indented statements run once for each selected element.

3956 Set `live` by subtract 1.0 and float(dones[t]). Compute and bind the right-hand value to live, naming an intermediate for the surrounding algorithm.

3958 Set `delta` by subtract rewards[t] + gamma ** durations[t] * next_values[t] * live and values[t]. TD residual = option reward + next value discounted across all primitive days - current value. live is zero at terminal boundaries, so no terminal bootstrap is added.

3960 Set `carry` by add delta and (gamma * lam) ** durations[t] * live * carry. Build backward GAE; discount and lambda are each adjusted for the option's duration, and live prevents advantages crossing episode boundaries.

3962 Set `advantage[t]` by evaluate carry. Compute the GAE estimate used by PPO and store it in advantage[t]; later checks/updates use this named intermediate.

3964 Return `(advantage, advantage + np.asarray(values))` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

9.156. train

Dispatch new method families to their dedicated trainers. For the original five methods, collect transitions, account for duration, and optimize the fleet-joint PPO objective.

Source lines 3967–4001.

```
3967  def train(config, method='conditional', steps=4096, seed=0, rollout_size=128):
3969      """Interaction budget counts actual simulated days, not manager decisions."""
3972      if method in HRL_PPO_METHODS:
3974          return train_aircraft_hrl(config, method, steps, seed, rollout_size)
3976      if method == 'maxq':
3978          return train_maxq(config, steps, seed)
3980      if method == 'option_critic':
3982          return train_option_critic(config, steps, seed)
3984      if method == 'hierarchical_vdn':
3986          return train_hierarchical_vdn(config, steps, seed)
3987      if method in DCMAC_METHODS:
3989          return train_dcmac(config, method, steps, seed, rollout_size)
3991      if method in MAPPO_METHODS:
3993          return train_mappo(config, method, steps, seed, rollout_size)
```

```
    if steps < 1 or rollout_size < 2:
        raise ValueError('positive budget and rollout>=2 required')
    torch.set_num_threads(1)
    torch.manual_seed(seed)
```

Line-by-line explanation.

3967 Define `train(config, method='conditional', steps=4096, seed=0, rollout_size=128)`. Dispatch new method families to their dedicated trainers. For the original five methods, collect transitions, account for duration, and optimize the fleet-joint PPO objective.

3969 Documentation text: Interaction budget counts actual simulated days, not manager decisions.. This states the contract or qualification for `train`; it does not update the simulator or model.

3972 Test `method in HRL_PPO_METHODS`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3974 Return `train_aircraft_hrl(config, method, steps, seed, rollout_size)` to the caller by call train_aircraft_hrl with the shown positional and keyword arguments. No further statements in this call execute.

3976 Test `method == 'maxq'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3978 Return `train_maxq(config, steps, seed)` to the caller by call train_maxq with the shown positional and keyword arguments. No further statements in this call execute.

3980 Test `method == 'option_critic'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3982 Return `train_option_critic(config, steps, seed)` to the caller by call train_option_critic with the shown positional and keyword arguments. No further statements in this call execute.

3984 Test `method == 'hierarchical_vdn'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3986 Return `train_hierarchical_vdn(config, steps, seed)` to the caller by call train_hierarchical_vdn with the shown positional and keyword arguments. No further statements in this call execute.

3987 Test `method in DCMAC_METHODS`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3989 Return `train_dcmac(config, method, steps, seed, rollout_size)` to the caller by call train_dcmac with the shown positional and keyword arguments. No further statements in this call execute.

3991 Test `method in MAPPO_METHODS`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3993 Return `train_mappo(config, method, steps, seed, rollout_size)` to the caller by call train_mappo with the shown positional and keyword arguments. No further statements in this call execute.

3995 Test `steps < 1 or rollout_size < 2`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

3997 Raise `ValueError('positive budget and rollout>=2 required')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

3999 Set the cpu thread count for these small models. The exact receiver and arguments are shown on this source line.

4001 Seed neural initialization and torch sampling. The exact receiver and arguments are shown on this source line.

Source lines 4003–4037.

```
    env = FleetMaintenanceEnv(config)
    model = make_model(env, method)
    optimizer = torch.optim.Adam(model.parameters(), lr=0.0003)
    rng = np.random.default_rng(seed)
    episode = 0
    obs, _ = env.reset(seed=seed)
    simulated = 0
    logs = []
    while simulated < steps:
        data = {k: [] for k in ('obs', 'action', 'logp', 'value', 'next_value', 'reward', 'done',
        ↪  'duration')}
        while len(data['obs']) < rollout_size and simulated < steps:
            with torch.no_grad():
                action, logp, _, value = model.act(torch.tensor(obs))
            chosen = action[0].numpy()
            if method in ('options', 'learned_options'):
                macro = int(chosen) if method == 'options' else int(chosen[0])
                plan = None if method == 'options' else chosen[1:].reshape(config.aircraft, 8)
                next_obs, reward, done, truncated, info, duration, _ = execute_option(env, macro, obs,
                ↪  max_days=min(14, steps - simulated), planned_action=plan)
```

Line-by-line explanation.

4003 Set `env` by call FleetMaintenanceEnv with the shown positional and keyword arguments. It holds the environment or wrapper that owns physical transitions. Compute and bind the right-hand value to env, naming an intermediate for the surrounding algorithm.

4005 Set `model` by call make_model with the shown positional and keyword arguments. It holds the chosen controller architecture. Compute and bind the right-hand value to model, naming an intermediate for the surrounding algorithm.

4007 Set `optimizer` by create an Adam optimizer for the specified parameter collection. Compute and bind the right-hand value to optimizer, naming an intermediate for the surrounding algorithm.

4009 Set `rng` by call np.random.default_rng with the shown positional and keyword arguments. Compute and bind the right-hand value to rng, naming an intermediate for the surrounding algorithm.

4011 Set `episode` by use the literal 0. Compute the current training episode index and store it in episode; later checks/updates use this named intermediate.

4013 Set `(obs, _)` by initialize a new episode using the stated seed/options. Unpack reset's pair into the starting observation and auxiliary info; the underscore means this caller does not need the info dictionary.

4015 Set `simulated` by use the literal 0. Compute and bind the right-hand value to simulated, naming an intermediate for the surrounding algorithm.

4017 Set `logs` by construct the shown list in its stated order. It holds episode/update diagnostics for later inspection. Compute and bind the right-hand value to logs, naming an intermediate for the surrounding algorithm.

4019 Repeat the indented block while `simulated < steps` remains true; this condition controls the simulation/collection loop here.

4021 Set `data` by construct the collection by iterating the shown comprehension and applying its filters. Compute and bind the right-hand value to data, naming an intermediate for the surrounding algorithm.

4023 Repeat the indented block while `len(data['obs']) < rollout_size and simulated < steps` remains true; this condition controls the simulation/collection loop here.

4025 Enter `torch.no_grad()`. The block records no gradients, keeping rollout/target calculations outside backpropagation.

4027 Set `(action, logp, _, value)` by call model.act with the shown positional and keyword arguments. Unpack the policy result into its selected action, likelihood, entropy, and value outputs; underscore positions deliberately discard unused statistics.

4029 Set `chosen` by expose this CPU tensor as a NumPy array. Compute and bind the right-hand value to chosen, naming an intermediate for the surrounding algorithm.

4031 Test `method in ('options', 'learned_options')`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

4033 Set `macro` by select int(chosen) when method == 'options', otherwise int(chosen[0]). Compute the manager's high-level option ID and store it in macro; later checks/updates use this named intermediate.

4035 Set `plan` by select None when method == 'options', otherwise chosen[1:].reshape(config.aircraft, 8). Compute the proposed or accepted maintenance plan and store it in plan; later checks/updates use this named intermediate.

4037 Set `(next_obs, reward, done, truncated, info, duration, _)` by call execute_option with the shown positional and keyword arguments. Unpack the return values from execute_option(env, macro, obs, max_days=min(14, steps - simulated), planned_action=plan) into next_obs, reward, done, truncated, info, duration; underscore entries are intentionally ignored.

Source lines 4039–4073.

```
4039            else:
4041                next_obs, reward, done, truncated, info = env.step(chosen)
4043                duration = 1
4045            terminal = done or truncated
4047            with torch.no_grad():
4049                nv = float(model.value(torch.tensor(next_obs).unsqueeze(0))[0])
4051            for key, item in [('obs', obs.copy()), ('action', np.asarray(chosen).copy()), ('logp',
                ↪  float(logp[0])), ('value', float(value[0])), ('next_value', nv), ('reward', reward),
                ↪  ('done', terminal), ('duration', duration)]:
4053                data[key].append(item)
4055            simulated += duration
4057            obs = next_obs
4059            if terminal:
4061                logs.append({'episode': episode, 'simulated_days': simulated, **info['metrics']})
4063                episode += 1
4065                obs, _ = env.reset(seed=seed + episode)
```

```
4067            advantages, targets = smdp_gae(data['reward'], data['value'], data['next_value'],
                ↪  data['done'], data['duration'])
4069            tensors = {k: torch.tensor(np.asarray(v)) for k, v in data.items() if k in ('obs', 'action',
                ↪  'logp')}
4071            advantages = torch.tensor(advantages)
4073            if len(advantages) > 1:
```

Line-by-line explanation.

4039 Execute this alternative only when the corresponding preceding condition did not select its branch.

4041 Set `(next_obs, reward, done, truncated, info)` by execute this object's step with the shown arguments. Unpack the environment step result in Gym order: next observation, reward, terminated, truncated, and info. The underscore field is intentionally discarded when present.

4043 Set `duration` by use the literal 1. It holds the number of primitive days in this decision. Compute the primitive days consumed by a job or option and store it in duration; later checks/updates use this named intermediate.

4045 Set `terminal` by evaluate done or truncated. It holds the episode boundary used to suppress continuation value. Compute and bind the right-hand value to terminal, naming an intermediate for the surrounding algorithm.

4047 Enter `torch.no_grad()`. The block records no gradients, keeping rollout/target calculations outside backpropagation.

4049 Set `nv` by convert the result to a Python floating-point number. Compute and bind the right-hand value to nv, naming an intermediate for the surrounding algorithm.

4051 Iterate `(key, item)` over `[('obs', obs.copy()), ('action', np.asarray(chosen).copy()), ('logp', float(logp[0])), ('value', float(value[0])), ('next_value', nv), ('reward', reward), ('done', terminal), ('duration', duration)]`. The following indented statements run once for each selected element.

4053 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

4055 Add `duration` to `simulated`, and store the result back in that same target. The change is visible to subsequent statements in this block.

4057 Set `obs` by evaluate next_obs. It holds the observed belief vector, rather than hidden physical health. Compute the current encoded state and store it in obs; later checks/updates use this named intermediate.

4059 Test `terminal`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

4061 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

4063 Add `1` to `episode`, and store the result back in that same target. The change is visible to subsequent statements in this block.

4065 Set `(obs, _)` by initialize a new episode using the stated seed/options. Unpack reset's pair into the starting observation and auxiliary info; the underscore means this caller does not need the info dictionary.

4067 Set `(advantages, targets)` by call smdp_gae with the shown positional and keyword arguments. Unpack smdp_gae(data['reward'], data['value'], data['next_value'], data['done'], data['duration']) into named values advantages, targets so each part can be used separately below.

4069 Set `tensors` by construct the collection by iterating the shown comprehension and applying its filters. Compute and bind the right-hand value to tensors, naming an intermediate for the surrounding algorithm.

4071 Set `advantages` by call torch.tensor with the shown positional and keyword arguments. It holds the policy credit signal with its retained aircraft/agent axis. Compute and bind the right-hand value to advantages, naming an intermediate for the surrounding algorithm.

4073 Test `len(advantages) > 1`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

Source lines 4075–4107.

```
            advantages = (advantages - advantages.mean()) / (advantages.std(unbiased=False) + 1e-08)
        targets = torch.tensor(targets, dtype=torch.float32)
        for _ in range(4):
            ids = rng.permutation(len(targets))
            for start in range(0, len(ids), 64):
                idx = ids[start:start + 64]
                _, newlog, entropy, values = model.act(tensors['obs'][idx], tensors['action'][idx])
                ratio = torch.exp(newlog - tensors['logp'][idx])
                surrogate = torch.minimum(ratio * advantages[idx], ratio.clamp(0.8, 1.2) *
                ↪  advantages[idx])
                loss = -surrogate.mean() + 0.5 * ((values - targets[idx]) ** 2).mean() - 0.001 *
                ↪  entropy.mean()
                if not torch.isfinite(loss):
                    raise RuntimeError('nonfinite PPO loss')
                optimizer.zero_grad()
                loss.backward()
                torch.nn.utils.clip_grad_norm_(model.parameters(), 0.5)
                optimizer.step()
    return (model, logs, simulated)
```

Line-by-line explanation.

4075 Set `advantages` by divide advantages - advantages.mean() and advantages.std(unbiased=False) + 1e-08. It holds the policy credit signal with its retained aircraft/agent axis. Compute and bind the right-hand value to advantages, naming an intermediate for the surrounding algorithm.

4077 Set `targets` by call torch.tensor with the shown positional and keyword arguments. Compute and bind the right-hand value to targets, naming an intermediate for the surrounding algorithm.

4079 Iterate `_` over `range(4)`. The following indented statements run once for each selected element.

4081 Set `ids` by call rng.permutation with the shown positional and keyword arguments. Compute and bind the right-hand value to ids, naming an intermediate for the surrounding algorithm.

4083 Iterate `start` over `range(0, len(ids), 64)`. The following indented statements run once for each selected element.

4085 Set `idx` by select the indexed/sliced entries of ids. Compute and bind the right-hand value to idx, naming an intermediate for the surrounding algorithm.

4087 Set `(_, newlog, entropy, values)` by call model.act with the shown positional and keyword arguments. Unpack the policy result into its selected action, likelihood, entropy, and value outputs; underscore positions deliberately discard unused statistics.

4089 Set `ratio` by exponentiate log values, including log-probability ratios. It holds the current-to-behavior probability ratio. Exponentiate the log-probability difference to obtain pinew(a|s)/piold(a|s), PPO's policy-update ratio.

4091 Set `surrogate` by call torch.minimum with the shown positional and keyword arguments. Take the smaller unclipped/clipped objective; clipping the ratio to 0.8–1.2 limits how far this minibatch moves the policy likelihood.

4093 Set `loss` by subtract -surrogate.mean() + 0.5 * ((values - targets[idx]) ** 2).mean() and 0.001 * entropy.mean(). Combine policy loss, half-weighted squared critic error, and a small negative entropy term that rewards exploration.

4095 Test `not torch.isfinite(loss)`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

4097 Raise `RuntimeError('nonfinite PPO loss')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

4099 Clear gradients from the previous optimization step. The exact receiver and arguments are shown on this source line.

4101 Differentiate the loss and accumulate parameter gradients. The exact receiver and arguments are shown on this source line.

4103 Cap the joint gradient norm before the optimizer changes weights. The exact receiver and arguments are shown on this source line.

4105 Apply accumulated gradients to the optimizer's registered parameters. The exact receiver and arguments are shown on this source line.

4107 Return `(model, logs, simulated)` to the caller by construct the shown tuple in its stated order. No further statements in this call execute.

9.157. evaluate

Dispatch hierarchy evaluation or roll out the original/request controllers on held-out seeds without gradient updates. Aggregate metrics retain the common environment definitions.

Source lines 4111–4145.

```
4111 def evaluate(config, method, model=None, seeds=range(10000, 10010)):
4113     """Evaluate fixed weights on held-out seeds; proposal methods use their common broker."""
4116     if method in HRL_METHODS:
4118         return evaluate_hrl(config, method, model, seeds)
4119     if method != 'threshold' and model is None:
4121         raise ValueError('evaluation requires a trained model')
4123     records = []
4125     for seed in seeds:
4127         if method in REQUEST_METHODS:
4129             env = MultiAgentFleetEnv(config, request_assignment(method))
4131             _, info = env.reset(seed=int(seed))
4133             obs = info['central_observation']
4135         else:
4137             env = FleetMaintenanceEnv(config)
4139             obs, _ = env.reset(seed=int(seed))
4141         decisions = 0
4143         while True:
4145             if method == 'threshold':
```

Line-by-line explanation.

`4111` Define `evaluate(config, method, model=None, seeds=range(10000, 10010))`. Dispatch hierarchy evaluation or roll out the original/request controllers on held-out seeds without gradient updates. Aggregate metrics retain the common environment definitions.

`4113` Documentation text: Evaluate fixed weights on held-out seeds; proposal methods use their common broker.. This states the contract or qualification for `evaluate`; it does not update the simulator or model.

`4116` Test `method in HRL_METHODS`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

`4118` Return `evaluate_hrl(config, method, model, seeds)` to the caller by call evaluate_hrl with the shown positional and keyword arguments. No further statements in this call execute.

`4119` Test `method != 'threshold' and model is None`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

`4121` Raise `ValueError('evaluation requires a trained model')` immediately. The invalid request/configuration is rejected rather than silently changing its meaning.

`4123` Set `records` by construct the shown list in its stated order. Execute this step of the multi-agent observation, proposal, or learning calculation.

`4125` Iterate `seed` over `seeds`. The following indented statements run once for each selected element.

`4127` Test `method in REQUEST_METHODS`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

`4129` Set `env` by call MultiAgentFleetEnv with the shown positional and keyword arguments. It holds the environment or wrapper that owns physical transitions. Execute this step of the multi-agent observation, proposal, or learning calculation.

`4131` Set `(_, info)` by initialize a new episode using the stated seed/options. Execute this step of the multi-agent observation, proposal, or learning calculation.

`4133` Set `obs` by select the indexed/sliced entries of info. It holds the observed belief vector, rather than hidden physical health. Execute this step of the multi-agent observation, proposal, or learning calculation.

`4135` Execute this alternative only when the corresponding preceding condition did not select its branch.

`4137` Set `env` by call FleetMaintenanceEnv with the shown positional and keyword arguments. It holds the environment or wrapper that owns physical transitions. Execute this step of the multi-agent observation, proposal, or learning calculation.

`4139` Set `(obs, _)` by initialize a new episode using the stated seed/options. Execute this step of the multi-agent observation, proposal, or learning calculation.

`4141` Set `decisions` by use the literal 0. Execute this step of the multi-agent observation, proposal, or learning calculation.

`4143` Repeat the indented block while `True` remains true; this condition controls the simulation/collection loop here.

4145 Test `method == 'threshold'`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

Source lines 4147–4183.

```
4147                obs, _, done, truncated, info = env.step(threshold_action(env, obs))
4149            else:
4151                with torch.no_grad():
4154                    if method in REQUEST_METHODS:
4156                        views = np.stack(list(env.observations().values())) if method in MAPPO_METHODS
                            ↪  else obs
4159                        action, _, _ = model.policy(torch.tensor(views).unsqueeze(0) if method in
                            ↪  MAPPO_METHODS else torch.tensor(views), deterministic=True)
4161                    else:
4163                        action, _, _, _ = model.act(torch.tensor(obs), deterministic=True)
4165                if method in REQUEST_METHODS:
4167                    _, _, done, truncated, info = env.step(model.action_dict(action[0]))
4169                    obs = info['central_observation']
4171                elif method in ('options', 'learned_options'):
4173                    chosen = action[0].numpy()
4175                    macro = int(chosen) if method == 'options' else int(chosen[0])
4177                    plan = None if method == 'options' else chosen[1:].reshape(config.aircraft, 8)
4179                    obs, _, done, truncated, info, _, _ = execute_option(env, macro, obs,
                        ↪  planned_action=plan)
4181                else:
4183                    obs, _, done, truncated, info = env.step(action[0].numpy())
```

Line-by-line explanation.

4147 Set `(obs, _, done, truncated, info)` by execute this object's step with the shown arguments. Execute this step of the multi-agent observation, proposal, or learning calculation.

4149 Execute this alternative only when the corresponding preceding condition did not select its branch.

4151 Enter `torch.no_grad()`. The block records no gradients, keeping rollout/target calculations outside backpropagation.

4154 Test `method in REQUEST_METHODS`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

4156 Set `views` by select np.stack(list(env.observations().values())) when method in MAPPO_METHODS, otherwise obs. Stack exactly the decentralized observations exposed by the adapter.

4159 Set `(action, _, _)` by call model.policy with the shown positional and keyword arguments. Run the proposal policies alone; the value network is not required for execution.

4161 Execute this alternative only when the corresponding preceding condition did not select its branch.

4163 Set `(action, _, _, _)` by call model.act with the shown positional and keyword arguments. Obtain the baseline action from its existing policy.

4165 Test `method in REQUEST_METHODS`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

4167 Set `(_, _, done, truncated, info)` by execute this object's step with the shown arguments. Execute this step of the multi-agent observation, proposal, or learning calculation.

4169 Set `obs` by select the indexed/sliced entries of info. It holds the observed belief vector, rather than hidden physical health. Execute this step of the multi-agent observation, proposal, or learning calculation.

4171 Test `method in ('options', 'learned_options')`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

4173 Set `chosen` by expose this CPU tensor as a NumPy array. Execute this step of the multi-agent observation, proposal, or learning calculation.

4175 Set `macro` by select int(chosen) when method == 'options', otherwise int(chosen[0]). Execute this step of the multi-agent observation, proposal, or learning calculation.

4177 Set `plan` by select None when method == 'options', otherwise chosen[1:].reshape(config.aircraft, 8). Execute this step of the multi-agent observation, proposal, or learning calculation.

4179 Set `(obs, _, done, truncated, info, _, _)` by call execute_option with the shown positional and keyword arguments. Execute this step of the multi-agent observation, proposal, or learning calculation.

4181 Execute this alternative only when the corresponding preceding condition did not select its branch.

4183 Set `(obs, _, done, truncated, info)` by execute this object's step with the shown arguments. Execute this step of the multi-agent observation, proposal, or learning calculation.

Source lines 4185–4193.

```
4185                decisions += 1
4187                if done or truncated:
4189                    break
4191            records.append({'seed': int(seed), 'decisions': decisions, 'availability':
                ↪  info['metrics']['flights'] / (config.aircraft * config.horizon), **info['metrics']})
4193        return records
```

Line-by-line explanation.

4185 Add `1` to `decisions`, and store the result back in that same target. The change is visible to subsequent statements in this block.

4187 Test `done or truncated`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

4189 Exit the nearest loop immediately; execution resumes after that loop.

4191 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

4193 Return `records` to the caller by evaluate records. No further statements in this call execute.

9.158. main

Parse command-line configuration and run requested methods, seeds, checkpoint saving and evaluation. The all-method path explicitly records the one-aircraft MAXQ restriction.

Source lines 4196–4230.

```
4196 def main():
4198     parser = argparse.ArgumentParser()
4200     parser.add_argument('--method', choices=[*ALL_LEARNED_METHODS, 'all'], default='all')
4202     parser.add_argument('--steps', type=int, default=4096)
4204     parser.add_argument('--aircraft', type=int, default=7)
4206     parser.add_argument('--crews', type=int, default=3)
4208     parser.add_argument('--workshops', type=int, default=2)
4210     parser.add_argument('--crew-time-factors', type=float, nargs='+', default=())
4212     parser.add_argument('--workshop-time-factors', type=float, nargs='+', default=())
4214     parser.add_argument('--horizon', type=int, default=90)
4216     parser.add_argument('--seed', type=int, default=0)
4218     parser.add_argument('--eval-episodes', type=int, default=10)
4220     parser.add_argument('--output', default='runs/smoke')
4222     args = parser.parse_args()
4224     config = Config(aircraft=args.aircraft, horizon=args.horizon, crews=args.crews,
        ↪  workshops=args.workshops, crew_time_factors=tuple(args.crew_time_factors),
        ↪  workshop_time_factors=tuple(args.workshop_time_factors))
4226     destination = Path(args.output)
4228     destination.mkdir(parents=True, exist_ok=True)
4230     result = {'config': asdict(config), 'training_seed': args.seed, 'requested_steps': args.steps,
        ↪  'observation_version': OBSERVATION_VERSION, 'status': 'preliminary teaching experiment; not a
        ↪  paper reproduction or convergence claim', 'comparison_note': 'new HRL methods share a
        ↪  restricted 22-action catalog and broker; hrl_flat matches planning_hierarchy authority;
        ↪  concurrent_options has a fixed worker; target/option/VDN clocks and objectives differ; MAXQ is
        ↪  single-aircraft; legacy and MAPPO/DCMAC use broader action spaces', 'evaluation':
        ↪  {'threshold': evaluate(config, 'threshold', seeds=range(10000, 10000 + args.eval_episodes))}}
```

Line-by-line explanation.

4196 Define `main()`. Parse command-line configuration and run requested methods, seeds, checkpoint saving and evaluation. The all-method path explicitly records the one-aircraft MAXQ restriction.

4198 Set `parser` by call argparse.ArgumentParser with the shown positional and keyword arguments. Compute and bind the right-hand value to parser, naming an intermediate for the surrounding algorithm.

4200 Call parser.add_argument with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

4202 Call parser.add_argument with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

4204 Call parser.add_argument with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

4206 Call parser.add_argument with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

4208 Call parser.add_argument with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

4210 Call parser.add_argument with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

4212 Call parser.add_argument with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

4214 Call parser.add_argument with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

4216 Call parser.add_argument with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

4218 Call parser.add_argument with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

4220 Call parser.add_argument with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

4222 Set `args` by call parser.parse_args with the shown positional and keyword arguments. Compute and bind the right-hand value to args, naming an intermediate for the surrounding algorithm.

4224 Set `config` by call Config with the shown positional and keyword arguments. Compute and bind the right-hand value to config, naming an intermediate for the surrounding algorithm.

4226 Set `destination` by call Path with the shown positional and keyword arguments. Compute and bind the right-hand value to destination, naming an intermediate for the surrounding algorithm.

4228 Call destination.mkdir with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

4230 Set `result` by construct the shown dict in its stated order. Compute and bind the right-hand value to result, naming an intermediate for the surrounding algorithm.

Source lines 4232–4264.

```
    methods = list(ALL_LEARNED_METHODS) if args.method == 'all' else [args.method]
    if args.method == 'all' and config.aircraft != 1:
        methods.remove('maxq')
        result['skipped_methods'] = {'maxq': 'requires aircraft=1'}
    for method in methods:
        model, logs, days = train(config, method, args.steps, args.seed)
        torch.save({'state_dict': model.state_dict(), 'config': asdict(config), 'method': method,
        ↪  'training_seed': args.seed, 'simulated_days': days, 'observation_version':
        ↪  OBSERVATION_VERSION}, destination / f'{method}.pt')
        result['evaluation'][method] = evaluate(config, method, model, seeds=range(10000, 10000 +
        ↪  args.eval_episodes))
        result[method + '_training'] = logs
        print(method, 'days', days, 'mean test return', np.mean([r['return'] for r in
        ↪  result['evaluation'][method]]), flush=True)
    (destination / 'results.json').write_text(json.dumps(result, indent=2), encoding='utf-8')
    summary = ['# Preliminary comparison', '', result['status'], '', result['comparison_note'], '', '|
    ↪  Method | Return | Availability | Failed flights | Rejected plans | Decisions |',
    ↪  '|---|---:|---:|---:|---:|---:|']
    for method, records in result['evaluation'].items():
        mean = lambda k: np.mean([r[k] for r in records])
        summary.append(f"| {method} | {mean('return'):.2f} | {mean('availability'):.3f} |
        ↪  {mean('flight_failures'):.1f} | {mean('rejections'):.1f} | {mean('decisions'):.1f} |")
    (destination / 'summary.md').write_text('\n'.join(summary) + '\n', encoding='utf-8')
    print('Saved', destination.resolve())
```

Line-by-line explanation.

4232 Set `methods` by select list(ALL_LEARNED_METHODS) when args.method == 'all', otherwise [args.method]. Compute and bind the right-hand value to methods, naming an intermediate for the surrounding algorithm.

4234 Test `args.method == 'all' and config.aircraft != 1`. Execute the indented branch only when it is true; subsequent alternatives handle the other case.

4236 Call methods.remove with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

4238 Set `result['skipped_methods']` by construct the shown dict in its stated order. Record why this method is not part of the selected fleet run.

4240 Iterate `method` over `methods`. The following indented statements run once for each selected element.

4242 Set `(model, logs, days)` by call train with the shown positional and keyword arguments. Unpack train(config, method, args.steps, args.seed) into named values model, logs, days so each part can be used separately below.

4244 Call torch.save with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

4246 Set `result['evaluation'][method]` by call evaluate with the shown positional and keyword arguments. Compute and bind the right-hand value to result['evaluation'][method], naming an intermediate for the surrounding algorithm.

4248 Set `result[method + '_training']` by evaluate logs. Compute and bind the right-hand value to result[method + '_training'], naming an intermediate for the surrounding algorithm.

4250 Call print with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

4252 Call (destination / 'results.json').write_text with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

4254 Set `summary` by construct the shown list in its stated order. Compute and bind the right-hand value to summary, naming an intermediate for the surrounding algorithm.

4256 Iterate `(method, records)` over `result['evaluation'].items()`. The following indented statements run once for each selected element.

4258 Set `mean` by evaluate lambda k: np.mean([r[k] for r in records]). Compute and bind the right-hand value to mean, naming an intermediate for the surrounding algorithm.

4260 Append one record without replacing existing records. The exact receiver and arguments are shown on this source line.

4262 Call (destination / 'summary.md').write_text with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

4264 Call print with the shown positional and keyword arguments. The exact receiver and arguments are shown on this source line.

9.159. Module declarations and integration

Source lines 4268–4270.

```
4268  OPERATE_UNTIL_LOW = 0
4270  REPLACE_NOW = 1
```

Line-by-line explanation.

4268 Set `OPERATE_UNTIL_LOW` by use the literal 0.

4270 Set `REPLACE_NOW` by use the literal 1.

10. Closing interpretation

The updated tutorial now contains genuine multi-agent learning interfaces and several explicitly scoped hierarchy implementations. Sharing weights, factorizing outputs, using global information, learning option duration, and coordinating resources are distinct design choices. The current code is most useful when these choices are inspected separately and compared under matched authority and information. Its software coverage is substantially broader than the earlier report, while physical calibration and controlled research conclusions remain future work.

References

[1] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*, second edition. MIT Press, 2018. http://incompleteideas.net/book/the-book-2nd.html.

[2] R. S. Sutton, D. Precup and S. Singh. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112:181–211, 1999. https://doi.org/10.1016/S0004-3702(99)00052-1.

[3] J. Schulman et al. Proximal Policy Optimization Algorithms. 2017. https://arxiv.org/abs/1707.06347.

[4] Stable-Baselines3 contributors. Custom Policy Network documentation. Accessed 26 October 2026. https://stable-baselines3.readthedocs.io/en/master/guide/custom_policy.html.

[5] C. P. Andriotis and K. G. Papakonstantinou. Managing engineering systems with large state and action spaces through deep reinforcement learning. *Reliability Engineering & System Safety*, 191:106483, 2019. https://doi.org/10.1016/j.ress.2019.04.036.

[6] C. P. Andriotis and K. G. Papakonstantinou. Deep reinforcement learning driven inspection and maintenance planning under incomplete information and constraints. *Reliability Engineering & System Safety*, 212:107551, 2021. https://doi.org/10.1016/j.ress.2021.107551.

[7] C. Yu et al. The Surprising Effectiveness of PPO in Cooperative Multi-Agent Games. *Advances in Neural Information Processing Systems*, 35, 2022. https://proceedings.neurips.cc/paper_files/paper/2022/hash/9c1535a02f0ce079433344e14d910597-Abstract.html.

[8] T. G. Dietterich. Hierarchical Reinforcement Learning with the MAXQ Value Function Decomposition. *Journal of Artificial Intelligence Research*, 13:227–303, 2000. https://www.cs.cmu.edu/afs/cs/project/jair/pub/volume13/dietterich00a.pdf.

[9] P.-L. Bacon, J. Harb and D. Precup. The Option-Critic Architecture. *Proceedings of AAAI*, 31, 2017. https://doi.org/10.1609/aaai.v31i1.10916.

[10] Z. Xu, Y. Bai, B. Zhang, D. Li and G. Fan. HAVEN: Hierarchical Cooperative Multi-Agent Reinforcement Learning with Dual Coordination Mechanism. *Proceedings of AAAI*, 37(10):11735–11743, 2023. https://doi.org/10.1609/aaai.v37i10.26386.

[11] P. Sunehag et al. Value-Decomposition Networks For Cooperative Multi-Agent Learning. *Proceedings of AAMAS*, 2018. https://arxiv.org/abs/1706.05296.